Ingegneria del Software

Appunti del corso

Sezione Prof. Alessandro Margara

Anno accademico 2023/2024

Lezione 1: Introduzione (14/09/2023 Lorenzo Battini)

Cos'è l'ingegneria del software?

- Ciclo di sviluppo, dall'idea/specifica al prodotto finito. In particolare, studia un approccio sistematico e ingegneristico allo sviluppo, alla messa in opera (come si installa, va caricato in un server), alla manutenzione del software.
- Il software può cambiare nel tempo, può essere necessario modificare quanto scritto in precedenza o riprendere codice scritto da altri.
- Processo che include vari aspetti, manageriali, tecnologici.
- Si studiano metodi empirici e sostenibili per prevedere i costi per tutta la vita del software e per ottimizzare:
    - Gestione tempo, stare nei tempi richiesti dal cliente.
    - Gestione risorse, per esempio le risorse umane richieste.
    - Gestione qualità.
- I sistemi studiati sono complessi, per funzionalità, evoluzione nel tempo, adattabilità (versioni personalizzate alle varie necessità dei clienti).

Rispetto all'ingegneria tradizionale dove c'è una grande fase di design iniziale, ma una volta costruito il prodotto resta pressoché invariato nel tempo (salvo manutenzione o simili), nell'ingegneria del software di solito il prodotto è in continua evoluzione, soggetto a continue modifiche. Ci sono rare eccezioni come software industriale legato a macchinari specifici (che quindi resta invariato per tutta la durata della macchina).

Inizialmente il software era "usa e getta", usato da scienziati/specialisti per fare calcoli in modo più rapido che non a mano (scritto e usato dalla stessa persona).

Nel tempo l'utilizzo si è evoluto, diventando pervasivo.

Verso gli anni '80-'90 si arriva al punto in cui si fa fatica a stare dietro con i metodi "artigianali" usati prima al crescere delle esigenze di utilizzo del software.

Ingegneria del software: come per le altre ingegnerie ci si occupa di trovare un processo efficace, diviso in fasi precise, per arrivare a un prodotto della qualità richiesta, nei tempi richieste.

Un'altra differenza rispetto ad altri tipi di ingegnerie (come l'ingegneria civile), è che queste ultime usano metodi standardizzati, studiati e stabiliti da tempo. Inoltre, come detto sopra, in tali ambiti è difficile, spesso quasi impossibile o comunque non praticabile, modificare il prodotto dopo la messa in opera.

A livello tradizionale, il software viene visto come innovativo. Lo sviluppatore crea un prodotto nuovo, si considera quasi un'artista e continua a modificarlo quando serve.

L'ingegneria del software mira ad "adattare" le tecniche delle altre ingegnerie per sviluppare software, cioè per trovare metodi standard, dei "building blocks" robusti e riutilizzabili per creare nuovo software.

Lo sviluppo del software non è solo l'implementazione, come è stato visto per esempio nel corso di fondamenti di informatica in cui i programmi da sviluppare sono piccoli.

È molto importante anche la progettazione, e i prodotti generati sono tutti gli artefatti necessari al ciclo di vita del software, quindi non solo il codice sorgente e/o l'eseguibile ma anche documentazione,

documenti interni per comunicare tra vari gruppi, script per installare o aggiornare, test case utili a verificare il corretto funzionamento del programma.

Spesso uno dei problemi iniziali del software è comprendere correttamente le specifiche da quanto richiesto da un cliente che proviene da un ambito diverso, magari specifico e non conosciuto dallo sviluppatore.

La tecnica object-oriented aiuta a scomporre il programma in modo più vantaggioso per questo ambito.

Processo di sviluppo del software

Dal metodo "code & fix", un po' a casaccio usato primo, un primo metodo più ordinato a cui si è pensato è il metodo a cascata (anni '70, la prima conferenza è stata nel 1968). Quest'ultimo è lineare, ispirato in un certo senso alla catena di montaggio. È diviso in varie fasi (sequenziali):

- Studio di fattibilità, studiare i costi e i vantaggi/difetti di un progetto, per comprendere se svolgere tale progetto (oppure potrebbero esserci più alternative tra cui scegliere);
- Analisi e specifica dei progetti, comprendere e scrivere quanto richiesto dai clienti (interfaccia grafica, numero di utenti): non ci occuperemo molto di questa parte.
- Progettazione: dividere in moduli, unità da distribuire tra vari team di sviluppo
- Implementazione e test di unità: test di unità vuol dire che ciascun team si occupa di validare la funzionalità dell'unità di cui si occupa
- Integrazione e test di sistema: spesso in modo bottom up, si integrano le componenti a partire dalle unità in aggregati sempre più grandi verificando a ogni passo il corretto funzionamento dell'aggregato
- Messa in opera: per esempio, installare sui computer/server del cliente, o sui server del centro di sviluppo software se si prevede di accedere da remoto
- Manutenzione: non solo correttiva, risolvere problemi, ma anche evolutiva, aggiungere nuove funzionalità.

Ci sono due aspetti di test: verifica, la correttezza rispetto alle specifiche che è quanto si fa fino alla messa in opera circa; la validazione, controllare se il risultato è proprio quello che prevedeva il cliente, svolta quando si arriva al test di sistema o prima della messa in opera, perché è necessario avere un prodotto quasi finito da mostrare a cliente. Questo è vero in questo modello, mentre in altri si cerca di validare anche prima, per evitare di dover implementare di nuovo grandi porzioni di codice e quindi incorrere in costi significativi.

Non vi è un unico processo di sviluppo che va bene per qualsiasi tipologia di software, dipende molto dall'ambito di utilizzo, per esempio cambia tra una web app, una applicazione industriale/aziendale o software per ambiti "critici" come dei sistemi per il pilotaggio di aeroplani. Ci sono alcuni design subottimali, che è meglio evitare a favore di altri, ma ce ne sono molti altri ugualmente validi, o con trade-off di peso abbastanza uguale, per un certo ambito.

Ci concentreremo sulle fasi evidenziate.

Uno dei limiti di questo modello è la lentezza dovuta a dover testare ogni componente nelle varie fasi, portando al passaggio di molto tempo prima di poter rilasciare un prodotto al committente.

La possibilità di essere modificato è uno dei vantaggi del software, ed è la ragione per cui adesso si sta cercando di spostare alcuni elementi dall'hardware al software.

Cicli di vita “agili”

Ce ne sono vari, l'idea è che il vero valore stia nel prodotto (software), e quindi l’obiettivo è garantire di avere pronte versioni del software che abbiano un valore per il committente molto frequentemente, avendo anche la possibilità di tornare indietro ed effettuare modifiche.

Scrum

Si concentra sulla parte manageriale, meno su quella tecnica.

Visto che il cambiamento si vuole, ma è anche fastidioso, si procede per fasi, dette sprint, nelle quali si effettuano alcune delle modifiche che si è deciso di svolgere in procedenza. All’interno dello sprint nessuno può chiedere di cambiare requisiti o chiedere modifiche, esse verranno effettuate nella prossima fase.

I team sono di 5-8 persone, e quindi si occuperanno di una porzione idonea del progetto. Queste dimensioni servono a permettere la comunicazione diretta, a voce, evitare la dispersione.

In ogni team vi è un rappresentante dei committenti, detto product owner, una persona tecnica che si interfaccia con i committenti, fa comunicare i due mondi. Quindi per esempio evidenzia al committente cosa sia fattibile nei tempi dati e cosa no.

Product backlog: le feature richieste dal committente (requisiti), sono scritte in modo sintetico, come user stories con linguaggio semplice. Viene poi assegnato a ogni user story un voto, quindi una priorità, con la supervisione di un tecnico che ne verifica la fattibilità. I cambiamenti devono essere divisi in pacchetti da completare in 4-5 settimane, e poi ancora in task da assegnare nei team. Al massimo si sbaglia leggermente la stima e quindi non si completano alcuni task, ma si cerca di tenere sotto controllo.

Tutti questi processi cercano di fornire qualcosa di tangibile al committente al seguito di ogni sprint il prima possibile (per questo è detto agile).

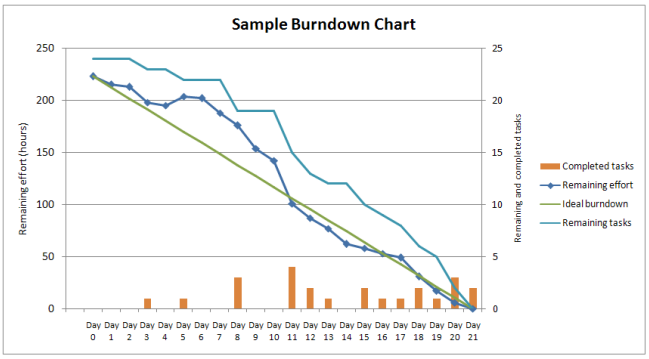
Si cerca di mantenere snello il processo.

All’interno del team non vi è un manager, ma un team leader, perché si favorisce la collaborazione e l’organizzazione orizzontale. Il team leader non è quindi di solito un manager, ma uno sviluppatore che facilita la comunicazione, ha una visione di livello più alto.

Stand up meeting, da fare “in piedi” nel senso che deve essere rapido, 15 minuti ogni giorno, serve solo per sapere se ci sono problemi e fissare meeting più lunghi se servono. Serve a minimizzare i tempi, facilitare la comunicazione (evitare che si debba andare a cercare persone per capire come risolvere i problemi).

Ci sono altre cerimonie (cioè incontri del team), come un meeting finale in cui si fa una demo tecnica di quanto si è fatto di solito con il committente per fare validazione, ma senza documentazione.
Quindi al termine dello sprint si mostra al cliente quanto si è fatto.

Retrospective meeting, ragionare sul processo fatto.



Tutto viene tenuto sotto controllo. Vi è un burndown chart, che mostra quanti task sono stati svolti in giorni e quanti ci si aspettava che venissero svolti.

Extreme Programming (XP)

Si concentra sulla parte tecnica.

Propone aspetti tecnici per fornire in modo efficiente il prodotto.

Una delle idee proposte è evitare design perché il software si evolve così rapidamente che conviene modificare di continuo il codice (refactoring) tenendolo anche in ordine. Un messaggio ragionevole è quello di non sprecare troppo tempo nella fase di design.

Pair programming: a turno uno scrive, l'altro da dietro controlla che non ci siano errori.

Il testing è integrato nel processo (test first), si simula l'utente per verificare il funzionamento nel corso dello sviluppo. È un aspetto talmente importante che talvolta si definiscono a priori i casi di test che devono essere superati.

AB testing: due versioni in contemporanea da confrontare, per esempio si propone a una percentuale di utenti una versione nuova e la si sceglie se è più proficua.

DevOps

È molto utilizzato attualmente. Un primo gap è tra il committente e lo sviluppatore. Un secondo gap che emerge soprattutto nel caso di software offerto come servizio, quindi quando la logica si trova in un server e poi l'utente si connette con una web app/thin client, è che quello tra developers e operators. I developers (sviluppatori) implementano il codice, gli operators si occupano di installare il codice. Esempio di problematiche possono essere la saturazione delle risorse (per esempio della memoria RAM) dopo alcuni giorni di utilizzo, oppure in un momento di utilizzo più intenso (per esempio il giorno di lancio di un prodotto).

DevOps mira a colmare questo secondo gap "unendo" in unico team developers e operators. Questo team si occupa quindi anche di verificare, per esempio, che il software sia scalabile e scalarlo in base alle necessità. Deploy, install; operate; monitor, osservare il software mentre viene eseguito e capire se ci sono problematiche o elementi che si possono ottimizzare.

Sistemi a microservizi: un sistema complesso viene scomposto in tanti piccoli microservizi (in modo tale da poter essere mantenuti ciascuno da un team da 5-8 persone) che interagiscono per creare il servizio più grande. L'interazione tra microservizi avviene spesso via web, come client server. I microservizi espongono verso l'esterno solo le funzionalità necessarie, il resto viene gestito internamente dal team che se ne occupa. Questo permette al sistema di evolversi rapidamente dato che i vari microservizi possono essere modificati in modo indipendente.

Fattori scatenanti: software offerte come servizi; la possibilità di accedere/spegnere macchine facilmente, in particolare grazie a macchine virtuali o container, il che rende possibile scalare in base alla necessità, in modo differente per i vari microservizi; sistemi di monitoraggio che funzionano molto rapidamente, ampiamente automatizzati, che per esempio una volta che viene sviluppata una

nuova versione viene in automatico testata e, nel caso in cui passi i test, la installa (continuos deployment).

I cicli di vita cercano di fronteggiare l'evoluzione del software, che non è necessariamente lineare. Non vi è un ciclo di vita usato da tutti, è una scelta fatta in base all'ambito dalle singole aziende.

Lezione 2: Programmazione Object-Oriented (18/09/2023 Antonino Ciancimino, Matteo Delogu)

La programmazione ad oggetti si fonda sul principio generale del **riuso**. Questo assume una sua connotazione specifica nell'ingegneria del software: si vuole suddividere il programma in piccoli segmenti componibili fra loro. Queste entità vengono chiamate **moduli** e, a questo livello di astrazione, sono sostanzialmente definibili come segmenti di codice riusabili, poi vedremo essere quelli che chiamiamo **oggetti**.

Il principio del riuso ha vari benefici, tutti derivanti dalla possibilità di utilizzare intere librerie di moduli scritti da entità terze:

- non dobbiamo reimplementare tali funzionalità => riduzione dei tempi di sviluppo;
- la responsabilità di tali moduli è dell'entità terze => minore manutenzione da fare;
- supponiamo che l'entità terze siano esperte nel loro campo => maggiore robustezza ed efficienza del codice;
- tempo e generica fatica risparmiata possono essere usate per scrivere il proprio codice => possibilità di sfruttare al massimo le proprie risorse.

Applicando questa filosofia di sviluppo si potrebbero, in ogni caso, incontrare delle difficoltà:

- i moduli del programma (scritti da noi o integrati tramite una libreria esterna) devono essere **adattabili** (se scritti da noi) e **adatti** (se scritti da terzi) alle nostre esigenze;
- non si ha mai la piena certezza che i moduli scritti da terzi siano effettivamente affidabili;
- Si potrebbe teoricamente **perdere d'efficienza**, dal momento che le soluzioni *preconfezionate* sono sempre più generiche di quelle che potremmo implementare noi, tagliate esattamente sul nostro problema e quindi ottimizzate *ad hoc*.

Il processo concettuale di divisione del problema in parti, con la conseguente implementazione in un certo linguaggio di programmazione, è detto **modularizzazione**.

PER POTER AVERE IL "RIUSO" HO BISOGNO DELLA MODULARIZZAZIONE.

Le parti devono godere di certe proprietà che ne assicurano la buona integrazione nel sistema:

- autonomia rispetto alle altre parti => **coesione;**
  - *I moduli devono fare "poche cose", essendo così di facile comprensione*
- ridotta interazione con le altre parti => **scarsamente connessi**;
  - *Le connessioni tra i moduli devono essere lineari e comprensibili*

In ogni modulo è fondamentale distinguere due aspetti:

- **Interfaccia**: cosa fa il modulo, quali servizi offre all'esterno, cosa mostra agli utilizzatori;

- **Implementazione**: come fa il modulo ad offrire un servizio, ossia le peculiarità interne al modulo, tutto ciò che egli compie al fine di andare dall'input fornito all'output desiderato.

Osservazione: Dal nostro punto di vista l'interfaccia di un modulo è più importante della sua implementazione, dal momento che questa definisce come il modulo deve essere usato e come si integra al resto del sistema software; tra l'altro questa definisce una sorta di **contratto** fra esso e gli utilizzatori: il fornitore del modulo di impegna a fornire un buona implementazione del servizio richiesto con una buona interfaccia, l'utilizzatore si impegna invece ad **usare il modulo adeguatamente, ossia solo e soltanto tramite l'interfaccia** e non tramite vie terze (ex. accedere direttamente ai campi di una *struct* in C, invece di usare le procedure scritte dal fornitore a tal fine).

La modularizzazione del programma ha due conseguenze positive:

- permette il riuso, infatti in un progetto modulare è possibile riusare ogni modulo più volte; questo ha un impatto significativo: permette di ridurre la complessità del problema, suddividendolo in problemi minori (secondo il principio del divide et impera);
- influenza positivamente altre qualità del software, soprattutto le seguenti:
    - verificabilità: è possibile testare ogni modulo separatamente ed indipendentemente;
    - manutenibilità: è possibile localizzare e circoscrivere molto chiaramente in quali moduli è necessario fare delle modifiche;
    - comprensibilità: una divisione in moduli dà una visione più chiara del significato del codice nel suo complesso.

La modularizzazione è possibile anche senza il concetto di oggetto e di programmazione *object-oriented*, ad esempio in C si può effettuare una *sorta* di modularizzazione tramite le procedure e le *struct*.

Questo approccio è detto *top-down* e si sviluppa così:

- la funzionalità principale è suddivisa ricorsivamente (ossia più volte) in funzionalità minori;
- si smette di scomporre quando le micro-funzionalità ottenute sono facilmente implementabili.

Questo modello è efficace ma solo nel caso si sviluppi un sistema di piccole dimensioni, ad esempio un singolo algoritmo; in sistemi di dimensioni sostenute questo degenera facilmente nella confusione, dato che si decidono precocemente i vincoli fra i vari moduli (che come detto prima dovrebbero godere di buona indipendenza l'uno dall'altro).

Per introdurre successivamente il concetto di **oggetto** bisogna chiarire ora cosa è un **tipo di dato**: formalmente è un insieme di valori (dominio) e un insieme di operazioni (operazioni interne al dominio). È importante associare al tipo anche le operazioni poiché queste, anche se definite dallo stesso simbolo o nome, possono avere effetti molto diversi se relative a domini diversi.

Ad esempio: il prodotto nei numeri razionali e nei numeri reali si comporta nella solita maniera intuitiva che conosciamo, il prodotto righe per colonne nel dominio delle matrici è invece ben diverso.

Il concetto di tipo porta con sé degli ottimi vantaggi:

- permette al programmatore di non doversi occupare delle celle di memoria;
- *astrae* la struttura di rappresentazione di un dato, vale a dire rende **trasparente all'utilizzatore** del tipo come questo è implementato nella macchina in uso;

- permette al compilatore di segnalare eventuali operazioni illecite sui dati (*type checking*).

```
Data d;
int g;
d = g; illegale! ... forse non si voleva scrivere così!
```

Come parzialmente accennato sopra, quando si lavora ad un progetto di grandi dimensioni le *astrazioni solo procedurali* (ossia la semplice divisioni del problema in procedure) non sono sufficienti per avere una buona modularizzazione

Esempio

Poniamo di voler realizzare un applicativo gestionale di discrete dimensioni. A questo progetto vengono affidati diversi programmatori, ognuno incaricato di realizzarne una parte.

- ***Alberto*** deve programmare l'interfaccia utente e definire il tipo "`Data`"

```
typedef struct {
   int giorno;
   int mese;
   int anno;
} Data;

void stampaData(Data d) {
printf("%d", giorno);
if (d.mese == 1)
  printf("Gen");
else if (d.mese == 2)
  printf("Feb");
...
printf("%d", anno);
}
```

- ***Bruno*** utilizza il tipo "`Data`" per scrivere una applicazione per il pagamento degli stipendi

```
Data d;
...
d.giorno=14;
d.mese=12;
d.anno=2000;
...
if (d.giorno == 27)
   pagaStipendi();
if (d.mese == 12)
...  pagaTredicesime();
```

→ ***Alberto***, nella prima versione della sua parte di codice, definisce "`Data`" come tre interi.

→ ***Bruno***, visto il codice di Alberto, inserisce i dati direttamente all'interno della struttura dati utilizzando tre interi e scrive la sua funzione per il pagamento degli stipendi.

Nel corso dello sviluppo Alberto si rende conto che avrebbe potuto snellire il suo codice cambiando la definizione di "`Data`"

```c
typedef struct {
         int giorno;
         char mese[4]; //ora mese é un array di char
         int anno;
   } Data ;

void stampaData(Data d) {
  printf("%d", d.giorno);
  printf("%s", d.mese); //infatti qua si stampa una stringa
  printf("%d", d.anno);
}
```

☹ Il codice scritto da ***Bruno*** non funziona più...

Il mese era un numero, ora è diventato una stringa, ci sarà bisogno quindi di modificare tutte le parti di codice dove veniva usato il mese come intero. Inserendo all'interno dell'`IF` una `strcmp`

Non sappiamo quante volte sia stata usato data all'interno del programma, ma ogni sviluppatore che ha usato la prima definizione (*a tre interi*) sarà costretto a modificare la propria parte di codice sulla falsa riga di ***Bruno***.

Realizziamo quindi che, quando si usa un tipo di astrazione procedurale, le **modifiche sono costose** sia in termini di tempo che, ovviamente, di soldi.

Soluzione

Una possibile soluzione è dare delle regole agli sviluppatori:

Alberto	***Bruno***
Dovrà introdurre una serie di operazioni sul tipo di dato affinché Bruno le possa usare	Non potrà accedere direttamente alla struttura dati

→ ***Alberto*** dovrà introdurre funzioni come:

```c
void inizializzaData(Data *d, int g, int m, int a) {
  d->giorno = g;
  d->mese = m;
  d->anno = a;
}

int leggiGiorno(Data d){return(d.giorno);}
int leggiMese(Data d){return(d.mese);}
int leggiAnno(Data d){return(d.anno);}
```

I modelli di funzione compongono così **l'interfaccia.** Quest'ultima non dovrà MAI cambiare nel tempo in modo tale da non dover far cambiare repentinamente codice a ***Bruno***	*Notazione alla Java* - Funzionalità offerte sono `pubbliche` - E l'implementazione interna può variare

-> **le modifiche diventano semplici, veloci e molto meno *costose*** <-

Osservazione: in questo caso, ovvero in C, tra gli sviluppatori posso avere solo una "*convenzione*", mentre Java mi fornisce un costrutto tecnico.

Introduciamo a questo punto il tipo di dato astratto (ADT).

Quest'ultimo può essere considerato sinonimo di oggetto. É una astrazione sui dati che **non** classifica i dati in base alla loro rappresentazione (`implementazione`), ma in base al loro comportamento atteso (`specifica`).

-> In altre parole, i dati vengono classificati in base alle operazioni applicabili su di essi, ovvero le operazioni appartenenti all'interfaccia.

La parte importante nella definizione di un modulo sono le sue **FUNZIONALITÀ**

Non a caso i sistemi enormemente complessi sono governabili poiché siamo stati in grado di costruire buone astrazioni.

→ Ma cos'è esattamente una **astrazione**?
- *È un tipo di ragionamento che mi permette di dimenticare cosa c'è sotto al cofano, consentendomi di ragionare ad un livello superiore in maniera precisa e senza commettere errori.*

Lezione 3: Programmazione Object-Oriented / UML (19/09/2023 Matteo Arrigo, Samuele Allegranza)

Object Orientation – Concetti di base

Oggetti

L'elemento di base della programmazione Object Oriented sono gli **oggetti**, istanze che rappresentano dati o concetti del mondo reale, portati all'interno di un programma. Gli oggetti sono descritti da 3 aspetti fondamentali:

- Stato: Lo stato consiste nell'insieme degli **attributi** dell'oggetto, i quali rappresentano le caratteristiche o le proprietà dello stesso. Sono l'equivalente dei parametri di una struttura dati in C. Gli attributi possono essere mutabili (possono essere modificati) o immutabili (non possono essere modificati).
- Comportamento: L'insieme dei **metodi** dell'oggetto, ovvero le funzionalità dello stesso. Sono l'equivalente delle funzioni in C.
- Identità: Ogni oggetto ne possiede una propria. Due oggetti si dicono uguali se hanno lo stesso stato. Tuttavia, non sono definibili **identici**, perché ognuno ha una propria identità. In "termini tecnici", ogni oggetto ha una propria allocazione di memoria, che lo rende indipendente dagli altri, seppur ne possano esistere altri con contenuto uguale. Posso quindi referenziare attraverso più variabili lo stesso oggetto.

È importante sottolineare che non è possibile modificare gli attributi direttamente, ma è necessario sfruttare dei metodi che modificano a loro volta internamente lo stato. Questi metodi sono detti **modificatori**. Esistono quindi due tipologie di metodi

- Modificatori: modificano lo stato
- Osservatori: non modificano lo stato

Esempio: Un venditore di automobili identifica l'oggetto Automobile, il quale possiede:

- Stato (attributi): targa, telaio, colore, tipo di motore, …

- Comportamento (metodi):
    - Modificatori: cambiaTarga, impostaTelaio, cambiaColore…
    - Osservatori: accendi, spegni, sterza, ...

Classi

È il modello (o "blueprint", "stampo") con il quale si costruiscono gli oggetti. Si tratta, perciò, di una **astrazione** della realtà. La classe è il "concetto", mentre l'oggetto è una specifica istanza di tale "concetto".

Parafrasando con il C, le variabili possono essere viste come le istanze di uno stesso tipo, dove il tipo assume invece il ruolo di classe.

Riprendendo l'esempio precedente del venditore di automobili, esisterà la classe Automobile che descriverà la struttura dello stato e del comportamento. Per ogni automobile che il venditore possiede in negozio verrà creata un'istanza della classe Automobile (ovvero un oggetto) che descrive tutte le sue caratteristiche (la propria targa, il seriale del telaio, il suo colore, …)

Le proprietà di una classe possono essere **pubbliche** o **private**. Le proprietà private non sono accessibili dall'esterno della classe, come se avessero un "guscio" che le protegge dall'esterno, mentre quelle pubbliche sì. In genere i linguaggi di programmazione permettono di scegliere per ogni proprietà di classe la visibilità, ma è buona norma, per mantenere l'incapsulamento, impostare gli attributi tutti come privati mentre i metodi come pubblici

Possono anche esistere metodi e attributi ***di classe***, i quali sono condivisi tra tutte le istanze della stessa classe. Possono essere utili per ottimizzare l'uso delle risorse in quei casi dove è inutile replicare la stessa proprietà per ogni oggetto istanziato.

È anche possibile creare delle classi che fanno riferimento ad altre classi. Le **relazioni** che possono svilupparsi tra queste ultime possono essere di:

- Aggregazione: ad esempio la classe Volante *appartiene* alla classe Automobile
- Uso: ad esempio la classe Persona *usa* la classe Automobile
- Ereditarietà: ad esempio la classe AutomobileElettrica è una classe che *specializza* la più generale classe Automobile. La sottoclasse (nell'esempio, AutomobileElettrica) eredita stato e comportamento delle sopraclassi (nell'esempio, Automobile). Le sottoclassi possono aggiungere proprietà o modificare quelle esistenti.

L'**ereditarietà** porta chiaramente una serie di vantaggi (primo fra tutti riuso ed estensibilità'), ma anche degli svantaggi. Si ha una località debole (dislocazione del codice) e una potenziale perdita di efficienza per via del binding dinamico.

Polimorfismo

Il polimorfismo è la capacità di un'entità di assumere forme diverse nel corso della propria esistenza. Infatti, per una variabile distinguiamo:

- Tipo statico della variabile: noto a compile-time.
- Tipo dinamico di variabile: noto solo a run-time.
  È il tipo specifico che la variabile poi assume in certo istante a programma avviato.

In sinergia con l'ereditarietà, se una variabile polimorfica ha una certa classe come tipo statico, a runtime potrà assumere come tipo dinamico quello di quella classe o di qualsiasi delle sue classi figlie. Quindi il polimorfismo sfrutta la gerarchia dei tipi risultante dall'ereditarietà.

Si può avere polimorfismo anche per i metodi. Questo è basato sulla distinzione di:

- Static binding: il legame fra il nome della funzione e il codice da eseguire viene stabilito in fase di compilazione
- Late / dynamic binding: il legame fra il nome della funzione e il codice da eseguire avviene a runtime

Usare il late binding permette così una gestione dinamica delle invocazioni dei metodi delle classi. Inoltre ogni sottoclasse che eredita metodi dalla classe madre può anche sovrascriverli. Quindi usando il late binding possono esistere molteplici metodi con lo stesso nome ma comportamento differente, in base alla classe di appartenenza nella gerarchia ereditaria.

Sfruttando il polimorfismo tra classi ereditate, all'invocazione di un metodo su un oggetto si cercherà il codice del metodo invocato prima nel corpo della classe specifica (tipo dinamico) di quell'oggetto, e se poi non si trova si risale la gerarchia ereditaria andando alla classe madre, etc.

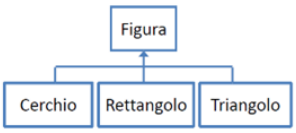


Soluzione tradizionale

```
rettangolo: Rettangolo;
cerchio: Cerchio;
triangolo: Triangolo;
figure: array[1..100] Of Figure;

figure[1] := rettangolo;
figure[2] := triangolo;
figure[3] := cerchio;

for i:= 1 to 100 do
  if figure[i] = "Rettangolo"
    // disegna rettangolo
  end;
  if figure[i] = "Triangolo"
    // disegna triangolo
  end;
  ....
end;
```



Soluzione Object-Oriented

```
rettangolo: Rettangolo;
cerchio: Cerchio;
triangolo: Triangolo;
figure: array[1..100] Of Figure;

figure[1] := rettangolo;
figure[2] := triangolo;
figure[3] := cerchio;

for i:= 1 to 100 do
      figure[i].disegna();
```

Esempio: costruiamo un array di figure

Struttura delle classi: 3 classi (Cerchio, Rettangolo, Triangolo) ereditano dalla classe madre comune Figura, e assumiamo che la classe Figura specifichi il metodo disegna, ereditato così da tutte le classi figlie.

L'array è dichiarato come array di Figura (tipo statico). Ma a runtime, ogni elemento dell'array, grazie al polimorfismo, potrà essere un'istanza di Figura, Cerchio, Rettangolo o Triangolo (tipo dinamico).

Inoltre, per disegnare la figura non c'è bisogno di distinguere la figura stessa per poi usare un metodo apposito (come nella soluzione tradizionale), ma si può sfruttare il late binding e invocare il metodo disegna() definito da Figura e ereditato (e magari sovrascritto) dalle classi figlie.

In più una soluzione del genere è facilmente estensibile per aggiungere figure specifiche, poiché basta aggiungere la classe figlia e non serve cambiare altro codice!

Unified Modeling Language

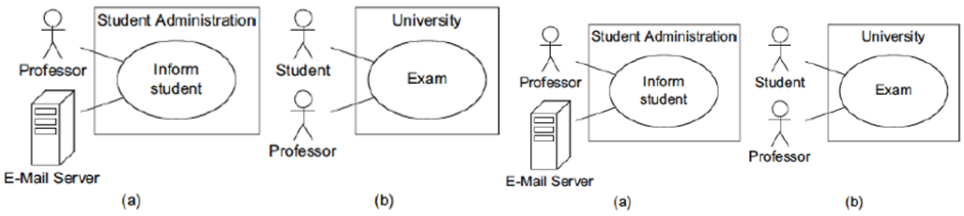
UML (= Unified Modeling Language) è un linguaggio grafico standardizzato usato per avere una progettazione grafica della realtà. Fornisce un modello, cioè una descrizione astratta del sistema da progettare e dell'ambiente in cui dovrà operare. Modellare il sistema aiuta gli analisti a capire le funzionalità del sistema e potrà poi essere usato per meglio comunicare con i committenti.

Si possono avere diversi punti di vista sul sistema da progettare, e questo dà luogo a diversi modelli grafici. Possiamo dividere i molti tipi di diagrammi UML principalmente in:

- Diagrammi di struttura: descrivono la struttura del programma. Sono usati come una foto statica del sistema. Possono avere granularità più o meno fine.
  Noi ci focalizziamo sui Class diagram, che hanno granularità fine, focalizzandosi su classi, attributi, etc.
- Diagrammi di comportamento: mostrano alcuni scenari interessanti di interazione tra classi. Sono usati come un video del sistema in azione. È utile per documentare come interagiscono tra loro i vari oggetti a runtime.
  Noi ci focalizziamo su Sequence diagram

Lo scopo comunque è solo modellare il sistema, e questo non basta per l'implementazione effettiva (l'implementazione non è una traduzione 1 a 1 dei diagrammi UML, anche se messi tutti insieme…)

Diagrammi di use case



(esempi)

Lo scopo dei diagrammi di use case è documentare ad altissimo livello le funzionalità attese, chi le usa e per quali intenti. Descrivono i requisiti di sistema, quindi equivalgono più o meno alle user stories di SCRUM

Class diagram

Un class diagram descrive la struttura delle classi del programma. Ha un livello di dettaglio maggiore rispetto i diagrammi di use case, poiché descrive l'effettiva struttura implementativa della nostra futura soluzione. Nell'implementazione con Java, le classi descritte definite in UML arrivano a corrispondere effettivamente con le classi Java, con un'operazione di traduzione del diagramma in codice.

Una classe UML è schematizzata da un box diviso in 3 parti:

- Nome della classe
- Lista attributi, con sintassi <visibilità> <nome>: <tipo>
- Lista metodi, con sintassi <visibilità> <nome>(<lista attributi>): <tipo di ritorno>
  In <lista attributi>, gli attributi sono divisi da ‘,’ e ognuno ha la sintassi <nome>: <tipo>

Tra i modificatori di visibilità, ci sono ‘-‘ per privato e ‘+’ per pubblico. Le proprietà di classe sono sottolineate.

Solitamente si usa la convenzione per cui i tipi definiti dall’utente (le classi) hanno la prima lettera maiuscola, mentre i tipi primitivi e il nome delle variabili hanno la prima lettera minuscola.

Class diagram - Associazioni

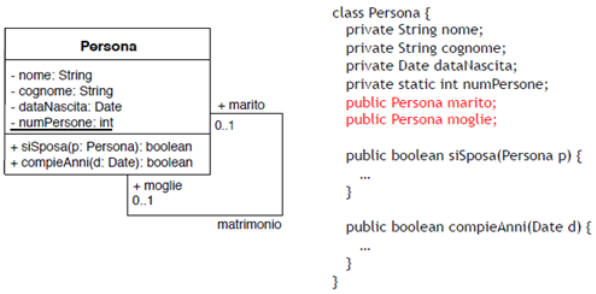
Le associazioni tra due classi sono definite da una linea continua che congiunge le classi stesse. In genere trovano una corrispondenza nel codice e aiutano a mettere in evidenza graficamente gli aspetti importanti della nostra modellizzazione. Possono avere un nome (label) a scopo solo documentativo.

L’associazione può avere un nome documentativo. Vicino la classe mettiamo la sua cardinalità nell’associazione, cioè il numero minimo e massimo di istanze che possono partecipare a quell’associazione.

Un’associazione implica l’esistenza di attributi impliciti nelle due classi per fare riferimento alle istanze dell’altra classe. Cioè, se la classe A è in associazione con la classe B, allora A dovrà avere un attributo di tipo ‘classe B’ che indica quali istanze della classe B sono legate alle istanze della classe A, e viceversa. In base alla cardinalità dell’altra classe, l’attributo sarà scalare o un array. Il nome di quest’attributo può essere messo vicino la classe, e ci si può mettere anche la visibilità dell’attributo. Se il nome dell’attributo non c’è, nell’implementazione gli si potrà dare un nome qualsiasi.

Se invece di una linea si ha una freccia, l’associazione ha solo una direzione. Quindi gli attributi usati come riferimenti nell’associazione sono da considerare solo per la classe alla base della freccia.

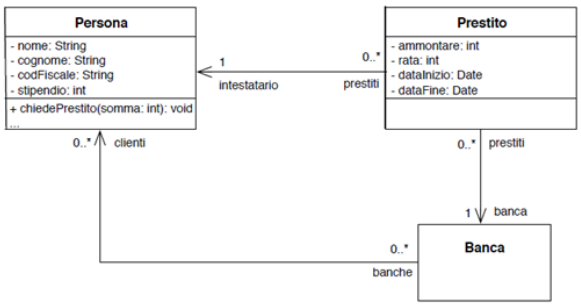
Esempi




```
class Persona {
  private String nome;
  private String cognome;
  private Date dataNascita;
  private static int numPersone;
  public Persona marito;
  public Persona moglie;

  public boolean siSposa(Persona p) {
    ...
  }

  public boolean compieAnni(Date d) {
    ...
  }
}
```




```
class Persona {
  private String nome;
  private String cognome;
  private String codFiscale;
  private int stipendio;
}

class Banca {
  private Persona[] clienti;
}
```

```
class Prestito {
  private int ammontare;
  private int rata;
  private Date dataInizio;
  private Date dataFine;
  private Persona intestatario;
  private Banca banca;
}
```

Lezione 4: UML (21/09/2023 Simone Curci, Piervito Creanza)

Aggregazioni



Le aggregazioni sono una forma particolare di associazioni, le quali consentono di specificare un oggetto come parte di un altro. Per fare un esempio, partendo da due oggetti: “ruota” e “automobile” è possibile aggregarli in modo da specificare che l’oggetto “ruota” sia part-of dell’oggetto “automobile”.

Composizioni



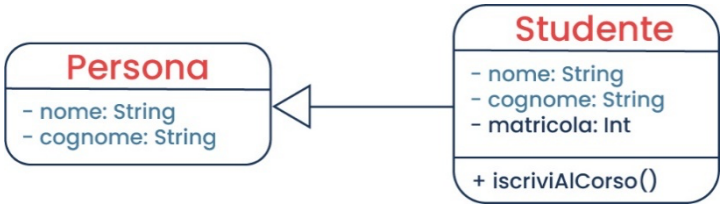
A differenza delle aggregazioni, un oggetto caratterizzato da una relazione di composizione condivide lo stesso ciclo di vita dell’oggetto padre a cui è legato. Questo comporta che nei linguaggi di programmazione che lo permettono, la deallocazione dell’oggetto padre determini anche la deallocazione dell’oggetto associato tramite composizione.

In questo modo, nel caso di una ipotetica rimozione dell’oggetto “auto” sarà automaticamente rimosso anche l’eventuale oggetto “ruote” ad esso associato.

Curiosità

Nel caso di Java, il ciclo di vita dei componenti non è esplicitamente gestito dall’utente, ma se ne occupa in automatico il garbage collector; tuttavia in C++, la mancanza di un garbage collector determina la necessità da parte del programmatore di deallocare gli oggetti non più utilizzati e rende la composizione ancora più comoda. (scomoda?)

Ereditarietà



L’ereditarietà permette di specializzare una classe già presente, aggiungendo alle sue specifiche originali nuovi metodi e attributi. Un esempio può essere una classe “persona” contenente gli attributi nome e cognome, la quale viene ereditata dalla sottoclasse “studente”. In questo modo, oltre agli attributi ereditati di nome e cognome, la classe “studente” potrà prevedere anche un campo aggiuntivo “matricola” e un metodo iscriviAlCorso().

In Java

Nel caso di Java non è possibile ereditare da più classi in quanto nelle classi ereditate potrebbero esserci attributi o metodi che condividono lo stesso nome, ma svolgono funzioni diverse. Esistono dei linguaggi di programmazione in cui tale possibilità è concessa.

Classi astratte

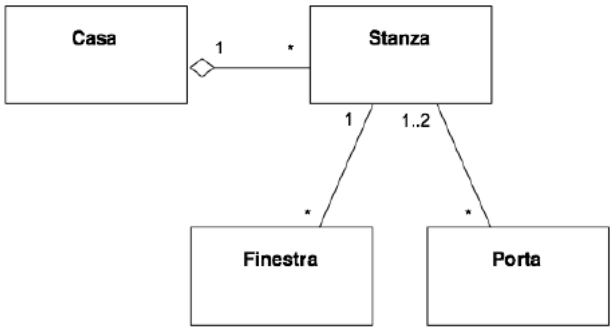
È una classe che non può essere istanziata direttamente, ma serve come modello o struttura per le sottoclassi da essa derivate che ne ereditano le caratteristiche e i comportamenti. Una classe astratta può contenere metodi astratti, che sono dichiarazioni di metodi senza una definizione concreta. Le sottoclassi devono implementare tutti i metodi astratti definiti nella classe astratta.

Curiosità

Nel caso di Java, quando si dichiara una classe astratta, poiché non è possibile conoscere a priori la dimensione della sottoclasse che sarà istanziata, la variabile che si riferisce al relativo oggetto è un puntatore associato a una porzione di memoria presente nello heap.

Diagramma degli oggetti

Per alcune particolari progettazioni la rappresentazione per classi potrebbe non essere la soluzione ideale. Talvolta può essere più chiaro rappresentare i singoli oggetti all'interno del nostro diagramma.



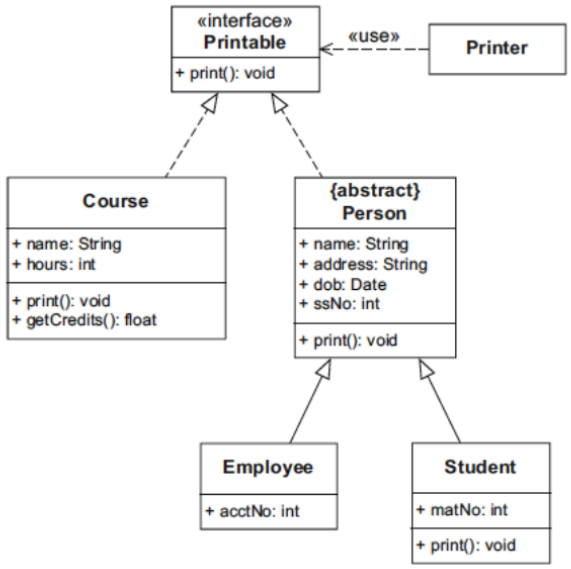
Package

I Package rappresentano delle cartelle che al loro interno includono diverse classi o anche altri packages. L'obiettivo è quello di garantire un adeguato ordine e confinamento. Raggruppare tra loro classi simili permette di semplificare la leggibilità facilitando l'individuazione dei singoli elementi.

Inoltre, attraverso i packages, è possibile introdurre il livello di visibilità per package oltre ai classici livelli pubblico e privato. In questo modo è possibile permettere alle classi interne al singolo package di comunicare tra loro attraverso metodi specifici non visibili dall'esterno.

Interfacce

Le interfacce, similmente alle classi astratte, sono strutture che non possono avere esemplari. Possiamo immaginarle come dei "progetti" che ci danno delle direttive su cosa una classe dovrà fare, ma senza specificare come dovrà farlo. Anche un'interfaccia avrà un nome e dei metodi ma con la differenza che tali metodi dovranno essere implementati dalle classi che vogliono far parte della suddetta interfaccia. Prendiamo come esempio l'interfaccia "Printable" con il metodo "print". Qualunque classe voglia far parte di questa interfaccia, dovrà implementare un suo metodo print. Di conseguenza un'interfaccia è una sorta di "promessa": se una classe vuole essere "Printable" dovrà necessariamente implementare il metodo "print".



Le classi possono relazionarsi a un'interfaccia in due modi differenti:

- attraverso una relazione d'uso
- attraverso una relazione di implementazione.

In questo caso "Printer" è in relazione d'uso perché sfrutta l'interfaccia "Printable" per stampare, mentre "Course" e "Person" sono in relazione d'implementazione dato che forniscono il loro metodo "print".

Diagrammi di interazione

I Diagrammi di interazione mostrano una sequenza temporale che scorre dall'alto verso il basso e descrivono la successione di operazioni e interazioni che avvengono durante l'esecuzione del codice. Questi diagrammi si dividono in tre tipi:

- Diagrammi di sequenza
- Diagrammi di comunicazione
- Diagramma degli stati

Tipi di messaggi

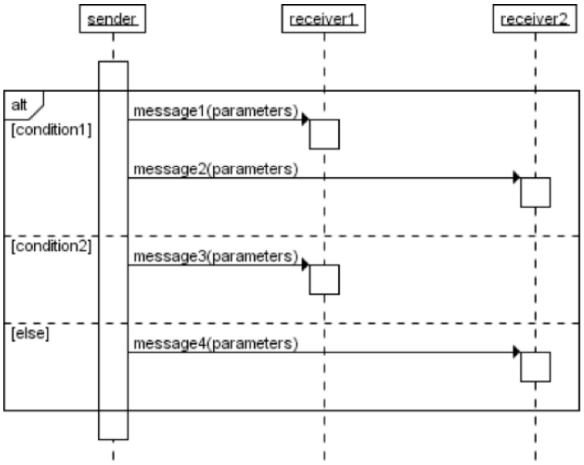


I diagrammi di sequenza sono utilizzati per rappresentare visivamente l'interazione tra oggetti nel tempo, concentrandosi su singoli use-case o problemi.



Nella rappresentazione è presente un attore esterno che interagisce con il sistema. In risposta alla sua interazione, viene invocato un metodo che a sua volta interagirà con una classe. I rettangoli azzurri rappresentano il "tempo di attività" di una classe cioè il tempo in cui rimangono in uso. Con una "X" si indica la deallocazione di un oggetto. Le risposte possono essere implicite, non appariranno nello schema, oppure esplicite, saranno rappresentate con una freccia tratteggiata.
A volte può essere utile utilizzare dei costrutti speciali che ci permettono di semplificare il diagramma.

- loop: ripeto una certa sezione del diagramma finché una condizione non viene soddisfatta
- opt: indico un'operazione opzionale che viene eseguita solo in una determinata condizione
- alt: propongo diverse alternative, ognuna avrà una sua condizione per essere eseguita e se ne seguo una, escludo le altre



Diagrammi di comunicazione

Nei diagrammi di comunicazione, si pone più enfasi sulla schematizzazione, sostituendo il diagramma temporale con label numeriche che seguono l'ordine di esecuzione.

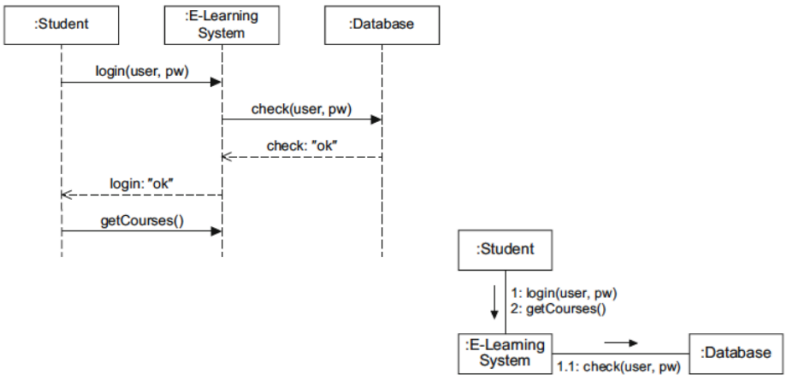
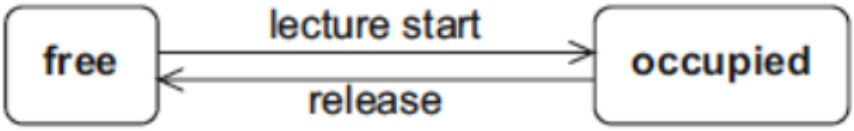


Diagramma degli stati

Il diagramma degli stati viene utilizzato per descrivere il comportamento dinamico di un oggetto o di una classe in risposta a eventi specifici e alle transizioni tra gli stati. Questo tipo di diagramma è particolarmente utile per modellare il comportamento di oggetti o componenti che hanno uno stato interno che può variare nel tempo.
Tale diagramma permette di rappresentare evoluzioni multiple concorrenti o esclusive. Per questo scopo è possibile utilizzare la decomposizione OR (Esclusiva) o la decomposizione AND (Concorrente).



Il diagramma degli stati di un'aula

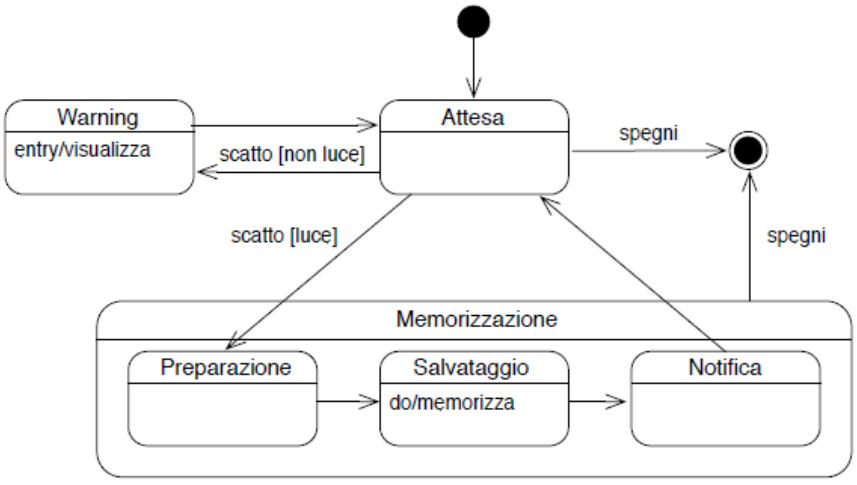
Per esempio, con questo diagramma stiamo rappresentando gli stati in cui si può trovare la classe "aula". Se la lezione sta iniziando, possiamo chiamare un metodo "occupy()" che cambia lo stato dell'aula da "free" a "occupied". Quando la lezione terminerà, chiameremo un altro metodo per liberarla: "release()".

Aula.java

```
public class Aula {
   private boolean free;

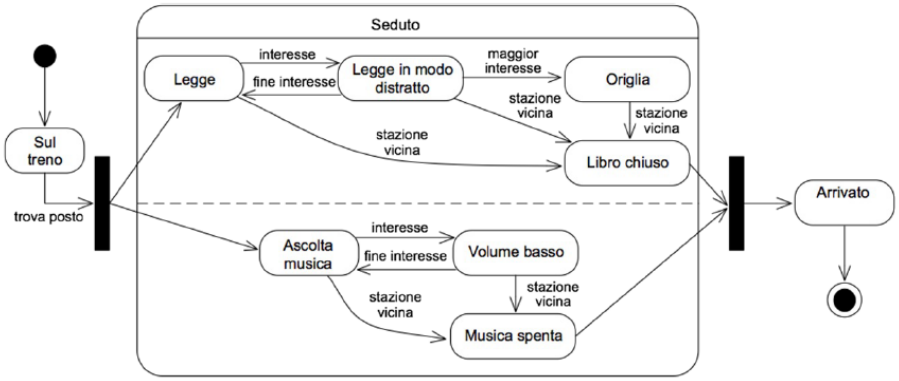
   public void occupy() {
      free = false;
   }

   public void release() {
      free  = true;
   }
}
```



Decomposizione OR

La decomposizione OR è esclusiva, perciò solo uno degli stati può essere attivo in un determinato istante. In questo esempio la classe "fotocamera" parte dallo stato "attesa", se viene chiamato il metodo "scatto()" bisognerà controllare se entra abbastanza luce nell'obiettivo. In caso affermativo si passerà allo stato "Memorizzazione", in caso contrario si passerà allo stato "Warning". L'esclusività è data dal fatto che gli stati "Memorizzazione" e "Warning" non saranno attivi contemporaneamente, o sarà attivo uno o l'altro.



Decomposizione AND (Prof Margara sul treno)

Nel caso dell'AND invece modelliamo attività concorrenti. In questo caso, dopo aver trovato posto, sarò contemporaneamente sia nello stato "Leggi" sia nello stato "Ascolta musica", chiamare il metodo "interesse()", modificherà entrambi gli stati contemporaneamente.

Lezione 5: Introduzione a Java (25/09/2023 Dario Crosa, Alessandro Del Fatti)

Java è un linguaggio creato da Sun Microsystems (ora Oracle).

È un linguaggio orientato agli oggetti molto popolare. Per questo corso utilizzeremo una qualsiasi versione di Java che sia maggiore o uguale alla 8, preferibilmente una versione LTS (long term support).

Linguaggi compilati e interpretati

I linguaggi di programmazione si possono suddividere in due grandi famiglie:

- **Linguaggi compilati**: questi linguaggi richiedono che i file sorgente vengano "tradotti" da un compilatore in codice macchina, che poi viene eseguito.
- **Linguaggi interpretati**: invece che un compilatore c'è un interprete che in tempo reale legge ed esegue il codice, riga per riga

Java adotta un modello ibrido: il codice sorgente viene compilato, ma il target della compilazione non è l'architettura del computer che si sta usando (come avviene per esempio in C) ma è la JVM (Java Virtual Machine). Al termine della compilazione, invece che ottenere un eseguibile che contiene codice macchina, otteniamo un file che contiene un codice specializzato, utilizzato dalla JVM, che si chiama bytecode.

Questo modello di compilazione ha come principale beneficio la portabilità dei programmi Java. infatti, una volta compilato un programma, questo potrà essere eseguito su qualsiasi piattaforma per la quale esista un'implementazione della JVM.

Inoltre, ci sono tutti gli altri benefici che la compilazione fornisce, come la possibilità di ottimizzare il codice in fase di compilazione.

Compilazione e Hello World

Creiamo, compiliamo ed eseguiamo il nostro primo programma Java.

Essendo Java un linguaggio OOP i programmi sono composti da classi, ogni classe deve essere scritta all'interno di un file .java che abbia lo stesso nome della classe contenuta.

Apriamo il terminale in una cartella a piacere e creiamo il file Prova.java, che quindi conterrà la classe Prova. Al suo interno scriviamo un semplice Hello Word.

Prova.java

```
public class Prova {
   public static void main(String[] args) {
      System.out.println("Hello World!");
   }
}
```

Tutti i programmi java iniziano la loro esecuzione dal metodo statico **main()**. (Ora non entriamo nel dettaglio del significato della keyword static).

Per compilare il nostro file in bytecode utilizziamo il comando **javac Prova.java**, questo produrrà nella stessa directory dove ci troviamo un nuovo file: **Prova.class**

Per eseguire il bytecode che abbiamo appena compilato utilizziamo il comando **java Prova** (non serve specificare l'estensione .class).

Tutti i comandi che abbiamo utilizzato sono forniti dal JDK (Java Development Kit) che è il pacchetto di tool per lo sviluppo java. Esiste anche il JRE (Java Runtime Environment) che non contiene tool per lo sviluppo (come javac) ma solo l'implementazione della JVM per eseguire il codice.

Il JDK è il pacchetto per sviluppatori mentre il JRE è il pacchetto pensato per gli utenti finali.

Per il resto del corso useremo un'IDE (come IntelliJ IDEA) che automatizzerà (e nasconderà) queste operazioni tediose.

Classi

Apriamo IntelliJ IDEA e creiamo un nuovo progetto Maven.

Creiamo una classe Data, come dice il nome questa classe deve rappresentare una data del calendario. Questa classe avrà tre attributi di tipo int: giorno, mese, anno e i relativi metodi osservatori (getter).

Aggiungiamo anche un metodo modificatore **capodanno()** che "porta" la data al Capodanno successivo.

Data.java

```
public class Data {
   private int giorno;
   private int mese;
   private int anno;

   public void capodanno() {
      giorno = 1;
      mese = 1;
      anno = anno + 1;
   }

   public int getGiorno() {
      return giorno;
   }

   public int getMese() {
      return mese;
   }

   public int getAnno() {
      return anno;
   }
}
```

Definire una classe definisce anche un tipo di dato (con lo stesso nome) quindi si può definire all'interno di **main()** una variabile di tipo Data.

Data.java

```
public class Main {
   public static void main(String[] args) {
      Data d;
   }
}
```

Tipi

I tipi in Java si suddividono in

- **Tipi primitivi**
  Sono tipi numerici, hanno nomi che iniziano con la lettera minuscola.

Tipo	**Dimensione (byte)**	**Valore di default**
byte	1	0
short	2	0
int	4	0
long	8	0L
float	4	0.0f
double	8	0.0d
char	2	\u0000’
boolean	1	false

- **Classi**
  Tutti gli altri tipi sono classi.
  I loro nomi iniziano con la lettera maiuscola.
  Il valore di default di una variabile che ha come tipo una classe è **null**.
  Non tutti i tipi forniti da Java sono tipi primitivi, per esempio il tipo String è una classe fornita da Java.

Gestione della memoria

Allocazione delle variabili

Quando dichiariamo delle variabili all’interno di un metodo queste occupano dello spazio in memoria; l’allocazione di questo spazio viene gestita in modo diverso a seconda che la variabile contenga un tipo primitivo o una classe.

I tipi primitivi si comportano "come in C": vengono allocate nello stack, all’interno del record di attivazione del metodo

es: int x = 10; definisce nel record di attivazione, sullo stack, uno spazio di 32bit per contenere la variabile x.

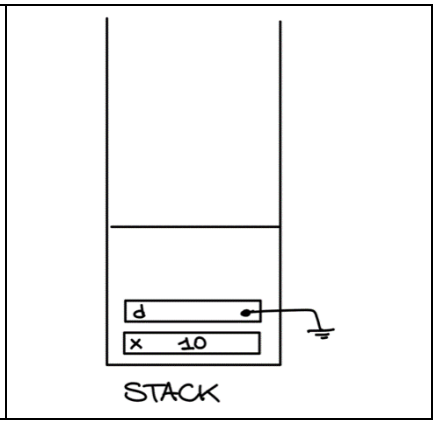
Gli oggetti invece non vengono allocati nello stack ma nello heap. Quindi le variabili che “contengono” oggetti in realtà contengono un puntatore alla zona di memoria nello heap che contiene l’oggetto.

es: Data d = null; definisce nel record di attivazione, lo spazio per un puntatore.

Esempio completo:

Main.java	

```
public class Main {
   public static void main(String[] args) {
      int x = 10;
      Data d = null;
   }
}
```



Attenzione: dichiarare la variabile non alloca immediatamente la memoria per l'oggetto; questo avviene invocando l'operatore **new.**

New e costruttori

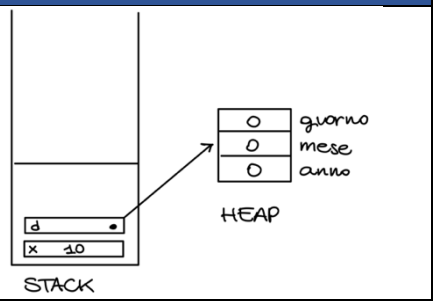
L'operatore new permette di creare un nuovo oggetto, quindi alloca la memoria necessaria e poi invoca il **costruttore**. Il costruttore è un metodo che ha lo scopo di inizializzare i campi della classe quando questa viene creata.

Se noi non definiamo un costruttore, Java ne fornisce uno di default che inizializza tutti i campi della classe con i loro valori di default. Se invece si definisce almeno un costruttore, quello di default non verrà fornito.

es: Data d = new Data(); alloca nello stack lo spazio per d, che è un puntatore e nello heap lo spazio per l'oggetto di tipo Data, poi invoca il costruttore di default di Data (perché noi non ne abbiamo definito uno)

Main.java

```
public class Main {
   public static void main(String[] args) {
      int x = 10;
      Data d = new Data();
   }
}
```



Definiamo un costruttore per la nostra classe Data. I costruttori si definiscono in modo simile ai metodi, ma:

- Devono avere lo stesso nome della classe
- Non hanno valore di ritorno

Possiamo anche definire più costruttori a patto che questi siano distinguibili tra di loro, ovvero che abbiano o un numero di argomenti diverso o alcuni degli argomenti siano di tipo diverso.

Main.java

```
public class Data {
   private int giorno;
   private int mese;
   private int anno;

   public Data(int parGiorno, int parMese, int parAnno) {
```

```
    giorno = parGiorno;
    mese = parMese;
    anno = parAnno;
  }

  public Data(int parMese, int parAnno) {
    giorno = 1;
    mese = parMese;
    anno = parAnno;
  }
  // metodi omessi per brevità
}
```

Ora che abbiamo definito noi un costruttore, il costruttore di default non ci sarà più; se volessi comunque averlo dovremmo definirlo noi manualmente.

This

This *è una keyword che possiamo usare in due modi:*

- Come un riferimento all'oggetto nel quale il metodo si trova
- Come riferimento ai costruttori della classe nel quale mi trovo

Possiamo usare il this per fare un refactor dei costruttori che abbiamo scritto prima:

Main.java

```
public class Data {
  private int giorno;
  private int mese;
  private int anno;

  public Data(int giorno, int mese, int anno) {
    this.giorno = giorno;
    this.mese = mese;
    this.anno = anno;
  }

  public Data(int mese, int anno) {
    this.giorno = 1;
    this.mese = mese;
    this.anno = anno;
  }
  // metodi omessi per brevità
}
```

In questo primo esempio abbiamo utilizzato il this per disambiguare tra i campi della classe (es: this.giorno) e i parametri dei costruttori (es: giorno). Questo modo di scrivere i costruttori è un idioma tipico del linguaggio Java.

Un altro idioma consiste nel creare un costruttore "principale" (con tutti i parametri) e altre versioni "semplificate" (con meno parametri). Questo viene realizzato usando la keyword this in questo modo:

Main.java

```
public class Data {
  private int giorno;
  private int mese;
  private int anno;
```

```
    public Data(int giorno, int mese, int anno) {
        this.giorno = giorno;
        this.mese = mese;
        this.anno = anno;
    }

    public Data(int mese, int anno) {
        this(1, mese, anno);
    }

    // metodi omessi per brevità
}
```

Aliasing

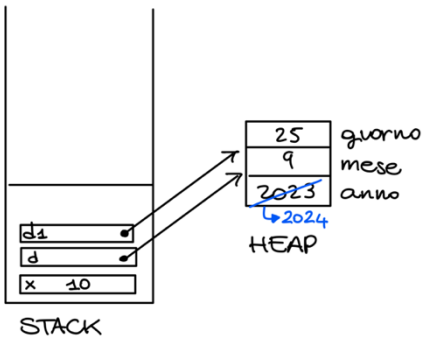
Main.java

```
public class Main {
    public static void main(String[] args) {
        Data d = new Data(25, 9, 2023);
        Data d1 = d;
        d1.capodanno();
        System.out.println(d.getAnno()); // stampa 2024
    }
}
```

Il codice qui sopra stamperà 2024, questo perché le variabili d e d1 (che sono dei puntatori) si riferiscono in realtà allo stesso oggetto.

Questo avviene perché l'operatore di assegnazione = assegna il valore dell'espressione a sinistra alla variabile a destra. Però a sinistra abbiamo la variabile d1, che contiene un riferimento (cioè un puntatore senza aritmetica e che viene dereferenziato automaticamente) alla zona di memoria dell'oggetto di tipo Data. Quindi quello che otteniamo è di copiare il puntatore e non l'oggetto puntato.

Quindi quando successivamente invochiamo il metodo modificatore **capodanno()** su d1, queste modifiche saranno visibile anche su d (in quanto d e d1 si riferiscono allo stesso oggetto).



Garbage collector

Creiamo una seconda variabile Data d2 = d;

Main.java

```
public class Main {
    public static void main(String[] args) {
        int x = 10;
```

```
        Data d = null;
        d = new Data(25, 9, 2023);

        Data d1 = d;
        d1.capodanno();

        Data d2 = new Data(12, 2023);
        d2 = d;
    }
}
```

L'operazione di assegnazione in fondo a main() sarebbe problematica in un linguaggio come C in quanto questa sovrascrive il valore della variabile d2, perdendo il riferimento precedente e quindi rendendo impossibile deallocare la zona di memoria creata per ospitare la seconda Data.

In java questo non è un problema perché è presente un componente chiamato Garbage Collector: il suo scopo è quello di trovare tutte le zone di memoria che non sono più utilizzate e deallocarle. Questo significa che in Java la memoria viene gestita in modo completamente automatico, a differenza di C.

Il Garbage Collector utilizza una strategia detta mark and sweep, che consiste in due passaggi:

1. marcare tutti gli oggetti raggiungibili (direttamente o indirettamente)
2. deallocare tutti quelli che non lo sono

Esempi

Accesso a variabili private

Main.java

```
public class Data {
    private int giorno;
    private int mese;
    private int anno;

    // costruttori omessi per brevità

    public void copiaDa(Data d) {
        this.giorno = d.giorno;
        this.mese = d.mese;
        this.anno = d.anno;
    }

    // altri metodi omessi per brevità
}
```

Nel metodo copiaDa() accediamo ai campi dell'oggetto **d** anche se questi sono privati. Possiamo farlo perché li metodo copiaDa() si trova all'interno della classe Data.

Quindi il controllo sull'accesso viene fatto in base alla classe e non all'oggetto.

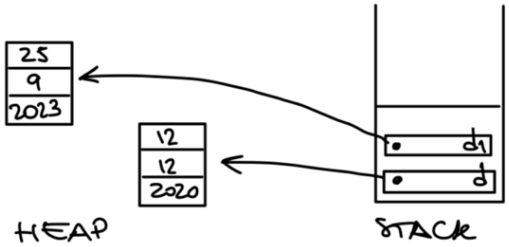
Esempio 2

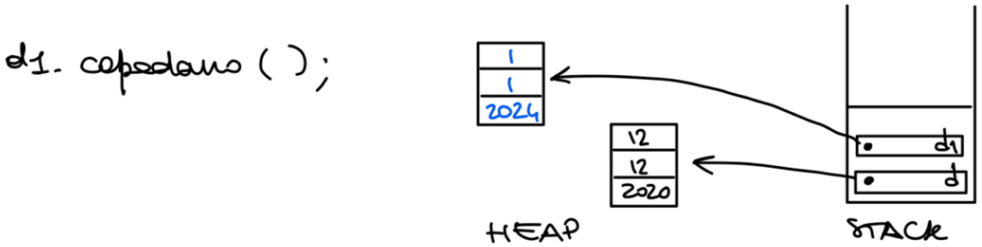
Main.java

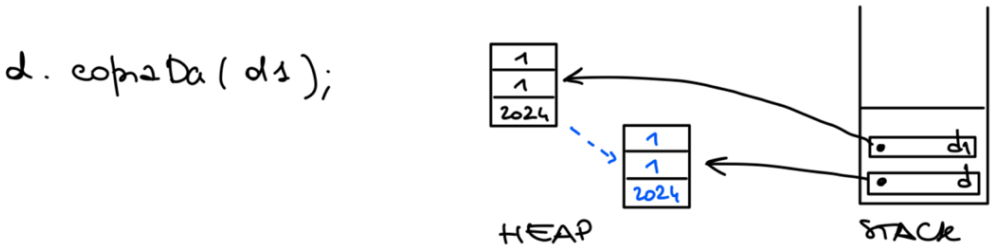
```
public class Main {
    public static void main(String[] args) {
        Data d = new Data(12, 12, 2020);
        Data d1 = new Data(25, 9, 2023);
```

```
        d1.capodanno();
        d.copiaDa(d1);
        System.out.println(d.getAnno()); // stampa 2024
    }
}
```

Vediamo l'esecuzione passo per passo:







Swap

Main.java

```
public class Main {
    public static void main(String[] args) {
        Data d = new Data(25, 9, 2023);
        Data d1 = new Data(12, 12, 2020);
        swap(d, d1);
    }

    public static void swap(Data a, Data b) {
        Data tmp = a;
        a = b;
        b = tmp;
    }
}
```

Quando invochiamo il metodo **swap()** nel **main()** questo in realtà non scambia i valori di d e d1.

Questo succede perché tutti i parametri vengono passati per copia; quindi, valori di **d** e **d1** (che sono dei puntatori alle due date) vengono copiati in **a** e **b**; dopo di che **a** e b vengono effettivamente

swappati ma questo non può influire in nessuna maniera sui valori di **d** e **d1**, che quindi rimangono immutate.

Lezione 6: Java (28/09/2023 Luca Dignatici, Marco Dolciotti)

Final

La keyword "**final**" permette di definire attributi **costanti**, che dunque non potranno essere modificati, una volta inizializzati.
Nel caso di un tipo primitivo, è il valore a essere costante, mentre per un oggetto lo sarà il riferimento a esso stesso.
Qualora "**final**" venga applicata all'attributo di una classe, allora esso dovrà essere necessariamente inizializzato nel costruttore della classe.

Esempio con una classe Persona:

esempio ERRATO:

Persona.java

```
public class Persona{
    private final String name;  //non posso cambiare la variabile con
keyword final
    private final String surname;
    //genera errore perché this.surname è final ma non viene
    inizializzato nel costruttore
    public Persona(String name){
        this.name = name;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;    //genera errore perché this.name è final
    }
}
```

esempio CORRETTO:

Persona.java

```
public class Persona{
    private final String name;  //non posso cambiare la variabile con
keyword final
    private final String surname;
    public Persona(String name, String surname){
        this.name = name;
        this.surname = surname;
    }
    public String getName() {
        return name;
```

```
    }
}
```

Ereditarietà tra classi

Quando si creano sottoclassi sfruttando l'ereditarietà, bisogna fare attenzione a non cercare di accedere ad attributi privati della sovraclasse nell'implementazione dei metodi della sottoclasse.

Questo problema si può raggirare usando metodi della sovraclasse per accedere a tali metodi.

Vediamo un esempio di ereditarietà tra classi creando una sottoclasse Studente di Persona:

esempio ERRATO:

Studente.java

```
public class Studente extends Persona{

    //genera errore perché this.name e this.surname sono membri privati
    della sovraclasse
    public Studente(String name, String surname){
        this.name = name;
        this.surname = surname;
    }

    public int getVoto() {
        return voto;
    }

}
```

esempio CORRETTO:

Studente.java

```
public class Studente extends Persona{

    public Studente(String name, String surname){
        //chiamo il costruttore della classe da cui eredita usando la
        keyword super
        super(name, surname);
        this.voto = voto;
    }

    public int getVoto() {
        return voto;
    }

}
```

Occorre prestare attenzione alla dichiarazione e alla costruzione degli oggetti, poiché è possibile costruire un oggetto appartenente a una sottoclasse e dichiararlo come la sovraclasse, ma non il contrario.

esempio ERRATO:

Main.java

```java
public static void main(String[] args){
    Persona p1 = new Student('A', 'B');
    //genera errore perché Persona è una sopraclasse di Studente
    Studente p2 = new Person ('C', 'D');
}
```

esempio CORRETTO:

Main.java
```java
public static void main(String[] args){
    Studente p1 = new Studente('A', 'B');
    //questo è possibile perché uno Studente è anche una Persona
    Persona p2 = new Studente('C', 'D');
}
```

Non è possibile invocare un metodo presente solo in una sottoclasse su un oggetto dichiarato come la sovraclasse, anche qualora l'oggetto sia stato costruito come la sottoclasse stessa.

esempio ERRATO:

Main.java
```java
public static void main(String[] args){
    Persona p2 = new Studente('C', 'D', 10);
    //genera errore poiché p2 è registrato dal compilatore come Persona e
    cerchiamo di invocare un metodo proprio di Studente
    System.out.println(p2.getVoto());
}
```

esempio CORRETTO:

Main.java
```java
public static void main(String[] args){
    Studente p2 = new Studente('C', 'D', 10);

    System.out.println(p2.getVoto());
}
```

Override dei metodi

Una sottoclasse può "sovrascrivere" il metodo di una sovraclasse, cosicché, una volta invocato il metodo, quest'ultimo sia quello appartenente alla sottoclasse e non quello della sovraclasse.

Ciò risulta essere vero anche quando l'oggetto è costruito come la sottoclasse, ma dichiarato come la sovraclasse (si tratta di un esempio di **polimorfismo**).

esempio CORRETTO:

Persona.java
```java
public class Persona{
    private final String name;
    private final String surname;
    public Persona(String name, String surname){
        this.name = name;
        this.surname = surname;
    }
    public String getName() {
        return name;
```

```
    }
    //questo metodo può essere usato da p1, p2, ma non p4 (poiché p4 ne
    ha uno con lo stesso nome in override)
    public void print(){
        System.out.println(name + ' ' + surname);
    }
}
```

Studente.java

```
public class Studente extends Persona{
    private int voto;
    public Studente(String name, String surname, int voto){
        super(name, surname);
        this.voto = voto;
    }

    public int getVoto() {
        return voto;
    }

    //definisco il metodo print() in Student usando l'override
    public void print(){
        System.out.println("Studente: " + getName() + " " + getSurname()
                           + getVoto());
    }
```

Main.java

```
public static void main(String[] args){
    Persona p1 = new Persona('A', 'B');
    Persona p2 = new Studente('C', 'D', 10);
    Studente p4 = new Studente('E', 'F', 30);
    //i seguenti metodi vanno tutti a buon fine
    p1.print();
    p2.print();
    p4.print();
    int voto = p4.setVoto();
}
```

In questo caso il main stampa:

```
A B
C D
Studente: E F 30
```

Anche se dal compilatore viene dichiarato Persona, durante il **runtime** a p4 viene assegnato il tipo dinamico Studente e quindi viene invocato il metodo **print()** associato a Studente, grazie all'**override.** Si tratta di un processo per il quale una sottoclasse può ridefinire i metodi delle sue superclassi, lasciando invariato numero e tipo dei parametri. Da non confondere con quest'ultimo, **l'overloading** di metodi, utile a definire funzioni con codice differente ma con effetti simili su tipi diversi, si ha con classi che presentano metodi con lo stesso nome, purché si distinguano per numero e/o tipo dei parametri.

Array

In Java, gli array di oggetti contengono i riferimenti ai suoi elementi e non gli elementi stessi, mentre gli array di tipi primitivi contengono direttamente gli elementi.

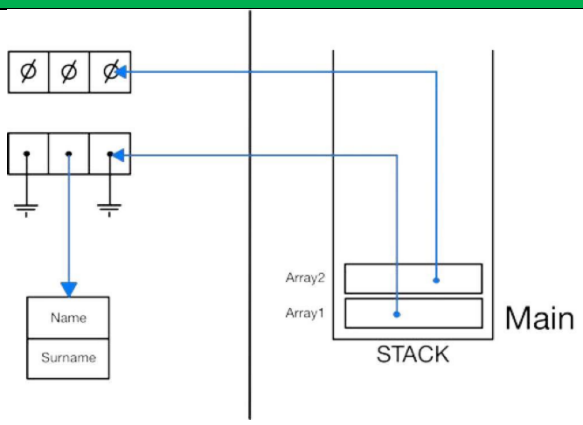
Notare inoltre il fatto che una variabile di tipo array (presente sullo staz1ck) è un riferimento all'array stesso (presente sullo heap).

Main.java

```
public class Corso{
    private final Student[]
students;
    private int numStudents;

    public Corso(int capacity){
        students = new
Student[capacity];
        this.numStudents = 0;
    }

    public void addStudent(Student
s){
        students[numStudents] = s;
        numStudents++;
    }
}
```



I seguenti esempi sono equivalenti, ma il secondo risulta essere meno efficiente, poiché prevede che si scorra tutto l'array.

esempio 1:

Studente.java

```
public float calcolaMedia(){
    float somma = 0;
    for(int i=0; i < numStudents; i++){
        somma += students[i].getVoto();
    }
    return numStudents == 0 ? 0 : somma / numStudents;
}
```

Esempio 2:

Studente.java

```
public float calcolaMedia(){
    float somma = 0;
    for(Studente s : students){
        somma += s.getVoto();
    }
    return numStudents == 0 ? 0 : somma / numStudents;
}
```

Enumerazione

È possibile dichiarare tipi **enumerati**, per modellare insiemi con cardinalità ridotta. A una classe enumerata si possono aggiungere costruttore, metodi e attributi.

I costruttori sono invocati solo quando vengono “costruite” le costanti; non si può costruire oggetti generici da classi enumerazioni.

StudentType.java

```
public enum StudentType {
    ING, ARCH, DES, WRONG
}
```

StudentType è una classe, costituita da **4** istanze.

Tutte le classi enumerate offrono i seguenti metodi:

- **Static Enum valueOf(String name)**: Restituisce la costante enumerata della classe indicata che ha quel nome
- **String toString()**: Restituisce il nome della costante enumerata
- **Size[] valori = Size.values();** Restituisce un array contenente tutti i valori della classe

…

Aggiunta personale di uno studente di un altro scaglione che segue queste lezioni.

Tipi statici e dinamici in Java.

Quando viene dichiarata una variabile di tipo non primitivo (quindi che non sia int, double ecc...) essa viene dichiarata in questo modo.

TipoStatico g = new TipoDinamico();

oppure

TipoStatico g;

TipoStatico e TipoDinamico possono coincidere come no.

Se coincidono, non c’è nessuna particolarità: sia in fase di compilazione, che in fase di runtime, la variabile avrà le proprietà di TipoDinamico.

Se non essi non coincidono, si configura uno dei seguenti scenari:

- **TipoStatico è superclasse di TipoDinamico:** il programma compila e in fase di compilazione la variabile è di TipoStatico. In fase di runtime, la variabile è di TipoDinamico. Se sono stati definiti override nella classe TipoDinamico, vengono considerati quelli. Overload e metodi non ridefiniti in TipoDinamico sono quelli di TipoStatico.
- **TipoStatico non è superclasse di TipoDinamico:** il programma dà errore.

Quando in un codice viene chiamato un metodo su un oggetto, il codice viene, per queste ragioni, analizzato prima in fase di compilazione e poi in fase di runtime. Durante la **compilazione**, il TipoStatico è il tipo della variabile, quindi se viene compilata la chiamata di un metodo chiamato su un oggetto di TipoStatico, la signature viene cercata solo tra i metodi della classe TipoStatico.

Nella fase di **runtime**, però, se esistono **override**, come già detto, vengono eseguiti quegli override.

Ipotizziamo che esista un metodo Metodo(TipoStatico s) definito nella classe TipoStatico e un suo overload Metodo(TipoDinamico s).

Allora, la sostanziale differenza tra

TipoStatico g = new TipoDinamico();

g.Metodo(g);

E la situazione in cui si ha

TipoDinamico g = new TipoDinamico();

g.Metodo(g);

È che, se ci sono **overload definiti solo in TipoDinamico**, come in questo caso, **non vengono considerati nel primo caso**. Questo può essere sicuramente utile se si vuole definire un overload di Metodo che permetta di prendere una qualsiasi variabile di TipoStatico e che io non voglia, per quella variabile, chiamare l'overload che accetta solo una variabile TipoDinamico.

Un esempio per fugare ogni dubbio.

Caso visto a lezione.

Polimorfismo: abbiamo una classe padre Punto2D e una classe figlio Punto3D. Abbiamo metodo public dist(Punto2D p);

Ipotizzando che la classe Punto3D abbia:

public dist(Punto2D p), override.

public dist(Punto3D p), overload;

Cerchiamo di capire quale metodo viene chiamato se l'istruzione è la seguente, con a e b punti 2D o 3D.

a.dist(b);

Oggetto su cui è chiamato il metodo (quello a sx del .)	Oggetto che viene passato come parametro.	Caso Numero #
È Punto2D ma non Punto3D.	È Punto2D (sia che sia Punto3D sia che non lo sia).	Caso1 Viene chiamato dist(Punto2D p) della classe Punto2D. **Metodo ereditato**
È Punto3D, ma di tipo statico Punto2D.	È di tipo statico Punto3D.	Caso 2 Viene chiamato il metodo dist(Punto2D p) della classe Punto3D. **override**
È Punto3D, ma di tipo statico Punto2D.	È Punto3D, ma di tipo statico Punto2D.	Caso 3 Viene chiamato il metodo dist(Punto2D p) della classe Punto3D. **override**
È Punto3D di tipo statico Punto3D.	È Punto2D, ma non Punto3D	Caso 4

		Viene chiamato il metodo dist(Punto2D p) della classe Punto3D. **override**
È Punto3D di tipo statico Punto3D.	È Punto3D di tipo statico Punto3D.	Caso 5 Viene chiamato il metodo dist(Punto3D p) della classe Punto3D. **overload**

Caso numero #	Spiegazione
1	Staticamente, in fase di compilazione, viene rilevato solo il metodo chiamato (a è Punto2D), in runtime viene confermato quello.
2	Staticamente, in fase di compilazione, viene rilevato solo il metodo dist(Punto2D p) della classe Punto2D. In Runtime, siccome si ha tipo dinamico Punto3D, si osserva l'override, che viene chiamato.
3	Come caso 2
4	Staticamente abbiamo un Punto3D, quindi rilevo i metodi della classe Punto3D (overload e override). Abbiamo un Punto2D che non è Punto3D, senza dubbio scelgo l'override, con parametro Punto2D.
5	Staticamente viene rilevato che esiste il metodo chiamato, l'overload che viene selezionato perché ho Punto3D (tipo statico per entrambe le variabili coinvolte)

….

Lezione 7: Java (02/10/2023 Daniel Bonardi, Abdallah Alkhetiar)

Enumerazione

Le enumerazioni sono delle classi con una particolarità, **il numero di istanze è predefinito**.

Dato che non posso creare una nuova istanza di un'enumerazione, l'unica notazione che posso utilizzare è quella puntata.

Main.java

```
public static void main(String[] args)
{
    StudentType t = StudentType.ING;
}
```

Essendo le enumerazioni delle classi posso darle degli attributi. Ad esempio, posso creare un attributo final ***name*** che quindi devo inizializzare, ma per farlo mi serve un costruttore, che avrà una particolarità, ossia che non è public ma private ed il motivo è semplice, infatti tutte le istanze di ***StudentType s***ono create all'interno della classe stessa e non possono essere create all'esterno, quindi non ci capiterà mai di utilizzare un costruttore all'esterno della classe stessa. Adesso rimane

un problema da risolvere, ossia che quando nella classe creo un'istanza quell'istanza deve avere l'attributo ***name*** inizializzato e per farlo devo passare la stringa che voglio associare all'istanza.

Oltre agli attributi, le interfacce possono anche avere dei metodi, l'esempio più semplice è quello di un getter per l'attributo ***name***.

StudentType.java

```
ING("Ingegneria"),
ARCH("Architettura"),
DES("Design"),
WRONG("Università sbagliata");


private final String name;

private StudentType(String name) {
    this.name = name;
}

public String getName() {
    return name;
}
```

Polimorfismo

In java ogni variabile ha un tipo statico (come è stato dichiarato), e un tipo dinamico (Come l'oggetto a cui si riferisce è stato istanziato), dove il tipo dinamico può essere uguale al tipo statico o qualcosa di più preciso. Per quanto riguarda i metodi posso chiamare tutti quelli che sono definiti nel tipo statico, in questo modo sono sicuro che l'istanza contenga il metodo (Possiamo ragionare come se fosse il minimo comune denominatore). Facendo l'override di un metodo della sopraclasse in una sottoclasse viene chiamato il metodo sovrascritto. Il nostro obbiettivo è quello di sperimentare il comportamento dei metodi a causa del polimorfismo, degli override e degli overload (che spiegheremo dopo). Iniziamo creando la classe ***Punto2D*** e la sottoclasse ***Punto3D*** che eredita dalla prima.

Punto2D.java

```
public class Punto2D
{
    private final int x;
    private final int y;

    public Punto2D(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public double dist(Punto2D other) {
        System.out.println("Metodo di Punto2D con parametro Punto2D");
        double res = 0; // Calcolo distanza omesso


        return Math.sqrt(res);
```

```
    }
```

Punto3D.java

```
public class Punto3D extends Punto2D
{
    private final int z;

    public Punto3D(int x, int y, int z) {
        super(x, y);
        this.z = z;
    }

}
```

Come già sappiamo ogni metodo creato da una classe viene ereditato da tutte le sottoclassi. Quindi per testare il funzionamento della funzione ***dist*** andiamo nel ***Main*** e creiamo i tre possibili tipi di variabili che possono esistere. A questo punto creiamo tutte le possibili combinazioni tra chiamate.

Main.java

```
Punto2D a = new Punto2D(1, 2);
Punto2D b = new Punto3D(2, 4, 6);
Punto3D c = new Punto3D(3, 4, 6);

a.dist(a);
a.dist(b);
a.dist(c);

b.dist(a);
b.dist(b);
b.dist(c);

c.dist(a);
c.dist(b);
c.dist(c);
```

Tutti le chiamate in questa condizione funzionano perché sia la classe ***Punto2D*** che ***Punto3D*** contengono il metodo ***dist***, e il parametro ***Punto3D*** è accettato perché è più specifico di ***Punto2D***. Quindi ovviamente quello che stampano queste 9 chiamate è sempre lo stesso messaggio (Il metodo è lo stesso). Introduciamo adesso il concetto di **overload** ovvero definire più versioni di un metodo, utilizzando lo stesso nome ma una signature diversa, dove con signature si intende il numero e/o tipo di parametri.

Importante: java sceglie la signature di un metodo sulla base del tipo statico. Vediamo come si comportano i metodi quando aggiungiamo un overload.

Punto3D.java

```java
public double dist(Punto3D other) {
    double res = 0; // Omesso calcolo distanza
    System.out.println("Metodo di Punto3D con parametro Punto3D");
    return res;
}
```

Main.java

```java
Punto2D a = new Punto2D(1, 2);
Punto2D b = new Punto3D(2, 4, 6);
Punto3D c = new Punto3D(3, 4, 6);

a.dist(a); // Metodo di Punto2D con parametro Punto2D
a.dist(b); // Metodo di Punto2D con parametro Punto2D
a.dist(c); // Metodo di Punto2D con parametro Punto2D

b.dist(a); // Metodo di Punto2D con parametro Punto2D
b.dist(b); // Metodo di Punto2D con parametro Punto2D

b.dist(c); // Metodo di Punto2D con parametro Punto2D

c.dist(a); // Metodo di Punto2D con parametro Punto2D
c.dist(b); // Metodo di Punto2D con parametro Punto2D
c.dist(c); // Metodo di Punto3D con parametro Punto3D !!
```

Complichiamo per un'ultima volta la situazione ed aggiungiamo a ***Punto3D*** l'override del metodo ***dist***, e poi controlliamo cosa cambia con le chiamate all'interno del ***Main***.

Punto3D.java

```java
@Override
public double dist(Punto2D other) {
    double res = 0; // Omesso calcolo distanza
    System.out.println("Metodo di Punto3D con parametro Punto2D");
    return res;
}
```

Main.java

```java
Punto2D a = new Punto2D(1, 2);
Punto2D b = new Punto3D(2, 4, 6);
Punto3D c = new Punto3D(3, 4, 6);

a.dist(a); // Metodo di Punto2D con parametro Punto2D
a.dist(b); // Metodo di Punto2D con parametro Punto2D
a.dist(c); // Metodo di Punto2D con parametro Punto2D

b.dist(a); // Metodo di Punto3D con parametro Punto2D
b.dist(b); // Metodo di Punto3D con parametro Punto2D
```

```
b.dist(c); // Metodo di Punto3D con parametro Punto2D

c.dist(a); // Metodo di Punto3D con parametro Punto2D
c.dist(b); // Metodo di Punto3D con parametro Punto2D
c.dist(c); // Metodo di Punto3D con parametro Punto3D !!
```

Ecco un riassunto del comportamento dei metodi nel caso di overload e override, tenendo presente che prima si guarda se c'è un overload compatibile con il tipo statico del parametro, poi si guarda se c'è un override del metodo e nel caso usare quello overridden dal tipo dinamico della variabile chiamante.

Parametro Chiamante	Statico: Sopraclasse Dinamico: Sopraclasse	Statico: Sopraclasse Dinamico: Sottoclasse	Statico: Sottoclasse Dinamico: Sottoclasse
Statico: Sopraclasse Dinamico: Sottoclasse	Base	Base	Base
Statico: Sopraclasse Dinamico: Sottoclasse	Override / Base	Override / Base	Override / Base
Statico: Sottoclasse Dinamico: Sottoclasse	Override / Base	Override / Base	Overload / Override /Overload

Classi astratte e interfacce

Le classi astratte sono caratterizzate dalla keyword **abstract** e la loro particolarità è che non possono essere istanziate. Ad esempio, posso creare una classe astratta Figura e una sua sottoclasse Quadrato.

Figura.java

```java
public abstract class Figura
{
    public abstract void draw();

}
```

Quadrato.java

```java
public class Quadrato extends Figura
{
    @Override
    public void draw() {

    }
}
```

Il metodo della classe astratta non contiene codice e deve essere sempre implementato nelle sue sottoclassi. Le interfacce invece sono una promessa che una classe implementerà dei metodi. Ad esempio, creo l'interfaccia ***Printable*** e faccio in modo che ***Figura*** la erediti.

Printable.java

```
public interface Printable
{
    public void print();
}
```

Figura.java

```
public abstract class Figura implements Printable
{
    public abstract void draw();

}
```

A questo punto la classe figura dovrebbe implementare tutte le funzioni di ***Printable***, ma essendo una classe astratta può fare ricadere la necessità di implementazione sui suoi figli, ossia su ***Quadrato***.

Quadrato.java

```
@Override
public void print() {

}
```

La classe Object

È la classe da cui derivano tutte le classi, a meno che specificato diversamente. La classe ***Object*** contiene dei metodi su cui posso fare override nelle sottoclassi, tra i più utili ci sono:

toString(): permette di fornire una rappresentazione testuale dell'oggetto. Java automaticamente utilizza questo metodo in tutti i punti nei quali si aspetta una stringa. Quando si utilizza il metodo di default sull'istanza di una classe composta da diversi attributi, ritorna semplicemente il riferimento dell'oggetto. Si può fare override del metodo toString in una classe così da avere una stringa opportuna come valore di ritorno del metodo chiamato su una sua istanza. Questo override viene ereditato dalle sue sottoclassi.

equals(Object obj): restituisce true quando due oggetti comparati sono istanze della stessa classe e hanno stesso contenuto, in questo caso si parla di uguaglianza. (Attenzione a non confondere con identità, infatti a volte l'uguaglianza può essere definita sulla base dell'identità). Java utilizza di default la posizione in memoria come identificatore di uguaglianza e quindi è semplicemente

l'identità. Come con il metodo toString si può fare l'override del metodo equals dentro una classe in modo da avere un'uguaglianza semantica che però deve utilizzare solo attributi immutabili (final) della classe per fare il confronto (per evitare problemi con gli insiemi in Java).

hashCode(): Serve per verificare l'uguaglianza di due oggetti, infatti se due oggetti sono uguali allora hanno lo stesso hash code. Tuttavia, non si verifica nel senso opposto, infatti due oggetti diversi possono avere lo stesso hashcode. (Quindi è condizione necessaria ma non sufficiente per uguaglianza).

Metodi e attributi statici

Un attributo può essere marcato statico o di classe con la keyword **static**. Questi attributi non appartengono ad una singola istanza ma alla classe stessa. Lo stesso vale per i metodi statici.

Person.java

```
private static int numPersons = 0;

public Person(String name, String surname) {
    this.name = name;
    this.surname = surname;
    numPersons++;
}

public static int getNumPersons() {
    return numPersons;
}
```

Dentro metodi statici non posso riferirmi ad attributi o metodi di istanza perché non so su che istanza verrebbero chiamati. Posso però riferirmi ad attributi statici in metodi non statici. Per poter chiamare il metodo statico devo riferirmi direttamente alla classe.

Main.java

```
int numPersons = Person.getNumPersons();
```

Lezione 8: Java (09/10/2023 Mattia Colombo, Vittorio Cozzoli)

Packages in Java

Un "package" in Java è una raccolta di classi e interfacce correlate. Funziona come un namespace tramite il quale raggruppare classi correlate in modo logico e permettendo di evitare conflitti di nomi tra le classi. I package sono anche utilizzati per fornire un meccanismo di controllo dell'accesso e per proteggere le classi da accessi non autorizzati.

Java, di default, parte da una certa cartella che dipende dalla configurazione iniziale che abbiamo dato al progetto (ad es. Maven oppure progetto Java semplice, dove la root directory del progetto è **`src`**), che rappresenta il punto in cui io inserisco tutto il mio codice sorgente

► *Def:* **package di default** (o package senza nome)

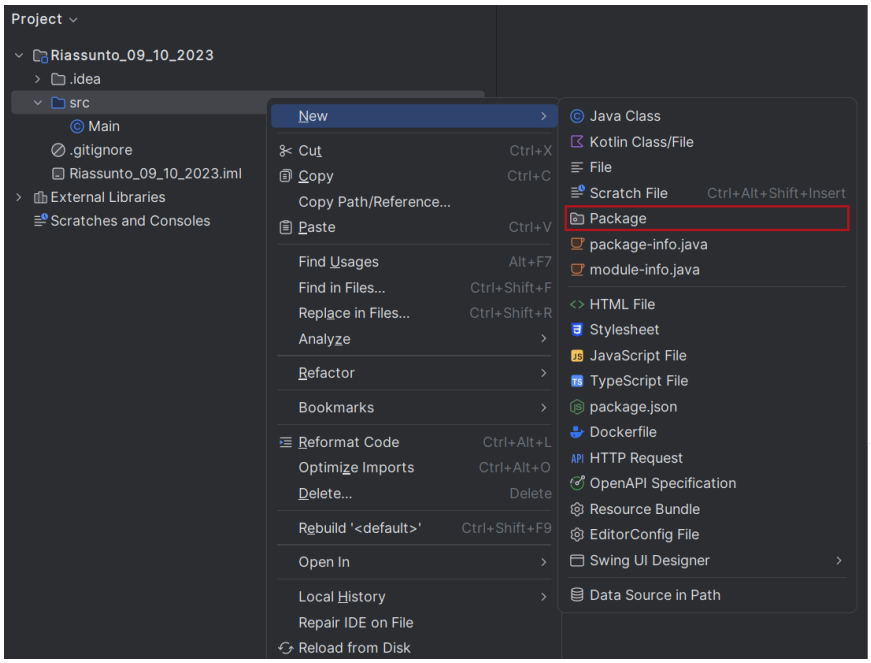
Quando compiliamo classi non esplicitamente incluse in un package, Java le tratta come se appartenessero ad un package di default, senza nome, che include tutte le classi e tutte le interfacce definite nella stessa directory. *(si consiglia tuttavia di evitarne l'uso al fine di prevenire eventuali conflitti di nomi e poter così mantenere un codice ben organizzato).*

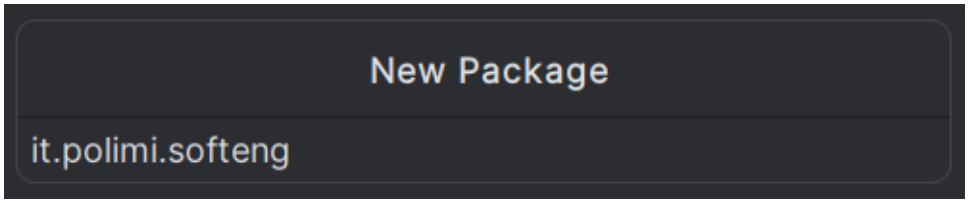
I package rappresentano dunque un insieme di cartelle che definiscono dei namespace separati (cartelle che contengono delle classi, dove classi appartenenti a package diversi sono classi diverse).

Creazione e dichiarazione di un package in Java:

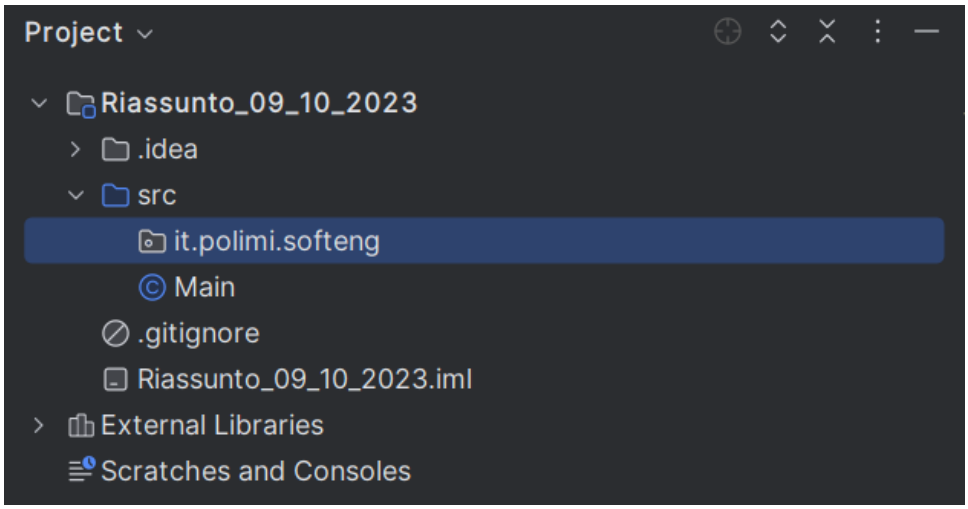
Codesnippet.java

```
package it.polimi.softeng.pkg;
```





CONVENZIONE: Java vuole che i nomi dei package vengano dichiarati con lettere minuscole e che non inizino per un valore numerico.



Quando dico ad **IntelliJ IDEA** di costruire un package, effettivamente sto creando una cartella nel mio filesystem. Tale costruzione consta di una struttura gerarchica: tendo a costruire cartelle che contengono altre cartelle che a loro volta ne contengono altre.

► *Def:* **convenzione di dominio inversa**

È la convenzione che adottiamo per definire precisamente la gerarchia di cartelle che creiamo durante la definizione del nostro package: ogni volta che utilizziamo la notazione ".", stiamo creando una nuova sottocartella nel nostro filesystem: **`it.polimi.softeng`** → cartella "**`it`**" → cartella "**`polimi`**" → cartella "**`softeng`**"

Se definisco una classe all'interno di un package, devo dichiararlo nella prima riga di codice relativa alla classe in cui sto lavorando:

Codesnippet.java

```java
package it.polimi.softeng.pkg;

public class Prova{

}
```

Il package serve come riferimento per **evitare ambiguità tra classi con lo stesso nome ma in namespace diversi**. Ad esempio, la prima classe "**`Prova`**" ha come nome completo **`it.polimi.softeng.pkg.Prova`**, mentre la seconda classe "**`Prova`**" è **`it.polimi.softeng.pkg2.Prova.`**
Quindi, i package sono fondamentali per distinguere tra classi omonime.

Un modo per utilizzare una data classe nel **`Main`** è chiamandola per il suo nome completo:

Dichiarazione di una variabile nel Main

```
//Dichiarazione di variabile
it.polimi.softeng.pkg.Prova prova = null;
```

Tuttavia, anziché dichiarare una variabile con il tipo di cui sopra, si preferisce generalmente importare il package all'interno della classe in cui stiamo lavorando (come per gli #include del C):

Import di una singola classe dal package

```
import it.polimi.softeng.pkg.Prova;
```

Import di tutte le classi presenti nel package

```
import it.polimi.softeng.pkg.*;
```

NOTAZIONE: Se uso "*", importo tutte le classi pubbliche (qualora ve ne fosse più di una) appartenenti al package

⚠ **:** Classi omonime NON possono esistere all'interno dello stesso package

⚠**:** Posso importare al più una classe omonima (altrimenti genereremmo ambiguità)

Dato che in un progetto si possono avere nomi comuni, l'uso di namespace separati evita conflitti. Ciò consente di sviluppare l'applicazione utilizzando elementi da diverse librerie senza preoccuparsi di possibili ambiguità.

L'altra motivazione per cui si usano i package è per **avere un livello in più di *information hiding***: è possibile avere delle visibilità esclusivamente a livello di package, oltre che alla classica distinzione tra attributi pubblici e privati ⇒ posso avere delle classi visibili a livello di package che espongono dei metodi pubblici che saranno tutti e soli quelli (visibili) che interessano all'esterno (è come se tali classi visibili mostrassero un'interfaccia utente molto ridotta che altri sviluppatori potranno successivamente utilizzare) che poi internamente al package vanno a chiamare le altre classi presenti al suo interno. Di contro, tutte le classi e i metodi che NON voglio esporre all'esterno, possono essere "ridefinite" come *classi di package* oppure *metodi di package.*

Per l'information hiding, si distinguono 4 "scale di grigio" in Java:

- **PUBLIC** := quando un elemento (come una classe, un metodo o una variabile) è dichiarato come **`public`**, significa che può essere accessibile da qualsiasi altra classe o metodo in qualsiasi pacchetto, purché questa abbia una referenza all'oggetto in questione o possa importare la classe di appartenenza di quell'elemento.
- **PRIVATE** := fa riferimento a metodi che generalmente servono esclusivamente per astrazioni procedurali (decomposizione procedurale come in C) —> sono pezzettini di codice della classe che uso soltanto io sviluppatore e che non voglio che vengano visti all'esterno.
- **FRIENDLY** := visibilità solo all'interno del package (altre classi al di fuori di quella in cui si trova ciò che abbiamo dichiarato come friendly non possono chiamare il nostro attributo/metodo, mentre possono farlo solo gli attributi situati all'interno dello stesso package in cui è presente il nostro metodo/attributo giustappunto friendly).
  - **NOTAZIONE:** Per definire metodi/attributi *friendly* **NON uso nessuna keyword.**
  - **FRIENDLY** non permette l'*override* di metodi
- **PROTECTED** := visibilità concessa a chiunque si trovi all'interno dello stesso package o a chiunque estenda quella data classe anche se locato in un altro package.

Esempio di utilizzo di tali livelli di visibilità:

Esempio di utilizzo di Public, Protected e Friendly

```
//CLASSE: Prova
package it.polimi.softeng.pkg;
public class Prova{
	private int a;
	private int b;

	public Prova(int a, int b){
		this.a = a;
		this.b = b;
		this.c = c;
	}
	public int getA(){
		return A;
	}
	protected int getB(){
		return b;
	}
	int getC(){ //friendly
		return c;
	}
}
```

Dato il codice di cui sopra, appare evidente come dal Main sarà possibile chiamare solamente *getA();*

esempio CORRETTO:

Snippet_Main.java

```
int aaa = prova.getA();
```

esempio ERRATO:

Snippet_Main.java

```
int bbb = prova.getB();
int ccc = prova.getC();
```

Scelte di Design & Best Practices

Le best practice in Java suggeriscono di mantenere gli attributi come privati. L'*eccezione* a questa regola si verifica principalmente quando gli *attributi rappresentano costanti che si desidera utilizzare in tutto il programma; in questi casi, possono essere definiti come pubblici.*
Per quanto riguarda i **metodi**, quelli **privati** sono **utilizzati principalmente per l'astrazione procedurale** (similmente a come avviene in C) e **non è necessario esporli all'esterno**. I **metodi friendly o protected, invece, sono utili quando si desidera che siano accessibili ai developers, ma non esposti ampiamente**. *Prendendo come esempio la gestione di un grafo: potremmo avere classi pubbliche come Edge e Vertex. All'interno di queste classi, potrebbero esserci metodi friendly o protected che permettono al grafo di eseguire operazioni tra i suoi vertici, ma questi metodi non dovrebbero essere visibili all'esterno.*

Il Main, una classe come le altre

Il Main è una classe come tutte le altre: innanzitutto, NON è detto che il metodo `public static void Main(String args[]){}` si trovi all'interno di una classe che si chiami **`Main`**; di fatti, è possibile collocare il metodo **`Main`** ovunque, in qualsiasi package lo sviluppatore preferisca.

È possibile avere due classi diverse che non si chiamano **`Main`** ma che implementano entrambe `public static void Main(String args[]){}` come metodo e posso decidere a quale far riferimento (cioè da quale classe partire) quando lancio il programma.
Tuttavia, come verrà spiegato fra breve, bisogna prestare attenzione al fatto che, per ogni file che definisco, è possibile avere **al più una classe pubblica**: tale classe può avere oppure no un metodo *public Main* e, nel caso in cui dovesse averlo, suddetto metodo può essere eseguito (a differenza del C, dove il *Main* è e deve essere unico). In Java, invece, è come se si potessero avere più Main all'interno di uno stesso package o all'interno di package diversi; è possibile dunque decidere da quale classe (che ovviamente implementi il metodo *Main*) far partire il mio programma. Per far questo si hanno a disposizione due semplici modi:

- Da riga di comando, invocando
  `javac NomeClasse.java`
- Da IntelliJ, cliccando "play" sulla classe corretta.

In Java, ogni file rappresenta una "**compilation unit**", o **unità di compilazione**. Questa unità di compilazione **può contenere al massimo una classe pubblica, e il nome di questa classe deve corrispondere al nome del file**. Questa **regola assicura una corrispondenza chiara tra il nome del file e la classe pubblica che contiene**. Sebbene ci possa essere solo una classe pubblica per file, **è possibile avere molteplici classi non pubbliche all'interno della stessa unità di compilazione.** *Pertanto, inserire più di una classe pubblica in un singolo file violerebbe la convenzione che stabilisce una chiara corrispondenza tra il nome del file e la sua classe pubblica principale.*

esempio ERRATO:

Snippet_Due_Classi_Pubbliche.java

```
public class Prova{}
public class Prova2{} //--> ERRORE
```

È possibile, tuttavia, definire più classi a livello di package, a patto che ovviamente al più una tra queste sia dunque public.

OSS: Per ogni compilation unit c'è un metodo Main ⇒ posso avere più compilation unit all'interno di file diversi ⇒ posso avere più Main all'interno di file diversi

Exceptions in Java

Le eccezioni in Java forniscono un mezzo strutturato per gestire errori e condizioni eccezionali in un'applicazione. A differenza di *linguaggi come C*, dove la gestione degli errori si basa spesso su *codici di ritorno* e *variabili globali*, **Java utilizza un approccio basato sull'orientamento agli oggetti attraverso il sistema di eccezioni**.

In Java, una Exception è un **oggetto restituito (in modo speciale) da un metodo.**

La classe **`Exception`** è definita nel core di Java, **`java.lang`; è dunque automaticamente importata dentro ogni Class file!**

Le Exception si propagano da un RDA *(Record di Attivazione)* di un metodo **chiamato** all'RDA del metodo **chiamante.** In generale:

Il **chiamato** **tira** / **solleva** / fa **`throw`** dell'Exception, mentre il **chiamante** **riceve** / **deve fare** **`catch`**.
(il significato di 'deve' verrà approfondito nella sezione successiva)

Esistono due tipi principali di Exceptions in Java: le **Checked** e le **Unchecked/Runtime Exceptions.**

Checked Exceptions

N. B: d'ora in poi assumeremo di utilizzare il package **`it.polimi.softeng`**

In Java, le eccezioni **Checked** sono quelle eccezioni che un metodo deve dichiarare nel suo **prototipo** / **signature** attraverso la clausola **`throws`** *se c'è la possibilità che tale eccezione* ***possa essere sollevata***.

Convenzione: il nome di tutte le classi che definiscono eccezioni deve terminare con '**`Exception`**' *(prassi ricorrente in Java codebases)*

Per lanciare situazioni eccezionali, si può procedere:

1. definendo una sottoclasse di **`Exception`**;
2. **Istanzio** tale classe utilizzando **`new`** *(ho a disposizione il costruttore di default)*
3. La ritorno utilizzando la keyword speciale **`throw`** *(segnalo "ritorno eccezionale")*
4. Quando un metodo solleva un'eccezione **checked** con **`throw`**, deve anche dichiararla con **`throws`** nel suo prototipo, **a meno che** l'eccezione non venga gestita all'interno del metodo con un blocco **`try-catch`**.
5. L'uso della keyword **`throw`** interrompe l'esecuzione del metodo corrente e restituisce il controllo al chiamante, propagando l'eccezione del tipo appena definito

MyCheckedException.java

```java
package it.polimi.softeng;

public class MyCheckedException extends Exception {
    public MyCheckedException(String message) {
        super(message); // vedasi sezione 'Logging' in seguito
    }
}
```

Demo.java

```java
package it.polimi.softeng;

public class Demo {// Exception non gestita in metodo
    public void metodoCheLanciaEccezione() throws MyCheckedException {
        throw new MyCheckedException("Errore!");
    }

    public void altroMetodo() { // Exception gestita dentro il metodo

        try {
            metodoCheLanciaEccezione();
        } catch (MyCheckedException e) {
            e.printStackTrace();
        }
    }
}
```

Vige una **condizione di obbligatorietà**: se un metodo lancia una di queste eccezioni **Checked** in qualsiasi punto del suo flusso di esecuzione, **il compilatore insiste affinché il chiamante gestisca tale eccezione**. Questa gestione può avvenire catturando l'eccezione tramite un **blocco catch**. Ecco perché si dice che il chiamante **deve fare `catch` (per questo si dicono Checked dal compilatore).** Questo significa che ***il chiamante deve anticipare e prepararsi a gestire potenziali problemi***; si procede come segue:

1. Quando si chiama un metodo che può lanciare un'eccezione, si può "catturare" quell'eccezione utilizzando un blocco **`try-catch`**. Questo significa che si tenterà di eseguire il codice all'interno del blocco **`try`**, e **se si verifica l'eccezione specificata**, il controllo passerà al blocco **`catch`** corrispondente. All'interno del blocco **`catch`**, si può decidere come gestire l'errore, ad esempio, stampando un messaggio o eseguendo una qualche logica di ripristino
2. Se ci sono n eccezioni che potrebbero essere lanciate, il chiamante può scegliere di gestire specificamente le prime x eccezioni **in modo individualizzato** e, successivamente, gestire tutte le altre $n - x$ in modo più generico usando la **classe madre `Exception`**. Questo

catturerà ***qualsiasi*** eccezione, permettendo di gestire situazioni impreviste in modo generico. *[si sfrutta polimorfismo].* È inoltre importante notare come **Java esamini i blocchi `catch` in ordine e si fermi al primo che corrisponde al tipo di eccezione lanciata: l'ordine in cui li scriviamo nel chiamante è rilevante!**

EccezioneSpecifica1Exception.java

```
class EccezioneSpecifica1Exception extends Exception {}
```

EccezioneSpecifica2Exception.java

```
class EccezioneSpecifica2Exception extends Exception {}
```

GestoreEccezioni.java

```
public class GestoreEccezioni {

    public static void Main(String[] args) {
        try {
            // Prova qualcosa che potrebbe lanciare un'eccezione...
            throw new EccezioneSpecifica2Exception(); // solo per
dimostrazione
        }

        // 2. Gestisci le eccezioni specifiche
        catch (EccezioneSpecifica1Exception e1) {
            System.out.println("Gestito EccezioneSpecifica1Exception");
        }
        catch (EccezioneSpecifica2Exception e2) {
            System.out.println("Gestito EccezioneSpecifica2Exception");
        }
        // ... gestisco altre eccezioni specifiche qui...

        // Poi, gestisci le eccezioni più generiche
        catch (Exception e) {
            System.out.println("Eccezione gestita in modo generico" );
        }
    }
}
```

Implementando il tutto nell'esempio delle precedenti lezioni:

NoStudentiException.java

```java
package it.polimi.softeng.exceptions;

public class NoStudentiException extends Exception {
    public NoStudentiException(String message) {
        super(message);
    }
}
```

MediaSottoSogliaException.java

```java
package it.polimi.softeng.exceptions;

public class MediaSottoSogliaException extends Exception {
    public MediaSottoSogliaException(String message) {
        super(message);
    }
}
```

Corso.java

```java
public float calcolaMedia() throws NoStudentiException {
    float somma = 0;
    for (int i = 0; i < numStudents; i++) {
        somma += students[i].getVoto();
    }
     /* Originariamente, il codice era:
     * return numStudents == 0 ? 0 : somma / numStudents;
     * Divisione senza preoccupazioni perché il 'throw'
     * interrompe la funzione qualora si abbiano 0 studenti per calcolo
     * Questo fa ritornare il controllo al caller, dove dovrà essere
     * implementata una soluzione per gestire l'exception. */
    if (numStudents == 0) {
        throw new NoStudentiException("Nessuno studente presente.");
    }
    return somma / numStudents;
}
```

Main.java

```java
//…
Corso corso = new Corso(20);
try {
    float media = corso.calcolaMedia();
} catch (NoStudentiException e) {
    System.out.println("Eccezione: Non ci sono studenti nel corso!");
} catch (MediaSottoSogliaException e) {
    System.out.println("Eccezione: La media è sotto la soglia!");
} catch (Exception e) {
    System.out.println("Si è verificata un'eccezione non prevista: " +
e.getMessage());//Vedasi sezione 'Logging'
}
```

Scelte di Design & Best Practices

Consideriamo una struttura gerarchica di chiamate: **`Main`** → **`Metodo1`** → **`Metodo2`**.

Quando **`Metodo2`** lancia un'eccezione, **`Metodo1`** ha due scelte principali:

1. **Propagazione dell'Eccezione**: **`Metodo1`** può decidere di non gestire l'eccezione e passarla al livello superiore (in questo caso, a **`Main`**).
2. **Gestione dell'Eccezione:** **`Metodo1`** può decidere di gestire l'eccezione utilizzando un blocco **`try-catch.`** In questo scenario, **`Main`** non verrà mai a conoscenza dell'errore avvenuto in **`Metodo2`**

Dal punto di vista del design, è fondamentale per ogni metodo distinguere tra **eccezioni proprie**, ovvero eccezioni che il metodo potrebbe lanciare a causa della sua logica interna, ed **eccezioni altrui**, cioè quelle che il metodo potrebbe incontrare a causa delle chiamate ad altri metodi.

Un principio cardine è che ***un metodo dovrebbe gestire automaticamente un'eccezione solo se possiede tutte le informazioni necessarie per risolverla correttamente.***

Una pratica comune nel design del software è quella di **gestire parzialmente** un'eccezione all'interno di un blocco **`catch`**, e poi lanciare un'altra eccezione più generica verso l'alto per riportare il sistema in uno stato più gestibile.

*Per esempio, durante una divisione per zero, si potrebbe identificare e gestire l'errore, ma poi informare il chiamante con un'eccezione più generica, indicando che l'operazione desiderata (come **`CalcolaMedia`**) non è stata completata a causa di un potenziale errore di divisione per zero.*

Runtime Exceptions

Le **eccezioni a runtime** in Java, note anche come "**unchecked exceptions**", rappresentano situazioni d'errore che si verificano **spesso** durante l'esecuzione di un programma e che potrebbero non essere anticipate dallo sviluppatore. Sono classificate come eccezioni non verificate perché **il compilatore Java non richiede che vengano esplicitamente gestite o dichiarate nel codice** (a differenza delle eccezioni controllate o "checked exceptions").

Le eccezioni a Runtime sono difatti così frequenti che se ogni volta che si effettuasse una di queste operazioni frequenti si dovesse dichiarare o gestire l'eccezione, il codice sarebbe sovraccaricato con blocchi `try-catch` e dichiarazioni, rendendo il codice meno leggibile e manutenibile.

In generale, le eccezioni a runtime derivano dalla classe `RuntimeException`, che a sua volta eredita dalla classe `Exception`.

Esempi comuni di `RuntimeException` sono:

1. `NullPointerException`: si verifica quando si tenta di accedere a metodi o attributi su una variabile di riferimento che non è stata ancora inizializzata. In Java, mentre le variabili di tipo primitivo sono allocate direttamente sullo stack, le variabili di tipo riferimento puntano a oggetti allocati nello heap. Se dichiariamo una variabile di riferimento, ma non la inizializziamo con `new`, tentando di accedervi si scatena questa eccezione.
2. `ArithmeticException`: si manifesta quando si eseguono operazioni aritmetiche illegali, quali la divisione per zero. Ad esempio, eseguendo un calcolo come `int risultato = 10 / 0;` scateneremmo questa eccezione.

Checked vs. Unchecked/Runtime Exceptions

	Checked	Unchecked
Origine	Ereditano da `Exception` (esclusa `RuntimeException`)	Sono sottoclassi di `RuntimeException`
Signature dei metodi	Siamo obbligati a dichiarare l'eccezione nella signature del metodo	**NON** Siamo obbligati a dichiarare l'eccezione nella signature del metodo
Try/Catch	**DOBBIAMO** gestire l'eccezione esclusivamente mediante `try/catch` o dichiarata con `throws`	**POSSIAMO** gestire l'eccezione mediante try/catch

Logging delle Exception

Le eccezioni in Java offrono metodi integrati per ottenere dettagli sul loro verificarsi, permettendo una diagnosi e un debug più efficaci.

La keyword **`super`** applicata alle Exception

Nelle lezioni precedenti abbiamo esplorato la keyword **`super`**, un riferimento alla superclasse. Con **`super`** possiamo **accedere ai costruttori e metodi della superclasse**, garantendo che la sottoclasse erediti correttamente le funzionalità parentali. È utile **istanziare una eccezione con una stringa come parametro in ingresso del costruttore**: infatti il tipo di oggetto ritornato dal metodo che fa throw dell'errore rappresenta il tipo di errore

Questa dinamica si estende anche alle Exception: usando **`super(message);`**, passiamo un messaggio specifico al costruttore dell'eccezione padre (in questo caso, appunto, la classe madre è **`Exception`**). Così, pur creando un'eccezione personalizzata, conserviamo il metodo standard di gestire e visualizzare il messaggio d'errore.

Metodi utili per il debug delle Exception

Tra i metodi più utilizzati ereditati da **`Exception`** troviamo **`e.getMessage()`** e **`e.printStackTrace()`**:

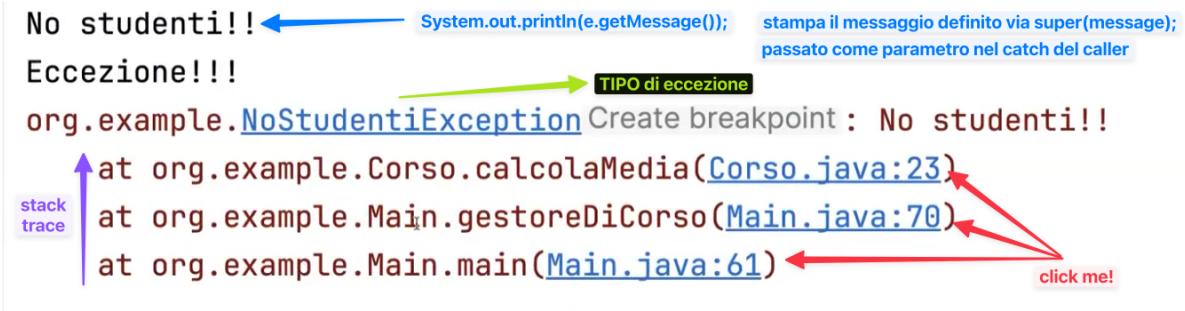
- **`e.getMessage()`**: restituisce il messaggio di dettaglio associato all'eccezione, definito con **`super`** come visto in precedenza
- **`e.printStackTrace()`**: stampa la traccia dello stack dell'eccezione, dalla sua origine alla sua effettiva generazione, sul flusso di errore standard.

Ad esempio:

Main.java

```java
try {
    float media = corso.calcolaMedia();
} catch (Exception e) {
    e.printStackTrace();
    System.out.println(e.getMessage());
    System.out.println("Si è verificata un'eccezione!");
}
```

Output:



IntelliJ IDEA *ci permette addirittura di cliccare collegamenti preposti per andare alla riga in cui si è verificata l'eccezione!*

Nel frammento UML visto in questo corso non si possono rappresentare eccezioni.

Exceptions e perdita del controllo del flusso di esecuzione

Nel processo di sviluppo, ci troviamo spesso di fronte a situazioni in cui, nonostante la presenza di eccezioni, dobbiamo garantire che certe operazioni vengano eseguite **indipendentemente** dall'eventuale manifestarsi di un'eccezione.

Esempio: GESTIONE DI FILE

*Una pratica comune è quella di assicurarsi che un file venga chiuso sia che le operazioni su di esso siano andate a buon fine, **sia che siano sorti degli errori**.*

Ecco dove interviene il blocco `finally`: esso viene eseguito **SEMPRE**, *indipendentemente dal fatto che nel blocco **`try`** sia stata sollevata un'eccezione o meno.*

In termini di flusso di esecuzione, questo significa che potremmo avere:

1. Sequenza **`try`** → **`finally`**
2. Sequenza **`try`** → **`catch`** → **`finally`**

esempio CORRETTO:

Main.java

```java
Corso corso = new Corso(20);
try {
    float media = gestoreDiCorso(corso);
} catch (NoStudentiException e) {
    e.printStackTrace();
    System.out.println(e.getMessage());
    System.out.println("Si è verificata un'eccezione!");
} finally {
    System.out.println("Questo codice verrà eseguito in ogni caso.");
}
```

⚠ **`finally` non è** superfluo!

Non basta ridondare il codice sopra e sotto, perché se dovessimo avere un'altra exception dentro il **`try`** o il **`catch`** perderemmo il controllo, e quindi **non saremmo certi di eseguire la parte di codice ridondante.**

esempio ERRATO:

Codesnippet.java

```java
Corso corso = new Corso(20);
try {
    float media = gestoreDiCorso(corso);
    System.out.println("Questo codice NON verrà eseguito in ogni caso.");
} catch (NoStudentiException e) {
    e.printStackTrace();
    System.out.println(e.getMessage());
    System.out.println("Si è verificata un'eccezione!");
    System.out.println("Questo codice NON verrà eseguito in ogni caso.");
}
```

Lezione 9: Java (12/10/2023 Perla Durante, Elisabetta Cometti)

COLLECTIONS

Generici

Per introdurre le collections e il concetto di tipo generico, iniziamo da una struttura dati molto semplice: la coppia, "**Pair**", ovvero una coppia di due oggetti dello stesso tipo. Partiamo ad esempio con una coppia di stringhe.

PairofStrings

Voglio creare una collection immutabile, così che una volta costruita non possa più essere modificata.

Creiamo una nuova classe PairofStrings in cui inseriamo due attributi final (per immutabilità) first e second di tipo String, aggiungendo i relativi costruttori e getters.

PairOfStrings.java

```
public class PairofStrings {
     private final String first;
     private final String second;

    public PairofStrings(String first, String second) {
          this.first = first;
          this.second = second;
     }

    public String getFirst() {
          return first;
     }

    public String getSecond() {
          return second;
     }

}
```

Pair <T>

Se volessimo creare una **coppia più generica**, sarebbe un errore creare una classe chiamata "PairofObject" poiché potremmo inserire qualsiasi tipo di oggetto al suo interno, risultando in una coppia non omogenea. Questo non è ciò che intendiamo ottenere, quindi "PairOfObject" non può essere la soluzione che stiamo cercando. I limiti di questa approccio sono che non possiamo garantire l'omogeneità dei dati nella coppia, e inoltre, non possiamo conoscere a priori il tipo di valore restituito dai getter.

Per non dover creare una classe per ogni tipo (non vogliamo dover scrivere PairofInteger, PairofDouble, PairofString ecc.), Java ci permette di definire delle **classi che sono generiche rispetto un tipo**. Possiamo quindi scrivere una classe che sia **Pair<T>** (letto "Pair of T") per un qualche tipo T e che si possa istanziare per un qualunque tipo T (classe generica o parametrica rispetto al tipo T). La scriviamo in questo modo:

Pair.java

```java
public class Pair <T>{

    private final T first;
    private final T second;



    public Pair(T first, T second) {
        this.first = first;
        this.second = second;
    }

    public T getFirst() {
        return first;
    }
    public T getSecond() {
        return second;
    }
}
```

In altre parole, <T> funge da placeholder, cioè rappresenta un tipo che andrò poi a definire nel momento in cui dichiarerò una variabile e creerò un'istanza della classe.

Nel Main, per istanziare la classe, quando definisco una variabile di tipo Pair, devo dare un valore a T, concretizzando cosa mi aspetto per T. Nella dichiarazione inserisco il tipo specifico che voglio assegnare a T nelle parentesi angolari <…>:

Main.java

```java
public class Main {
    public static void main(String[] args) {
       Pair<String> stringPair = new Pair<String>("ciao", "hello");

    }

}
```

È un errore dichiarare come tipo <String> e passare come argomento un int, infatti Java ce lo segnala:

Main.java

```java
public class Main {
    public static void main(String[] args) {
      Pair<String> stringPair = new Pair<String>("ciao", 1);
    }
}
```

I generici devono essere dei tipi riferimento, quindi non posso inserire nelle parentesi <> dei tipi primitivi. Esistono però degli oggetti che sono la corrispondenza lato oggetto dei tipi primitivi. Ad esempio, per il tipo “int” c’è il tipo “Integer”, per il tipo “double” c’è il tipo “Double” ecc. . Per quanto ci riguarda sono praticamente interscambiabili, perché Java fa automaticamente le operazioni di boxing e unboxing. Infatti, “Integer” non è altro che un oggetto stanziato sullo heap che contiene solamente un intero. Tutte le volte che ho un codice che prende un “int”, posso passargli un “Integer” e Java esegue l’unboxing, “aprendo” la scatola e prendendo l’intero e, viceversa, se passo un “int” a un “Integer” fa boxing, prendendo l’”int” e mettendolo dentro “Integer”.

Nel main istanzierò quindi la coppia di interi in questo modo:

Main.java
```
Pair<Integer> intPair; //sarebbe sbagliato Pair<int> intPair;
intPair = new Pair <Integer>(1,2); //qui java fa automaticamente boxing
```

DifferentPair <T, U>

È possibile creare una classe che sia **generica rispetto a più di un tipo**. Qui stiamo definendo una classe che rappresenta una coppia composta da due tipi generici distinti:

DifferentPair.java
```
public class DifferentPair <T,U>{
    private final T first;
    private final U second;

    public DifferentPair(T first, U second) {
        this.first = first;
        this.second = second;
    }

    public T getFirst() {
        return first;
    }

    public U getSecond() {
        return second;
    }
}
```

La classe DifferentPair consente di lavorare con coppie di tipi generici diversi, il che significa che è possibile utilizzarla con qualsiasi combinazione di tipi T e U. Questo offre una maggiore flessibilità nel codice, consentendo di creare coppie di dati di tipo diverso senza la necessità di creare nuove classi specifiche per ciascuna combinazione di tipi.

Nel Main la istanziamo così:

Main.java
```
DifferentPair<String, Double> dp;
dp = new DifferentPair<String, Double>("Ciao", 3.4);
```

In molti casi ovvi, come questo, posso omettere il tipo: infatti sarà Java a fare inferenza di tipo (type inference). Per farlo uso il cosiddetto operatore diamond <> in questo modo:

Main.java
```
DifferentPair<String, Double> dp;
dp = new DifferentPair<>("Ciao", 3.4);
```

Se lo si utilizza in casi di ambiguità, Java lo segnalerà.

Collections

In Java raramente si utilizzano gli array e, invece, si preferiscono le **collezioni**. Tra le numerose collezioni presenti nei manuali, ci concentreremo su tre delle più comunemente utilizzate:

- **List** —> sequenza di oggetti omogenei in cui è definito un ordinamento e dove sono ammessi duplicati.
- **Set** —> insieme di oggetti (non sequenza, non è definita la posizione) in cui non sono ammessi duplicati.
- **Map** —> dizionari, associano a una chiave un valore, hanno un modo rapido per ricavare il valore dalla chiave.

List

List è una classe astratta. Ha due implementazioni concrete che ci interessano.

- **ArrayList**: internamente alloca un array di una certa capacità, quando gli elementi che inserisco superano quella capacità dealloca e rialloca un array più grande. (Come array dinamico in C)
- **LinkedList**: serie di elementi contenenti il valore e il riferimento all'elemento successivo. (Come lista in C)

Implementazione di liste di oggetti di tipo String:

Main.java
```
public class Main {
    public static void main(String[] args) {
        List<String> stringList = new ArrayList<>();
        //tipo statico List, tipo dinamico ArrayList (sottotipo di List)

        List<String> stringList1 = new LinkedList<>();
        //tipo statico List, tipo dinamico LinkedList (sottotipo di List
    }
}
```

Alcune delle **operazioni** su List: add, remove, size

Main.java
```
List<String> stringList = new ArrayList<>();

List<String> stringList1 = new LinkedList<>();
```

```java
stringList.add("Ciao"); //aggiunge stringa
stringList.remove("Ciao"); //rimuove stringa
int size = stringList.size(); //ritorna numero di elementi
```

Set

Come implementazione concreta della classe astratta Set utilizziamo il suo sottotipo **HashSet**, un set che per controllare se un elemento è già presente usa l'hash code.

Implementazione di liste di oggetti di tipo String:

Main.java
```java
Set<String> stringSet = new HashSet <>();
stringSet.add("Ciao");
stringSet.add("Ciao"); //non la inserisce perchè è un duplicato
stringSet.add("Hello");
int setSize = stringSet.size(); //setSize = 2
```

Quando facciamo la seconda chiamata a add("Ciao"), la stringa non viene inserita nel set poiché è già presente. I set, come HashSet, si basano sulla verifica di uguaglianza per garantire che non siano presenti duplicati all'interno del set. Questa verifica viene effettuata utilizzando il metodo equals. Quindi, nel nostro caso, setSize vale 2 poiché il set contiene solo "Ciao" e "Hello".

Map

Hasmap è l'implementazione concreta della classe astratta Map. Utilizza un hash per trovare il valore associato a una chiave in tempo costante ammortizzato (O (1)).

Main.java
```java
Map<String, Integer> contacts = new HashMap<>();
contacts.put("Alessandro", 340); //inserisce una chiave e un valore
Integer num = contacts.get("Alessandro");
```

Scorrere su una collezione

Per scorrere tutte le collezioni di Java posso utilizzare il for generalizzato:

Main.java
```java
//List
 for (String s : stringList) {
     System.out.println(s);
 }

//Set
 for (String s : stringSet) {
      System.out.println(s);
 }
```

```java
//Maps
for (String s : contacts.keySet()) { //per le Maps uso l'operazione keySet
   System.out.println(s + " " + contacts.get(s)); //stampo la stringa (chiave s)
//e il valore associato alla stringa (contacts.get(s))
   }
```

Invarianza dei tipi generici

Domanda: se è vero che String è sottotipo di Object, è vero anche che List<String> è sottotipo di List<Object>? Dipende dalla scelta del singolo linguaggio di programmazione.

- Scelta **covariante**: se è vero che String è sottotipo di Object, è vero anche che List<String> è sottotipo di List<Object>
- Scelta **controvariante**: il contrario di covariante, se è vero che String è sottotipo di Object, List<String> è sopratipo di List<Object>
- Scelta **invariante**: se è vero che String è sottotipo di Object, List<String> non è né sopratipo né sottotipo di List<Object>

Java fa la scelta invariante. Dunque, List<String> non è sottotipo di List<Object> e List<Object> non è sottotipo di List<String>, perché se non fosse così potrei scrivere sbagliando:

Main.java
```java
List<String> ls;
List<Object> lo = null;
lo = ls;
```

In apparenza, sembra che l'assegnamento sia valido, ma in realtà, Java impedisce questo tipo di assegnamento perché permetterebbe un inserimento errato. Per dimostrare questo, proviamo ad aggiungere un oggetto che non è una stringa (ad esempio, un intero) a lo:

Main.java
```java
lo.add(12); //Errore! Lo è una List<String> e ora contiene un intero.
```

Se fosse consentito l'assegnamento, avremmo inserito un intero in una lista che dovrebbe contenere solo stringhe. Questo comportamento non è la semantica desiderata e potrebbe causare problemi durante l'esecuzione del programma. Pertanto, Java mantiene l'invarianza per garantire la sicurezza e la coerenza del tipo durante l'assegnamento di oggetti tra liste di tipi generici diversi.

Quindi i generici sono invarianti: list di String e list di Object non sono né sopra tipo né sottotipo l'uno dell'altro, mentre un ArrayList di String è un sottotipo di List di String.

Iteratori

In Java posso iterare su una collezione.

Un oggetto su cui posso iterare è iterabile, ovvero implementa un'interfaccia **Iterable<T>** (iterando sull'oggetto ottengo valori di tipo T). Su tutto ciò che è Iterable<T> io posso usare un for generalizzato.

Il metodo definito nell'interfaccia Iterable<T> è il metodo **iterator()**, che mi ritorna un Iterator<T>.

Iterator<T> è un oggetto che mi espone due metodi, che mi permettono di generalizzare il concetto di scorrimento su una collezione:

- hasNext(): booleano che ci indica se ci sono ancora elementi.
- next(): se è vero hasNext(), ritorna il prossimo elemento.

Come abbiamo già visto, List è iterabile perché la classe Collection in Java, di cui è sottoclasse, lo è. Posso quindi utilizzare il for generalizzato:

Main.java
```
for(String s : stringList){
            System.out.println(s);
    }
```

Java lo traduce automaticamente in questo modo (codice equivalente a quello sopra):

Main.java
```
Iterator<String> it = stringList.iterator();

        while (it.hasNext()){//continuo a iterare finchè ho ancora elementi

        System.out.println(it.next());

        }
```

Vogliamo implementare ora un iteratore per scorrere la nostra collection Pair<T> con un for generalizzato.

Innanzitutto, vogliamo che la classe Pair sia iterabile:

Pair.java
```
public class Pair<T> implements Iterable<T>{...}
```

L'interfaccia Iterable<T> ci promette di implementare il metodo iterator() che mi ritornerà un Iterator<T>.

Definiamo prima un iteratore specifico per la coppia implementando una **classe privata PairIterator,** interna alla classe Pair (questa classe non sarà esposta nemmeno al package perchè interessa solo a Pair, non deve essere visibile dall'esterno).

Pair.java
```
public class Pair<T> implements Iterable<T>{

...
     private class PairIterator implements Iterator<T>{
             private int pos;

             @Override
             public boolean hasNext(){
                 return pos<2;
             }

             @Override
```

```
            public T next(){
               if(pos == 0){
                    pos++;
                    return first;
               } else if (pos ==1 ){
                    pos ++;
                    return second;
               }

               return null;
    }
}
```

Questo PairIterator deve implementare un Iterator<T> (implements Iterator<T>) e dunque avere i due metodi di Iterator, hasNext() e next().

Costruito il nostro iteratore specifico, possiamo passarlo a chiunque chieda un iterator dall'esterno chiamando il metodo iterator() definito in Pair:

Pair.java

```
public class Pair<T> implements Iterable<T>{
        ...
        @Override
        public Iterator<T> iterator(){
             return new PairIterator();
        }

        private class PairIterator implements Iterator<T>{
             ...
}
```

Istanziamo il ciclo for generalizzato nel Main:

Main.java

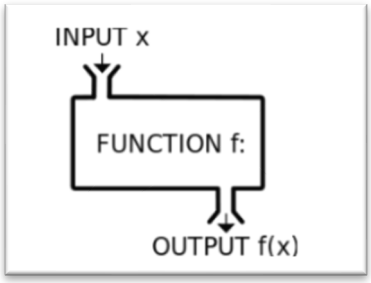
```
Pair<String> prova = new Pair("Ciao", "Hello");
for (String s : prova) {
    System.out.println(s);
}
```

1. Lezione 10: Java funzionale (16/10/2023 Gianluca Di Paola, Matteo Bergagna)

Introduzione al paradigma di Programmazione Funzionale

Esistono diversi paradigmi di programmazione, tra cui quelli:

- **Object Oriented**: si bassa sul fatto che esistano strutture dati che **mutano nel tempo** e che interagiscono con le funzioni, mediante **metodi associati** che ne mutano lo stato.
  Ad esempio, se in una lista volessi aggiungere un elemento, il suo contenuto verrebbe modificato.
- **Programmazione Funzionale**: si basa sull'assunzione che **i dati siano immutabili** (almeno, nel paradigma funzionale puro) e che una computazione venga espressa attraverso una **trasformazione dei dati** (mediante funzioni, intese in senso matematico ovvero l'output dipende soltanto dall'input).



Il paradigma Object Oriented permette la modifica delle parti interne dell'oggetto, da parte dei metodi chiamati, mentre quello della Programmazione Funzionale non muta la natura degli attributi. Oggigiorno quasi tutti i linguaggi supportano (almeno parzialmente) la programmazione funzionale.

Vantaggi e Side Effects

Vantaggi del paradigma della programmazione funzionale:

1. È più **facile**, in alcuni contesti, ragionare sul codice **se non si hanno side effects**, che rendono difficile testare le applicazioni.
   - Un esempio di *side effect* è il fatto che la funzione potrebbe comportarsi in modo diverso, in base al momento in cui viene chiamata (stati differenti del sistema possono generare risultati differenti).
2. **Semplifica il parallelismo**: la programmazione concorrente potrebbe annullare l'atomicità dei metodi.
   - Anni fa non era frequente fare uso di tecniche di programmazione parallela (limitazioni hardware), mentre le implementazioni più di recenti prediligono un approccio di programmazione concorrente.
   - Infatti, il grosso problema del parallelismo è che, se ci sono **più flussi di esecuzione** che lavorano in contemporanea **sugli stessi dati**, si potrebbero **generare problemi di accesso concorrente**.
3. **Stessi parametri** in ingresso implicano lo **stesso risultato**. Questa proprietà non è garantita dai linguaggi object-oriented (l'oggetto su cui lavoro o i parametri possono mutare tra due identiche chiamate).

Programmazione Funzionale

Nei linguaggi di programmazione **funzionali puri**, **non esistono variabili**. Le strutture dati sono immutabili; infatti, se volessi ampliare una lista, dovrei ri-generarla completamente, spostando i dati all'interno da quella pre-esistente a quella nuova.

📌 Programmazione funzionale e metodo matematico sono molto vicini, quanto a metodo di ragionamento!

Note (esempi di programmazione funzionale):

- *list.add(...)* non esiste! Ogni volta che voglio cambiare una lista, ne devo creare una nuova
- *for (int i=0; i<10; i++)* non esiste! **Non posso mutare *i*!** I cicli sono sostituiti da invocazioni ricorsive di funzioni

Osservazioni:

- **Astrazione sui dati**: il paradigma Object-Oriented tende a favorire l'incapsulamento
- **Astrazione sulle funzioni**: il paradigma Functional tendono a favorire la composizione delle funzioni

Programmazione Funzionale in Java

Da Java 8, è stato introdotto uno "zucchero sintattico" che agevola la programmazione funzionale. Ad esempio:

📄 java

```java
class Persona {
    private String nome;
    private int eta;
    ...
    int getEta() {
        return eta;
    }
}
```

📄 Main.java

```
list<Persona> list=...
list.add(...);
list.add(...);
```

⁉️ Problema: come ordino le liste in modo che le persone appaiano in ordine crescente di età? Uso ***Collections***, in particolare il metodo statico ***Collections.sort()***.

📄 sort

```
public static <T> void sort(List<T> list, Comparator<? super T> c);
//ordina la lista specificata, secondo l'ordine introdotto dallo specifico comparatore
```

⚠️ Attenzione alla sintassi: quando una classe non è parametrica rispetto a un tipo generico T, **si può parametrizzare** un metodo al suo interno con la l'espressione *<T>* come sopra.

Osserviamo quindi che *Sort* è un metodo generico rispetto a un parametro T che prende in ingresso una lista di oggetti di tipo *T* e la ordina tramite un *Comparator<T>*, ovvero un'interfaccia che ha come unico metodo *compare* che dice al chiamante **come** ordinare i due oggetti.

Inoltre, **deve** (non ne esisterebbe, altrimenti, un'implementazione) **essere implementata l'interfaccia *Comparator***, che fornisce maggiore controllo sulla metodologia di ordinamento degli oggetti, sovrascrivendo il metodo di comparazione di default.
Ad esempio, volendo fornire (a livello di astrazione) a Java la funzione per confrontare due oggetti (qui, due persone), potremmo fare:

📄 Main.java

```
MyComparator c = new MyComparator();
Collections.sort(list, c)
```

📄 MyComparator.java

```
class MyComparator implements Comparator<Persona> {
   @Override
   public int compare(Persona p1, Persona p2) {
      if (p1.getEta() < p2.getEta()) return -1;
      else if (p1.getEta() > p2.getEta()) return 1;
      else return 0;
   }
}
```

- ⚠️ Java **non** abbraccia **del tutto** il paradigma funzionale puro infatti, nell'esempio, la lista viene modificata!!
- La classe MyComparator implementa la sola funzione compare che, semplicemente, codifica il confronto tra due elementi. Di conseguenza ***Sort*** sarà parametrico rispetto a tale funzione di *MyComparator*.

Creare una classe a se stante solo per ridefinire il metodo *compare* non ha molto senso e quindi alternativamente si usano le **Classi Anonime**: class che **definisco** e **istanzio contemporaneamente** (come illustrato qui sotto).

📄 Main.java

```
Collections.sort(l, new Comparator<Persona>() {
   @Override
   public int compare(Persona p1, Persona p2) {
   if (p1.getEta() < p2.getEta()) return -1;
   else if (p1.getEta() > p2.getEta()) return 1;
   return 0;
   }
});
```

- ✅ Vantaggio: non devo creare nuove classi
- ❌ Svantaggio: usando le classi anonime posso usare tale classe in un solo punto e non in altri del mio codice

Interfacce Funzionali e Lambda Expression

Da Java 8, quando ho una interfaccia che implementa un solo metodo (come *Comparator*) sto usando le cosiddette **Interfacce Funzionali** (**Functional Interfaces**). La sintassi delle Interfacce Funzionali è la **Lambda Expression** che aiuta nella **leggibilità** del codice.
Vediamone un esempio:

Main.java

```
Comparator<Persona> c = (Persona p1, Persona p2) -> {
   if (p1.getEta() < p2.getEta()) return -1;
   else if (p1.getEta() > p2.getEta()) return 1;
   else return 0;
};
```

- Abbiamo dichiarato una classe anonima per *Persona* nella quale è implementato come unico metodo quello di *Comparator*.

💡 Abbiamo appena costruito un nuovo **metodo di astrazione**: le **Funzioni**. Anche da un punto di vista matematico, *c* è proprio una funzione!

⁉️ Domanda: "Come faccio a sapere che il metodo che sto implementando è proprio *Compare*?" Perché questa **sintassi** funziona solo con le **interfacce funzionali**… quindi non posso sbagliarmi!

📌 Se si hanno possibili eccezioni nel codice dell'interfaccia è bene creare una classe specifica per questo ruolo senza usare la notazione Lambda Expression

Osservazioni sintattiche:

- Dato che sto implementando il metodo *Compare* di *Persona* (rif. *Comparator<Persona> c=(p1, p2)*), posso anche omettere i tipi di dato, come ad esempio:

Main.java

```
Comparator<Persona> c = (p1, p2) -> {
   if (p1.getEta() < p2.getEta()) return -1;
   else if (p1.getEta() > p2.getEta()) return 1;
   else return 0;
};
```

- Quando abbiamo due o più input li inseriamo in parentesi tonde (se solo uno, si possono omettere);
- Dopo la "freccia" della **Lambda Expression** si può mettere un singolo **statement** oppure più **statements** fra parentesi graffe.

Patterns di programmazione funzionale

Java fornisce dei pattern funzionali all'interno delle sue librerie. All'interno di ***java.utils.funtion*** troviamo delle Interfacce funzionali che aiutano a definire i seguenti concetti:

Interfacce	Tipo degli argomenti	Tipo del risultato
Function<T,R>	T	R
BiFunction<T,U,R>	T, U	R
Consumer<T>	T	non torna nulla
Predicate<T>	T	Boolean

Optional *<T>*

Un elemento importante del paradigma di programmazione funzionale è il tipo ***Optional<T>***, che permette di ignorare il fatto che alcuni dati possano essere assenti (*null*). Ipotizziamo un esempio classico utilizzando *null* quando qualcosa non esiste:

Indirizzo.java

```
public class Indirizzo {
   final String via;
   final String commenti;

   public Indirizzo(String via, String commenti)
   {
      this.via = via;
          this.commenti = commenti;
   }

   public Indirizzo(String via) {
      this.via = via;
      this.commenti = null;
   }
   ...
   public String getCommenti() {
      return commenti;
   }
}
```

Persona.java

```
public class Persona {
   final String nome;
   final Optional<Indirizzo> indirizzo;

   public Persona(String nome, Indirizzo indirizzo)
   {
      this.nome = nome;
      this.indirizzo = Optional.of(indirizzo);
   }

   public Persona(String nome) {
      this.nome = nome;
      this.indirizzo = Optional.empty();
   }
   ...
   public Optional<Indirizzo> getIndirizzo() {
   return indirizzo;
   }
}
```

✏️ Problema: Vogliamo stampare, se ci sono, i commenti; altrimenti, la stringa "no comment".

Main.java

```
Persona p = …
…
Indirizzo i = p.getIndirizzo();
if (i == null) {
   System.out.println(“No comment”);
} else {
   String c = i.getCommenti();
   if (c == null) {
      System.out.println(“No comment”);
   } else {
      System.out.println(c);
   }
}
```

⚠ Questo codice è **problematico** e **poco comodo**:

- ad ogni passo, dobbiamo preoccuparci del fatto che il risultato potrebbe essere *null*;
- troppo codice aggiunto;
- se mi dimentico un controllo, causo un’eccezione a runtime;
- le cose peggiorano all’aumento del numero di classi.

⁉ Come può aiutarmi la programmazione funzionale? *Optional<T>* **nasconde** il fatto che un elemento possa essere *null*:

- ***Optional.of(val)***: crea un Optional con **quel valore**
- ***Optional.empty()***: crea un Optional **vuoto**

Modifichiamo quindi il codice adottando la programmazione funzionale:

Indirizzo.java

```
public class Indirizzo {
   final String via;
   final Optional<String> commenti;

   public Indirizzo(String via, String commenti){
this.via = via;
      this.commenti = Optional.of(commenti);
   }

   public Indirizzo(String via) {
      this.via = via;
      this.commenti = Optional.empty();
   }
   …
   public Optional<String> getCommenti() {
    return commenti;
   }
}
```

Persona.java

```
public class Persona {
   final String nome;
   final Optional<Indirizzo> indirizzo;

   public Persona(String nome, Indirizzo indirizzo){
      this.nome = nome;
      this.indirizzo = Optional.of(indirizzo);
   }

   public Persona(String nome) {
      this.nome = nome;
      this.indirizzo = Optional.empty();
   }
    …
   public Optional<Indirizzo> getIndirizzo() {
      return indirizzo;
   }
}
```

⁉ Sorge un altro problema: come accedo al valore interno di un *Optional<T> o* ? In particolare come possiamo far si che l'oggetto ci dia un commento, se esiste, e in caso contrario Empty? Esistono varie possibilità:

- *o.get()*: è sconsigliato, **non** segue l'approccio funzionale!
- *o.ifPresent()*: prende in ingresso una funzione che accetta T e ci fa qualcosa, venendo invocata solo se non è vuoto
- ***o.flatMap()*** è un metodo che:
    1. Prende in ingresso una funzione da T a Optional<U>;
    2. Ritorna un Optional<U>;
    3. Se o è empty ritorna empty, altrimenti applica la funzione al contenuto e ritorna il risultato della funzione.
- ***o.orElse(T val)*** prende un Optional<T> e:
    1. Se o non è vuoto ritorna il suo valore;
    2. Altrimenti ritorna val.

📌 Gli ultimi tre metodi sono quelli che più si avvicinano all'approccio funzionale.

💡 L'idea è di avere un *Optional* che funge da **scatola con due stati: vuoto o pieno**. Passiamo quindi una funzione alla *flatMap* che si preoccupa di controllare se la scatola è piena e, in tal caso, applica la funzione al contenuto (esempio: *getIndirizzo*) e ne ritorna il risultato, altrimenti ritorna la scatola vuoto.

⁉ La gestione del codice è migliorata? Certo, basta paragonare il con e senza Optional<T>:

📄 Indirizzo.java

```
//SENZA Optional<T>

Persona p = ...
...
if (p == null) {
   System.out.println("No comment");
} else {
   Indirizzo i = p.getIndirizzo();
   if (i == null) {
      System.out.println("No comment");
   } else {
      String c = i.getCommenti();
      if (c == null) {
         System.out.println("No comment");
      } else {
         System.out.println(c);
      }
   }
}
```

📄 Persona.java

```
//CON Optional<T>

Optional<Persona> o = ...
...
String commenti = o
   .flatMap(p -> p.getIndirizzo())
   .flatMap(i -> i.getCommenti())
   .orElse("no comment");

System.out.println(commenti);
```

p -> p.getIndirizzo() è una funzione che passo alla *flatMap*. Lo stesso vale per la riga seguente.

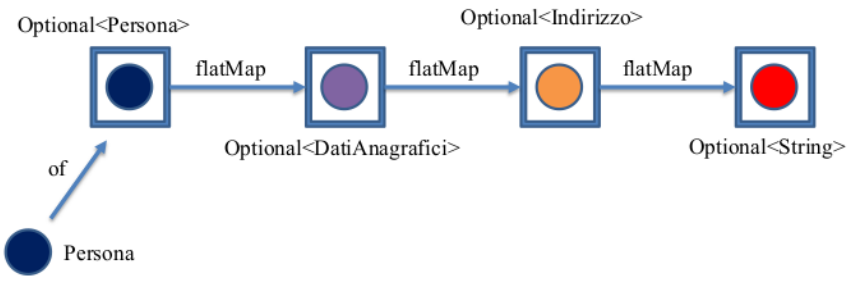
Ogni metodo *flatMap* ritorna un nuovo optional che poi viene usato per il *flatMap* oppure l'*orElse* seguente (che ritornano un Optional). Si ha quindi una concatenazione di Optional e funzioni.

Introduzione alla Composizioni di Funzioni

Quello appena sviscerato è uno schema ricorrente nella programmazione funzionale (ma non l'unico: ne vedremo altri).

💡IDEA: **inglobare un valore in una struttura**, che permette di **concatenare funzioni**, mascherando alcuni aspetti. La combinazione di *Optional* e *flatMap* premette di mantenere una visione esterna su degli "scatolotti" e non semplici oggetti.

Ad esempio, ciò avviene nel seguente **Monade:**



Lezione 11: Java funzionale (19/10/2023 Gianmarco Dezi, Riccardo Alì)

Intro: L'idea di base della programmazione funzionale è avere delle trasformazioni di dati immutabili.

Le Collection sono strutture dati mutabili: hanno metodi che mi permettono di modificare il loro contenuto.

Da java 8 posso trasformare le collection in uno Stream.

Stream <T>

Uno Stream è il corrispondente immutabile di una Collection. La modifica di tali strutture dati avviene con delle funzioni di libreria che mi permettono di trasformare uno Stream in un altro Stream.

Qualunque sia la collection (List, Set, ecc), esiste un metodo ''.stream'' che trasforma la Collection in uno Stream.

Essendo immutabili, non esiste alcun metodo stream.add, .delete ecc.

Si usano le seguenti funzioni di Stream:

forEach()

Parametri in ingresso:

Consumer<T>

(ossia una funzione che prende in ingresso T e ritorna void.

ATTENZIONE: Function<T, void> NON è una notazione accettata in Java)

Valore di ritorno:

void

Cosa Fa:

serve per iterare attraverso tutti gli elementi dello stream e applicare un'operazione a ciascun elemento.

(forEach astrae sulla moltitudine dell'elemento)

Note ed esempi:

È un'operazione terminale che viene utilizzata per eseguire un'azione su ogni elemento dello stream, tipicamente utilizzata per operazioni che non necessitano di un risultato, come stampare gli elementi su una console o eseguire calcoli senza la necessità di restituire un valore.

Se si necessita di un risultato, si consideri l'uso di altre operazioni terminali che generano un risultato, come collect(), reduce(), o sum()

Dopo che forEach() è stata eseguita, lo stream non può più essere utilizzato perché è stato "consumato" dall'operazione terminale.

📄 Stampa tutti gli elementi.java

```
…
List<String> list = …
list.stream().forEach(x -> System.out.println(x));
```

✏️ Data una lista di stringhe:

1)il comando .stream crea uno stream

2) il comando .forEach itera su ogni elemento x, e stampa a schermo x

DIGRESSIONE SINTATTICA

RIFERIMENTO A METODO

In Java 8 è stata introdotta un'ulteriore sintassi chiamata riferimento a metodo, che consiste in una versione più concisa e leggibile delle espressioni lambda, pensata per quelle espressioni che chiamano solamente un metodo specifico.

L'utilizzo di riferimento a metodo è da preferire tuttte le volte che io ho un parametro o un insieme di parametri e voglio SOLAMENTE applicare una funzione a tutti i parametri nell'ordine in cui li ho passati.

Le sintassi possibili sono le seguenti:

1) object::instanceMethod

2) Class::staticMethod

3) Class::instanceMethod

4) Class::new

📄 EsempioRiferimentoAMetodo.java

```
list.stream().forEach(x -> System.out.println(x));
list.stream().forEach(System.out::println)
```

✏️ Le due righe di codice sono equivalenti

Il riferimento a metodo che utilizzeremo di più è quello a metodi statici.

FINE DIGRESSIONE SINTATTICA

filter()

Parametri in ingresso:

Predicate<T>

(ossia una funzione che prende un <T> e restituisce un booleano)

Valore di ritorno:

Stream<T>

Cosa Fa:

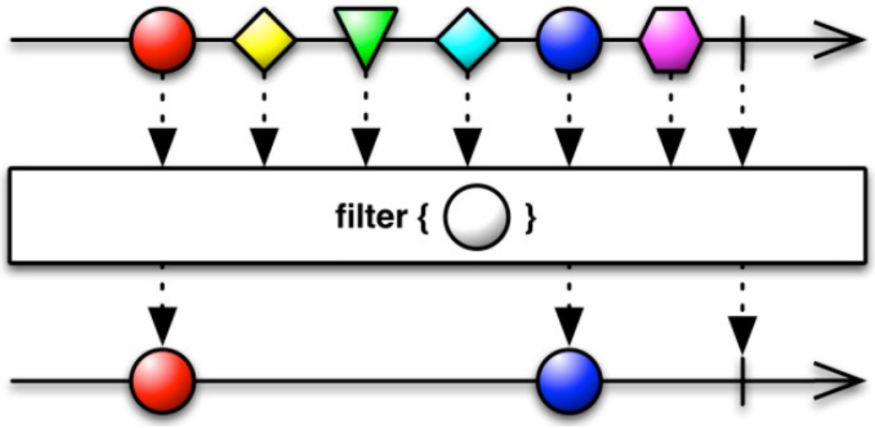
La filter() di uno Stream<T> viene utilizzata per filtrare i dati sulla base di una condizione specifica: solo gli elementi dello Stream iniziale che soddisfano la condizione (cioè, per i quali il predicato ritorna `true`), vengono aggiunti nello Stream risultante;

Note ed esempi:

📄 Stampa solo i numeri pari.java

```
…
List<Integer> list = …
list.stream()
    .filter(x -> x%2 == 0)
    .forEach(System.out::println);
```

✏️ è buona norma andare a capo per ogni metodo; filter itera su ogni elemento x: se x%2 è uguale a 0 (se x è pari) allora x farà parte dello stream risultante



map()

Parametri in ingresso:

Function <T, U>

Valore di ritorno:

Stream<U>

Cosa Fa:

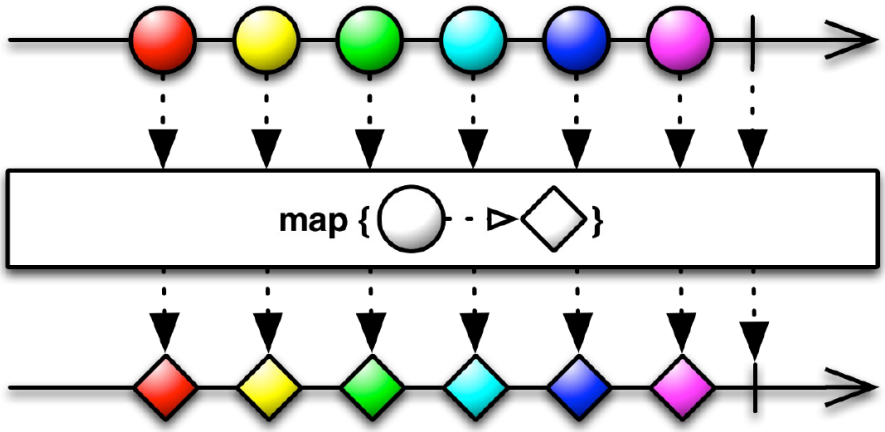
La map di uno Stream<T> trasforma tutti gli elementi dello stream iniziale tramite la trasformazione passata in ingresso e ritorna uno Stream<U> composto dai suddetti elementi trasformati

Note ed esempi:

📄 Stampa la lunghezza di ogni stringa.java

```
…
List<String> list = …
list.stream()
    .map(x -> x.length())
    .forEach(System.out::println);
…
```

✏️ .map trasforma ogni elemento x (che è una stringa), nella sua corrispondente lunghezza (che è un intero)



distinct()

Parametri in ingresso:

void

Valore di ritorno:

Stream<T>

Cosa Fa:

Elimina i duplicati (a e b sono duplicati se a.equals(b))

Note ed esempi:

Garantisce che tutti gli elementi nello stream di output siano unici secondo la loro implementazione del metodo `equals`.

`distinct` è un'operazione stateful che potrebbe quindi richiedere memoria significativa perché deve conservare gli elementi precedentemente visti per garantire che non ci siano duplicati.

sorted()

Parametri in ingresso:

void

oppure un Comparator<T> (vedi sotto)

Valore di ritorno:

Stream<T>

Cosa Fa:

ordina gli elementi con uno dei seguenti criteri:

- Ordine naturale per interi e stringhe
- Usa Comparable (se la struttura dati <T> implementa Comparable<T>)
- Usa un overload del metodo sorted (in questo caso la sorted prenderà in ingresso un Comparator<T>)

Note ed esempi:

Se il tipo di elemento richiede un Comparator che però non viene passato, si cade in un RunTime error

limit(x)

Parametri in ingresso:

long

Valore di ritorno:

Stream<T>

Cosa Fa:

Ritorna uno Stream con i primi X elementi dello Stream iniziale (X è il long passato per input)

Note ed esempi:

Il metodo `limit()` funziona solo per assicurare che lo stream non superi il numero massimo di elementi stabilito. Quindi se il limite specificato è SUPERIORE al numero di elementi presenti nello stream, non verrà estesa la dimensione dello stream per raggiungere il limite specificato.

Pertanto, se hai uno stream di 8 elementi e applichi il metodo `.limit(10)`, lo Stream risultante avrà 8 elementi (e non 10, con gli ultimi due inizializzati all'elemento nullo)

L'operazione di limitazione è una operazione di elaborazione a corto circuito, ossia non necessariamente elabora tutti gli elementi dello stream di ingresso per produrre il nuovo stream di uscita. Piuttosto si ferma non appena ha raggiunto il numero necessario di elementi

flatMap()

Parametri in ingresso:

Function(T, Stream<U>)

Valore di ritorno:

Stream<U>

Cosa Fa:

La flatmap per uno Stream<T> applica la funzione passata in ingresso a tutti gli elementi dello Stream iniziale, generando in questo modo uno Stream di Stream.

Poi appiattisce (flatten) lo Stream di Stream ottenuto in un semplice Stream, inserendo nell'UNICO Stream di uscita tutti gli elementi di tutti gli Stream<U> generati dalla funzione in ingresso.

Note ed esempi:

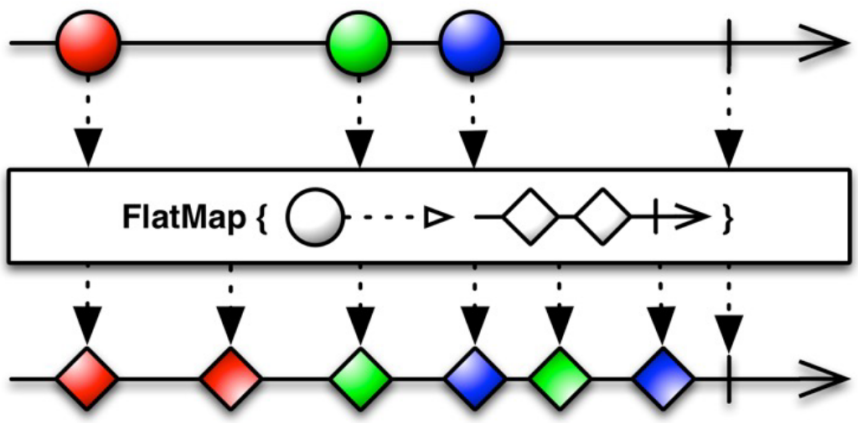
Non è garantita alcuna forma di ordinamento nello Stream che viene ritornato

Main.java

```
…
list.stream()
    .flatMap(str -> str.lines())
    .mapToInt(line -> line.length())
    .sorted()
    .limit(10)
    .forEach(System.out::println);
…
```

✏️ La funzione in ingresso della flatmap va da una stringa a uno stream, quindi si deduce che .lines, seppur non conoscendola, deve sicuramente ritornare uno Stream

✏️ .mapToInt è un metodo che mi permette di creare un IntStream. Un IntStream ha tutti i metodi di Stream (più altri metodi specifici per gli interi) ed è uno Stream con dentro gli int (piuttosto che Int). Questo risponde al problema di non poter avere uno Stream di int(minuscolo), poichè in Java non esiste un generico di tipo primitivo. ATTENZIONE: IntStream è una classe DIVERSA da Stream ma che ha i suoi stessi metodi.



reduce

Parametri in ingresso:

- Identità: Un valore che non cambia il risultato della riduzione (come `0` per l'addizione o `1` per la moltiplicazione). Questo è il valore di partenza della riduzione.

- Accumulatore: Una funzione binaria che prende due parametri: il risultato parziale della riduzione (di tipo T) e l'elemento successivo dello stream (anch'esso di tipo T). Produce un nuovo valore che sarà utilizzato come input parziale per la prossima operazione.

Valore di ritorno:

<T> (se viene fornita un'dentità come input)

Optional<T> (se non viene fornita un'identità)

Cosa Fa:

La `reduce()` riduce tutti gli elementi di uno Stream a un singolo valore che è il risultato della combinazione di tutti gli elementi dello stream secondo la funzione di accumulo specificata.

Alcuni esempi di riduzione sono la somma e la media.

Note ed esempi:

Utilizzare operatori che godano della proprietà associativa se si sta lavorando in contesti con possibile implementazione parallela

SommaOppureMedia.java

```
…
Integer somma = list.stream()
.mapToInt(parola -> parola.length())
.sum();


double media = list.stream()
.mapToInt(parola -> parola.length())
.average()
.getAsDouble();
// Average ritorna un OptionalDouble …
```

✏️ Si noti che da un intStream, .sum ritorna un intero

✏️ .average non ritorna un double, ma un optionalDouble (che è l'equivalente di Optional, per i double (d minuscola))per evitare errori qualora non ci siano elementi (Evita la divisione per 0)

La reduce può anche essere implementata con un input binario (un valore identità e un accumulatore, ossia una funzione che prende a sua volta come input il risultato parziale della riduzione e l'elemento corrente dello stream, restituendo così un'altra riduzione parziale.

ImplementazioneManualeSomma.java

```
Integer somma = list.stream()
.mapToInt(parola -> parola.length())
.reduce(0, (a, b) -> a + b);
```

collect():

Come detto prima reduce() crea un nuovo valore per ogni elemento dello stream, in modo da poter far ritornare solo quel valore e non l'intero stream, la funzione collect() invece va a modificare sempre lo stesso valore, andando a salvare e a riconvertire uno stream in una lista riempiendone i suoi campi. Questa procedura non è sempre possibile farla con un forEach in quanto alcune volte risulta essere poco efficiente e può provocare alcuni problemi. Questi problemi riguardano la concorrenza, in quanto potrebbe capitare che più thread vadano a processare i dati e successivamente a scriverli nello stesso stream che verrà poi riconvertito in una lista, andando però a creare una race.

Il suo funzionamento è legato alla presenza di tre funzioni in ingresso:

- Supplier che crea il valore di ritorno
- Accumulator che incorpora un elemento nel valore di ritorno
- Combiner che combina due risultati parziali

Gli stream possono essere valutati in parallelo e i risultati vengono poi uniti grazie a una funzione di 'addAll' . Questa funzione fa parte del metodo collect() che viene implementata nel collector di libreria predefinito, quindi andando a chiamare: `.collect(Collectors.toList());` ci assicuriamo di riunire tutti i risultati della gestione degli stream che vengono poi convertiti in una lista.

Esempio di utilizzo del metodo collect(): salvare in una lista le parole che iniziano per 'a'

Main.java

```
…
List<String> list = …
List<String> paroleConA = list.stream()
         .filter(parola -> parola.startsWith("a"))
         .collect(ArrayList :: new,
                     ArrayList :: add,
                     ArrayList :: addAll);
…
```

Nel caso di strutture dati "standard" ci sono dei collectors predefiniti di libreria che permettono di velocizzare la scrittura del codice, dato che contengono al proprio interno l'implementazione delle funzioni utilizzate:

Main.java

```
…
List<String> list = …
List<String> paroleConA = list.stream()
         .filter(parola -> parola.startsWith(“a”))
         .collect(Collectors.toList());
…
```

Collectors in questo caso è un Overload di collect() che prende al suo interno tutto ciò che è necessario per passare le funzioni presenti all'interno del caso generale di collect().

I due casi possibili di implementazione di funzioni di libreria sono:

- `.collect(Collectors.toList())`
- `.collect(Collectors.toSet())`

collect() sarà la funzione inversa di stream, in quanto mi permette di tornare ad avere una lista una volta che ho terminato di utilizzare gli stream.

Stream <T>: Osservazioni:

- Generalmente uno stream non memorizza i suoi elementi dato che possono essere memorizzati in una collezione o generati on demand, l'unico caso in cui uno stream potrebbe dover memorizzare i dati è durante la sua valutazione.
- Le operazioni su stream:
    - Non possono modificare lo stream a cui vengono applicate, ma andranno a utilizzare solo il suo contenuto.
    - Possono restituire nuovi stream che contengono i risultati, ovvero sono concatenabili e permettono quindi di andare a eseguire un'altra intermediate operation.
    - Sono “Lazy” quando è possibile, cioè sono pigre, infatti vengono eseguite solo quando sono necessarie per eseguire una terminal operation che porta al risultato (ad esempio: se si stanno cercando le prime 5 parole che iniziano per N, allora il filtro si fermerà dopo il quinto successo di ricerca).

Closures:

Fino a questo punto abbiamo visto solo funzioni, ma java è un linguaggio molto più potente dove le funzioni sono in realtà delle Closures: ovvero possono catturare l'ambiente in cui si trovano.

Ad esempio, se si definisce una variabile Predicate di string, ovvero una funzione che va da una stringa a un booleano, in particolare questa funzione parte da una stringa e valuta true se questa stringa inizia per “L”.

Main.java

```
…
Predicate<String> pred = s -> s.startsWith(“L”);
…
List.stream().filter(pred). …
…
```

StartsWith è dunque un metodo di String poichè s è una stringa.

Il problema è: “Come si fa a definire un predicate generale che non filtra sulla base L come in questo caso, ma su una base generica, come si procede?”

La soluzione è che è possibile andare a catturare il contesto (lo scope) all’interno delle funzioni, cioè posso andare ad utilizzare delle variabili che sono definite fuori dalla funzione considerata andando così a definire (chiudere) il contesto --> questo è ciò a cui ci si riferisce quando si parla del concetto di Closures. In questo modo si va a generare un metodo che ritorna una funzione lambda parametrizzata dove: Predicate è anch’esso un tipo di dato e può essere il valore di ritorno del metodo. Nell’esempio seguente si può notare che il codice è un metodo dove è visibile, e quindi è in Scope, una variabile (String prefix) che non è l’input della funzione ma è qualcosa che è definito nel contesto e che quindi posso usare nella funzione lambda.

Esempio di Closures:

Main.java

```
…
Public static Predicate<String> pred(String prefix){
Return s -> s.startsWith(prefix);
}
…
List.stream().filter(pred(“X”)). …
```

In questo modo si può andare a chiamare il metodo tutte le volte che si vuole e ogni volta ritornerà un predicate di string diverso partendo dal prefix che gli ho passato, quindi questo è un metodo che va a generare delle funzioni dal parametro in ingresso che gli viene passato perché quella funzione si chiude rispetto al contesto.

N.B.: ci sono delle regole affinchè questo processo funzioni e si vada ad evitare di avere side effects: la più importante è che quella variabile sia di tipo **final** o **effectively final** cioè che in ogni caso non può essere modificata fino alla fine del programma una volta che viene dichiarata. Non è infatti consentito richiamare un metodo modificatore nello scope in cui sto utilizzando quella variabile.

Un altro esempio di closure:

Main.java

```
…
final String prefix = “Z”
Predicate<String> pred = s -> s.startsWith(prefix);
…
```

In questo caso pred chiude sulla variabile prefix definita al di fuori della funzione lambda.

parallel:

Capita frequentemente che gli elementi di uno stesso stream siano indipendenti tra loro e per questo possono essere analizzati in parallelo e per poterlo fare basterà aggiungere un .parallel() nella porzione di codice che richiama la list.stream().

Esempio:

Main.java

```
…
List<String> list = …
List.stream().parallel()
    .map(x -> x.size())
    .forEach(System.out::println);
…
```

Parallel: side effects

Bisogna però stare attenti alla gestione in parallelo degli elementi in uno stream, perché in alcuni casi dove non è consentito si vanno a generare dei side effects ovvero degli errori nell’esecuzione del codice (a runtime); ad esempio:

Main.java

```
…
final List<Integer> result = new ArrayList<>();
List.stream().parallel()
    .map(x -> x.size())
    .forEach(x -> result.add(x));
…
```

In questo caso c’è un problema perché non è thread-safe, in quanto, mettendo parallel, più thread in parallelo stanno processando i dati e più thread in parallelo stanno scrivendo sulla lista. In questo modo definisco un accesso concorrente alla stessa struttura dati da parte di più thread e questo genera una race.

Main.java

```
…
final List<Integer> result = new ArrayList<>();
list.stream().parallel()
    .map(x -> x.size())
    .collect(Collectors.toList());
…
```

Invece se si va a eseguire lo stesso procedimento, ma si utilizza la parte dichiarativa di java con la

thread una propria lista e poi le va a unire insieme alla fine con un “addAll” (già compreso nella parte dichiarativa).

Esercizi tratti da temi d’esame sulla programmazione funzionale:

Esercizio 1 (appello 12.02.2023):

Main.java

```
public class Shop{
       public List<Product> getProducts();
       …
}
public class Product{
       public int getPrice();
       public String getName();
       …
}
//Si completi il metodo getAllProductsNames, che ritorna una lista con
//tutti i prodotti in vendita in almeno uno Shop presente nella lista
//shops (senza duplicati)

Public static List<String> getAllProductsNames(List<Shop>shops){
       return shops.stream()
       .flatMap(s -> s.getProducts().stream())
       .map(p -> p.getName())
       .distinct()
       .collect(Collectors.toList());
}
```

- Shops.stream() ritorna uno stream di shop
- .flapMap è una funzione che prende ogni elemento di shop e fa ritornare uno stream di qualcosa in questo caso per ogni shop fa ritornare lo stream dei prodotti degli shop
- poi mappo da prodotti il nome dei prodotti per ottenere uno stream di stringhe che saranno tolte dallo stream con distinct e poi inseriti in una lista con collect().

Esercizio 2 (appello 10.07.2023):

Main.java

```
public class Drawing{
       public List<Shape> getShapes() {}
       public String getName() {}
       public int getDiagonal() {}
}

public class Shape{
       public Color getColor() {}
}

//Completare l'implementazione del seguente metodo printNameOfBiggest //che
prende in ingreosso una lista di dipinti (Drawing) e stampa il nome //del
dipinto avente la diagonale maggiore tra quelli nella lista. Se la //lista è
vuota, il metodo non stampa nulla. Se esistono più dipinti con //la stessa
diagonale, il metodo stampa comunque un solo nome (non viene //specificato
quale).

public static void<String> getAllProductsNames(List<Shop>shops){
      drawings.stream()
       .max((d1, d2) -> Integer.compare(d1.getDiagonal(), d2.getDiagonal()))
       .flatMap(s -> s.getProducts().stream())
       .map(Drawing::getName)
       .distinct()
       .ifPresent(System.out::println);
}
```



- Drawings è una lista che con .stream() viene trasformata in uno stream
- .max è un comparator che è un metodo che prende in ingresso due drawing che devono essere confrontati
- Posso usare il metodo della diagonale, ma se non lo conosco posso dire:
  if d1.getdiagonal > d2.getdiagonal return 1;
  else if d2.getdiagonal > d1.getdiagonal
  return -1;
  else if if d1.getdiagonal = 2.getdiagonal
  return 0;
- Dopo .max dato uno stream<T> ottengo un Optional<T> perché se la lista o lo stream erano vuoti non mi ritornerà niente
- Anche l'Optional ha una map che è un metodo che prende in ingresso una funzione da T a U che ritorna un Optional di U. Qui parto da un Optional di Drawing e ottengo uno string che viene scritta in questo modo poiché è una funzione che chiama un solo metodo
- ifPresent è un metodo che prende in ingresso un consumer e se l'Optional è presente il metodo chiama una funzione su quell'elemento se non è presente non fa niente.

Lezione 12: Java concorrente (23/10/2023 Francesco Caracciolo, Matteo Colombi)

Programmazione Concorrente

L'idea della programmazione concorrente è quella di **avere più flussi in esecuzione** che vanno avanti (possibilmente) **in parallelo** (se si dispone di più core per l'esecuzione). La programmazione concorrente svolge un ruolo chiave anche per quanto riguarda l'esecuzione su un solo core, infatti permette di mettere in attesa determinati flussi (come ad esempio la ricezione dell'input da parte dell'utente) senza dover bloccare tutti gli altri.

Il modello di thread di Java è **preemptive,** cioè ogni thread riceve un certo quanto di tempo, assegnatogli dallo scheduler del sistema operativo, alla scadenza del quale viene sospeso. Non è quindi il programma a decidere quale thread gira.

Classe Thread

Per implementare un thread e lanciarlo è necessario utilizzare la classe Thread (java d_ocs).

Ci sono due modi per implementare i Thread in Java. Il primo è creando una sottoclasse che estende la classe Thread:

MyhThread.java

```
package it.polimi.softeng.concurrency;

public class MyThread extends Thread {
    private final int num;

    public MyThread(int num) {
        this.num = num;
    }
    @Override
    public void run() {
        System.out.println("Numero: " + num);
    }
}
```

Il metodo run, in override, **contiene il codice che verrà eseguito nel thread** una volta lanciato.

Nota: run non accetta argomenti, è una funzione da void a void. Per "passargli degli argomenti" è necessario definirli come attributi e impostarli con il costruttore, come nel caso di *num*.

Per lanciarlo, dobbiamo prima instanziare un oggetto della classe MyThread e poi chiamare il metodo *start*:

Main.java

```
package it.polimi.softeng.concurrency;

public class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread(10); // Instanzio un oggetto della classe
MyThread
        t.start(); // Avvio il thread (viene eseguito il metodo run)
```

```
        System.out.println("Ciao");
    }
}
```

Questo codice crea un'instanza MyThread con attributo num a 10, poi chiama il metodo start per avviare il thread.

Questo codice ha tre possibili output, il codice è quindi **non deterministico**:

Output 1	Output 2	Output 3
Numero 10 Ciao	Ciao Numero 10	Ciao

- Output 1: Viene eseguito prima il codice nel thread, poi la println nel Main, e in seguito il programma finisce la propria esecuzione
- Output 2: Viene eseguito prima la println nel main, poi viene eseguito il codice nel Thread, e in seguito il programma finisce la propria esecuzione
- Output 3: Viene eseguito il main e in seguito il programma termina la propria esecuzione, uccidendo il thread prima ancora che potesse eseguire il proprio codice

Estendere la classe Thread non è però il modo più comune per implementare i thread in Java, questo soprattutto perché Java non ha ereditarietà multipla, ma solo singola. Dato che MyThread estende già Thread, non è possibile estendere nessun'altra classe.

Interfaccia Runnable

Di norma, per implementare i thread, si vuole creare una classe che **implementa un'interfaccia Runnable** (Java docs).

Per creare una classe che ha lo stesso comportamento di *MyThread* creiamo il seguente codice:

MyRunnable.java

```
package it.polimi.softeng.concurrency;

public class MyRunnable implements Runnable {
    private final int num;

    public MyRunnable(int num) {
        this.num = num;
    }
    @Override
    public void run() {
        System.out.println("Numero " + num);
    }
}
```

Da notare come, tralasciando la prima riga, il codice sia identico a *MyThread*.

Main.java

```
package it.polimi.softeng.concurrency;

public class Main {
    public static void main(String[] args) {
```

```
        Runnable r = new MyRunnable(10);
        Thread t = new Thread(r);
        t.start();
        System.out.println("Ciao");
    }
}
```

Nel main:

1. Instanzio un oggetto r di classe MyRunnable
2. Instanzio un oggetto di classe Thread, con argomento il Runnable *r*
3. Avvio il thread come in precedenza

Il codice è del tutto equivalente al codice precedentemente implementato utilizzando la classe *MyThread*.

Volendo è anche possibile avviare il thread in una sola riga:

Main.java
```
package it.polimi.softeng.concurrency;

public class Main {
    public static void main(String[] args) {
        new Thread(new MyRunnable(10)).start();
        System.out.println("Ciao");
    }
}
```

Oppure, se volessi avviare 500 thread:

Main.java
```
package it.polimi.softeng.concurrency;

public class Main {
    public static void main(String[] args) {
        for (int i=0; i < 500; i++) {
            Runnable r = new MyRunnable(i);
            Thread t = new Thread(r);
            t.start();
        }
        System.out.println("Ciao");
    }
}
```

Non è garantito che i threads verranno eseguiti in ordine oppure che verranno eseguiti prima della print di “Ciao”.

Runnable con programmazione funzionale

Runnable è un’interfaccia funzionale, è quindi possibile farlo con la programmazione funzionale senza dover creare una classe apposita.

Main.java
```
package it.polimi.softeng.concurrency;
```

```java
public class Main {
    public static void main(String[] args) {
        for (int i = 0; i < 500; i++){
            final int a = i; // Le closures accettano solo variabili final o
effectively final
            Thread t = new Thread(() -> System.out.println(a));
            t.start();
        }
    }
}
```

Nota: La funzione lambda passata al costruttore di Thread è compatibile con Runnable, per questa motivazione, Java tra i costruttori in overload di Thread, sceglie *Thread(Runnable target)*.

Lo stesso codice si può scrivere in forma più ridotta.

Main.java

```java
package it.polimi.softeng.concurrency;

public class Main {
    public static void main(String[] args) {
        for (int i = 0; i < 500; i++){
            final int a = i;
            new Thread(() -> System.out.println(a)).start();
        }
    }
}
```

Join: aspettare i thread

Per aspettare la terminazione di un thread, si può utilizzare il metodo *join* (docs Java) di *Thread*.

Join è un metodo che non prende argomenti e non ritorna alcun valore (void), ma può lanciare l'eccezione *InterruptedException*.

Scriviamo un codice che crea e avvia 500 thread, e poi aspetta la loro fine prima di terminare il programma.

Main.java

```java
package it.polimi.softeng.concurrency;

public class Main {
    // InterruptedException lanciata da join
    public static void main(String[] args) throws InterruptedException {
        List<Thread> = l = new ArrayList<>();

        for (int i = 0; i < 500; i++){
            final int a = i;
            Thread t = new Thread(() -> System.out.println(a));
            t.start();
            l.add(t); // Salvo tutti i riferimenti a tutti i thread
        }
        for (Thread t : l) {
            t.join(); // Aspetto che tutti i thread terminino
        }
```

```
        System.out.println("Ciao");
    }
}
```

Questo codice garantisce che “Ciao” sia stampato per ultimo, tuttavia non garantisce che i numeri da 1 a 500 vengano stampati in ordine.

Lo stesso codice lo possiamo esprimere anche in programmazione funzionale:

Main.java

```
package it.polimi.softeng.concurrency;

public class Main {
    public static void main(String[] args) {
        List<Thread> = l = new ArrayList<>();

        for (int i = 0; i < 500; i++){
            final int a = i; //Le closures accettano solo final o
effectively final
            Thread t = new Thread(() -> System.out.println(a));
            t.start();
            l.add(t); // Salvo tutti i riferimenti a tutti i thread
        }
        // Oppure in programmazione funzionale
        l.stream().forEach( t -> {
            try {
                t.join();
            } catch (InterruptedExcetpion e) {
                throw new RuntimeException();
            }
        });
        //L'eccezione va gestita per forza peché è gestita dalla funzione
che ho passato
        // Aspetto tutti i thread prima di printare ciao
        System.out.println("Ciao");
    }
}
```

Conflitti tra thread

Consideriamo la classe seguente:

BankAccount.java

```
public class BankAccount {
    private int amount;

    public void deposit(int x) {
        amount += x;
    }

    public void withdraw(int x) {
        amount -= x;
    }
```

```
    public int getAmount() {
        return amount;
    }
}
```

Se proviamo a creare due thread che chiamano un metodo di `BankAccount` contemporaneamente si potrebbero creare dei conflitti. Ad esempio, se T1 e T2 sono due thread che stanno eseguendo il metodo `deposit(10)` su un `BankAccount` che inzialmente ha `amount=0`, potremmo avere diversi risultati:

- Alla fine dell'esecuzione di entrambi i thread, ci aspettiamo che `amount` sia pari a `20`. In alcuni casi, questo è il risultato che otteniamo.
- Se l'esecuzione dei thread si intreccia opportunamente, possiamo ottenere come risultato anche `10`: l'istruzione `amount += 10`, infatti, non è un'istruzione atomica, e si può scomporre in tre passaggi:
    1. lettura di `amount` corrente;
    2. operazione aritmetica (+10);
    3. scrittura del nuovo valore di `amount`.

    Osserviamo che, in Java, le uniche operazioni atomiche sono operazioni di lettura e scrittura di variabili a 32 bit. Segue quindi che la maggioranza delle operazioni non sono atomiche, anche se a prima vista potrebbero sembrarle.

    Ritornando all'esempio, se i thread si alternano come segue (altre opzioni sono possibili), otteniamo un risultato scorretto:
    1. T1 legge `amount` (legge `0`);
    2. T1 calcola `amount+10=10`;
    3. T2 legge `amount` (legge `0`);
    4. T2 calcola `amount+10=10`;
    5. T1 scrive `amount` (scrive `10`);
    6. T2 scrive `amount` (scrive `10`).

Sincronizzazione

Il problema che si è venuto a creare è tipico della programmazione concorrente, ed è generato dal tentativo di accedere ad una risorsa condivisa (in questo caso la variabile `amount`) con più thread contemporaneamente. Per risolverlo, diversi linguaggi adottano metodi differenti. L'idea di base è quella di rendere artificialmente "atomici" alcuni blocchi di codice, nel senso di impedire che l'esecuzione di porzioni di codice che operano sulla stessa risorsa sia intrecciata tra più thread. In Java, questo si fa di solito con la parola chiave **`synchronized`** (java doc 1, java doc 2). Ad esempio, possiamo aggiungere `synchronized` alla signature dei metodi di `BankAccount` che accedono alla risorsa condivisa `amount`.

BankAccount.java
```
public class BankAccount {
    private int amount;
```

```
    public synchronized void deposit(int x) {
    // o anche: public void synchronized deposit(int x)
        amount += x;
    }

    public synchronized void withdraw(int x) {
        amount -= x;
    }

    public synchronized int getAmount() {
        return amount;
    }
}
```

Dichiarare i metodi come `synchronized` impedisce che due thread possano eseguire altri blocchi di codice sincronizzato contemporaneamente (quindi non necessariamente la stessa porzione di codice!), eliminando la possibilità che si generino situazioni come quella delineata sopra. Osserviamo che la sincronizzazione avviene solo su uno specifico oggetto. Se creiamo ad esempio due istanze di `BankAccount`, diciamo `b1` e `b2`, e due thread eseguono rispettivamente `b1.deposit(10)` e `b2.deposit(10)`, le loro esecuzioni possono procedere contemporaneamente senza problemi, in quanto i metodi sono eseguiti su due oggetti differenti.

Dichiarare del codice come `synchronized` è utile per evitare conflitti nella gestione di risorse condivise; tuttavia, a volte è preferibile non sincronizzare interi metodi, in quanto sincronizzare significa limitare il parallelismo, e quindi diminuire i benefici della programmazione concorrente. Possiamo quindi (e spesso è preferibile, specie se un metodo deve eseguire calcoli onerosi su variabili locali, e quindi non condivise) dichiarare singoli blocchi critici di codice come `synchronized`, e lasciare invece che altre porzioni di codice proseguano in parallelo la propria esecuzione. Nell'esempio sopra, possiamo ad esempio evitare conflitti nel metodo `withdraw` anche come segue:

BankAccount.java

```
...
public void withdraw(int x) {
    // Qui ci sono operazioni non sincronizzate
    synchronized (this) {
        amount -= x;
    }
    // Qui ci sono altre operazioni non sincronizzate
}
...
```

Il `this` è il riferimento all'oggetto con cui vogliamo sincronizzarci, cioè l'oggetto di cui vogliamo usare le risorse condivise in mutua esclusione. In questo caso, usiamo le risorse dell'oggetto stesso su cui lavoriamo per evitare conflitti con altri thread che eseguono metodi su questa istanza di `BankAccount`.

Deadlock

Analizziamo ora una problematica che si può presentare quando si lavora con blocchi di codice sincronizzati. Supponiamo di voler aggiungere un metodo `transfer` alla classe `BankAccount`. Un primo tentativo di scrittura di tale metodo può essere il seguente:

BankAccount.java

```
...
public void transfer(BankAccount other, int x) {
    synchronized (this) {
        synchronized (other) {
            other.amount -= x;
            amount += x;
        }
    }
}
...
```

Supponiamo ora di avere due istanze di `BankAccount`, diciamo b1 e b2, che sono gestite da due thread T1 e T2. Consideriamo ora il caso che segue:

T1: `b1.transfer(b2,10)`

`synchronized(this)` → blocca b1

Ora interviene lo scheduler, che fa proseguire l'altro thread:

T2: `b2.transfer(b1,10)`

`synchronized(this)` → blocca b2

`synchronized(other)` → T2 si ferma in quanto T1 ha già bloccato la risorsa b1

Poiché T2 è bloccato, T1 torna in esecuzione:

T1: `synchronized(other)` → T1 si ferma in quanto T2 ha già bloccato la risorsa b2

Si è creata una situazione da cui non possiamo più uscire, in quanto tutti e due i thread sono in attesa di una risorsa che è già bloccata dall'altro. Questa situazione è detta **deadlock**.

Risolvere i casi di deadlock non è banale. Nel caso specifico, senza introdurre metodi di gestione più avanzati, non c'è un modo per risolvere il deadlock senza cambiare la semantica del metodo. Accettando che le operazioni sui due oggetti non siano più necessariamente eseguite insieme, possiamo eliminare il deadlock modificando transfer come segue:

BankAccount.java

```
...
public void transferNoDeadlock(BankAccount other, int x) {
    synchronized (this) {
        amount += x;
    }
    // Qui in mezzo possono avvenire altre cose!!!
    synchronized (other) {
        other.amount -= x;
    }
}
...
```

Tra le modifiche di `amount` e di `other.amount` possono intervenire altri thread. Questo può essere un problema, ad esempio, se volessimo effettuare dei controlli prima di eseguire le istruzioni (ad esempio, che `other.amount` non sia negativo).

Vedremo più avanti metodi avanzati per evitare i deadlock.

I metodi `wait` e `notifyAll`

Supponiamo ora di voler aggiungere un controllo su `withdraw` in modo che, nel caso in cui nel conto non ci siano soldi a sufficienza, rimanga in sospeso finché ciò non avviene. Un primo tentativo può essere il seguente:

BankAccount.java

```
...
public synchronized void withdrawPositive(int x) {
    while (amount < x);
    amount -= x;
}
...
```

Il codice sopra ha due problemi molto gravi:

1. Contiene uno spinlock, cioè un ciclo `while` con corpo vuoto che controlla continuamente una condizione. Questo spreca risorse ed è, tranne in rari casi, una pratica da evitare.
2. Se la condizione `amount < x` è verificata inizialmente, e quindi si entra nel ciclo, non si uscirà mai dal ciclo stesso: infatti, `withdrawPositive` è dichiarato come `synchronized`, dunque nessun altro thread può aggiungere ad `amount` in quanto anche il metodo `deposit` è sincronizzato.

Modifichiamo allora il codice come segue:

BankAccount.java

```
...
public synchronized void withdrawPositive(int x) throws InterruptedException {
    while (amount < x)
        wait();
    amount -= x;
}
...
```

Il metodo `wait` (java doc) si può invocare solo in blocchi di codice sincronizzati, e mette in attesa il thread corrente fino a quando non viene risvegliato, rilasciando inoltre la risorsa condivisa. Un thread può essere risvegliato o per l'invocazione da parte di un altro thread del metodo `notifyAll` (vedi sotto), oppure periodicamente (e apparentemente casualmente), come indicato nella documentazione di Java:

> "A thread can also wake up without being notified, interrupted, or timing out, a so-called *spurious wakeup*." (nella pagina della documentazione indicata sopra)

È opportuno usare un `while` e non un `if` per due ragioni: sia perché il thread si può risvegliare in qualunque momento, sia perché, anche se viene risvegliare perché è avvenuto un deposito, non è detto che il saldo del conto corrente sia ora sufficiente per effettuare il prelievo.

Osserviamo che il metodo `wait` può lanciare una `InterruptedException` (java doc). Citando di nuovo la documentazione di Java, "thrown when a thread is waiting, sleeping, or otherwise occupied, and the thread is interrupted, either before or during the activity."

Aver invocato `wait` sistema entrambi i problemi che aveva la prima versione di `withdrawPositive`: non c'è più lo spinlock e un altro thread può intervenire modificando il valore di `amount`.

Per fare in modo che i thread in attesa nel metodo withdrawPositive vengano risvegliati quando potrebbe essere disponibile abbastanza denaro, modifichiamo il metodo `deposit` come segue:

BankAccount.java

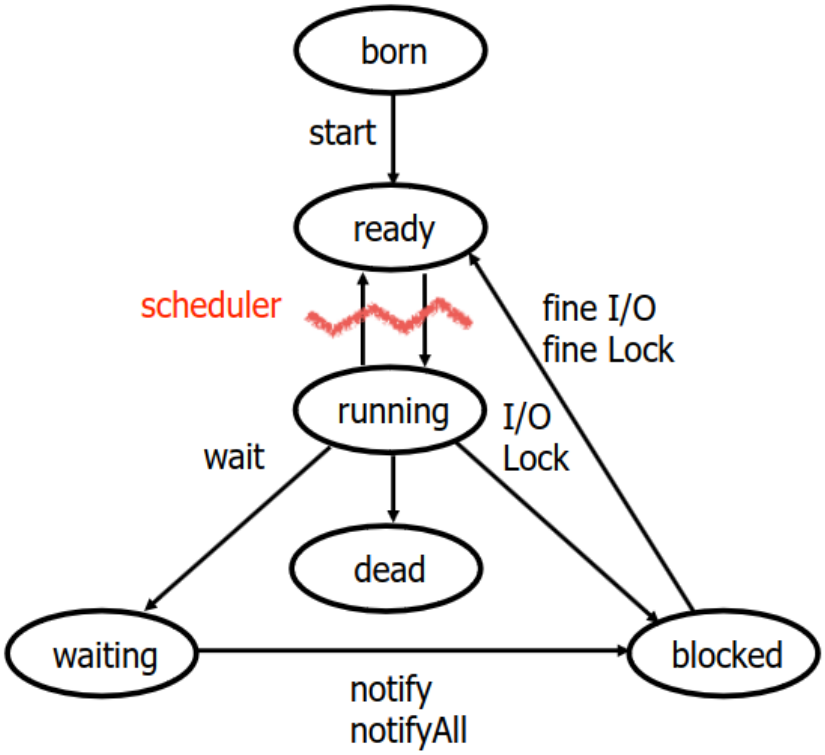
```
...
public synchronized void deposit(int x) {
    amount += x;
    notifyAll();
}
...
```

Come accennato sopra, il metodo `notifyAll` (java doc) risveglia tutti i thread in attesa sull'oggetto che lo ha invocato. Al pari di `wait`, `notifyAll` si può invocare solo in blocchi di codice `synchronized`.

Lezione 13: Java concorrente (24/10/2023 Matteo Benedic, Roberto Alessandro Bertolini)

Ciclo di vita di un thread

Ciclo di vita di un thread



Questo grafico presenta il ciclo di vita di un thread, cioè tutti gli stati in cui si può trovare e tutte le primitive che permettono il cambio di stato.

Lo stato Born indica la creazione del thread. Per creare un thread posso creare una classe che estende thread e crearne un'istanza, o creare un thread generico a cui passo nel costruttore un Runnable o, in modo equivalente, passare una funzione lambda.

Tramite la funzione **start()** possiamo passare allo stato Ready, cioè lo stato nel cui si trovano tutti i thread che possono essere scelti dallo scheduler per esser lanciati.

Dopo che lo scheduler lancia il thread esso entra nello stato Running, ovvero il thread viene eseguito.
Da cui il cambiamento di stato può avvenire in diversi modi:

- esso può terminare la sua funzione (DEAD) *[che non può esser terminato in alcun modo dall'esterno];*
- può far scadere il quanto di tempo e a quel punto viene riportato in stato di READY;
- può finire nello stato BLOCKED tramite I/O (Attesa di un evento esterno) o se deve ottenere un Lock che è già preso da un altro thread;
- infine, può finire in stato di attesa (WAITING) se si attiva la funzione **wait().**

Dallo stato di Waiting il thread può solo passare allo stato BLOCKED, se viene risvegliato da una **notify()** o **notifyAll()** attivata da un altro thread.
La funzione notifyAll() risveglia tutti i thread che sono in stato di waiting rispetto a quel lock intrinseco, mentre notify() risveglia in modo non deterministico uno solo di questi thread.

Da Blocked, infine, si può tornare allo stato di Ready quando il Lock o l'evento esterno atteso sono liberi.

Esempio Produttore/Consumatore

In questo esempio ci occupiamo della gestione di una coda di stringhe:

- I produttori saranno i thread che scrivono stringhe nella nostra struttura dati
- I consumatori invece stampano in stdout ciò che pescano dalla coda

Costruiamo inizialmente la risorsa condivisa, la nostra coda FIFO *(First In First Out),* in modo tale che sia Thread Safe. Rispetto al C, dove ci concentriamo direttamente sui Thread, Java mantiene una visione Object-Oriented, e si concentra sulle risorse (in questo caso la coda) e su come questi oggetti condivisi debbano essere costruiti in modo tale che i Thread possano lavorarci in sicurezza.

MyQueue.java

```
import java.util.LinkedList;
import java.util.Queue;

public class myQueue {
  private final Queue<String> queue;

  public MyQueue() {
      queue = new LinkedList<>();
  }

  public void add(String s) {
      queue.add(s);
  }

  public String pop() {
      return queue.poll();
  }
}
```

Possiamo utilizzare come attributo un oggetto di classe Queue<T> offerta direttamente dalla documentazione di Java.
Dobbiamo inoltre inizializzarla a final, e possiamo farlo anche se dobbiamo aggiungere e togliere stringhe dalla coda, perché ciò che inizializziamo a final è il riferimento, che punterà sempre alla stessa struttura dati.

Oltre all'attributo, e il costruttore, abbiamo creato il metodo *add* per il produttore, per aggiungere stringhe in testa, e il metodo *pop* per il consumatore, per estrarle in coda.

Ora dobbiamo sincronizzare la nostra risorsa. Per permettere l'accesso ai metodi a più Thread lasciando tutto così, permetteremmo modifiche concorrenti e potremmo raggiungere uno stato inconsistente. Se succedesse, Java lancerebbe una ConcurredModificationException.

Una possibilità è inserire synchronized nell'intestazione dei metodi:

MyQueue.java

```
public synchronized void add(String s) {
      queue.add(s);
}

public synchronized String pop() {
```

```
        return queue.poll();
}
```

Così facendo non permettiamo a due Thread contemporaneamente di accedere al metodo ***add***, a due Thread di accedere contemporaneamente a ***pop***, e inoltre ancora più importante non permettiamo l'accesso concorrente ai metodi add e pop trasversalmente.
È importante che non avvenga ciò perché la coda al suo interno è un oggetto complesso, e permettere un accesso concorrente può portare ad uno stato non consistente.

In alternativa possiamo anche scrivere in quest'altro modo, che è semanticamente equivalente:

MyQueue.java

```
public void add(String s) {
        synchronized (this) {
                queue.add(s);
        }
}

public synchronized void String pop() {
        return queue.poll();
}
```

Anche in questo caso il thread che accede al metodo add acquisisce il lock intrinseco all'oggetto su cui aggiunge la stringa, lo stesso lock che otterrebbe un Thread che accede al metodo pop.

Invece, questa scrittura

MyQueue.java

```
public void add(String s) {
        synchronized (queue) {
                queue.add(s);
        }
}

public synchronized String pop() {
        return queue.poll();
}
```

non è semanticamente equivalente. Infatti, in questo caso, un Thread che accede al metodo add acquisisce un Lock diverso da quello intrinseco sull'oggetto, ma ottiene il lock sull'attributo queue. In questo modo, fino a due Thread possono accedere contemporaneamente al nostro oggetto: uno che dal metodo pop ottiene l'unico Lock intrinseco sull'oggetto di classe MyQueue, e uno tramite il metodo add che ottiene l'unico Lock sull'attributo queue.

Tra le prime due alternative di scrittura non c'è alcuna differenza, e in questo caso anche se entrambi i Lock fossero su queue sarebbe uguale.
Se si vuole, da un punto di vista paradigmatico si preferisce inserire sempre il synchronized nell'intestazione del metodo se dobbiamo utilizzare un singolo lock su più metodi, se invece si hanno più lock si utilizza il blocco interno al metodo così da nominare l'oggetto su cui vogliamo ottenere il lock.

Ora vogliamo aggiungere una condizione di blocco al metodo Pop se la coda è vuota.

MyQueue.java

```
public synchronized void add(String s) {
        queue.add(s);
        this.notifyAll();
}
```

```
public synchronized String pop() throws InterruptedException {
        while (queue.isEmpty()) {
                this.wait();
        }
        return queue.poll();
}
```

In questo modo, se un Thread entra nel metodo pop e la coda è vuota, si attiva wait() che rilascia il lock su this, e mette in attesa il Thread. Quest'ultimo sarà risvegliato solo quando un altro Thread, tramite il metodo add, aggiungerà una nuova stringa e attiverà la notifyAll().

In quali casi il thread viene risvegliato anche se è in wait() e quando si sveglia ha la coda vuota?
Quando ci sono due thread in waiting e un terzo aggiunge un oggetto. In questo caso, i due Thread si svegliano contemporaneamente, e uno dei due otterrà il lock intrinseco entrando nel metodo pop. Quando il lock sarà rilasciato, l'ultimo thread entrerà nella pop, ma ritroverà nuovamente la coda vuota, ed entrerà nello stato di wait().
Per questo motivo è importante utilizzare uno while per la nostra condizione, e non è possibile sostituirlo ad esempio con un if.

Ora creiamo i nostri Thread produttore e consumatore.

Main.java

```
MyQueue queue = new MyQueue();

Thread producer = new Thread(() -> {
	for (int i = 0; i < 100; i++) {
		queue.add("Ciao");
		try {
			// questa sleep si prende un secondo
			Thread.sleep(1000);
		} catch (InterruptedException e) {
			throw new RuntimeException(e);
		}
	}
});

Thread consumer = new Thread(() -> {
	for (int i = 0; i < 100; i++) {
		try {
			System.out.println(queue.pop());
		} catch (InterruptedException e) {
			throw new RuntimeException(e);
		}
	}
});

producer.start();
consumer.start();
producer.join();
consumer.join();
```

Da quando parte la producer.start(). Il produttore inizia a inserire in coda un “Ciao” al secondo, data la funzione sleep() che mette in attesa il thread ogni secondo, per 100 volte.
Il consumatore da quando parte, invece, tenta ogni volta di stampare l’ultimo elemento (il primo inserito) della coda. Se quest’ultimo “supera” il producer, e tenta di stampare qualcosa quando la coda è vuota, questo Thread beccherà la wait, e aspetterà l’inserimento di un nuovo elemento in coda da parte del producer.

Infine, date le producer.join() e consumer.join(), il Thread principale aspetterà che sia il producer sia il consumer finiscano la loro funzione prima di continuare con il suo codice. L’output che ci aspettiamo sarà di esattamente 100 “Ciao”.

Questo esempio di sistema produttore-consumatore rappresenta uno schema frequentemente utilizzato per lo scambio di dati tra due Thread che eseguono operazioni diverse all’interno di un programma: un produttore può essere un servizio che continuamente prova a scaricare da un server un dato aggiornato, e lo pubblica nella coda, mentre il consumatore non ha idea dell’origine del dato, ma appena risvegliato prende il dato e lo mostra a video. In questo modo si può compartimentare e separare l’applicazione in più moduli che svolgono funzioni diverse, come da politica di programmazione Object-Oriented.

Correzione Tema d’Esame del 16 Gennaio 2020

Account.java

```java
public class Account {
    private static List<Double> log = new ArrayList<Double>();
    private double balance;

    public Account() {
        balance = 0.0;
    }

    public synchronized void deposit(double val) {
        balance = balance+val;
        log.add(val) ;
    }

    public synchronized void withdraw(double val) {
        balance = balance-val;
        log.add(-val);
    }
}
```

a) La sincronizzazione è corretta? Motivare la risposta

La sincronizzazione per l'accesso all'attributo balance è corretta, in quanto entrambe le funzioni deposit e withdraw sono sincronizzate prendendo il lock intrinseco dell'istanza di Account.

Occorre però prestare attenzione all'attributo log, che presenta il modificatore static nella sua dichiarazione. Come visto durante la Lezione 7 del corso, gli attributi marcati come static non appartengono ad una singola istanza della classe ma alla classe stessa.

Si consideri il seguente frammento di codice:

Example.java

```java
public class Example {
    public static void main(String[] args) {
        Account account1 = new Account();
        Account account2 = new Account();

        new Thread(() -> {
            while (true) { account1.deposit(1.0f); }
        }).start();

        new Thread(() -> {
            while (true) { account2.deposit(2.0f); }
        }).start();
    }
}
```

I due thread stanno operando su due istanze diverse della classe `Account`, non c'è quindi possibilità di eccezioni ConcurrentModificationException durante la modifica dell'attributo balance.

Essendo però l'attributo `log` con modificatore `static`, e quindi condiviso tra le due istanze, i due thread stanno contemporaneamente modificando la stessa lista. Questo potrebbe naturalmente portare a ConcurrentModificationExceptions o undefined behavior.

È quindi necessario che nei due membri `withdraw` e `deposit` si prenda un lock anche sull'attributo `log` quando viene modificato, ad esempio in questo modo:

Account.java

```
public class Account {
	private static List<Double> log = new ArrayList<Double>();
	private double balance;

	public Account() {
		balance = 0.0;
	}

	public synchronized void deposit(double val) {
		balance = balance+val ;
		synchronized (log) { log.add(val); }
	}

	public synchronized void withdraw(double val) {
		 balance = balance-val ;
		 synchronized (log) { log.add(-val); }
	}
}
```

Ora la classe è correttamente sincronizzata.

Si noti che non è un problema il prendere due lock diversi dentro la stessa funzione in questo caso. Il lock su `log` è sempre il secondo lock che viene preso dai metodi, quindi non c'è la possibilità di giungere in una situazione di deadlock in cui un thread aspetta di poter lockare `log` mentre un altro thread sta aspettando di poter lockare l'istanza della classe.

b) Modificare la classe affinché il metodo withdraw venga sospeso in mancanza di fondi per il prelievo

Per sospendere l'operazione di `withdraw`, è sufficiente chiamare la funzione `wait` dentro un ciclo while finché il `balance` non supera la quantità di fondi da prelevare.

Così facendo, si libera il lock preso sull'istanza e si mette il thread in stato di attesa, in modo che un altro thread possa incrementare la quantità di fondi. È importante accertarsi che:

- La funzione .`wait`() sia chiamata sullo stesso oggetto su cui avevo preso il lock
- La chiamata a .`wait`() sia dentro un while loop che verifica la condizione imposta. Non è garantito che il risveglio del thread avvenga quando la condizione richiesta viene raggiunta; il thread può essere risvegliato dallo scheduler in qualsiasi momento
- La funzione .`wait`() può triggerare una InterruptedException: è quindi necessario gestire questo caso, catchando l'eccezione o aggiungendo un `throws` nella dichiarazione del metodo

Di seguito una valida implementazione della richiesta:

Account.java

```
public synchronized void withdraw(double val) {
        while (balance < val) {
                try {
                        wait();
                } catch (InterruptedException e) {
                        throw new RuntimeException(e);
                }
        }
        balance = balance-val ;
        synchronized (log) { log.add(-val); }
}
```

c) Si aggiunga un metodo void printLog() che stampa l'intero log in maniera asincrona rispetto al chiamante

Account.java

```
public void printLog() {
        new Thread(() -> {
                log.stream().forEach(System.out::println);
        }).start();
}
```

Questa prima implementazione, benché tecnicamente risponda alla richiesta, presenta un grande problema: non prendendo nessun lock sull'oggetto log, potrebbe portare a una ConcurrentModificationException nel caso in cui un altro thread modificasse il contenuto della lista aggiungendo o rimuovendo oggetti.

Account.java

```
public void printLog() {
    synchronized (log) {
            new Thread(() -> {
                    log.stream().forEach(
                            System.out::println
                    );
            }).start();
    }
}
```

Account.java

```
public void printLog() {
        new Thread(() -> {
                synchronized (log) {
                        log.stream().forEach(
                                System.out::println
                        );
                }
        }).start();
}
```

Entrambe le precedenti implementazioni prendono un lock sull'attributo `log`, ma solo la seconda lo fa correttamente: nel primo caso il lock viene preso nello scope della funzione `printLog()`, mentre l'operazione di lettura sull'oggetto viene fatta nello scope del Thread creato da `printLog()`; nel secondo esempio invece la funzione `printLog()` crea un nuovo Thread, nel cui scope viene preso il lock su `log` e dentro il lock viene effettuata l'operazione di lettura.

La soluzione di sinistra ha, a tutti gli effetti, lo stesso behavior della prima implementazione che non acquisiva nessun lock, perché variando lo scope tutto ciò che viene eseguito dentro il thread non viene sincronizzato.

C'è però un ulteriore dettaglio da considerare: tutte le operazioni di lettura e scrittura da terminale richiedono l'interazione del programma con il sistema operativo, che è un'operazione molto dispendiosa in termini di tempo.

Ciò significa che, nella seconda soluzione presentata prima, anche se il programma prende correttamente il lock su `log`, lo potrebbe mantenere per un'elevata quantità di tempo, interrompendo il lavoro degli altri thread e rallentando l'esecuzione del programma.

Si consideri la seguente implementazione:

Account.java
```
public void printLog() {
        List<Double> copy;
        synchronized (log) {
                copy = new ArrayList<>(log);
        }
        new Thread(() -> {
                copy.stream().forEach(System.out::println);
         }).start();
}
```

In questo caso il lock viene preso solo per effettuare un'operazione di copia in memoria, molto più veloce della stampa su stdout. Il thread creato poi è libero di printare il contenuto di `copy` senza prendere nessun lock. Con questa soluzione il programma può raggiungere un miglior parallelismo,

Si noti che `copy` in questo caso è effectively final, e per questo motivo si può usare dentro il thread.

Correzione Tema d'Esame del 3 Febbraio 2020

MultiLock.java
```
public interface MultiLock {
        // Sospende il chiamante se ci sono state più di 10 invocazioni del medesimo metodo (dall'ultimo reset)
        public void acquire();

        // Re-inizializza l'oggetto, permettendo altre 10 chiamate di acquire (incluse quelle attualmente sospese)
        public void reset();
}
```

a) Si fornisca un'implementazione dell'interfaccia MultiLock

Per implementare la classe secondo le specifiche date, avrò bisogno di un atttributo che tiene conto del numero di invocazioni del metodo che ci sono state. Avrò inoltre bisogno di sincronizzare le invocazioni delle funzioni per evitare problemi di concorrenza nell'caso di uso in ambito multithreaded.

Una possibile implementazione dell'interfaccia è la seguente:

MultiLockImpl.java

```
class MultiLockImpl implements MultiLock {
	private int numAcquires;

	@Override
	public synchronized void acquire() {
		while (numAcquires >= 10) {
			try {
				wait();
			} catch (InterruptedException e) {
				throw new RuntimeException(e);
			}
		}
		numAcquires++;
	}

	@Override
	public synchronized void reset() {
		numAcquires = 0;
		notifyAll();
	}
}
```

In questa soluzione, l'attributo `numAcquires` memorizza il numero di invocazioni effettuate. I due membri hanno il necessario decoratore `@Override` in quanto implementazione di un'interfaccia, e sono inoltre sincronizzati con la keyword synchronized. È interessante notare che `synchronized` non compariva nella dichiarazione dell'interfaccia ma può essere aggiunto nella sua implementazione: questo perché la sincronizzazione è un dettaglio puramente implementativo, semanticamente equivalente a racchiudere il body della funzione dentro un `synchronized(this)`.

b) Si consideri il seguente codice. Quali valori sono ammissibili per le variabili numAcquire e numReset affinché tutti i thread prima o poi proseguano senza che ne rimanga uno in sospeso?

Test.java

```
class Test {
    public static void test() {
        final MultiLock lock = null;
        int numAcquire = 0;
        int numReset = 0;
        for ( int i =0; i<numAcquire ; i ++) {
            new Thread(() -> lock.acquire()).start();
        }
        for ( int i =0; i<numReset ; i ++) {
            new Thread(() -> lock.reset()).start();
        }
    }
}
```

Una volta acquisito il lock come imposto dalla keyword `synchronized`, la funzione `reset()` non contiene nessuna invocazione al metodo `wait()` che potrebbe sospendere l'esecuzione. Non c'è quindi upper bound al numero possibile di chiamate a `reset()`, che succederanno sempre senza sospendere il thread.

Impostando come numero di invocazioni `numAcquire` a un valore massimo di 10 abbiamo la garanzia che la funzione `acquire()` non entri mai nella condizione *`numAcquires >= 10`* che comporterebbe una wait del thread in attesa che il contatore sia resettato. Con un numero di invocazioni `numAcquire` più alto, come ad esempio 11, affinché tutti i thread terminino è richiesto che avvenga anche una chiamata a `reset()` per riportare il contatore a 0 almeno una volta. Per il comportamento dello scheduler di Java non è però possibile garantire che questo avvenga sempre: esso potrebbe decidere di schedulare prima ogni invocazione di `reset()` e poi quelle di `acquire()`.

Per questa ragione, l'upper bound di `numAcquire` è pari a 10. Di eseguito un esempio con `numAcquire` maggiore di 10 in cui un thread rimane sospeso.

Test.java	Possible ordine di scheduling che non termina

int numAcquire = 11;
int numReset = 2;
for (int i = 0; i<numAcquire; i++) {
new Thread(() ->
lock.acquire()
).start();
}
for (int i = 0; i<numReset; i++) {
new Thread(
() -> lock.reset()
).start();
}
	> reset(); <
	> reset(); <
	> acquire(); <
	> acquire(); <
	> acquire(); <
	> acquire(); <
	> acquire(); <
	> acquire(); <
	> acquire(); <
	> acquire(); <
	> acquire(); <
	> acquire(); <
	> acquire(); STUCK WAITING

Correzione Tema d'Esame del 18 Giugno 2021

GPSTrack.java

```
public class GPSTrack {
	private double latitude[];
	private double longitude[];
	private int numPoints;

	public GPSTrack () {
		latitude = new double[1000];
		longitude = new double [1000];
		numPoints = 0;
	}

	public void addPoint(double latitude , double longitude) {
		synchronized(this.latitude) {
			this . latitude [numPoints] = latitude ;
			this.longitude[numPoints] = longitude;
			numPoints ++;
		}
	}

	public double getLatitude (int point) {
		synchronized (this.latitude) {
			return this.latitude[point];
		}
	}

	public double getLongitude(int point) {
```

```
                synchronized(this.latitude) {
                        return this.longitude[point];
                }
        }
}
```

a) La classe è correttamente sincronizzata? Motivare la risposta

Affinché la sincronizzazione della classe sia corretta è necessario che:

- Tutti i metodi che accedono agli attributi della classe siano sincronizzati
- La sincronizzazione di tutti i metodi che modificano o accedono ad un determinato attributo avvenga sempre sullo stesso oggetto

In questo caso l'implementazione soddisfa questi requisiti, ed è quindi sincronizzata correttamente. Non vi è alcuna differenza nell'usare l'attributo `latitude` per la sincronizzazione, rispetto a `this` o ad un altro oggetto non primitivo.

b) Si aggiunga alla classe un metodo wait100 che sospende il chiamante fino a che non siano stati raggiunti almeno 100 punti alla traccia.

Una prima risposta potrebbe essere la seguente:

GPSTrack.java

```
public synchronized void wait100() throws InterruptedException {
        while (numPoints < 100) {
                wait();
        }
}
```

Il codice presentato però è sbagliato, perché gli altri metodi prendevano il lock su `this.latitude`, mentre qui il lock viene preso su `this` utilizzando la keyword `synchronized` nella definizione del metodo.

GPSTrack.java

```
public void wait100() throws InterruptedException {
        synchronized(latitude) {
                while (numPoints < 100) {
                        latitude.wait();
                }
        }
}
```

Questa soluzione invece utilizza correttamente il lock, prendendolo sullo stesso oggetto su cui viene preso negli altri metodi.

Attualmente però nessuna altra funzione chiama `notify()` o `notifyAll()` dopo aver modificato il contatore `numPoints`, quindi `wait100()` potrebbe rimanere in attesa indefinitamente anche se il contatore è stato incrementato a sufficienza.

È quindi necessario modificare anche la funzione `addPoint()` per notificare ai thread in attesa l'incremento del contatore `numPoints`, come ad esempio:

GPSTrack.java

```
public void addPoint(double latitude , double longitude) {
        synchronized(this.latitude) {
```

```
                this.latitude [numPoints] = latitude;
                this.longitude[numPoints] = longitude;
                numPoints ++;
                this.latitude.notifyAll();
        }
}
```

È importante che `notifyAll()` (o `notify()`) sia chiamata sullo stesso oggetto su cui si sta prendendo il lock: in questo caso è `this.latitude`.

Correzione Tema d'Esame del 28 Agosto 2023

SensorReadings.java

```
public class SensorReadings {
    private int id;
    private List<Double> data;
    private int daysInUse;

    public SensorReadings(int id) {
            this.id = id;
            data = new ArrayList<Double>();
            daysInUse = 0;
     }

     public void addReading(double d) {
            synchronized(data) {
                    data.add(d);
            }
     }

     public double getReading(int index) {
            synchronized(data) {
                    return data.get(index);
            }
     }

     public void changeAllReadings(List<Double> newData) {
            synchronized(data) {
                     data = newData;
            }
     }

     public synchronized void newDayStarted() {
            daysInUse++;
     }
}
```

a) La classe è correttamente sincronizzata? Motivare la risposta

La classe non è correttamente sincronizzata perché nel metodo `changeAllReadings()` si modifica il puntatore dell'attributo data sostituendolo con `newData`. Questo fa sì che ogni successiva chiamata ad una funzione sincronizzata prenda un lock su un oggetto diverso da quello originale. È necessario sincronizzare i metodi della classe su un oggetto che rimanga invariato in tutto il lifetime dell'istanza.

Una possibile soluzione è la seguente, che sfrutta un nuovo attributo `lock`, di tipo `Object`, utilizzato solo ed esclusivamente per sincronizzare i tre metodi `addReading`, `getReading` e `changeAllReadings`:

SensorReadings.java

```
public class SensorReadings {
	private int id;
	private List<Double> data;
	private int daysInUse;
	private Object lock;

	public SensorReadings(int id) {
		this.id = id;
		data = new ArrayList<Double>();
		daysInUse = 0;
		lock = new Object();
	}

	public void addReading(double d) {
		synchronized(lock) {
			data.add(d);
		}
	}

	public double getReading(int index) {
		synchronized(lock) {
			return data.get(index);
		}
	}

	public void changeAllReadings(List<Double> newData) {
		synchronized(lock) {
			data = newData;
		}
	}

	public synchronized void newDayStarted() {
		daysInUse++;
	}
}
```

Alternativamente, si potrebbero sincronizzare i tre metodi su `this`, ma ciò potrebbe cambiare il behavior dell'oggetto, in quanto `newDayStarted` prende già un lock su this e potrebbe operare in contemporanea agli altri tre metodi poiché non va ad accedere all'attributo data. La sincronizzazione su `this` è quindi solo parzialmente corretta.

b) Aggiungere un metodo waitFor30, che sospende il chiamante fino a che non ci sono 30 letture disponibili

Una possibile soluzione, che naturalmente tiene conto delle modifiche fatte al punto a), è la seguente:

SensorReadings.java

```
public void waitFor30() {
        synchronized (lock) {
                while (data.size() < 30) {
                        try {
                                lock.wait();
                        } catch (InterruptedException e) {
                                throw new RuntimeException(e);
                        }
                }
        }
}
```

Il metodo è sincronizzato su `lock`, come gli altri tre metodi che accedono alla stessa risorsa, e chiama `wait` finché la condizione non è soddisfatta.

È inoltre necessario aggiungere una chiamata a `notifyAll`() alla fine dei metodi `changeAllReadings`() e `addReading`().

c) Aggiungere un metodo readyToBeReplaced, che sospende il chiamante se ci sono meno di 10000 letture oppure sono passati meno di 365 giorni di utilizzo del sensore

Si noti che il numero di letture viene modificato dai metodi `addReading`() e `changeAllReadings`(), che prendono un lock sull'attributo `lock`, mentre il numero di giorni di utilizzo del sensore è incrementato dal metodo `newDayStarted`(), che è invece sincronizzato intrinsecamente su `this`.

Un possibile approccio potrebbe essere il seguente:

SensorReadings.java

```
public void readyToBeReplaced() throws InterruptedException {
        synchronized (this) {
                synchronized (lock) {
                        while (daysInUse < 365 || data.size() < 10000) {
                                lock.wait();
                        }
                }
        }
}
```

Quindi sincronizzare il metodo sia su `this` che su `lock`. Il problema poi sta nella chiamata a `wait`, che serve a mettere il thread in stato Suspended e rilasciare le risorse lockate in attesa di un `notify`() o `notifyAll`(): non è possibile chiamare wait() su più di un oggetto. L'esempio qui sopra quindi non rilascia mai il lock acquisito su `this`, che quindi non può più venir preso da nessuna invocazione a `newDayStarted`(), possibilmente bloccando permanentemente il thread.

In questo esercizio, la soluzione al problema è modificare il comportamento della classe in modo che tutti i metodi siano sincronizzati sullo stesso riferimento, ad esempio così:

SensorReadings.java

```
public class SensorReadings {
	private int id;
	private List<Double> data;
	private int daysInUse;

	public SensorReadings(int id) {
		this.id = id;
		data = new ArrayList<Double>();
		daysInUse = 0;
	}

	public synchronized void addReading(double d) {
		data.add(d);
		notifyAll();
	}

	public synchronized double getReading(int index) {
		return data.get(index);
	}

	public synchronized void changeAllReadings(List<Double> newData) {
		data = newData;
		notifyAll();
	}

	public synchronized void newDayStarted() {
		daysInUse++;
		notifyAll();
	}

	public synchronized void rdyToBeReplaced() throws InterruptedException{
		while (daysInUse < 365 || data.size() < 10000) {
			wait();
		}
	}
}
```

Lezione 14: Java concorrente (26/10/2023 Camilla Balzarotti, Lisa Cardini)

Costrutti avanzati sulla programmazione concorrente

Tutta la semantica di Java relativa alla programmazione concorrente è esprimibile con synchronized, join, wait e notify ma a volte, al posto di implementarle esplicitamente, potrebbe essere utile usare altre funzionalità già fornite dalle librerie standard di Java.

Una prima motivazione risiede nella volontà di ridurre il costo di creazione dei thread.

Finora (si veda lezione precedente) task svolto da lavoratore e lavoratore stesso coincidono.

Ogni volta che si voleva lanciare qualcosa in maniera asincrona, ossia un task che si vuole delegare ad un altro thread/flusso di esecuzione che non sia il main, venivano fatte due cose

1. creare il thread

2. passare la funzione da eseguire (il Runnable)

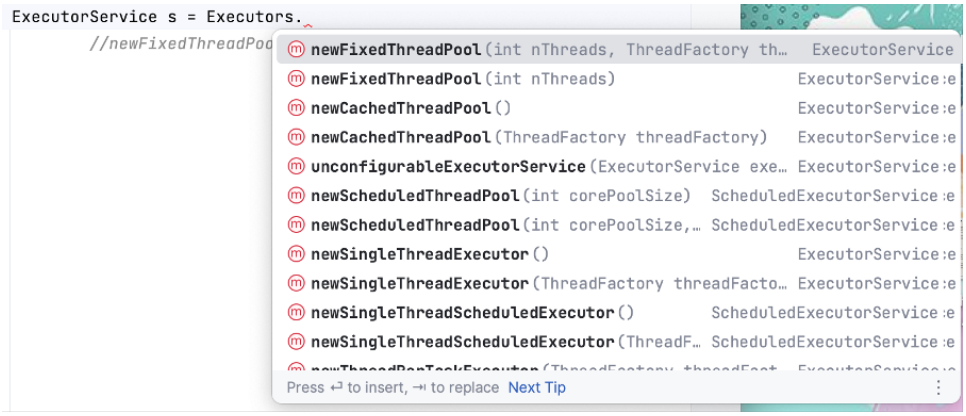
Concettualmente ha senso tenerle separate così da riuscire a distinguere chiaramente

- task = lavoro da svolgere
- thread = lavoratore

Al termine, il thread viene licenziato. Continue operazioni di inizializzazione ed eliminazione di thread hanno un costo piccolo ma non nullo, che può però diventare esoso se si continua a creare nuovi lavoratori per i task.

Si vorrebbe quindi definire una **thread pool** o **executor service**, ossia un insieme di lavoratori ben controllato a cui si continuano a sottomettere dei lavori. In questo modo, terminato un task, il thread non viene licenziato ma, essendo libero, estrae, se presente, un altro compito dalla pila dei lavori da svolgere e lo esegue.

Per quanto riguarda la sintassi, si dichiara un oggetto di classe ExecutorService, che viene costruito usando un metodo statico della classe Executors che consente di avere vari tipi di executor, diversi per caratteristiche.



Si tratta di metodi statici che istanziano internamente l'oggetto senza chiamare esplicitamente il costruttore con new (**design pattern Factory**).

Ad esempio

- **NewFixedthreadpool** (numero di thread) = costruire una thread pool di dimensione fissa
- **SingleThreadExecutor** = pool con un solo thread lavoratore
- **Costrutti con Scheduled** = permettono di reschedulare, far fare più volte periodicamente la stessa cosa al lavoratore

Per ulteriori esempi si rimanda alla documentazione di Java

https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/Executors.html

Main.java

```
//per dichiarare una thread pool con quattro lavoratori fissi
ExecutorService s = Executors.newFixedThreadPool(4);
```

Una volta creata la pool, avendo disaccoppiato i lavorati dai task da eseguire, per sottomettere i task si usa un metodo messo a disposizione dalla classe ExecutorService detto **submit**, da void a void, che riceve come parametro un Runnable.

Main.java

```
//si sottomettono 5 lavori, 4 dei quali verranno affidati “immediatamente” a 4 thread, mentre 1 rimarrà in attesa che uno dei thread termini
s.submit( () -> System.out.println("Ciao1"));
s.submit( () -> System.out.println("Ciao2"));
s.submit( () -> System.out.println("Ciao3"));
s.submit( () -> System.out.println("Ciao4"));
s.submit( () -> System.out.println("Ciao5"));
```

Fondamentalmente una thread pool è una coda bloccante, a cui si aggiunge un Runnable ogni volta che si fa submit, mentre i thread sono dei consumer che, quando liberi, pescano dalla coda un nuovo job, lo svolgono.

Il vantaggio è quello di poter disaccoppiare task e thread lavoratore e aggiungere nuovi task in diversi punti del codice.

Per chiudere l’Executor Service e rilasciare le risorse aassociate ai thread, vi sono due opzioni

- **Shut down now**, che chiude istantaneamente la pool, anche se non tutti i task sono stati portati a termine
- **Shut down**, che attende che tutti i task siano stati completati prima di chiudere la pool

Shut down è un metodo non bloccante, ossia ordina semplicemente all’Executor Service di terminare in modo disciplinato, ma il codice non si blocca in attesa che sia effettivamente finita la thread pool.

Fare submit di task ad una pool dopo una shutdown relativa alla stessa pool genera un’eccezione.

Per aspettare invece, occorre usare il metodo **awaitTermination**, che prende in ingresso due parametri, un timeout scelto dal programmatore e la sua unitài di misura, indicata usando TimeUnit (https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/TimeUnit.html).

AwaitTermination è un metodo bloccante che termina in due casi

- Lo shutdown è andato a buon fine prima dello scadere del time out, e in quel caso ritorna true
- È scaduto il time out, e in quel caso ritorna false

Quindi in caso si voglia aspettare effettivamente la terminazione (anche se va oltre il timeout), si usa un while.

Main.java

```
while(!s.awaitTermination(10, TimeUnit.SECONDS));
ExecutorService s1 = Executors.newFixedThreadPool(4);
```

In questo modo non si prosegue finché awaitTermination non ritorna true, ossia, avendo messo la negazione, la condizione del while diventa false.

Lo svantaggio dei pattern usati finora è ci si può bloccare solo alla fine dell’intera esecuzione. Si vorrebbe invece un modo per bloccarsi solo sui singoli task.

Ciò è in realtà possibile sfruttando un overload di submit in cui in ingresso non viene passato **un oggetto di classe** Runnable ma un **Callable<T>**.

Nello specifico, Callable<T> è un’interfaccia funzionale con un unico metodo call, che è un Supplier<T>, ossia non prende nulla come parametro in ingresso ma, computando in maniera

asincrona, ritorna un oggetto che, quando la computazione asincrona sarà terminata, sarà di tipo T. Per questo viene detto **Future<T>**.

(https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/Callable.html)

(https://docs.oracle.com/javase%2F7%2Fdocs%2Fapi%2F%2F/java/util/concurrent/Future.html)

Main.java

```
ExecutorService s1 = Executors.newFixedThreadPool(4);

s.submit(() -> "Ciao");
String f2 = "Ciao";
Future<String> f1 = s1.submit(() -> "Ciao");
// si noti la differenza tra f2 ed f1
// in f2 il valore della stringa è calcolato in modo sincrono, ossia assegnato alla variabile dallo stesso thread che sta eseguendo il codice, cioè il main
// nel secondo caso invece la stringa viene calcolata in modo asincrono
// f1 quindi è un Future di T e a submit viene passata la funzione che permette di ottenerne il valore
```

Il Callable è quindi non bloccante, permette al "manager" di passare ad altro.

Quando poi il manager ha bisogno il valore computato dal Callable esegue un'istruzione **get.**

get

```
V get()
      throws InterruptedException,
             ExecutionException
```

Waits if necessary for the computation to complete, and then retrieves its result.

Returns:

the computed result

Throws:

`CancellationException` - if the computation was cancelled

`ExecutionException` - if the computation threw an exception

`InterruptedException` - if the current thread was interrupted while waiting

Si tratta di un'istruzione bloccante, in quanto vi sono due possibili scenari

- Al momento della chiamata del get, il Callable aveva già terminato e scritto il risultato
  -> chiamata molto veloce
- In caso contrario, il manager deve aspettare finché il risultato della computazione del Callable non viene calcolato e scritto nell'oggetto risultato

Main.java

```
String f1String = f1.get();
```

Sincronizzare strutture dati

Tutte le collezioni di dati viste precedentemente non sono thread safe/concurrent, ossia potenzialmente non funzionano correttamente se più thread vi accedono contemporaneamente.

Per questo quando si implementano, come è stato fatto con PippoQueue, si pone sull'oggetto una sorta di wrapper esterno.

Questo pattern tuttavia è diventato talmente pervasivo che Java offre dei metodi per sincronizzare, in qualsiasi punto del codice, le strutture dati.

Main.java
```
List<String> l1 = new ArrayList<>();
//Per renderla thread safe
List<String> l2 = Collections.synchronizedList(l1);
```

In design pattern, synchronizedList è un **Decorator** = metodo che prende oggetto, espone la stessa interfaccia dell'oggetto passato come parametro ma aggiunge delle funzionalità (in questo caso lo rende thread safe).

Si procede ugualmente per mappe e set.

⚠ usare l1 ed l2 in realtà non è corretto. Infatti facendo l1.add() si accederebbe, mediante il puntatore l1, in maniera non più thread safe al cmontenuto puntato da l2. Per evitarlo, avendo sempre stesso tipo statico, si si usa lo stesso puntatore cosi da perdere il reference vecchio.

Main.java
```
List<String> l1 = new ArrayList<>();
//Per renderla thread safe
l1 = Collections.synchronizedList(l1);
```

Non tutta la libreria di Java è già sincronizzata automaticamente perché sarebbe costoso ma Java mette a disposizione dei metodi per rendere th\read safe gli oggetti, così che, per risparmiare, questo venga fatto SOLO quando strettamente necessario mentre ci si trova in programmazione concorrente.

Esistono invece strutture dati che nascono per essere efficienti già sincronizzate/thread safe, soprattutto per le code. Ad esempio producer e consumer con una coda in comune sono un pattern molto usato.

Main.java
```
//Per code già thread safe, implementate internamente come Linked List
Queue<String> q1 = new ConcurrentLinkedQueue<>();
//metodo non bloccante in quanto se il consumatore pesca e la coda è vuota viene ritornato -1
Queue<String> q2 = new LinkedBlockingQueue<>();
//metodo bloccante perché se la coda è vuota si rimane in attesa

//per le mappe
Map<String, Integer> m1 = new ConcurrentHashMap<>();
```

Per ulteriori esempi si rimanda alla documentazione di Java (https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/package-summary.html).

Equivalenti atomici

Nella lezione precedente BankAccount è stato creato come una sorta di guscio protettivo per l'intero amount perché in deposit e withdraw c'erano delle operazioni non atomiche, in particolare amount += o -= x (in assembly richiedono una load, una add ed una store).

Tuttavia, in realtà per tutti i tipi primitivi Java offre il corrispondente/**equivalente atomico** = offre tutta una serie di operazioni atomiche, per cui non occorre più usare il synchronized

Si rimanda alla documentazione di Java per ulteriori dettagli (https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/atomic/package-summary.html).

Ad esempio, per gli interi vi è **AtomicInteger**, e tra le funzioni che si possono svolgere in maniera atomica ci sono

- **addAndGet** = aggiunge valore e ritorna il risultato
- **get** = per avere il valore
- **new AtomicInteger(value)** per inizializzare il valore nel costruttore

BankAccountAtomic.java

```
package Concurrency;

import java.util.concurrent.atomic.AtomicInteger;

public class BankAccountAtomic {
    private AtomicInteger amount;

    public BankAccountAtomic() {
        amount = new AtomicInteger(0);
    }

    public void deposit(int x){
        amount.addAndGet(x);
    }
    public void withdraw(int x){
        amount.addAndGet(-x);
    }

    public AtomicInteger getAmount() {
        return amount;
    }
}
```

È conveniente quando si tratta istruzioni semplici perché solitamente sono lock free, ossia non acquisisce il lock ma vengono sfruttate direttamente le istruzioni atomiche già scritte nell'ISA della CPU.

⚠ Le operazioni di addAndGet sono atomiche singolarmente ma se vengono eseguite tante operazioni di addAndGet una dietro l'altra non possono essere considerate atomiche in toto

→ occorre usare il synchronized per renderle atomiche in blocco

Esempio -Rubrica telefonica concorrente (ContactBook)

Una rubrica telefonica è una struttura dati che, dato un nome, ritorna un numero di telefono: si vuole creare una lista telefonica al cui interno sarà presente una MAPPA che va da STRINGA (nome) a INTEGER (numero di telefono).

NOTA: nel codice si sta ignorando il caso in cui il nome cercato non sia presente nella rubrica.

Per renderlo thread safe, si possono sfruttare due metodi:

1. Mettere *synchronized* intorno ai due blocchi in modo che, quando bisogna entrare, viene acquisito un lock sulla ContactBook;
2. Utilizzare una struttura dati *concurrent* : invece di inizializzare con map = new HashMap <>(), si inizializza come map = new **concurrent** HashMap<>().

NOTA: thread safe implica che solo un thread alla volta può svolgere un'operazione, indipendentemente dalle operazioni che vengono fatte (lettura o scrittura ; ad esempio, mettendo synchronized e avendo solo due operazioni (get e put), è possibile avere al più un thread che svolge una di queste sulla ContactBook (scenario IDEALE).

Entrambi i metodi vanno quindi tendenzialmente bene ma, nella realtà, è possibile avere scenari diversi da quello ideale: si supponga, ad esempio, di avere uno scenario in cui "leggo sempre", ovvero si hanno molti thread che leggono e non scrivono quasi mai; in questo caso, sarebbe ottimale andare in PARALLELO, ovvero riuscire a fare in modo che, nel blocco critico di lettura, si possa entrare a meno che qualcuno stia modificando (conflitto read/write o write/write : le write devono essere esclusive, ovvero se qualcuno sta scrivendo, solo lui sta scrivendo ma, se si hanno più che leggono, li lascio leggere).

Per il parallelo viene usata la struttura della **LOCK**: a questi fini, viene introdotto il

READ-WRITE-LOCK

private final ReentrantReadWriteLock lock è una struttura dati al cui interno sono presenti due lock:

- **readLock** => non esclusivo
- **writeLock** => esclusivo

NOTA: "Reentrant" implica che se si ha un writeLock di livello più alto, è possibile acquisire un lockRead di livello più basso senza prima rilasciare il write.

Per scriverlo, viene tolto "synchronized" e si ha:

- PUT : la chiamata **lock.writeLock.lock()** e **lock.writeLock().unlock()** permette di acquisire e rilasciare un lock in scrittura;
- GET : la chiamata **lock.readLock().lock** e **lock.readLock.unlock()** permette di acquisire e rilasciare un lock in lettura.

ContactBook.java

```
package Concurrency;

import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantReadWriteLock;
```

```java
public class ContactBook {
    private final Map<String, Integer> map;
    private final ReentrantReadWriteLock lock;

    public ContactBook() {
        map = new HashMap<>();
        lock = new ReentrantReadWriteLock();
    }

    public void put (String name, int num){
        lock.writeLock().lock(); //acquisire un lock in scrittura
        map.put(name, num);
        lock.writeLock().unlock(); //rilasciare un lock in
scrittura
    }

    public int get(String name){
        lock.readLock().lock();
        int retVal = map.get(name);
        lock.readLock().unlock();
        return retVal;
    }
}
```

Il Lock è quindi una struttura dati che mi permette, nel caso in cui ci fosse un costo associato alla lettura, di *non serializzare* le letture ovvero nel momento in cui un thread è in lettura, e quindi tiene il lock perché costa tanto tempo, altri possono prendere il lock => il readLock è non esclusivo.

Tema d'esame (13 febbraio 2023)

Products è una struttura dati contenente due mappe:

1. **idToName** : da intero a stringa
2. **nameToPrice** : da un nome a un prezzo

Si hanno quindi dei prodotti salvati in modo che la prima mappa va dall'identificativo univoco del prodotto al suo nome e la seconda che va dal nome al suo prezzo (es. 123 computer prezzo 1000, 123 => Computer e Computer =>1000).

lastId serve in quanto ogni volta che viene inserito un nuovo prodotto, l'ID viene incrementato di 1 e viene ritornato al chiamante: in questo modo il chiamante stesso sa di aver inserito il prodotto e ha a disposizione l'ID sia per poter avere informazioni sul prodotto, sia per poterlo utilizzare.

Products.java

```java
package it.polimi.softeng.concurrency.exercises.ex2023_02_13;

import …
```

```
public class Products {
    private Map<Integer, String> idToName;
    private Map<String, Integer> nameToPrice;
    private int lastId
    public Products() {
        idToName = new HashMap<>();
        nameToPrice = new HashMap<>();
        lastId = 0;
    }
```

addProduct : viene fatto synchronized sulla prima mappa e, successivamente, ad assignedId(valore attuale ritornato) viene assegnato il lastId incrementato di 1 (principalmente "incremento di 1 lastId e prendo assignedId", che viene ritornato, quindi alla prima chiamata assignedId valeva 0, ritorno 0 e incremento di 1 e così via in modo che il chiamante ottiene l'ID associato al prodotto che sta inserendo).

Quindi dopo aver assegnato l'ID al prodotto, sulla prima mappa viene sincronizzata la corrispondenza ID/nome e, separatamente sull'altra mappa, viene sincronizzato nome/prezzo.

Products.java

```
    public int addProduct(String name, int price) {
        int assignedId = 0;
        synchronized (idToName) {
            assignedId = lastId++;
            idToName.put(assignedId, name);
        }

        synchronized (nameToPrice) {
            nameToPrice.put(name, price);
        }
        return assignedId;
    }
```

C'è poi il secondo metodo che ritorna il prezzo del prodotto avente l'id passato per parametro: **getPriceById**, se la corrispondenza da id a nome (idToName) contiene l'identificativo che ho passato, ottiene il nome e usa il nome per andare a cercare nell'altra mappa, altrimenti ritorna -1.

Producsts.java

```
    public int getPriceById(int id) {
        synchronized (idToName) {
            if (idToName.containsKey(id)) {
              synchronized (nameToPrice) {
                String name = idToName.get(id);
                return nameToPrice.get(name);
            } else {
                 return -1;
            }
```

```
            }
        }
}
```

Domanda a) *Si dica se la classe Products è opportunamente sincronizzata per permettere l'invocazione concorrente dei suoi metodi da parte di più thread. In caso negativo, si modifichi la classe per correggere la sua sincronizzazione.*

Ciò che bisogna sempre chiedersi è:

1. Ci sono strutture dati in cui faccio a accesso (contemporaneamente in scrittura e lettura) e non sono sincronizzato?
2. Quando ci sono due lock, una dentro l'altro, è possibile che ci sia qualche RUN del programma che porti ad accedervi al contrario? (Nel nostro caso non si possono verificare deadlock);
3. È troppo sincronizzato? Ci sono metodi che accedono a blocchi di attributi separati e potrebbero esserci accessi parallelizzati;
4. Ho violazioni di ATOMICITÀ?

(1) Notiamo che non è opportunamente sincronizzata: servirebbe un synchronized (nameToPrice) qui in quanto si sta accedendo in lettura e non è sincronizzato.

Questo non basta: il metodo addProduct non è atomico (4) in quanto si sta erroneamente assumendo che la chiamata a tutto il metodo sia atomica (== l'aggiornamento delle due mappe avviene senza che nessuno si possa mettere in mezzo): infatti, nella riga di codice che precede synchronized (nameToPrice), non si ha nessun lock intrinseco, e questo potrebbe far sorgere dei problemi.

È possibile, ad esempio, che un chiamante entri e acquisisca idToName (per poi inserire in idToName), e poi venga interrotto (preemption) dopo aver aggiornato metà delle sue strutture dati: a questo punto potrebbe entrare un altro thread che in idToName trova qualcosa e che quindi, assumendo l'atomicità del metodo, sia convinto che anche nameToPrice non sia vuoto; tuttavia, nel momento in cui fa nameToPrice.get(name), ritorna un'eccezione perché non aveva dentro il nameToPrice.

Principalmente ciò che viene fatto è aggiornare due strutture dati: una viene aggiornata, viene rilasciato il lock e poi viene aggiornata l'altra.

→ ATOMICITY VIOLATION

Quindi per fare in modo che i due blocchi vengano eseguiti in modo atomico, bisogna mettere un synchronized che inglobi tutto e togliere quello messo precedentemente.

Products.java

```
    public synchronized int addProduct(String name, int price) {
        int assignedId = 0;
        assignedId = lastId++;
        idToName.put(assignedId, name;
        nameToPrice.put(name, price);
        return assignedId;
    }
```

NOTA: i problemi in concorrenza sono:

1. RACE CONDITIONS : corse critiche con violazione di mutua esclusione (accedo in lettura e scrittura contemporaneamente);
2. ATOMICITY VIOLATION : blocchi che si suppone siano eseguiti in maniera atomica in realtà non lo sono;
3. DEADLOCK;
4. LIVELOCK: si è bloccati perché l'applicazione ci obbliga a rimanere bloccati non a causa di lock ma per com'è scritta, ovvero non si verifica mai la condizione per cui è possibile evolvere.

NOTA: normalmente si hanno oggetti confinati in una classe e, se questa è threadsafe, allora lo sono anche i suoi oggetti; tuttavia, può capitare che delle classi condividano dei references (si cerca di evitarlo per fare encapsulation dello stato): in questo caso si tende ad avere oggetti immutabili ma, se devono essere mutabili, bisogna sincronizzarli.

NOTA: se si vuole rendere un oggetto thread safe e quindi si ipotizza che tutto il suo stato sia incapsulato nell'oggetto, allora si usa l'oggetto come lock e si mette synchronized sui metodi; si usano synchronized su appositi oggetti nel caso in cui si vuole fare una sincronizzazione a granularità più fine.

Domanda b) *A partire dalla soluzione data alla domanda (a), si modifichi il metodo getPriceById in modo che, invece di ritornare -1, sospenda il chiamante se il prodotto non esiste. Se necessario, si indichi come modificare altri metodi.*

Il chiamante deve essere bloccato, quindi viene aggiunto il while e tolto l'if e in addProduct, prima del return, viene messa notifyAll().

Producsts.java

```
    public synchronized int getPriceById(int id) {
        while (! IdToName.containsKey(id)){
            wait();
        }
        String name = idToName.get(id);
        return nameToPrice.get(name);
        }
    }
}
```

Domanda c) *Si scriva un metodo void copyProduct(Products other) che, per tutti i prodotti di this che sono anche contenuti in other sostituisce i prezzi contenuti in this con i prezzi contenuti in other. Si presti particolare attenzione a evitare il rischio di possibili deadlock.*

Bisogna acquisire il lock anche sull'altro oggetto per poterci accedere in lettura in modo che nessuno possa scriverci contemporaneamente; inoltre, se mettessimo synchronized int e syncrhronized adder, si creerebbe un deadlock.

Ciò che viene fatto è quindi acquisire il lock su pthread, copiarlo, prendere il lock su this e svolgere l'operazione; dopo synchronized(this), bisogna aggiornare: si scorre l'altra mappa e, se ciò che c'è nell'altra mappa è contenuto nella mappa presa in considerazione, va sostituito.

Products.java

```
    public void copyProducts(Products other) {
```

```
        Map<String, Integer> otherPriceCopy;
        synchronized (other){
            otherPriceCopy = new HashMap<>(other.nameToPrice);
        }
        synchronized (this) {
            for (String name : otherPriceCopy.keySet()){
                if (this.nameToPrice.containsKey(name)) {
                  this.nameToPrice.put(name,
                  otherPriceCopy.get(name));
                }
            }
        }
}
```

Lezione 15: Design patterns (30/10/2023 Alesio Demiri, Federica Currò Dossi)

Design Patterns

I design patterns propongono soluzioni astratte e riutilizzabili, descrivendo un insieme di classi e interfacce, al fine di risolvere un problema ricorrente incontrato durante la fase di programmazione o sviluppo di un sistema software. La loro astrazione permette di essere adattabili a risolvere problemi specifici. Essi migliorano l'estendibilità (nuove funzionalità da implementare non richiedono la modifica del codice esistente), il riuso (possibilità di riutilizzare componenti in diverse parti del codice o al di fuori del progetto stesso sotto forma di librerie da altri programmi) e la manutenibilità del codice (correzione di errori).

I design patterns si dividono in quattro categorie:

1. Creazionali: definiscono metodologie per la creazione di oggetti. È fondamentale gestire la dipendenza tra il creatore di un oggetto e l'oggetto creato, in quanto la classe che crea un oggetto deve conoscere il tipo dinamico dell'oggetto da creare. Esempi di patterns creazionali: Abstract factory, Factory method, Pattern Builder, Prototype pattern e il Singleton.

2. Strutturali: risolvono problematiche inerenti alla struttura delle classi e degli oggetti, quali la composizione di oggetti a runtime attraverso quindi l'associazione. Esempi di patterns strutturali: Adapter, Bridge, Composite, Decorator, Facade e Proxy.

3. Comportamentali: definiscono il comportamento e le modalità di interazione tra oggetti. Essi si occupano della gestione delle dipendenze nelle funzionalità tra classe e sottoclasse (una superclasse potrebbe necessitare del comportamento di una sottoclasse per effettuare una determinata operazione), oppure tengono traccia delle azioni compiute da un oggetto, ad esempio per poterne cancellare l'effetto. Si pensi, per esempio, a una funzionalità 'modifica' o 'ritorna allo stato precedente'.

4. Architetturali: definiscono metodologie per la creazione di applicazioni e sistemi software. Sono quindi pattern di alto livello. I pattern architetturali sono spesso alla base di framework. Un framework non è altro che una libreria che serve per costruire sistemi e applicazioni e spesso impone un particolare stile per lo sviluppo di questi. Gestiscono più in generale la comunicazione tra diversi livelli logici di un sistema, sia tra la parte grafica e il modello, ma anche tra il modello e il networking, se si tratta di un'applicazione web.

Esiste una classificazione secondaria, che divide i Design patterns in Class patterns e Object patterns.

I Class patterns definiscono comportamenti e relazioni statiche tra classi che sono rilevanti a compile time. Gli Object patterns invece definiscono comportamenti e relazioni tra oggetti a runtime e tengono conto della possibilità di avere dinamicità, quindi che i comportamenti e relazioni possono essere modificate a runtime.

Factory pattern (esercizio Studente)

Il Factory pattern è un pattern di tipo creazionale, il cui scopo principale è di definire metodologie per la creazione di oggetti. Oltre alla gestione della dipendenza tra creatore e creato a compile time, limita il numero di istanze di una classe e gestisce la creazione di tipi complessi e parametrici.

Nell'esercizio proposto abbiamo una classe astratta 'Studente', una classe astratta 'Esame' e un metodo di Studente chiamato produceExam(), il quale mi ritorna un Esame. Ci sono due tipi di studenti, quello preparato (PreparedStudent) che produrrà un esame sufficiente (SufficientExam) e quello non preparato (LetsTryItAnyway) che produrrà un esame ampiamente insufficiente (InsufficientExam). L'obiettivo è di implementare il metodo produceExam in modo che mi crei un esame a seconda del tipo di studente in questione.

Soluzione: l'idea è che Studente abbia un metodo (examFactory) che verrà overridato in tutte le istanze specifiche e lo specifico override verrà chiamato dinamicamente. Nel momento in cui lo Studente deve creare un Esame lo delego ad un Factory che è un metodo astratto.

Student.java

```
package it.polimi.softeng.pattern.factory;


public abstract class Student {

public final Exam produceExam() {
        // Create exam (depending on the type of student)
        Exam myExam = examFactory();
        // Return the exam for evaluation
        return myExam;
    }
protected abstract Exam examFactory();
}
```

La creazione di un esame viene delegata al metodo astratto Factory, quindi ogni sottoclasse gestirà la creazione del proprio Esame, implementando il metodo examFactory(). Il metodo examFactory() è protected quindi non è visibile dall'esterno, serve a 'completare' I metodi pubblici, infatti all'interno del metodo produceExam potrebbero esserci righe di codice 'generale' che vale per qualunque istanza di studente, indipendentemente dallo specifico tipo; le parti specifiche di codice

le metto in un metodo (in questo caso examFactory), che verrà poi chiamato nella specifica istanza. In questo modo evito l'ovverride del metodo produceExam() perché potrebbero esserci grossi frammenti di codice che sono comuni a tutti gli Studenti.

PreparedStudent.java

```
package it.polimi.softeng.pattern.factory;


public class PreparedStudent extends Student {
    @Override
    protected Exam examFactory() {
        return new SufficientExam();
    }
}
```

LetsTryItAnywayStudent.java

```
package it.polimi.softeng.pattern.factory;

public class LetsTryItAnywayStudent extends Student {
    @Override
    protected Exam examFactory() {
        return new InsufficientExam();
    }
}
```

Abstract Factory pattern (esercizio su Interfaccia grafica)

L'abstract factory pattern è un pattern di tipo creazionale, fornisce un'interfaccia per creare famiglie di oggetti connessi o dipendenti tra loro, in modo che non ci sia necessità da parte degli utilizzatori di specificare i nomi delle classi concrete all'interno del proprio codice.

L'esercizio tratta della creazione di un'interfaccia grafica che in questo esempio, per semplicità, ha solo due componenti Window e Menu

Il problema da risolvere è che gli elementi grafici per la creazione di un'interfaccia possono essere creati con diversi stili, a seconda delle esigenze. Istanziando esplicitamente i tipi concreti di ciascuno stile (light e dark) la modifica dello stile stesso sarebbe complessa e lunga.

Soluzione: voglio istanziare nel Main un'interfaccia grafica che può essere o Light o Dark.

L'idea è di creare un'interfaccia o una classe astratta al cui interno pongo i Factory method che creano i componenti necessari.

L'idea è che nel main io istanzi un'interfaccia dark (o light) con Window e Menu, e trovare un modo per compattare la creazione di tutti gli elementi di un'interfaccia Dark o di un'interfaccia Light, all'interno di opportune classi. L'abstract Factory mi servirà per istanziare la versione dark (o light) di tutti i componenti dell'interfaccia.

Per prima cosa creiamo un Abstract Factory, che chiamiamo 'InterfaceFactory'. Ogni volta che voglio costruire un'interfaccia utilizzo i factory method che mi creano Window e Menu. Ho delegato la

parte di creazione dell'Interface, ovvero la creazione dei componenti che mi servono, ad un unico componente Abstract Factory.

InterfaceFactory.java

```
package it.polimi.softeng.pattern.abstractfactory;

public abstract class InterfaceFactory {


    public abstract Window createWindow();

    public abstract Menu createMenu();
}
```

Avrò delle sottoclassi DarkInterfaceFactory e LightInterfaceFactory che implementano i componenti corretti.

DarkInterfaceFactory.java

```
package it.polimi.softeng.pattern.abstractfactory;

public class DarkInterfaceFactory extends InterfaceFactory {

    @Override
    public Window createWindow(){
        return new DarkWindow();
    }

    @Override
    public Menu createMenu(){
        return new DarkMenu();
    }
}
```

LightInterfaceFactory.java

```
package it.polimi.softeng.pattern.abstractfactory;

public class LightInterfaceFactory extends InterfaceFactory {
    @Override
    public  Window createWindow(){
        return new LightWindow();
    }
    @Override
    public Menu createMenu(){
        return new LightMenu();
}
}
```

Nel Main è tutto istanziato in maniera generale, ho solo una riga in cui istanzio il factory desiderato.

Main.java

```
package it.polimi.softeng.pattern.abstractfactory;


public class Main {
    public static void main(String[] args) {
        // TODO: si vuole inizializzare un'interfaccia Dark con una finestra e un menu.
        InterfaceFactory factory = new DarkInterfaceFactory();
        Menu menu = factory.createMenu();
        Window window = factory.createWindow();
        System.out.println(menu + " " + window);
    }
    }
```

Adapter pattern (esercizio su PDFFile)

L'Adapter pattern è di tipo strutturale, converte l'interfaccia di una classe in una interfaccia diversa.

Inizialmente abbiamo dichiarato un'interfaccia Printable e una sottoclasse TextFile che implementa l'interfaccia Printable e rappresenta gli oggetti concreti stampabili.

Printable.java

```
package it.polimi.softeng.pattern.adapter;

public interface Printable {
    // Print with the given font size
    // Size from 0 to 10
    public void print(int size);
}
```

TextFile.java

```
package it.polimi.softeng.pattern.adapter;

public class TextFile implements Printable {
    @Override
    public void print(int size) {
        // Code to print ...}
}
```

Il committente mi chiede di stampare anche i file pdf, trovo una libreria esterna e una sua classe LibPDFFile. Questa classe però risulta problematica perché non implementa la mia interfaccia Printable, quindi devo adattare la struttura del mio codice. Obiettivo è quindi di stampare un documento pdf Printable usando la libreria LibPDFFile.

LibPDFFile.java

```java
package it.polimi.softeng.pattern_solutions.adapter;
// Class to manage PDF files (from an external library)
public class LibPDFFile {
    // Print the PDF file with the given size
    // Size from 0 to 20
    public void stampa(int size) {
    }
}
```

Soluzione: voglio creare una classe simile a TextFile, che chiameremo PDFFile e che implementa l'interfaccia Printable, all'interno deve usare la libreria LibPDFFile.

Ho due possibili soluzioni:

1. Soluzione che utilizza composizione: all'interno del mio PDFFile mi tengo il riferimento alla classe di libreria libPDFFile; quindi, ho un'istanza libPDFFile che inizializzo nel costruttore. Quindi ho scritto un 'guscio', un adapter, che dall'esterno mi fa sembrare quel pdf come lo vorrei, ovvero printable, nel momento in cui chiamo print delego esecuzione effettiva a libpdffile.

PDFFile.java

```java
package it.polimi.softeng.pattern_solutions.adapter;

public class PDFFile implements Printable  {
    private final LibPDFFile libPDFFile;

    public PDFFile() {
        this.libPDFFile = new LibPDFFile();
    }
    @Override
    public void print(int size) {
        libPDFFile.stampa(size*2);
    }
}
```

Main.java

```java
package it.polimi.softeng.pattern_solutions.adapter;

public class Main {
    public static void main(String[] args) {
        Printable file = new PDFFile();
        file.print(10);
    }
}
```

2. Soluzione che prevede l'estensione di LibPDFFile da parte di PDFFile oltre all'implementazione dell'interfaccia printable. In questo caso il Main non muta, la classe PDFFile sarà la seguente:

PDFFile.java

```
package it.polimi.softeng.pattern_solutions.adapter;

public class PDFFile extends LibPDFFile implements Printable  {
    private final LibPDFFile libPDFFile;
    public PDFFile() { this.libPDFFile = new LibPDFFile();
    }
    @Override
    public void print(int size) {
        super.stampa(size*2);
    }
}
```

Decorator (esercizio su Message Printer)

Decorator è un pattern strutturale, consente di aggiungere nuove funzionalità ad oggetti già esistenti (quindi a runtime). Partendo da una classe MessagePrinter (ha come metodo print, che prende in ingresso un messaggio e lo stampa), vorrei costruire tanti MessagePrinter, in particolare un DoubleMessagePrinter che stampa il messaggio due volte, un CapitalMessagePrinter che stampa il messaggio tutto in maiuscolo e soprattutto vorrei poter stampare tutte le combinazioni possibili tra i due tipi di messaggi (solo doppio, solo maiuscolo, doppio e maiuscolo, etc.) senza dover definire un numero esponenziale di classi.

MessagePrinter.java

```
package it.polimi.softeng.pattern_solutions.decorator;

public class MessagePrinter {
    public void print(String message) {
        System.out.println(message);
    }
}
```

Soluzione: Staticamente io posso definire la gerarchia delle sottoclassi, il decorator mi permette di estendere un oggetto a runtime, ovvero dinamicamente. Questo mi permette di poter comporre decorazioni, in questo caso combinare una decorazione che mi rende la stringa maiuscola con una decorazione che mi stampa la stringa due volte.

L'idea è che la decorazione sia componibile, quando dichiaro un oggetto decorato voglio poterlo decorare nuovamente e in diversi modi.

CapitalMessagePrinter.java

```
package it.polimi.softeng.pattern_solutions.decorator;

public class CapitalMessagePrinter extends MessagePrinter {
    private final MessagePrinter printer;
```

```
    public CapitalMessagePrinter(MessagePrinter printer) {
        this.printer = printer;
    }

    @Override
    public void print(String message) {
        printer.print(message.toUpperCase());
    }
}
```

Il CapitalMessagePrinter deve essere un MessagePrinter, così se posso decorare un message printer, allora posso decorare un capital message printer. Ogni decorazione costruisce un attributo tale che sia un altro message printer.

In DoubleMessagePrinter ho lo stesso codice (a meno di nomi di variabili e classe), creando un tipo statico printer di tipo MessagePrinter posso sfruttare nel metodo print anche il CapitalMessagePrinter, essendo lui stesso un MessagePrinter.

DoubleMessagePrinter.java

```
package it.polimi.softeng.pattern_solutions.decorator;

public class DoubleMessagePrinter extends MessagePrinter {
    private final MessagePrinter printer;

    public DoubleMessagePrinter(MessagePrinter printer) {
        this.printer = printer;
    }
    @Override
    public void print(String message) {
        printer.print(message + message);
    }
}
```

Main.java

```
package it.polimi.softeng.pattern_solutions.decorator;
public class Main {
    public static void main(String[] args) {
        MessagePrinter messagePrinter = new MessagePrinter();
        messagePrinter.print("Hello");

        MessagePrinter printer = new MessagePrinter();
        printer.print("Hello");
        MessagePrinter doublePrinter = new DoubleMessagePrinter(new
MessagePrinter());
        doublePrinter.print("Hello");
        MessagePrinter capitalPrinter = new CapitalMessagePrinter(new
```

```
MessagePrinter());
        capitalPrinter.print("Hello");
        MessagePrinter doubleCapitalPrinter = new DoubleMessagePrinter(new
CapitalMessagePrinter(new MessagePrinter()));
        doubleCapitalPrinter.print("Hello");
    }
}
```

Lezione 16: Design patterns (9/11/2023 Luigi Cardone, Lorenzo Cecchettini)

Design Patterns 2

In questa seconda lezione dedicata ai design patterns ci spostiamo dai pattern di tipo creazionale e strutturale a quelli di tipo **comportamentale** e **architetturale**.

Command

Supponiamo di voler scrivere un programma che implementi un menu e le sue voci, alle quali corrisponde un'azione da eseguire.

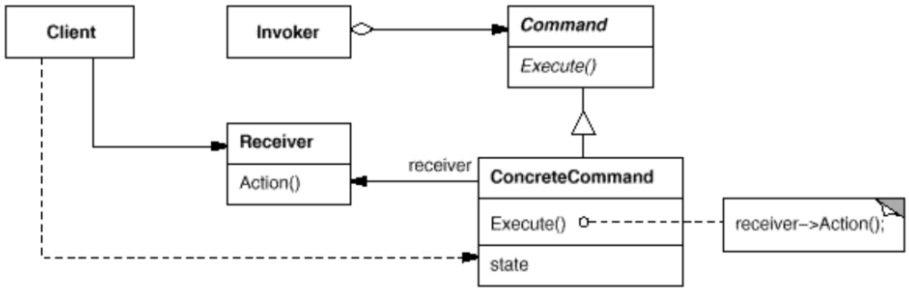
Command è un pattern di tipo comportamentale che ha l'obiettivo di gestire un insieme di **comandi** (le voci del menu) senza che questi siano conosciuti a priori e allo stesso tempo, separare **l'azione** in sé del comando selezionato.

In pratica, fa in modo di **separare le istanze** del menu dai comandi e l'azione dal comando stesso. Così facendo, si rende più flessibile il programma, permettendo di creare più menu con comandi o azioni diverse, e si lascia la possibilità di cambiare l'azione associata al comando a **runtime**.

Una possibile implementazione del pattern command è data dalla creazione di:

- Command*:* un *interface* (o *abstract class*), che nella struttura più semplice ha un *metodo* Execute che le sottoclassi di Command useranno per implementare la logica applicativa dell'istanza da compiere.
- ConcreteCommand: il comando in sé, implementa Command e compie l'azione stabilita. Di solito ci sono più classi di questo tipo, una per ogni azione che si vuole avere.

Possibile struttura per l'implementazione del Menu



Esercizio - Game Controller

La classe GameController è da interdersi come il controller fisico della console, composto da tasti (GameControllerButton) che agiscono attraverso delle azioni sul gioco, che a loro volta possono cambiare a seconda del gioco che si sta giocando.

Si chiede di completare il codice implementando la possibilità di cambiare gioco e di mappare le azioni sui tasti, facendo in modo che il GameController sia svincolato dal gioco e che i comandi non siano già mappati di default, per conferire la possibiltà di personalizzare le varie azioni sui button.

Per prima cosa si implementa la classe GameControllerButton che avrà come variabili **game** e **command**, essendo variabili private si creano i metodi set() e get() ed infine si aggiunge un metodo **onClick()** che esegue il comando del gioco nel caso non sia nullo, in questo modo il gioco è reso indipendente dal Game Controller e la sua influenza è limitata ai bottoni sulla quale veranno mappati i vari comandi.

GameControllerButton.java

```
package it.polimi.softeng.pattern_solutions.command;

public class GameControllerButton {

    private GameCommand command;

    private Game game;


    public void setGame(Game game) {

        this.game = game;

    }


    public GameCommand getCommand() {

        return command;

    }


    public void setCommand(GameCommand command) {
```

```
        this.command = command;
    }


    public void onClick() {
        if (command != null) {
            command.execute(game);
        }
    }
}
```

In seguito, si aggiunge il metodo **setButtonCommand()** in GameController che dà la possibilità di **mappare** le azioni sui bottoni adeguati al gioco, utilizzando set() e get().

GameController.java

```
public void setButtonCommand(int buttonId, GameCommand command) {
        controllerButtons.get(buttonId).setCommand(command);
    }
```

Per finire, si implementa **changeGame()** che permette di cambiare gioco nella classe GameController, facendo in modo che ogni oggetto GameControllerButton si aggiorni con esso, mantendo sempre indipendenti tra loro la classe GameController ed il gioco, di conseguenza anche i comandi del gioco con i bottoni del controller.

GameController.java

```
public void changeGame(Game game) {
        this.game = game;
        for (GameControllerButton button : controllerButtons) {
            button.setGame(game);
        }
    }
```

Observer

Observer è uno dei pattern comportamentali tra i più diffusi e noti, il cui scopo è quello di:

- Creare un legame ***one-to-many*** tra oggetti senza creare strette dipendenze tra di essi.
- Al cambiare dello stato di un oggetto, **aggiornare automaticamente** un insieme definito di oggetti dipendenti da esso.

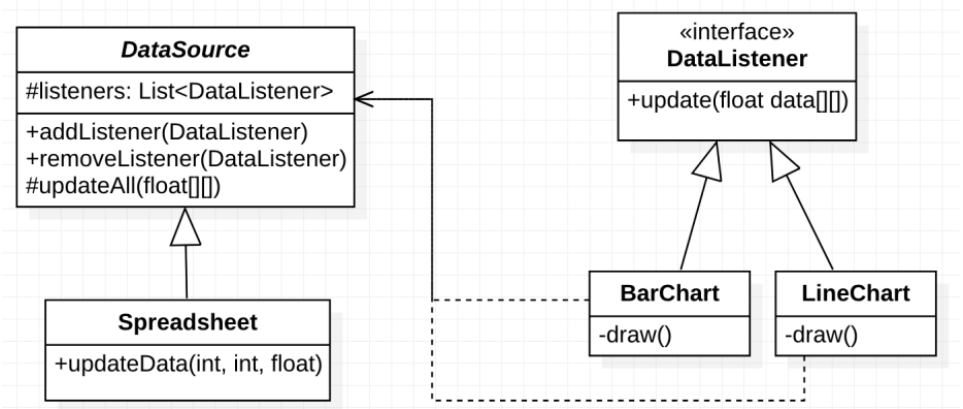
Supponiamo di voler creare un'applicazione per manipolare fogli di calcolo. Avremo bisogno di una classe **Spreadsheet,** contenete i dati e le formule inserite dall'utente, una classe **BarChart**, che permetterà di visualizzare i dati su un diagramma a istogrammi, e un'altra, **LineChart**, che li mostrerà con una curva su un cartesiano.

Vogliamo quindi gestire le interazioni tra la **sorgente di dati**, Spreadsheet, e le **classi utilizzatrici** dei dati, LineChart e BarChart, senza che queste dipendano strettamente l'uno dall'altra. Inoltre, è bene sottolineare che la sorgente dei dati non deve avere dipendenze forti con gli utilizzatori.

Una possibile implementazione del pattern Observer è data dalla creazione di:

- **Observable***:* una *abstract class* che rappresenta la sorgente dati, letteralmente la classe sotto osservazione, che contiene una lista di Observer che verranno allertati quando desiderato. Per permettere una gestione dinamica del numero di Observer sono presenti anche un metodo addObserver() e removeObserver().
- **Observer**: un'interfaccia che permette alle classi iscritte di implementare un metodo update() quando allertati dall'Observable.

Possibile struttura per l'implementazione dell'applicazione



Esercizio – User interface

Dato un ipotetico input da parte dell'utente si segnali a tutti i componenti necessari di svolgere la loro azione.

Questa è la classe astratta Observable che si occupa di creare la lista di observers e di notificare ognuno di essi nel momento in cui riceve un input.

ClickSource.java

```
package it.polimi.softeng.pattern_solutions.observer;
```

```
import java.util.ArrayList;
import java.util.List;
public abstract class ClickSource {
    private final List<ClickObserver> clickObservers = new ArrayList<>();
    public void addClickObserver(ClickObserver observer) {
        clickObservers.add(observer);
    }
    public void notifyClickObservers() {
        for (ClickObserver observer : clickObservers) {
            observer.onClick();
        }
    }
}
```

Successivamente, creiamo l'interfaccia ClickObserver che implementerà il metodo onCLick().

ClickObserver.java

```
package it.polimi.softeng.pattern_solutions.observer;



public interface ClickObserver {



    // Codice chiamato da una sorgente quando si verifica l'evento click
```

```
    public void onClick();



}
```

A questo punto, creamo la sottoclasse di ClickSource, che quindi, eredita la lista di observers e implementa il metodo **notifyClickObservers()** che permette di chiamare metodo su di essi.

UserInterface.java

```
package it.polimi.softeng.pattern_solutions.observer;


public class UserInterface extends ClickSource {


    public void click() {
        this.notifyClickObservers();
    }
}
```

Questa è la classe di ClickObservers che implementa l'interfaccia ClickObservers.

LogClickObservers.java

```
package it.polimi.softeng.pattern_solutions.observer;


public class LogClickObserver implements ClickObserver {
    @Override
    public void onClick() {
        // Salvo in un log
    }
}
```

Il Main si occupa di istanziare gli oggetti UserInterface e LogClickObserver ed infine iscrive LogClickObservers ad UserInterface .

Main.java

```
package it.polimi.softeng.pattern_solutions.observer;


public class Main {

    public static void main(String[] args) {

        LogClickObserver observer = new LogClickObserver();

        UserInterface userInterface = new UserInterface();

        userInterface.addClickObserver(observer);

    }

}
```

State

Come suggerisce il nome, il pattern State permette di implementare una maniera **estendibile**, **flessibile** e **riusabile** i processi riconducibili ad un Automa a Stati Finiti (ASF).

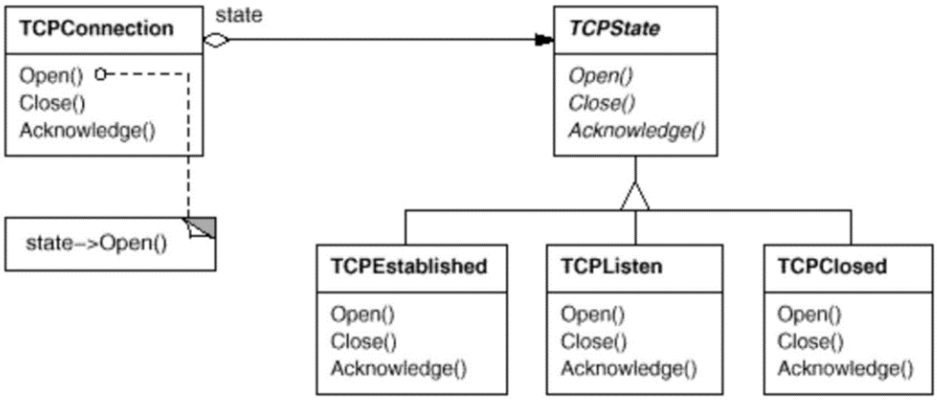
Viene utilizzato quindi, quando si vuole gestire un oggetto dove sono presenti una serie di operazioni complesse dipendenti da uno stato interno. Per cambiare il comportamento dell'oggetto, e quindi il suo stato, vengono introdotte una serie di variabili di stato e un blocco di statement condizionali per compiere azioni diverse in base ad esse

Supponiamo di voler creare un'implementazione del TCP (Transmission Control Protocol). Abbiamo quindi bisogno di una classe **TCPConnection** che si occupi della gestione della connessione e che, in base al suo stato, scelga un'azione pertinente ad esso.

Una possibile implementazione del pattern State è data dalla creazione di:

- State*:* una *abstract class* che verrà implementate da tutti gli stati di cui si necessita.
- StateConcrete: una *subclass* di State specializzata per un determinato stato che gestirà le operazioni relative allo stato e le eventuali transizioni a stati successivi.

Possibile struttura per l'implementazione dell'applicazione



Esercizio – The phone

Dato un telefono che può ricevere messaggi e chiamate e che può trovarsi in stati diversi come **Normale**, **Silenzioso** e **Notte.**

Scrivere il codice affinché il telefono suoni sempre se si trova in modalità Regolare, suoni solo con chiamate e vibri con messaggi se in modalità Silenzioso e vibri sempre se si trova in modalità Notte.

L'interfaccia PhoneState dispone delle funzioni **receiveCall()** e **receiveMessage()**, le quali verranno implementate dalle classi Regular, Silent e Night; è così possibile utilizzare questa interfaccia astratta per raggruppare delle funzioni che, una volta chiamate dalla classe Phone , restituiranno risultati diversi a seconda del valore della sua variabile stato.

PhoneState.java

```
package it.polimi.softeng.pattern_solutions.state;

public interface PhoneState {

    public void receiveCall();

    public void receiveMessage();
}
```

Come anticipato, la classe SilentState implementa l'interfaccia PhoneState ed approfondisce i metodi receiveCall() e receiveMessage().

SilentState.java

```java
package it.polimi.softeng.pattern_solutions.state;


public class SilentState implements PhoneState {
    @Override
    public void receiveCall() {
        // Ring
    }


    @Override
    public void receiveMessage() {
        // Silent
    }
}
```

La classe Phone ha come variabile privata **State**, inizializzata a Silent e per poter cambiare stato implementa un metodo Set(). Infine, si implementano i metodi recieveCall() e recieveMessage() che a loro volta chiamano il metodo della classe State.

Phone.java

```java
package it.polimi.softeng.pattern_solutions.state;


public class Phone {
    private PhoneState state;


    public Phone() {
        this.state = new SilentState();
    }


    public void receiveCall() {
        state.receiveCall();
    }
```

```
    public void receiveMessage() {
        state.receiveMessage();
    }

    public void setState(PhoneState state) {
        this.state = state;
    }
}
```

Model-View-Controller (MVC)

MVC è un patter architetturale tra i più usati nel mondo, solitamente per lo sviluppo di interfacce grafiche, che divide la logica del programma in tre elementi interconnessi:

- **Model**: i dati interni al programma. A volte, se lo stato persiste, e quindi i dati conservano lo stato oltre la singola esecuzione, questo può essere salvato su un database.
- **View**: l'interfaccia che si occupa dell'interazione con l'utente.
- **Controller**: il software che lega i due livelli.

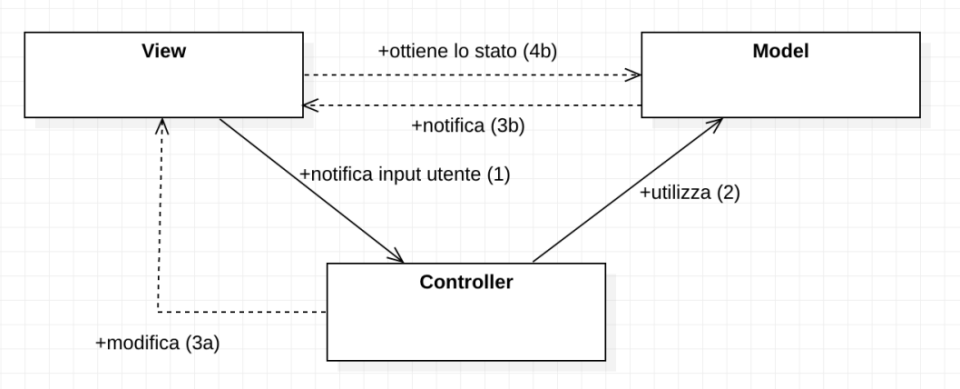
Come detto, MVC divide la parte **logica dei dati** e quella **dell'interazione** con l'utente seguendo il principio di **singola responsabilità**: ogni componente/classe deve svolgere una ed una sola funzione. Questo serve soprattutto perché i due livelli spesso operano su livelli di astrazione diversi, vengono sviluppati da team di sviluppo diversi e seguono una differente frequenza di aggiornamento. Tutti fattori che giovano, soprattutto su grande scala, della modularità di questo approccio che permette di aumentare la **manutenibilità** e **ridurre le interdipendenze**.

Va inoltre sottolineato che, dato il grande utilizzo del MVC, ne esistono diverse declinazioni e forme utilizzate in una grande varietà di contesti. Essi cambiano per terminologia e per forma ma mai per il principio di fondo che rimane sempre quello della separazione del Model e della View.

Per implementare il pattern MVC avremo bisogno di dividere i tre livelli attraverso la creazione di tre componenti:

- **Model**: una classe contenente lo stato e la logica applicativa e priva di dipendenze con gli altri componenti
- **View**: una classe contenente la logica di visualizzazione del Model con il quale può avere un riferimento
- **Controller**: una classe che fa da mediatore tra i due. Il Controller riceve gli input dalla View e coordina le interazioni tra la View e il Model. La sua responsabilità principale è interpretare gli input utente, tradurli in azioni sul Model e aggiornare la View di conseguenza. È privo di logica di specifica e interagisce con il Model attraverso un'interfaccia ben definita, mantenendo così la separazione delle responsabilità tra le tre componenti del pattern MVC.

Struttura generica del MVC



La struttura può avere delle varianti, notiamo infatti che il modello può avere:

- Controller con un riferimento a View che gli permette di modificare lo stato del modello e subito dopo modificare la View in base al nuovo stato (3a).
- Implementare un Observable(Model) – Observer(View) pattern dove al variare dello stato del Model, la View iscritta al Model cambi il suo stato di conseguenza (caso 3b – 4b)

Quale delle due implementare dipende molto dal peso che lo sviluppatore vuole dare alla View. Nel caso in cui si preferisca renderla un componente più **passivo**, quindi più facile da essere riutilizzato, allora è preferibile utilizzare il caso 3a dove questa ha solo un riferimento a Controller ed è completamente slegata da Model. Altrimenti, se la View è **specifica** per quel determinato modello, e quindi la riutilizzabilità non gioca un ruolo fondamentale, è preferibile il caso 3b – 4b dove la logica è distribuita su più classi, riducendo la complessità del Controller.

Esercizio – The music player

In questo esercizio l'obiettivo è quello di implementare la possibilità di passare alla prossima canzone tramite un metodo Skip e di aggiornare la View in modo tale che il cambiamento venga notificato e sia visibile all'utente.

L'elemento fondamentale è la classe **Song** che dispone della variabile privata **title** e dei metodi constructor della classe e **getTitle()**.

Song.java

```
package it.polimi.softeng.pattern_solutions.mvc;


public class Song {
    private final String title;


    public Song(String title) {
        this.title = title;
    }


    public String getTitle() {
```

```
        return title;
    }
}
```

Si crea una classe MusicLibraryModel (il nostro model) con il constructor e soprattutto il metodo getNextSong() che salva in result il titolo della canzone successiva, imposta nextSong alla canzone successiva a result nella lista e ritorna result, in questa classe si svolgono i calcoli i quali risultati saranno utilizzati quando la funzione onClick() della classe MusicAppController verrà chiamata per mostrare all'utente la nuova View.

MusicLibraryModel.java

```
package it.polimi.softeng.pattern_solutions.mvc;

import java.util.List;

public class MusicLibraryModel {
    private final List<Song> songs;
    private int nextSong;

    public MusicLibraryModel(List<Song> songs) {
        this.songs = songs;
        this.nextSong = 0;
    }

    public Song getNextSong() {
        Song result = songs.get(nextSong);
        nextSong = (nextSong+1) % songs.size();
        return result;
    }
}
```

A questo punto rimane solo gestire lo skip e il mostrare i cambiamenti nella classe MusicAppView. Quindi, si implementano i metodi **clickNext()** che funziona da notifica per tutti gli observers iscritti, i

quali chiamano il metodo **onClick(),** la tecnica Observer è utilizzata per sincronizzare l'aggiornamento di tutte le componenti iscritte, permettendo di chiamare la funzione onClick() senza dover istanziare tutte le classi che dovrebbero aggiornarsi nella classe MusicAppView.

MusicAppView.java

```
package it.polimi.softeng.pattern_solutions.mvc;

import java.util.ArrayList;
import java.util.List;

public class MusicAppView {
    private final List<ClickObserver> observerList = new ArrayList<>();

    public void addClickObserver(ClickObserver clickObserver) {
        observerList.add(clickObserver);
    }
    public void clickNext() {
        for (ClickObserver clickObserver : observerList) {
            clickObserver.onClick();
        }
    }

    public void updateCurrentSong(Song s) {
        System.out.println(s.getTitle());
    }
}
```

In seguito alla chiamata ClickNext() da parte della classe MusicAppView, la classe MusicAppController che implementa l'interfaccia observer ClickObservers utilizza il metodo onClick() e tramite le reference a model e view aggiorna quest'ultime.

MusicAppController.java

```
package it.polimi.softeng.pattern_solutions.mvc;



public class MusicAppController implements ClickObserver {
    private final MusicLibraryModel model;
    private final MusicAppView view;



    public MusicAppController(MusicLibraryModel model, MusicAppView view) {
        this.model = model;
        this.view = view;
    }



    @Override
    public void onClick() {
        Song newSong = model.getNextSong();
        view.updateCurrentSong(newSong);
    }
}
```

Lezione 17: Specifiche procedure (13/11/2023 Federico Bulloni, Simone Delbuono)

Specifica di astrazioni procedurali:

La specifica è un'**astrazione** di un frammento di codice che descrive in maniera rigorosa cosa fa, a prescindere dall'implementazione di questo frammento di codice. Essendo un'astrazione, la specifica formalizza "cosa fa" il metodo senza indicare "come" un metodo affronta il problema, ossia l'implementazione.

Il concetto di specifica è quindi strettamente legato all'approccio Object Oriented, dove l'interfaccia pubblica (CHE COSA) è separato dal metodo/oggetto prodotto dallo sviluppatore (COME).

Alcuni vantaggi della specifica sono:

- Località: per esempio, per capire come funziona il codice di una libreria basta leggere la specifica senza andare ad esaminare l'implementazione. Questo comporta maggiore

indipendenza dei programmatori che devono semplicemente mettersi d'accordo sulla specifica.

- Modificabilità: diverse implementazioni della specifica non impattano sulle altre specifiche che la "usano". Questo rende la manutenibilità del codice semplice ed economica.

Astrazione procedurale

Specifica di una procedura (funzione) che prende in ingresso un input ed emette un output senza dipendere dallo stato specifico dell'oggetto sul quale viene chiamato. In Java le procedure sopra descritte sono i **metodi statici** (di utilità) poiché questi non dipendono dallo stato (attributi) di un oggetto.

Esempio - valore assoluto

```
// Implementazioni diverse che rispondono alla STESSA SPECIFICA.
public static void int abs(int p) {
   if (p<0) return –p;
   return p;
}

public static void int abs(int p) { return (p<0) ? -p : p; }

public static void int abs(int p) { return p * sgn(p); }
```

JML e Programmazione per Contratto

Il linguaggio rigoroso e formale che utilizziamo per le specifiche delle astrazioni procedurali è **JML** (Java Modeling Language). Si basa su un modello di **programmazione per contratto,** che è un modo di progettare il software basato sulla metafora del contratto legale.

Quando si specifica un modulo viene "virtualmente" firmato un contratto tra:

- L'utilizzatore che si impegna a rispettare delle condizioni sui parametri sotto le quali la specifica è valida (VINCOLI SUL CHIAMANTE: **requires** o **precondizione**).
- Chi scrive il modulo garantisce che alla fine dell'esecuzione dell'astrazione ho il risultato desiderato dalla specifica (VINCOLI SUL FORNITORE: **ensures** o **postcondizione**) sotto l'ipotesi che la precondizione sia verificata.

Quindi per ciascun metodo va definito cosa richiede (**requires**) e cosa assicura (**ensures**)

Un primo esempio di contratto per il software

```
/*@ requires x >= 0.0                          //argomento del metodo >= 0
* @ ensures (x- \result * \result) < eps;     //radice approssimata corretta
*/

public static double sqrt(double x){}
```

Il **cliente** ha l'obbligo di passare un parametro non negativo e il diritto di ottenere una radice quadrata approssimata. Il **fornitore** ha l'obbligo di ritornare la radice quadrata e il diritto di ricevere argomenti non negativi.

Usare dei contratti come documentazione ha diversi vantaggi:

- Possono essere soddisfatti in vari modi --> diverse implementazioni per una stessa specifica.

- Non sono necessariamente operazionali: non è codice eseguibile ma formule logiche
- Sono verificabili meccanicamente: tramite teoremi di correttezza della logica posso testare una specifica (molto utile negli ambienti *mission critical* dove il codice non deve fallire).
- Attribuzione delle responsabilità; è facile individuare chi ha commesso un errore in caso di bug:
    - Se la precondizione è violata => è colpa dell'utilizzatore
    - Se la postcondizione è violata => è colpa di chi ha scritto il modulo
- Evita inefficienti controlli difensivi: i controlli sui valori ammissibili per i parametri vengono delegati al chiamante grazie al meccanismo delle precondizioni (approfondito in seguito).

Pre- e Post- condizioni: un riassunto

- La precondizione (**requires**) di un metodo indica il dominio della funzione, ossia delle condizioni sui parametri sotto le quali la specifica definita è valida.
- La postcondizione normale (**ensures**) indica cosa deve essere vero quando il metodo ritorna normalmente (senza sollevare eccezioni).
- La postcondizione eccezionale (**signals**) indica cosa deve essere vero quando il metodo ritorna sollevando un'eccezione (il chiamante utilizzi la funzionalità nel DOMINIO VALIDO, quindi con precondizioni valide, ma venga sollevata un'eccezione)

Esempio – Specifica del metodo **sqrt** con precondizioni ed eccezioni

Si consideri il metodo **sqrt** visto precedentemente. Lo sviluppatore del metodo ha a disposizione due approcci diversi per realizzarne le specifiche:

1. Porre come precondizione x >= 0, assicurando al chiamante che per ogni valore che soddisfi tale richiesta la post-condizione sarà verificata (si restituirà la radice quadrata approssimata corretta). In questo caso, lo sviluppatore non si preoccupa di gestire eventuali errori dati da un valore di ingresso inaccettabile, in quanto la precondizione richiede al chiamante l'inserimento di uno specifico dominio di valori. Nel caso in cui la precondizione non venisse rispettata, il valore di ritorno non sarebbe definito.

Specifica del metodo sqrt, utilizzo della precondizione

```
/*@ requires x >= 0.0                          //pre-condizione
* @ ensures (x- \result * \result) < eps;      //post-condizione
*/

public static float sqrt(float x)
```

2. Non utilizzare nessuna precondizione, ma fornire una post-condizione eccezionale che specifichi il caso in cui il valore inserito sia negativo. In tal modo il dominio dei valori accettati in ingresso verrebbe esteso a tutti i valori, ma la normale esecuzione del metodo sarebbe assicurata solo per un sottoinsieme di questi stessi valori. Con la post-condizione eccezionale lo sviluppatore garantisce la corretta gestione degli errori tramite eccezioni fornite al chiamante.

Specifica del metodo sqrt, utilizzo della post-condizione eccezionale

```
/*@ ensures x>=0 && (x- \result * \result) < eps;  //post-condiz. (normale)
* @ signals (IllegalArgumentException e) x < 0;   //post-condiz. eccezionale
*/
```

```
public static float sqrt(float x)
```

Scelta di un Approccio: precondizioni o eccezioni?

Quello che resta da chiedersi è come stabilire quale dei due approcci sopra descritti sia più conveniente adottare.

Si tenga presente che l'introduzione di una post-condizione nelle specifiche richiede l'implementazione da parte dello sviluppatore di controlli sui valori. In molti casi, tali controlli hanno un costo computazionale non indifferente, che può addirittura superare quello dell'esecuzione della funzionalità stessa.

> ### Esempio – Binary Search
>
> Una **binary search** viene eseguita correttamente solo se l'array su cui viene effettuata è ordinato. L'utilizzo di un array ordinato è quindi condizione da inserire nelle specifiche del metodo **binary search.** Inserire tale condizione come post-condizione eccezionale richiederebbe allo sviluppatore di implementare un controllo sull'ordine dell'array, mentre inserirla come precondizione non necessiterebbe di controlli, in quanto lo sviluppatore assumerebbe di avere un array già ordinato. Si noti come il costo temporale di un controllo sull' ordine di un array (O(n)) sia maggiore di quello del solo binary search (O(log(n))); utilizzare una precondizione risulterebbe quindi molto più conveniente rispetto a generare eccezioni.

Approcci generali

Normalmente, laddove possibile, i metodi di interfaccia pubblici dovrebbero avere specifiche totali (prive di precondizioni). Essendo scritte esclusivamente nelle specifiche, infatti, le precondizioni rischierebbero di indurre in errore il chiamante. L'utilizzo delle post-condizioni eccezionali è molto più sicuro, oltre ad essere utile a raccogliere eccezioni nel testing (eccezioni che nel caso di specifiche parziali non verrebbero fornite e non informerebbero l'utente sugli errori generati).

Le funzioni private prevedono invece un ambito di visibilità ristretto, vengono utilizzate da un numero minore di sviluppatori (lo scope è più piccolo) ed è quindi lecito utilizzare specifiche parziali (con precondizioni).

JML (specifiche in logica proposizionale per procedure)

JML utilizza formule di logica proposizionale e del primo ordine.
Le specifiche JML sono contenute in annotazioni, rappresentate con la sintassi dei commenti

Sintassi di JML
```
//@ specifica single line
/*@ specifica multilinea */
```

Per il momento, il nostro approccio a JML consisterà nella scrittura di condizioni tramite proposizioni logiche elementari che vadano a descrivere procedure. In particolare, andremo ad utilizzare proposizioni semplici che predicano sui valori in ingresso e sui valori in uscita di una funzione.
Esempi di proposizioni:

- X = 0
- Y < 3
- \result > 0 \result rappresenta il valore di ritorno della funzione

Per comporre proposizioni semplici si utilizzano i connettivi logici della logica proposizionale, quali la negazione (!), la congiunzione (&&), la disgiunzione (||), l'implicazione (==>) e la doppia implicazione (<==>).
Oltre a questi, JML introduce il termine " **\old(E)** ", che rappresenta il pre-stato di un certo elemento E, cioè lo stato in cui E si trovava prima dell'invocazione del metodo.
In JML non è consentito utilizzare metodi modificatori di oggetti, poiché dipendenti dallo stato dell'oggetto, o modificatori di tale stato (non sono funzioni matematiche, il loro risultato può variare). È invece consentito l'uso di metodi accessori o di generiche librerie (come java.lang.Math).
Come già mostrato, la specifica di un metodo non richiede necessariamente l'utilizzo di precondizioni (non inserire la clausola requires equivale a scrivere '@requires true'); le post-condizioni invece devono essere specificate, in quanto predicano sul funzionamento del metodo.

Può capitare che funzioni complesse non siano completamente specificabili; nel caso lo sviluppatore non sia in grado di realizzare una specifica per un metodo o un suo aspetto, infatti, è buona norma almeno documentarlo utilizzando i commenti (possibilmente mantenendo una struttura simil-JML).
Una post-condizione JML che sia più debole di quella corretta, abbinata ad un commento, è spesso più utile del solo commento, perché permette di effettuare verifiche formali di correttezza logica (meno complete perché non riguardanti la totalità della funzione, ma comunque utili).

Assertions in Java

Le assertions sono formule booleane abilitabili in Java.
Si tratta di espressioni che devono essere verificate durante l'esecuzione del codice. Un'asserzione che a runtime risulta 'falsa', produce un'eccezione *'AssertionError'* che termina l'esecuzione. In sostanza, le asserzioni sono ciò che lo sviluppatore pensa che debba succedere in un determinato punto del codice e quale pensa che sia lo stato del sistema in quel momento. Un utilizzo efficace delle assertions prevede di inserirle sempre nel programma ed abilitarle nel caso in cui si riscontri un bug, fungono quindi da controlli efficaci sul comportamento atteso del codice. Le asserzioni sono perciò uno strumento utile a scrivere precondizioni, anche non in JML.

Sintassi delle Asserzioni

```
assert <espressione booleana>
```

Oggetti Modificabili

Come già detto, JML non consente l'uso di metodi modificatori. L'uso di tali metodi, infatti, trasformerebbe JML in un linguaggio di logica operazionale, mentre l'obiettivo è unicamente quello di predicare sui valori in ingresso e in uscita ad una funzione.
E' lecito invece utilizzare metodi osservatori, in quanto forniscono informazioni sullo stato del sistema senza modificarlo.

Esempio – utilizzo metodo osservatore nella specifica

```
//metodo che inverte lo stato di accensione di un'automobile

//@ ensures p.accesa() <==> !\old(p.accesa())
```

```
public static int invertiStato (Automobile p) {}
```

Assignable

La keyword "*assignable*" viene usata per segnalare che un parametro può essere modificato dalla funzione (side-effect). Se una funzione non ha side-effect si può scrivere "*@assignable \nothing*".

Esempio di assignable e specifica parziale tramite commenti:

```
//@ assignable a;
//@ ensures (* a è una permutazione del valore originale di a *) &&
//@ (* a è ordinato per valori crescenti *);
```

Signals: gestione delle eccezioni

Esempio signals: ricerca di un elemento in un array

```
//@ ensures x == a[\result];                    // post-condizione normale
//@ signals (NotFoundExeption e) (* x non è in a *); //post-cond.
eccezionale

public static int cerca(int x, int a[]) throws NotFoundException
```

Nell'esempio qui sopra, al termine dell'esecuzione della funzione si può essere in uno dei due casi:

- La post-condizione normale è vera (x == a[\result]) e nessuna eccezione viene lanciata.
- Viene lanciata un'eccezione e è vero che x non è presente in a[].

Può capitare che la condizione della post-condizione normale sia verificata e venga comunque lanciata un'eccezione. Questo può essere risolto ponendo la post-condizione normale in modo che neghi quella della post-condizione eccezionale: in questo caso il fatto che x non sia presente in a[] esclude la possibilità che x sia uguale ad a[i] (per ogni indice i), quindi non ci sono problemi.

Occorre includere una clausola *signals* per ogni eccezione che il metodo può lanciare (sia checked che uncked), altrimenti ogni eccezione non menzionata verrebbe lanciata in ogni caso (come scrivere una signals con post-condizione true).

Quantificatori in JML

Per parlare degli elementi di una collezione si possono usare i metodi pubblici che non hanno side-effect (non modificano lo stato dell'oggetto), come *equals*, *contains*, *containsAll*, *get*, *sublist*, …

Esempio di accesso ad elementi di una collezione

```
//@ ensures a.containsAll(\old(a)) && \old(a).containsAll
//@ && (* a[] è ordinato per valori crescenti *)

public static void sort(List<Integer> a)
```

Convenzione: se un elemento x è di tipo *riferimento* (vettori, array, …), allora **\old(x)** è un riferimento all'oggetto nello stato al momento precedente alla chiamata del metodo.

Oltre a metodi che accedono alle collezioni, JML supporta diversi quantificatori:

- Universale: indicato da "**\forall**" e la seguente sintassi (\forall variabile; range; condizione)
  Per tutti i valori della variabile che soddisfano range, condizione deve essere vera.
- Esistenziale: indicato da "**\exists**" e la seguente sintassi (\exists variabile; range; condizione)
  Esiste almeno una variabile che soddisfa range per la quale condizione deve essere vera
- Quantificatore numerico: indicato da "**\num_of**" e la sintassi (\num_of int i; P(i); Q(i))
  Il numero totale (cardinalità) della variabile i per cui vale sia P(i) che Q(i).

- Funzioni quantificative: come i quantificatori sopra ma con un termine al posto della condizione
    - Sommatorie ("**\sum**") e produttorie ("**\product**") che rispettivamente sommano e moltiplicano tutti i valori del termine per i quali la variabile soddisfa il range.

      La sintassi è la stessa per entrambi: (\sum variabile; range; termine;)
    - Massimo ("**\max**") e minimo ("**\min**") che calcolano rispettivamente il massimo e il minimo valore assumibile dal termine per il quale la variabile soddisfa il range.

      La sintassi è la stessa per entrambi (\max variabile; range; termine;)

Esempio – utilizzo di quantificatori e funzioni quantificative

```
//Quantificatore universale: a[] ordinato per valori crescenti
//@ ensures (\forall int i; 0<=i && i<a.length-1; a[i]<=a[i+1]);

//Quantificatore esistenziale: ricerca di un elemento x in un array a[]
//@ ensures (\exists int i; 0<=i && i<a.length; a[i]==x);

//Quantificatore numerico: numero di elementi positivi in un array a[]
//@ ensures (\num_of int i; 0<=I && i<a.length; a[i]>0);

//Sommatoria e produttoria: somma e prodotto degli elementi di un array a[]
//@ ensures (\sum int i; 0<=i && i<a.length; a[i]);
//@ ensures (\product int i; 0<=i && i<a.length; a[i]);

//Massimo e minimo: massimo e minimo degli elementi di un array a[]
//@ ensures (\max int i; 0<=i && i<a.length; a[i]);
//@ ensures (\min int i; 0<=i && i<a.length; a[i]);
```

Esercizio – Specifica del metodo **sort**

Specifica del metodo sort

```
//@ requires a!=null &&
//@ (* non ci sono elementi ripetuti in a *)
//@ ensures
//@ (* a è una permutazione del valore iniziale *)
//@ && (* a ordinato per valori crescenti *);

public static void sort(int[] a) {}
```

Analizziamo il metodo sort e passiamo dal linguaggio naturale alle formule logiche.

Sotto l'ipotesi della non ripetizione degli elementi:

- L'array fornito in ingresso non deve contenere elementi ripetuti. Quindi, per ogni elemento presente nell'array, tutti gli altri elementi devono essere diversi da esso.
- L'array che viene fornito in uscita è lo stesso array ricevuto in ingresso, ma ordinato. Ciò equivale a dire che il secondo array è una permutazione del primo ed è ordinato con valori crescenti (ogni elemento è minore del successivo).
- Sulla base dell'assunzione che l'array non abbia elementi ripetuti, dire che il secondo array è permutazione del primo equivale a dire che tutti gli elementi inizialmente in a sono alla fine in a, e che per ogni elemento di a esiste un elemento uguale inizialmente presente in a.

Specifica del metodo sort - da linguaggio naturale a formule logiche

```
(*non ci sono elementi ripetuti in a*)
<==>
(per ogni elemento in a non si ha un elemento uguale ad esso)
<==>
(\forall int i; 0<=i && i<a.length;
    (\forall int j; i<j && j<a.length;
        a[i] != a[j]))

(* a ordinato per valori crescenti *)
<==>
(ogni elemento in a è minore del successivo)
<==>
(\forall int i; 0<=i && i<a.length-1;
    a[i] <= a[i+1])

(*a è una permutazione del valore iniziale*)
<==>
(\forall int i; 0<=i && i<a.length;
    (\exists j; 0<=j && j<a.length; a[i]==\old(a[j]))
    &&
    (\exists h; 0<=h && h<a.length; \old(a[i])==a[h])
)

<==>(formula equivalente)
(a.length == \old(a.length))
&&
(\forall int i; 0<=i && i<a.length;
    (\exists j; 0<=j && j<a.length; a[i]==\old(a[j])))
```

Specifica del metodo sort (senza ripetizioni)

```
/*@ requires a != null &&
  @ (\forall int i; 0<=i && i<a.length;
  @     (\forall int j; i<j && j<a.length; a[i] != a[j]))
  @
  @ ensures
  @ (\forall int i; 0<=i && i<a.length-1; a[i] <= a[i+1])
  @ &&
  @ (\forall int i; 0<=i && i<a.length;
  @     (\exists j; 0<=j && j<a.length; a[i]==\old(a[j]))
  @     &&
  @     (\exists h; 0<=h && h<a.length; \old(a[i])==a[h])
  @ )
  @
  @ assignable a[*];
*/
public static void sort (int[] a) {}
```

Specifica del metodo sort con ripetizione degli elementi

Si supponga ora che l'array a possa contenere dei duplicati. Come cambia la specifica?

- Non è più necessaria la "requires" che richiede come input un array senza duplicati
- Non è più sufficiente assicurare che un elemento nell'array iniziale esista anche in quello finale e viceversa.
- Bisogna aggiungere la post-condizione secondo cui ogni elemento che compare nel "vecchio" array compare anche nel "nuovo" in ugual quantità, e viceversa. Questo significa che per ogni elemento del vecchio array (analogamente del nuovo), il numero di posizioni nel vecchio array che presentano valori uguali a quell'elemento devono essere uguali al numero di posizioni nel nuovo array che presentano valori uguali allo stesso elemento.

Specifica del metodo sort (con ripetizioni)

```
/*@ requires a != null
  @
  @ ensures
  @ (\forall int i; 0<=i && i<a.length-1; a[i] <= a[i+1])
  @ &&
  @ (a.length = \old(a.length))
  @ &&
  @ (\forall int i; 0<=i && i<a.length;
  @     (\num_of int j; 0<=j && j<a.length; a[i]==\old(a[j])) ==
  @     (\num_of int h; 0<=h && h<a.length; a[i]==a[h])
  @ )
  @
  @ assignable a[*];
*/
public static void sort (int[] a) {}
```

Si noti il fatto che senza "assignable a[*]" la specifica sarebbe incorretta, perché il nuovo array potrebbe avere un numero maggiore di elementi del vecchio e non contraddire la post-condizione. Gli elementi in aggiunta al nuovo array non verrebbero "considerati" nel \forall perché in posizioni maggiori della lunghezza del vecchio array. "assignable a[*]" risolve il problema, perché impedisce la modifica dell'array e quindi della sua lunghezza, mentre permette solo la modifica dei suoi elementi.

Lezione 18: Specifiche ADT (16/11/2023 Diaferio Andrea, Emanuele Caruso)

Introduzione

Nella Lezione di oggi ci spostiamo dalle specifiche di astrazioni su metodi a quelle sui dati. Il nostro obiettivo sarà di specificare le astrazioni sui dati, ovvero classi e oggetti.

Partiamo subito con un esempio.

Esempio: metodo `add()` di un elemento in una lista.

- accede allo stato attuale della lista e lo modifica aggiungendo un elemento.

Ci occuperemo, quindi, di specificare il comportamento di un oggetto.

Specifiche delle astrazioni sui dati

Si vuole descrivere un nuovo tipo di dato indicandone valori e operazioni possibili; per fare questo si usa un tipo di dato astratto, ADT (abstract data type).

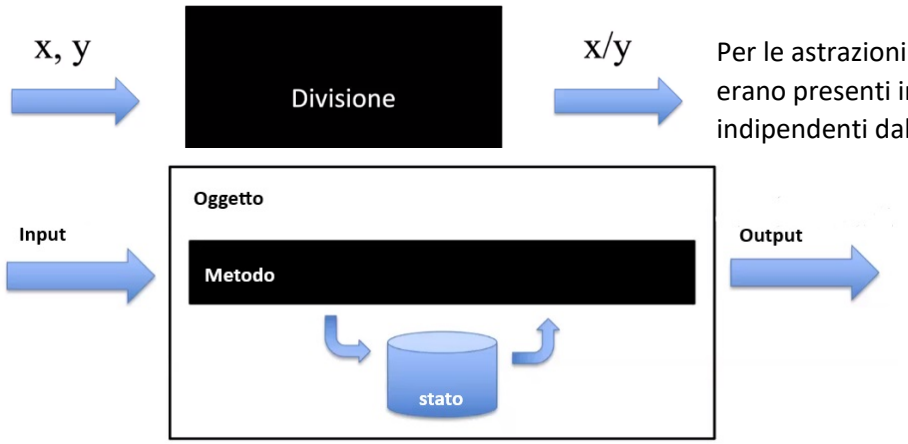
***Astraendo dai dettagli di rappresentazione dei valori e di implementazione delle operazioni, il resto del programma dipende solamente dalla specifica del tipo.*

L'ADT è "astratto" perché fornisce una visione indipendente dall'implementazione. L'utente del tipo di dato non ha bisogno di sapere come quel tipo di dato sia implementato

Sintassi e Semantica

La signature di un metodo definisce **solo** la sintassi.

Per quanto riguarda la semantica non basta più quanto visto nella lezione precedente:



Per le astrazioni procedurali infatti erano presenti input e output, indipendenti dallo stato dell'oggetto

Negli ADT invece, i metodi agiscono anche sulle variabili di stato dell'oggetto (potendole modificare)

JML: Visibilità e Metodi Puri

Analizziamo adesso come scrivere le nostre specifiche in JML.

All'interno delle pre- e post- condizioni (requires & ensures) di un metodo pubblico può apparire tutto ciò che è pubblico del metodo stesso e della classe (metodi **osservatori**). Non sono invece ammessi metodi pubblici **non** dichiarati come **puri.**

Naturalmente le regole analizzate in precedenza sulle specifiche dei metodi statici sono sempre valide.

Quando un metodo è definito "Puro" ?

Un metodo viene detto puro se non ha effetti collaterali, ovvero non modifica lo stato di un oggetto:

- Per questi "assignable" vale \nothing (si continuano a trattare formule logiche anche a seguito di una loro chiamata)
- Si preferisce usare la notazione ***/@Pure@/*** perché consente di richiamare il metodo in JML
- Un metodo puro può chiamare solo altri metodi puri
- In JML, i **metodi osservatori** sono definiti **come** metodi **puri**.

Anche i costruttori possono essere dichiarati come "pure". Questo implica che sono in grado di inizializzare gli attributi dichiarati nella classe, ma non possono modificare nient'altro.

Es: Il metodo `size()` che restituisce il numero di elementi di un contenitore è puro!

Per delineare un metodo puro si utilizza la notazione:

Notazione pure method

```
//@ ensures (*\result è cardinalità di this *)
public int /*@ pure @*/ size() { … }
```

Per poter osservare un dato oggetto, ci serviamo dei metodi osservatori (che vengono trattati al paragrafo seguente), e non di metodi privati.

Quello che è possibile fare per mezzo di pre- e post-condizioni è di predicare sull'input, sull'output e, in aggiunta, anche su tutti i metodi pubblici che permettono di osservare lo stato dell'oggetto dall'esterno.

Riprendiamo ora l'esempio dell'introduzione, l'add di una lista (questo metodo non è puro. È MODIFICATORE !). Per scrivere la sua specifica è possibile usare i metodi puri definiti nella classe.

- Un possibile approccio è tramite size(), un metodo puro che ritorna la dimensione della lista.

Infatti, prima di chiamare il metodo add, la size della lista è più piccola di 1 elemento rispetto a DOPO L' INVOCAZIONE della add():

```
size = old(size)+1
```

RICORDA SEMPRE: Una buona specifica consente di capire il funzionamento di un oggetto senza conoscerne i dettagli di implementazione.

Categorie di Metodi

- **Creatori:** producono nuovi oggetti
  - sono tutti dei costruttori
  - non necessariamente vale il contrario (se hanno oggetti del proprio tipo come argomenti sono classificati come produttori)
- **Produttori:** generano altri oggetti del proprio tipo (utilizzati per generare oggetti immutabili (esempio unione di due insiemi: un metodo produttore crea un nuovo insieme che è l'unione die due iniziali, anziché modificare l'oggetto di partenza))
- **Modificatori:** modificano l'oggetto su cui sono stati invocati
- **Osservatori:** restituiscono risultati di altri tipi (tipicamente quelli dichiarati puri in JML)

Mutabilità e Immutabilità

Un oggetto si dice mutabile se ha dei metodi modificatori (oltre a metodi osservatori), altrimenti viene detto **Immutabile**

La **mutabilità** è una **proprietà** che sussiste a livello **della specifica**, e non dell'implementazione. Il metodo è modificatore già nella specifica (quindi se è presente l'oggetto è mutabile).

Come decido se è mutabile??

La mutabilità è utile maggiormente per oggetti che modellano entità nel mondo reale (ad esempio auto, persone …) in generale oggetti che cambiano dinamicamente.

Viene spesso utilizzato per contenitori come array, insiemi …

Per essere ben formata, una classe, se mutabile, deve avere sia metodi osservatori che metodi trasformatori.

Esempio di dati mutabili: IntSet

- **Tipo:** "insieme di numeri interi" (IntSet)
- **Valori possibili**:

    - tutti gli insiemi finiti di numeri interi (di cardinalità qualsiasi) (NB: un insieme non contiene elementi ripetuti)
    - es: insieme vuoto, {1,10,35}, {7}, {50,-1,48,18,33,6,15,1000}
- **Operazioni** possibili su un dato insieme I
    - costruzione: crea I e lo inizializza come insieme vuoto
    - `void insert(int x)`: aggiunge x agli elementi di I
    - `void remove(int x)`: elimina x da I
    - `boolean isIn(int x)`: dice se x appartiene ad I o meno
    - `int size()`: restituisce la cardinalità di I
    - `int choose()`: ritorna un elemento qualsiasi di I

Come si può scrivere la specifica per queste operazioni?

In primis, notiamo che sono presenti sia metodi osservatori: `isIn(), size(), choose()`, sia metodi modificatori: `insert(), remove()`(classe mutabile).

IntSet.java

```
public class IntSet {
      // Insiemi di interi illimitati e modificabili;
      // Esempio: {1, 2, 10, -55}
      // (questo sopra esula da JML, è cosa buona aggiungere i
      // commenti descrittivi, ma non è necessario)

      // Osservatori: (JML)
      //@ ensures (\result == true) <==>
      //@ (*x è fra gli elementi di this*);
      public /*@ pure @*/ boolean isIn (int x){}

      //@ ensures (\result è cardinalità di this);
      public /*@ pure @*/ int size(){}

      //@ ensures this.isIn(\result) && this.size()>0;
      //@ signals (EmptyException) this.size()==0;
      public /*@ pure @*/ int choose() throws EmptyException { }
}
```

Nel nostro esempio non sono necessarie precondizioni:

- Se avessimo avuto `Integer x,` anziché `int x,` non sarebbe stata una funzione totale e quindi avremmo dovuto specificare la non nullità di x.

Dal momento che la nostra funzione è totale, non sono presenti eccezioni => non ho bisogno di specificare alcuna \signals

Non è possibile fare molto meglio di così per metodi puri…

Per questo motivo ci saranno dei metodi, chiamati "**ground**" (che verranno assunti veri), sui quali ci si baserà per definire tutti gli altri. Il nostro obbiettivo sarà quello di avere il minor numero possibile

di suddetti metodi (che sono definiti in linguaggio naturale) e che questi siano facili da definire, per non lasciare spazio ad ambiguità.

Se analizziamo il metodo `Size()` ,ad esempio, ci rendiamo conto che non è un metodo ground. È infatti possibile definire la cardinalità a partire da `isIn()`, sfruttando la funzione `num_of` di JML per contare le occorrenze usando la seguente notazione:

Esempio di specifica per il metodo size()

```
//@ \result == \num_of(int i; ; isIn(i));
```

La notazione "`; ;`" indica che non sono presenti restrizioni sul range => si prendono in considerazione tutti gli interi.

Ma allora ci si potrebbe chiedere: *"Non è controproducente una scrittura della specifica fatta così ??, perché dovrei implementarla in questo modo??"*

In realtà questa è solo una descrizione del comportamento della classe, l'implementazione è libera, e dunque chi di dovere potrà fare nel modo che preferirà.

La presenza di un'eccezione non implica la necessità di avere precondizioni, anzi spesso le due sono usate in mutua esclusione: la prima lascia il compito di verificare la condizione eccezionale al *chiamato*, la seconda al *chiamante*.

Nel nostro caso, non mettendo alcuna (pre-) condizione sulle dimensioni dell'insieme, quando andiamo a definire le post- condizioni dobbiamo specificare sia il comportamento atteso *normalmente*, sia il comportamento *eccezionale* (caso in cui l'insieme passato sia vuoto).

Se osserviamo la specifica di `choose()`:

Specifica del metodo choose()

```
//@ ensures this.isIn(\result) && this.size()>0;
//@ signals (EmptyException) this.size()==0;
public /*@ pure @*/ int choose() throws EmptyException { }
```

La seconda condizione nell'*ensures* implica *"l'altrimenti"* che invoca l'eccezione (infatti le post condizioni normali e quelle in eccezione sono in ***OR*** fra loro)

Costruttori e Modificatori della classe IntSet:

Costruttori:

```
//@ ensures this.size()==0;
```

```java
public IntSet() {
    /* inizializza this come insieme vuoto */
}
```

Modificatori:

```java
//@ ensures this.isIn(x) &&
//@ (\forall int y; x!=y; \old(isIn(y)) <==> isIn(y));
public void insert(int x) {
    /* inserisce x in this */
}

//@ ensures !this.isIn(x) &&
//@ (\forall int y; x!=y; \old(isIn(y)) <==> isIn(y));
public void remove(int x) {
    /* rimuove x da this */
}
```

Come possiamo specificare il metodo `insert()` ?

- Scegliamo come metodo ground sempre `isIn()`.
    - È facile da descrivere in linguaggio naturale e non lascia spazio ad ambiguità
- Non avendo restrizioni sul parametro che passiamo al metodo non sono presenti precondizioni
- Non sono presenti comportamenti eccezionali => no signals
- Procediamo col definire le post condizioni

Attenzione! In prima battuta si potrebbe pensare di usare `size()`, sempre definito a partire da `isIn()`, ragionando sul fatto che, a seguito di un inserimento, la dimensione finale debba essere maggiore di 1 rispetto a quella iniziale.
Questo però non è sempre vero: stiamo infatti lavorando con set di interi, quindi in caso di richiesta di inserimento di un elemento già presente, questo non viene aggiunto e di conseguenza la dimensione dell'insieme resta invariata!

È possibile usare più metodi puri all'interno della specifica, anche se questi non sono ground, a patto che siano definiti a partire da metodi ground e che la catena di dipendenze delle definizioni abbia una struttura ad albero (Es: `insert` definita da `size`, definita a sua volta da `isIn`) e non di un grafo (non si vogliono strutture cicliche. Es: `insert` definita da `size`, che è definita da `insert`).
Nel nostro caso è possibile usare `size()` per scrivere la specifica, ma questo rende le cose più complesse.

Un miglior approccio può essere invece quello di ragionare su cosa si vuole ottenere a seguito della chiamata del metodo:

- Banalmente se un elemento manca, a seguito della chiamata del metodo l'elemento sarà presente,
- Se l'elemento è già presente, l'elemento resta presente.

Es: `insert (4);`

Se ci limitiamo alla seguente specifica,

ensures per la insert()

```
//@ ensures this.inIn(x)
```

dato l'insieme di partenza I = {1,2,3}, a seguito dell'invocazione di `insert (4);` abbiamo che

{1,2,3} ---> {1,2,4}

Notiamo però che l'elemento "3" non è più presente all'interno dell'insieme. Per questo bisogna aggiungere che tutto quello che è presente nel vecchio insieme venga preservato.

Un modo possibile di scriverlo potrebbe essere:

ensures per la insert()

```
//@ ensures this.inIn(x) &&
//@ \forall(int i; i!=x; isIn(i) <==> \old(isIn(i)) )
```

Errori comuni:

`&& \old(isIn(x))` Questa implementazione non è corretta: implica infatti che l'elemento *x* NON possa già essere presente all'intero del nostro insieme I. Noi sappiamo però che il caso appena descritto è accettato, e semplicemente, a fronte di un'invocazione di `insert()` su tale elemento, l'insieme di partenza e l'insieme risultato coincideranno.

`\forall(int i; \old(isIn(i)); isIn(i))` Altro errore tipico è la sostituzione del blocco \forall di sopra con questo! Non va bene perché si è tolta la doppia implicazione; così facendo si è imposto che ci siano sempre gli elementi vecchi, senza dare alcuna restrizione sugli elementi che vengono aggiunti. Es: {1,2,3} insert (4) *NON* posso avere {1,2,3,4,**5**}

metodo remove()

```
public void remove(int x) {
    /* rimuove x da this */
}
```

È come per l'aggiunta: tutto quello che c'era prima dell'invocazione del metodo deve essere presente anche in seguito all'esecuzione, a meno dell'elemento da togliere.

Specifiche per la remove()

```
//@ ensures !this.isIn(x) &&
//@ (\forall int i; i!=x; \old(isIn(i)) <==> isIn(i))
```

Utilizzo degli ADT

La specifica deve essere sufficiente per poter usare l'astrazione senza conoscerne l'implementazione. Si deve quindi essere sicuri del comportamento dell'oggetto.

Es: guardando la specifica

specifica di IntSet

```
//@ ensures
//@ (*restituisce IntSet con tutti e soli gli elementi in a*);
 public static IntSet getArray (int[] a) throws NullPointerException {}
```

ne segue l'implementazione banale

IntSet.java

```
// Dalla specifica, s diventa insieme vuoto
IntSet s = new IntSet();
 for (int i=0; i<a.length; i++) {
      // Da specifica, a[i] è inserito in s
      s.insert(a[i]);
}
return s;
```

Esempio di struttura immutabile: Poly

- Tipo: "Polinomi a coefficienti interi" (Poly)
- Esempio di possibile valore: 1+2x+3x^3-2x^4...
- Costruzione: come costruire un polinomio?
    - Conviene partire dal polinomio più semplice: il monomio
    - Monomio ha coeff intero e grado >=0
- I polinomi si ottengono poi combinando monomi con + - *
- Costruzione: crea monomio di grado e coefficiente dati
- Come caso particolare, il polinomio zero (grado 0 coeff 0)

- `public int degree()`: restituisce il grado del polinomio
- `public int coeff(int d)` restituisce il coefficiente del termine di grado d
- `public Poly add(Poly q)` restituisce la somma di this e q
- `public Poly sub(Poly q)` restituisce la differenza di this e q
- `public Poly mul(Poly q)` restituisce il prodotto di this e q

Se si osserva la signature dei metodi, si scopre che gli ultimi tre sono dei produttori (non cambiano direttamente il polinomio ma ne creano uno nuovo), questo implica che la classe Poly è **immutabile.**

Guardando ora la specifica dei metodi

Specifica di Poly

```
  /*@ pure @*/ public class Poly{

//Osservatori:
//@ ensures (*\result è grado del polinomio*);
public int degree( ){}

//@ requires d> =0;
//@ ensures (d>degree() ? \result ==0 : (*\result è coeff del termine di
grado d *));
public int coeff(int d){}
```

Questi due sono i nostri metodi “ground”. È possibile notarlo dal fatto che parte della specifica è scritta in linguaggio naturale:

- `degree()` ritorna il grado del polinomio
- `coeff()` restituisce il coefficiente del termine di grado d
  - È necessario che il grado sia maggiore o uguale a zero (Se ad esempio prendessimo in ingresso –2, il metodo non funzionerebbe perché il comportamento non è definito…)

Costruttori della classe Poly

```
  //Costruttori:
//@ ensures this.degree()==0 && this.coeff(0)==0;
public Poly() {} //costruisce il polinomio zero

//@ ensures n>=0 && this.coeff(n)==c && this.degree()==n &&
//@ (\forall int i; 0<=i && i<this.degree(); this.coeff(i) == 0);
//@ signals(NegativeExponentException e) n<0;
public Poly(int c, int n) throws NegativeExponentException{}
//costruisce un monomio
```

Per definire i due costruttori sono stati usati solo i metodi ground.

Si analizza ora nel dettaglio la stesura di una singola specifica, evidenziandone i processi logici che è buona norma seguire:

- Se presenti eccezioni, è bene partire da questa perché più semplice

Specifica Poly(int c, int n)

```
//@ signals(negativeExponentException e) n<0
```

- Il passo successivo è assicurarsi che l'ensures che stiamo per scrivere sia disgiunta dall'exception, quindi si esprime la negazione dell'eccezione e la si mette in && con il resto delle ensures:

```
//@ ensures n>=0 &&
```

- È importante ricordarsi di specificare, oltre a tutto quello che si vuole ottenere, anche tutto ciò che si vuole NON accada, e, in caso di metodi modificatori, cosa debba rimanere invariato.

```
//@ ensures n>=0 && this.coeff(n)==c && this.degree()==n
```

Se ci limitassimo a quanto scritto fino ad ora, il polinomio 3x^2+4x+2 verrebbe accettato, non si è infatti detto nulla riguardo il fatto che tutti glia altri coefficienti debbano essere zero. Così facendo però si andrebbe a costruire un polinomio e non il monomio che si voleva inizialmente. Si aggiunge allora un controllo sui coefficienti:

```
//@ ensures n>=0 && this.coeff(n)==c && this.degree()==n &&
//@ (\forall int i; 0<=i && i<this.degree(); this.coeff(i) == 0);
```

Andando ora a completare la parte commentata del metodo `add()`:

Specifica add(Poly q)

```
//@ ensures q != null && (* \result == this + q *);
//@ signals (NullPointerException e) q == null;
public Poly add(Poly q) throws NullPointerException {…}
```

- Come prima, si parte dall'eccezione

```
 //@ signals (NullPointerException e) q == null;
```

In alternativa alla signals, per NullPointerException, si sarebbe potuto usare una \requires

- Avendo descritto l'eccezione tramite signals, creo nelle \ensures una disgiunzione:

```
 //@ ensures q != null &&
```

- Si conclude poi con la stesura del resto delle \ensures

```
//@ \result.degree()==this.degree>other.degree ? this.degree() :
other.degree()
//@ (\forall int i; ;  \result.coeff(i)==this.coeff(i)+other.coeff(i))
```

Con questo si è detto che il grado è il massimo fra i due, e che il coefficiente risultato è dato dalla somma di `this.coeff` e `other.coeff`

Lezione 19: Specifiche ADT (20/11/2023 Caravaggio Myriam, Rida Abbas)

Nella lezione di oggi, continuiamo e concludiamo la spiegazione in merito alla specifica di astrazione sui dati.

Nell'ultima lezione abbiamo visto in particolare la specifica di tipi di dato astratto: con quest'ultimo si intendono le strutture dati che hanno uno stato e che sono potenzialmente mutabili, ovvero la specifica dei loro metodi, in questo caso, deve considerare, oltre a quello che prendono in ingresso e producono in uscita, anche lo stato a cui accedono e che potrebbero modificare. Come esempio abbiamo visto una IntSet, dove abbiamo visto che lo stato interno del set cambia nel tempo e perciò

quando dobbiamo specificare il suo comportamento a fronte dell'invocazione di un metodo, dobbiamo poter dire come cambia lo stato.

C'è da specificare però un problema a cui si potrebbe andare in contro: la specifica di un dato di tipo astratto potrebbe avere delle limitazioni. Ciò si può costatare attraverso un esempio, in particolare sul funzionamento di una pila. Prendiamo una pila ti tipo LIFO (last in, first out): l'ultimo elemento che si aggiunge ad essa è il primo ad essere preso quando eseguiamo il comando di rimozione di un elemento. I metodi pubblici che possiamo utilizzare nella gestione di una pila sono:

- **PUSH**: permette di aggiungere un elemento, il quale verrà messo in cima;

specifica push

```
//@ ensures size() == \old(size()+1) && top() == x &&
//@ (*elementi sotto top invariati*)
Public void push (T x) {...}
```

- **POP**: permette di togliere l'ultimo l'elemento (quello in cima);

specifica pop

```
//@ requires size() > 0;
//@ ensures size() == \old(size()-1) && (elimina elemento da cima della pila*)
Public void pop() {...}
```

- **TOP**: permette di osservare quale elemento c'è in cima;

specifica top

```
//@ requires size() > 0;
//@ ensures (* \result è cima della pila*)
Public /*@ pure @*/ T top ();
```

- **SIZE**: permette di conoscere il numero di elementi della pila considerata.

specifica size

```
//@ ensures (*\result è il numero di elementi in this*);
```

```
public /*@pure@*/ int size() {...}
```

La specifica della pila è comprensibile, ma non è completamente formalizzata: infatti i metodi osservatori non ci permettono di definire il resto della specifica in maniera esaustiva. Per esempio, prendiamo in analisi cosa succede all'invocazione della Push. Sicuramente potremo dire che l'elemento aggiunto è in cima, e che la dimensione della pila è aumentata, grazie ai due metodi puri Size() e Top(). Quello che però non riesco a dire è se la pila, ad esempio, è stata riorganizzata (quindi se è cambiata), oppure se andassi ad inserire NULL nella pila, avrei che la Size aumenta comunque. Il problema intrinseco di questo discorso è proprio che il sapere qual è la dimensione della pila o qual è l'elemento in cima non ci basta per specificare tutti i comportamenti che la pila potrebbe avere.

Oggetto astratto tipico

Come possiamo risolvere questi problemi? Abbiamo due diverse soluzioni:

1. Utilizzare i commenti in linguaggio naturale, che in JML non sono codificati ma vengono trattati come TRUE: questo metodo non risolve completamente il problema ma almeno permette di conoscere informazioni aggiuntive;
2. Definire, nella sola specifica, quello che viene chiamato "**oggetto astratto tipico**" (OAT): esso definisce in modo preciso l'oggetto astratto definito dalla specifica della classe. È sostanzialmente la più semplice implementazione possibile, solitamente è una lista, un set o una stringa.

Per chiarire questo concetto, supponiamo che l'implementazione di una pila sia data mediante una lista. Utilizziamo tale lista per definire tutto ciò che non riusciamo mediante i soli metodi pubblici della pila. In questa ipotesi, avremo un oggetto astratto tipico che è rappresentato da un insieme di elementi tali per cui ho zero o un numero maggiore di zero di elementi e l'n-esimo elemento è quello che ottengo attraverso Top (). A questo punto possiamo definire ciò che prima non potevamo grazie all'OAT: possiamo ad esempio specificare tutti i metodi puri, anche quelli Ground, perché ci stiamo basando su un'implementazione che è la più semplice possibile. Questa semplicità è necessaria poiché l'obiettivo è di documentare e specificare più cose possibili nel modo più chiaro possibile. Questo ci mostra come questo tipo di implementazione abbia dei requisiti diversi dall'implementazione concreta, che solitamente è più complessa e utilizza strutture dati più evolute.

Definendo la pila tramite una lista, ci accorgiamo che i risultati dei metodi pubblici puri utilizzati su lista, corrispondono a quelli che si ottengono utilizzandoli sulla pila: ad esempio, il metodo pubblico puro Size applicato alla lista produce lo stesso risultato della Size applicata alla pila.

Di seguito, riportiamo la specifica di pila con oggetto tipico.



```
public class Pila<T> {
  //@ spec_public List<T> oat;
  // Oggetto tipico OAT e'<x_1, x_2, ..., x_n>, n>=0, x_i in T, x_n è il top

  //@ ensures \result == oat.size();
  public /*@ pure @*/ int size(){}

  //@ requires size()>0;
  //@ ensures \result== oat.get(oat.size()-1);
  public /*@ pure @*/ T top() throw EmptyException;

  //@ ensures size()==0;
  public Pila() { .. }

  //@ ensures size()==\old(size()+1) && top()==x &&
  //@ \old(oat).equals(oat.sublist(0, oat.size()-2));
  public void push(T x) {...}

  //@ requires size()>0;
  //@ ensures size()==\old(size()-1) &&
  //@ oat.equals(\old(oat.sublist(0, oat.size()-2))
  public void pop(){...}
}
adt
```

Oggetto tipico per IntSet

```
Public class IntSet {
  //@ spec_public List<Integer> oat;

  //@ ensures \result == oat.contains (x);
  Public /*@ pure @*/ boolean isIn (int x) {...}

  //@ ensures \result == oat.size ();
  Public /*@ pure @*/ int size () {...}
}
```

Adeguatezza di un tipo

Un tipo si dice **adeguato** se fornisce operazioni sufficienti perché esso sia utilizzabile in modo semplice ed efficace. Per verificare l'adeguatezza di un tipo, possiamo farlo attraverso le seguenti azioni:

- Verificare che l'interfaccia non sia troppo piccola: questo perché sarebbe sconveniente da utilizzare.
- Per contro al punto precedente, l'interfaccia non dev'essere eccessivamente grande: questo perché le interfacce troppo grandi rischiano di rendere l'oggetto più complesso del dovuto; esso diverrebbe meno comprensibile e coeso. Inoltre, un'interfaccia troppo ampia andrebbe a vincolare l'implementatore a modifiche future: più metodi si aggiungono, più è probabile che questi possano in qualche modo forzare alcune scelte di implementazione, riducendo la libertà di noi implementatori.

- Verificare se l'oggetto è mutabile, quindi assicurarsi che esso abbia metodi creatori, osservatori e mutatori necessari.
- Al contrario, verificare se l'oggetto è immutabile, quindi se dispone di metodi creatori, osservatori e produttori necessari.
- Il tipo deve essere totalmente popolato (fully populated): utilizzando creatori, osservatori e mutatori/produttori (in base alla mutabilità/immutabilità dell'oggetto), deve essere possibile ottenere ogni possibile istanza di stato astratto.
- Anche se fully populated, verificare che si può migliorare l'efficienza fornendo ulteriori operazioni in modo ragionevole.
- Non bisogna includere troppe operazioni che non si adattino allo scopo del tipo: l'astrazione, infatti, potrebbe diventare più difficile da comprendere, implementare e mantenere.

In generale, l'adeguatezza è un concetto informale e dipendente dal contesto d'utilizzo: se il contesto è limitato, bastano poche operazioni.

Proprietà astratte

Le proprietà astratte sono proprietà osservabili, con i metodi observer. Esse si dividono in due categorie:

1. **Proprietà evolutiva**: si tratta di una relazione tra uno stato astratto osservabile e lo stato astratto osservabile successivo. Non sono proprietà esprimibili in JML, ma possono essere dedotte guardando alle proprietà invarianti (punto 2) e ai metodi pubblici che vengono chiamati: questo perché JML non ci permette di ragionare sull'evoluzione temporale. Ad esempio, supponiamo di avere un IntSet che non possiede il metodo remove. Una proprietà ovvia è il fatto che la size () cresce in modo monotono crescente, perché senza remove la size () non potrà mai diminuire. Questo però non è esprimibile in JML, se non attraverso la dimostrazione del teorema che ci dice che size continua ad aumentare.

2. **Proprietà invariante**: è una proprietà degli stati astratti osservabili, che ci dice cose sempre vere in tutti gli stati che posso osservare. Ad esempio, la size () di un IntSet è sempre ≥ 0, in qualunque stato si trovi il nostro oggetto, oppure il grado di un polinomio è sempre positivo. Queste proprietà sono rappresentabili in JML, il quale ci permette di predicare sullo stato di un oggetto attraverso la logica del primo ordine.

Invariati astratti

Ora ignoriamo l'implementazione e usiamo solo la specifica per dimostrare gli **invarianti astratti.** Questi ultimi sono la formalizzazione di una proprietà astratta, in particolare un abstract invariant è una condizione che deve sempre essere verificata per l'oggetto astratto, come ad esempio il fatto che la dimensione di un IntSet è sempre ≥ 0. In JML ci sono i public invariant, che possono utilizzare solo attributi e metodi pubblici della classe. Ad esempio:

specifica top

```
//@ public invariant this.size () > 0;

//@ public invariant this.degree () > 0;
```

Di seguito, un esempio su un invariante:

Triangolo

```
public /*@ pure @*/ class Triangolo {

    //@ ensures (* costruisce un triangolo non degenere *)
    //@ signals (* TriangoloInvalidoException e) vertici.length !=3 ||
    //@ (* i vertici [] costituiscono un triangolo degenere*);
    public class Triangolo (Punto vertici []) throws
                                    TriangoloInvalidoException

    //@ ensures (*restituisce la lunghezza del lato minore *)
    public float latoMinore ()

    //@ ensures (*restituisce la lunghezza del lato intermedio *)
    public float latoMedio ()

    //@ ensures (*restituisce la lunghezza del lato maggiore *)
    public float latoMaggiore ()
}
```

Il triangolo non è degenere: vale la disuguaglianza triangolare e i lati hanno lunghezza > 0.

Invariante astratto

```
//@ public invariant
//@ latoMaggiore () < latoMinore() + latoMedio () &&
//@ latoMinore() > 0 &&
//@ latoMedio() > 0&&
//@ latoMaggiore () > 0;
```

Le proprietà astratte sono molto utili, poiché gli utilizzatori della classe possono usarle come assunzioni sul comportamento di una classe. Alcuni esempi sono:

- Uno stack ha proprietà evolutiva LIFO;
- Un triangolo non è degenere;
- Un IntSet non ha size () negativa;
- Il grado di un Poli è sempre ≥ 0.

Per essere davvero utili, bisogna avere verificare che gli invarianti siano verificati, il che è possibile solo in base alla specifica.

Tornando all'esempio del triangolo, bisogna chiedersi: l'invariante triangolo è verificato dalla specifica? Per rispondere è necessario avere la specifica precisa del costruttore. Se il costruttore non consente di costruire un triangolo degenere, allora abbiamo la certezza che l'invariante vale, in quanto la classe è immutabile: una volta costruito, il triangolo non potrà essere modificato. Si può notare che questa verifica non utilizza l'implementazione, ma solo la specifica del costruttore; quindi tale "prova" potrebbe essere dimostrata prima di implementare, e se l'implementazione sarà corretta rispetto alla specifica, l'invariante resterà verificato anche sull'implementazione.

Implementazione delle Astrazioni

Definiamo la differenza tra oggetto astratto e oggetto concreto:

- La specifica di una astrazione descrive un **Oggetto Astratto**
- L'implementazione di una astrazione descrive un **Oggetto Concreto**

Es: (**IntSet**)

- L'oggetto astratto è un "Insieme di interi"
- L'oggetto concreto, utilizzato per implementare la classe *Intset* , potrebbe assumere diverse forme , come un ArrayList , un albero binario o un'altra struttura dati.

Es: (**Poly**)

- L'oggetto astratto è un polinomio
- L'oggetto concreto, utilizzato per implementare la classe Poly , può assumere varie forme , tra cui array , una lista delle adiacenze o un'altra struttura dati conveniente.

Implementazione di un ADT

Per implementare un tipo, è necessario fornire una rappresentazione (rep) per gli oggetti del tipo, ovvero una struttura dati che rappresenti i loro valori.

Questo insieme si basa su un insieme di "instance variables" dichiarate private, che costituiscono lo stato interno dell'oggetto.

Es: (**IntSet**)

Gli elementi nell'insieme sono soggetti a cambiamenti, indicando che l'astrazione è mutabile, allora può essere una scelta conveniente utilizzare un ArrayList per memorizzare gli elementi.

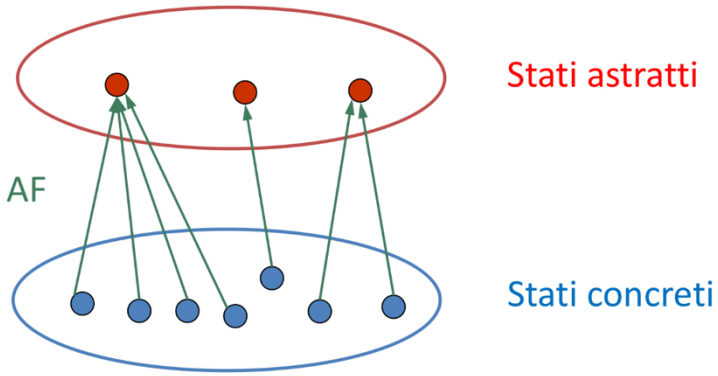
Relazioni fra Astrazione e Implementazione

Fino a questo momento, la nostra specifica si era concentrata esclusivamente sulla parte pubblica. Tuttavia, attualmente stiamo esplorando la prospettiva della parte privata, ossia l'implementazione. Quali proprietà sarebbe possibile definire per la nostra implementazione?

Sono 2:

1. la Abstraction Function (AF)
2. la Representation Invariant (RI)

Abstraction Function



Questa figura rappresenta la funzione di astrazione (Abstraction Function) e come notiamo abbiamo 2 stati:

Gli stati Astratti (della specifica) e gli Stati concreti (della rappresentazione).

Proviamo a capire quale sia la differenza fra i due stati mediante l'esempio dell'IntSet

Es: (**IntSet**)

L'insieme {1,2,5} è uno strato astratto, mentre il valore [1,2,5] (dell'ArrayList della rappresentazione) è uno stato concreto.

Come notato la funzione di astrazione associa a ogni stato concreto **uno e un solo** stato astratto, e di solito questa funzione non è iniettiva (ad esempio, per IntSet , [1,2,5] e [2,1,5] corrispondono allo stesso stato astratto {1,2,5}.

Definizione: La funzione di astrazione (AF) definisce il significato della rappresentazione (rep), cioè stabilisce come ogni oggetto concreto di una certa classe (in questo caso IntSet) implementa un oggetto dell'astrazione (in questo caso Insieme di interi).

Come definirla?

La posso definire come **invariante**.

Un invariante dichiarato come 'private' è autorizzato a utilizzare anche le parti private, (metodi puri cioè osservatori e attributi), della classe. Inoltre, è possibile impiegare un invariante per descrivere una relazione (funzione) tra le parti private e i metodi osservatori della classe.

Es: (**IntSet**)

Supponiamo di aver utilizzato come rappresentazione (rep) un ArrayList e chiamiamolo "arrayRep" per rappresentare il nostro insieme di interi.

Tengo a mente quali sono i metodi ground perché tutti gli altri metodi pubblici sono definiti a partire da quelli.

La funzione di astrazione (AF) di IntSet può essere scritta come:

```
/*@private invariant
  @(\forall int i ;  ; this.isIn(i) < ==>
  @arrayRep.contains(i))
  @*/
```

L'invariante afferma che la relazione tra l'oggetto corrente, il metodo **isIn** , e l'array rappresentato da **arrayRep** deve essere consistente per tutti gli interi **i**.

L'invariante sta dichiarando che la presenza di un elemento **i** nell'insieme (verificata da isIn(i) deve essere consistente con la presenza di **i** nell'array rappresentato (**arrayRep.contains(i)**).

Es: (**Poly**)

Il Poly di grado n è implementato con un vettore trms di dimensione n+1 che contiene il coefficiente del termine di grado i-esimo in i-esima posizione, con 0<=i<=n

La funzione di astrazione (AF) di Poly può essere scritta come:

```
  /*@private invariant
  @(\forall int i ; 0<=i && i<=deg ; coeff(i)==trms[i]) &&
  @(\forall int j ;  j<0 || j>deg ; coeff(j)==0);
  @*/
```

Es: (**classe Pila**)

La funzione di astrazione (AF) di Pila richiede in questo caso di usare l'oggetto astratto tipico (OAT) , e può essere scritta come:

```
     /*@private invariant oat.size( ) == cima &&
     @(\forall int i ; 0<=i && i<cima ; oat.get(i)=el[i]);
     @*/
```

toString() e AF

Il metodo **toString( )** è una funzione predefinita della classe Object che richiede una ridefinizione personalizzata (Override). La sua implementazione di base restituisce una stringa composta dal nome dell'oggetto seguito dal suo hash-code.

Tuttavia, lo scopo di toString( ) è fornire una rappresentazione testuale significativa dell'oggetto astratto. Pertanto, è possibile sfruttare tostring( ) per mappare l'oggetto concreto **this** nell'oggetto astratto, ottenendo così una rappresentazione testuale più comprensibile e personalizzata.

Es: (**IntSet: {1,7,3}**)

```
 public String toString() {
 if (els.size()==0)
return "IntSet: {}";
 String s = "IntSet: {" + \
                              arrayRep.elementAt(0).toString();
for (int i = 1; i<els.size(); i++)
     s = s + ", "+ arrayRep.elementAt(i).toString();
 return s +"}";
}
```

Come funziona?

Questo codice definisce il metodo *toString( )* per la classe *IntSet*. Se l'insieme (*els*) è vuoto, viene restituita la stringa "IntSet: {}". Altrimenti, viene costruita una rappresentazione testuale

dell'insieme in forma di stringa, iniziando con "IntSet: {" e aggiungendo successivamente ciascun elemento separato da virgola fino a quando l'intero insieme è stato attraversato.

Es: (**Poly**)

```
public  class Poly {
...
public String toString() {
   String s = "Poly: " + trms[0];
   for (int i = 1; i<=deg; i++)
      s = s + " + " + trms[i] + "x^" + i;
   return s;
  }
}
```

In sintesi, l'abstraction function (AF) definisce l'interpretazione della rappresentazione di un oggetto, associando a ogni istanza concreta l'oggetto astratto che si vuole rappresentare.

Questa relazione può essere formalizzata in JML attraverso **private invariant** , che collega gli attributi privati agli osservatori pubblici. La realizzazione pratica di questo concetto può avvenire tramite l'implementazione del metodo ***toString( )*** .

Non tutti gli oggetti concreti di una classe sono rappresentanti legali (validi) degli oggetti astratti.

Es: (**IntSet**)

Il valore [1,2,2] per il vettore *arrayRep* è scorretto, perché *IntSet* non contiene elementi duplicati.

Dunque, vogliamo descrivere in modo preciso delle ipotesi (condizioni) e inserirle nella documentazione della classe

Quindi è per questo che introduciamo le Invarianti di Rappresentazione.

Representation Invariant

L'invariante di rappresentazione (RI) è un invariante privato che mi dice le proprietà che devono valere per una rappresentazione tale per cui quella rappresentazione sia valida.

Es: (**IntSet**)

Supponiamo che nelle scelte implementative dell'IntSet ci sia la condizione "Non inserisco mai duplicati"

Dunque, posso definire una invariante di rappresentazione (una funzione) **J: Stato Concreto—>Boolean**

E si ha, J([1,2,2]) = false

e J([1,3,2]) = true

E in JML diventa,

```
//@ private invariant arrayRep != null &&
//@ !arrayRep.contains(null) &&
//@ (\forall int i; 0 <= i && i<arrayRep.size();
//@   (\forall int j; i < j && j<arrayRep.size();
//@     !(arrayRep.get(i).equals(arrayRep.get(j))));
```

N.B: L'invariante è private perché riguarda solo l'implementazione!

Si può implementare anche tramite un metodo booleano repOk()

```
public boolean repOk() {
    // Ensures: restituisce true se rep invariant
    // vale per this , altrimenti restituisce false
}
```

Lezione 20: Specifiche ADT (21/11/2023 Antony Brun, Jie Chen)

Link utile: metodi di Collection

Tema d'esame 28/08/2023

Si consideri la seguente classe `CityMap` che memorizza i punti di interesse di una città (`Site`), organizzati per tipo (ristoranti, hotel, musei, etc.).

```
public class CityMap {
   // Ritorna l'insieme di tutti i punti di interesse memorizzati
   public /*@ pure @*/ Set<Site> getSites() { ... }

   // Aggiunge il punto di interesse passato per parametro
   public void addSite(Site site) { ... }

   // Ritorna tutti i punti di interesse del tipo passato per parametro,
   // ordinati per distanza crescente rispetto alla location loc indicata.
   // Lancia una NotFoundException se non esiste alcun punto di interesse del tipo indicato.
   public /*@ pure @*/ List<Site> getSitesByType(Type type, Location loc) throws NotFoundException { ... }

   // Ritorna i k punti di interesse piu' vicini alla location loc indicata
   // (per semplicita', si assuma che non possano esistere due punti equidistanti da loc).
   // Se i punti di interesse sono meno di k, vengono ritornati tutti.
   // La mappa ritornata contiene i punti di interesse come chiave e la loro distanza da loc come valore.
   public /*@ pure @*/ Map<Site, Double> getNearestSites(int k, Location loc) { ... }
}

public /*@ pure @*/ class Site {
   // Ritorna la posizione geografica del punto di interesse
   public Location getLocation() { ... }

   // Ritorna il tipo del punto di interesse (ristorante, hotel, museo, ...)
   public Type getType() { ...}
... }

public /*@ pure @*/ class Location {
   // Calcola la distanza rispetto alla Location passata per parametro
   public double getDistance(Location other) { ... }
... }
```

Iniziamo l'analisi del problema considerando la classe "Site":

Class Site

```
public /*@ pure @*/ class Site {
       // Ritorna la posizione geografica del punto di interesse
       public Location getLocation() { ... }

       // Ritorna il tipo del punto di interesse (ristorante, hotel, museo, ...)
       public Type getType() { ...}
... }
```

La dichiarazione **/*@ pure @*/** all'interno dell'intestazione di una classe indica che essa è pura; una classe pura è una classe per cui tutti i metodi sono puri. Dichiarando un'intera classe come pura, possiamo evitare di dichiarare i singoli metodi al suo interno come metodi puri.

Questa notazione, inoltre, sottolinea il fatto che oggetti di questa classe sono immutabili (dato che abbiamo solo metodi osservatori).

Lo stesso ragionamento vale anche per la classe "Location":

Class Location

```
public /*@ pure @*/ class Location {
       // Calcola la distanza rispetto alla Location passata per parametro
       public double getDistance(Location other) { ... }
... }
```

In questo caso, entrambi le classi sono classi 'accessorie', ossia classi di supporto per la specifica della classe principale. I metodi di queste classi, infatti, potranno essere utilizzati nel caso in cui ci servissero durante la fase di specifica dei metodi della classe principale "CityMap".

Class CityMap

```
public class CityMap {
	// Ritorna l'insieme di tutti i punti di interesse memorizzati
	public /*@ pure @*/ Set<Site> getSites() { ... }

	// Aggiunge il punto di interesse passato per parametro
	public void addSite(Site site) { ... }

	// Ritorna tutti i punti di interesse del tipo passato per parametro,
	// ordinati per distanza crescente rispetto alla location loc indicata.
	// Lancia una NotFoundException se non esiste alcun punto di interesse del tipo
	indicato.
	public /*@ pure @*/ List<Site> getSitesByType(Type type, Location loc) throws
	NotFoundException { ... }

	// Ritorna i k punti di interesse piu' vicini alla location loc indicata
	// (per semplicita', si assuma che non possano esistere due punti equidistanti da
	loc).
	// Se i punti di interesse sono meno di k, vengono ritornati tutti.
	// La mappa ritornata contiene i punti di interesse come chiave e la loro
	distanza da loc come valore.
	public /*@ pure @*/ Map<Site, Double> getNearestSites(int k, Location loc) { ...
	}
}
```

Voltiamo ora l'attenzione alla classe CityMap. Questa classe presenta 3 metodi osservatori `getSites`, `getSitesByType`, `getNearestSites` (quelli denotati con la keyword "pure") e 1 metodo modificatore `addSite`.

Domanda a) Si specifichi in JML il metodo getSitesByType.

Per prima cosa, dobbiamo **definire i metodi 'base' (anche detti 'ground')**. Questi sono metodi che assumiamo come esistenti, e che utilizzeremo come la base per la specifica di tutti gli altri metodi della nostra classe (In modo simile a come gli assiomi matematici vengono usati per definire i teoremi matematici).

⚠ Attenzione!
Solo i metodi puri possono essere utilizzati come metodi base

Il passo di selezionare i metodi base è un passo molto importante, perché determinerà non solo come verranno specificati gli altri metodi, ma avrà un impatto diretto sulla difficoltà dell'esercizio; infatti, se scegliessimo un metodo 'scomodo' come metodo base, avremmo difficoltà nella specifica degli altri metodi.

In questo caso, è ragionevole selezionare come metodo base il metodo `getSites`, infatti, da esso potremo non solo specificare `getSitesByType` filtrando i Sites di un certo tipo, ma potremo anche specificare `getNearestSites`. Questo perché, attraverso il metodo `getSites` possiamo far riferimento all'intera lista di oggetti Sites.

Una 'good rule of thumb' è di iniziare **specificando le \requires**, ossia le condizioni che il chiamante deve garantire affinché le condizioni di \ensures siano soddisfatte dal metodo chiamato.

Normalmente si cerca di ridurre le \requires al minimo numero di clausole possibile per non restringere troppo l'utilizzatore. Clausole tipicamente inserite nelle \requires sono condizioni di non nullità.

⚠ Attenzione!

Nelle \requires NON bisogna mettere condizioni di non nullità nel caso in cui ci si riferisce ad un parametro di tipo primitivo (es. int).
Esempio errato: \requires t != null
public void foo(int t)
Questo perché i ltipi primitivi non possono assumere valore di NULL

Scriviamo ora le \requires per getSitesByType:

JML Code - \requires di getSitesByType

```
//@ requires type != null && loc != null
```

Dopo aver aggiunto le \requires, **si passa alle \signals**, che in generale sono abbastanza semplici da ricavare. Per scrivere le \signals, devo **mettere in AND le condizioni che mi portano all'eccezione, il fatto che è scattata un'eccezione, e il fatto che lo stato della classe non è stato cambiato**; Questo si fa perché, in generale, quando scatta un'eccezione, vogliamo 'abort' un qualsiasi insieme di modifiche applicate sull'istanza della classe su cui il metodo è stato chiamato. Si nota che, per un generico metodo puro, indicare che lo stato dell'istanza non è cambiato in situazioni eccezionali non è sbagliato, ma è ridondante a causa del fatto che un metodo puro non può per definizione portare modifiche sull'istanza. (Parte ridondante sottolineata)

JML Code - \signals di getSitesByType

```
//@ signals (NotFoundException e)
//@ (\forall Site s; getSites().contains(s);
//@ !s.getType().equals(type)) &&
//@ \old(getSites()).containsAll(getSites()) &&
//@ getSites().containsAll(\old(getSites())
```

⚠ Attenzione!

Nel pezzo di JML precedente, non è corretto scrivere !s.getType() == type. Il programmatore, infatti, potrebbe avere ridefinito l'uguaglianza diversamente dall'identità.

La parte finale della specifica consiste nello **specificare le \ensures**. Qui è importante assicurarsi che le \ensures e le \signals sono in disgiunzione; se nella condizione di eccezione ci finivamo quando non esisteva nessun elemento dell'insieme di Sites che aveva il tipo specificato, nella condizione di

non eccezione ci finiremo quando esiste almeno un elemento dell'insieme di Sites con il tipo specificato.

JML Code - \ensures di getSitesByType (Errato)

```
//@ ensures
//@ \result != null &&
//@ (\exists Site s; getSites().contains(s);
//@ s.getType().equals(type)) && (\forall Site s;
//@ \result.contains(s); getSites().contains(s) &&
//@ s.getType().equals(type))
```

Lasciando la \ensures così, stiamo specificando che tutti i Sites restituiti dal metodo sono quelli del tipo giusto e che sono contenuti nella lista originale dei Sites, ma dobbiamo anche specificare che sono TUTTI E SOLI i Sites del tipo richiesto. Scritta così, non escludiamo la possibilità di restituire un sottoinsieme stretto dei Sites del tipo richiesto. Per esempio, supponiamo di avere un insieme di Sites rappresentati da (ID, TYPE):

getSites() = {(Site1, type1), (Site2, type2), (Site3, type1), (Site4, type3)}

Se chiamassimo getSitesByType(), sul tipo type1, ci aspetteremmo in uscita tutti i Sites del tipo 1, ossia {(Site1, type1), (Site3, type1)}, ma se il metodo ci restituisse {(Site1, type1)}, la specifica scritta sopra sarebbe comunque soddisfatta. Per correggere la nostra specifica, dobbiamo quindi modificarla aggiungendo una condizione aggiuntiva:

JML Code - \ensures di getSitesByType (Corretta)

```
//@ ensures
//@ \result != null &&
//@ (\exists Site s; getSites().contains(s);
//@ s.getType().equals(type)) && (\forall Site s; ;
//@ \result.contains(s)
//@ <==>
//@ getSites().contains(s) && s.getType().equals(type))
```

Dato che il metodo `getSitesByType` restituisce un List<Site> e non un Set<Site>, dobbiamo stare attenti a non introdurre duplicati nella lista restituita. Dobbiamo dunque aggiungere una condizione aggiuntiva nel codice JML che specifica l'assenza di duplicati, anche se questa non è esplicitamente espressa nel testo (intuitivamente, ha senso che la lista restituita non abbia duplicati dato che gli elementi in essa contenuti sono stati estratti da un oggetto di tipo Set che non ha duplicati):

JML Code - \ensures di getSitesByType

```
//@ ensures
//@ \result != null &&
//@ (\exists Site s; getSites().contains(s);
//@ s.getType().equals(type)) && (\forall Site s; ;
//@ \result.contains(s) <==> getSites().contains(s) &&
//@ s.getType().equals(type)) &&
//@ (\forall int i; 0<=i && i < result.size(); (\forall int j;
//@ 0<=j && j<i; !\result.get(i).equals(\result.get(j)))
```

⚠ Attenzione!

Se il return type di getSitesByType fosse stato Set<Site>, non dovevamo e non potevamo includere la \forall clausola specificata nel codice JML precedente. Dato che un Set<Site> object non è un oggetto posizionale come una List<Site>, non e' possibile 'scorrere' i suoi elementi utilizzando un indice

Adesso, bisogna aggiungere la clausola di ordinamento crescente:

JML Code - \ensures di getSitesByType

```
//@ ensures
//@ \result != null &&
//@ (\exists Site s; getSites().contains(s);
//@ s.getType().equals(type)) && (\forall Site s; ;
//@ \result.contains(s) <==> getSites().contains(s) &&
//@ s.getType().equals(type)) &&
//@ (\forall int i; 0<=i && i < result.size(); (\forall int j;
//@ 0<=j && j<i; !\result.get(i).equals(\result.get(j))) &&
//@ (\forall int i; 0<=i && i<result.size()-1;
//@ \result.get(i).getLocation().getDistance(loc) <=
//@ \result.get(i+1).getLocation().getDistance(loc)
```

La condizione \result != null, anche se non strettamente necessaria per la risoluzione degli esercizi, permette ad un verificatore automatico di non andare in 'crash' nel caso in cui il metodo venisse implementato in modo tale da ritornare il valore null. Supponiamo infatti, che questa specifica viene data ad un programmatore che implementa questo metodo come un metodo che restituisce sempre il valore null. In quel caso, se non avessi la condizione di non nullita', un verificatore automatico non sarebbe in grado di eseguire l'istruzione \result.contains(s) nella clausola \exists, dato che un valore null non ha nessun metodo contains(), andando in crash. Dunque, avere la condizione di non nullita' permette alla mia specifica di essere piu' robusta e flessibile, permettendo ad un verificatore di gestire correttamente un insieme di scenari piu' ampi.

Domanda b) Si specifichi in JML il metodo getNearestSites.

Osserviamo che il metodo `getNearestSites` non lancia nessuna eccezione. Questo si traduce dall'assenza della \signals all'interno della specifica.

Procediamo come per il punto a, specificando prima le requires:

JML Code - \requires di getNearestSites

```
//@ requires loc != null && k > 0
```

Dato che non è esplicitamente specificato nel testo che il metodo gestisce esclusivamente k > 0, sarebbe stato possibile anche aver omesso quella condizione nella \requires, avendo inserito invece nelle \ensures, una clausola che garantisce che, nel caso in cui k <= 0, il risultato ritornato sarebbe stato una mappa vuota.

Scriviamo ora le \ensures. Partiamo specificando la dimensione del risultato restituito:

JML Code - \ensures di getNearestSites

```
//@ ensures
```

```
//@ \result != null && \result.size() == Math.min(k,
//@ getSites().size())
```

⚠ Attenzione!

La riga scritta nella \ensures, esprime il fatto che, la dimensione del risultato restituito dal metodo dovrà avere una dimensione pari al minimo tra il parametro k e al numero di Sites al momento in cui viene chiamato il metodo. Normalmente, useremmo \old.getSites().size(), ma dato che siamo in un metodo puro, utilizzare getSites().size() è equivalente. Infatti, siamo sicuri che l'insieme di Sites non verrà mai modificato dal metodo, e quindi, prendere la dimensione della lista di Sites all'inizio dell'invocazione del metodo o in qualunque altro stato di esecuzione del nostro metodo puro è irrilevante.

Dobbiamo inoltre specificare che tutto quello presente nel risultato è un elemento valido (ossia contenuto nella lista di Sites). In questo caso, bisogna portare particolare attenzione a gestire \result, dato che è un oggetto di tipo Map<Site, Double>, e non una semplice lista.

JML Code - \ensures di getNearestSites

```
//@ ensures
//@ \result != null && \result.size() == Math.min(k,
//@ getSites().size()) && (\forall Site s;
//@ \result.keySet().contains(s); getSites().contains(s) &&
//@ \result.get(s) == s.getLocation().getDistance(loc))
```

La clausola aggiunta dice il seguente: per ogni Site che è una chiave del \result object di tipo Map, quel Site deve essere contenuto nella lista di tutti i Sites, e il valore associato a quella chiave nella mappa è proprio la distanza di quel Site dal Site specificato dal parametro loc.

Esempio: (Supponiamo di avere un insieme di Sites rappresentati da (ID, distanzaDaSitoInteresse))

getSites() = {(Site1, 15), (Site2, 21), (Site3, 14), (Site4, 50)}

Risultato visualizzato di getNearestSites(3, Site4):

Key	Value
Site1	15
Site2	21
Site3	14

⚠ Attenzione!

Anche se *(\forall Site s; \result.keySet().contains(s); getSites().contains(s) && \result.get(s) == s.getLocation().getDistance(loc))* è equivalente a *(\forall Site s; ; \result.keySet().contains(s) ==> getSites().contains(s) && \result.get(s) == s.getLocation().getDistance(loc))*, si preferisce la prima forma perché un eventuale verificatore automatico JML avrebbe un bound. Se avessi usato la seconda forma, un verificatore avrebbe dovuto eseguire la verifica su "tutti i Sites s dell'universo" dato che il dominio non è stato ristretto.

Adesso dobbiamo dire che prendiamo i k più vicini (supponendo che tutti i Site abbiano una distanza diversa da loc per semplicità).

JML Code - \ensures di getNearestSites

```
//@ ensures
//@ \result != null && \result.size() == Math.min(k,
//@ getSites().size()) && (\forall Site s;
//@ \result.keySet().contains(s); getSites().contains(s) &&
//@ \result.get(s) == s.getLocation().getDistance(loc)) &&
//@ (\forall Site s; getSites().contains(s); (\num_of Site s1;
//@ getSites.contains(s1) && !s.equals(s1);
//@ s1.getLocation().getDistance(loc) <
//@ s.getLocation().getDistance(loc)) < k ==>
//@ \result.keySet().contains(s))
/*per ogni Site s contenuto nella lista di tutti i Sites, se il numero
di altri Site s1 (diversi dal s) che sono più vicini di s da loc è
minore di k, allora s è contenuto nel risultato.*/
```

⚠ Attenzione!
Non avrebbe senso ordinare una mappa dato che gli elementi contenuti in essa non sono associati ad indici, ma a chiavi.

Domanda c)

Si consideri un'implementazione che fa uso di una mappa per memorizzare i punti di interesse organizzati per tipo, come mostrato di seguito.

```
public class CityMap {
   private Map<Type, Set<Site>> sites;
... }
```

Per tale implementazione di forniscano l'invariante di rappresentazione e la funzione di astrazione.

Richiami teorici

Representation Invariant(rep invariant o RI): è una funzione che va da stati concreti ad un booleano

J: ConcSt -> Boolean

- È **private**, perché riguarda **solo** l'**implementazione**: non deve contenere niente della parte pubblica.
- Ci dice il **dominio degli stati concreti**: ragiona su che cosa sono i **valori validi** dei miei attributi (ad esempio: valori diverso da zero, valori senza duplicati, valori ordinati ecc.) cioè tutte le assunzioni che mi servono perché lo stato sia corretto.

Abstraction function(AF): è una funzione che mappa stati concreti a stati astratti

ConSt-> AbsSt

Sia RI sia AF sono rappresentate con una forma che inizia con *Private Invariant*:

- È **private** perché parla di qualcosa dell'implementazione.
- È un **invariante** perché ci dice le **relazioni che non cambiano mai** ovvero che sono **sempre vero** tra *stato concreto* (il valore degli attributi) e *stato astrato* (i metodi puri).

C.1. l'invariante di rappresentazione:

1. Non nullità: la mappa non sia nulla, non contenga chiavi nulle, non contenga valori nulli o vuoti.
2. Ogni punto di interesse deve essere non nullo e memorizzato nell'insieme corrispondente al proprio tipo.

JML code - RI

```
//@ private invariant
//@ sites!=null && !sites.containsKey(null)
&&!sites.containValue(null)&&
//@ (\forall Type t; sites.containsKey(t); !sites.get(t)!=null
&&!sites.get(t).isEmpty() &&
//@ (\forall Site s; sites.get(t).contains(s); s!=null
&&s.getType().equals(t))
/*per ogni tipo tale per cui il tipo t è la chiave di sites, se prendiamo
un qualunque sito che è nell'insieme ottenuto da quel tipo, allora il
tipo di quel sito è proprio il tipo di quella chiave.*/
/*sites.get(t).contains(s) ritorna un set di siti del tipo t*/
```

N.B.:

- Private invariant è un insieme di assunzioni che il programmatore ritiene valide.
- NULL vs EMPTY: la differenza sta sullo heap, **null** non punta niente, **empty** punta una zona dello heap che è un set vuoto (l'oggetto esiste ma è vuoto).
- Struttura mappa(dizionario): in questo caso Map<Type, set<Sites>> sites, devo verificare che il tipo di ogni sito che è nel set<Sites> sia proprio il tipo dell'indice Type, cioè tutti i siti devono essere nel posto giusto quindi il loro tipo deve essere quello della chiave.

C.2. la funzione di astrazione:

1) Individuamo quali sono i valori che vediamo tramite i valori pubblici in funzioni dei valori che abbiamo nei nostri attributi privati.
2) Stabiliamo una **relazione di uguaglianza** oppure una **relazione di se e solo se** tra i valori degli attributi e i valori dei nostri metodi pubblici puri: *"Tutti i siti che il metodo pubblico puro ground assume un certo valore <==> quelli siti sono contenuti nella mappa"*

- metodo puro pubblico: *Set<Site> getSites( );*
- metodo privato: *Map<Type, Set<Site>> sites;*
- fare la corrispondenza tra ciò che pubblico e ciò che privato

JML code - AF

```
//@ private invariant
//@ (\forall Site s; ; // qualunque Site s,
//@ getSites().contains(s) <==> // s è contenuto dei valori ritornati
dal metodo pubblico sse
//@ sites.containsKey(s.getType()) && // il suo tipo è una chiave
della mappa privata e
//@ sites.get (s. getType()). contains(s))
// quel sito è contenuto nell'insieme dei siti indicati con quella
chiave
```

Tema d'esame 26/08/2022

Esercizio 1

Si consideri la seguente classe `SeriesStore` per memorizzare episodi di serie TV. Gli episodi sono rappresentati dalla classe immutabile `Episode` che espone tra gli altri un metodo per ottenere la serie di appartenenza di un episodio.

```
public class SeriesStore {
  // Ritorna il primo episodio della serie passata per parametro.
  // Ritorna null se la serie non esiste.
  public /*@ pure @*/ Episode getFirstEpisode(String series);

  // Ritorna l'episodio successivo a quello passato per parametro.
  // Ritorna null se tale episodio non esiste, ovvero se quello passato per parametro
  // e' l'ultimo della serie.
  public /*@ pure @*/ Episode getNextEpisode(Episode ep);

  // Ritorna il numero di episodi della serie successivi a quello passato per parametro.
  // Ritorna 0 se la serie non esiste.
  public /*@ pure @*/ int numNextEpisodes(Episode ep);

  // Aggiunge l'episodio in fondo alla sua serie di appartenenza. Se la serie ancora
  // non esiste, l'episodio viene aggiunto come il primo di quella serie.
  // Lancia una IllegalEpisodeException se l'episodio passato come parametro e' null.
  public void addEpisode(Episode ep) throws IllegalEpisodeException;
}

public /*@ pure @*/ Episode {
  // Ritorna la seria di appartenenza dell'episodio.
  public /*@ pure @*/ String getSeries();
... }
```

Prima della risoluzione, ci conviene individuare le classi accessorie e i metodi accessori, in questo caso, abbiamo una classe mutabile e una classe immutabile:

- Class *SerieStore* è una classe mutabile che ha tre metodi puri (*getFirstEpisode, getNextEpisode* e *numNextEpisodes*) e un metodo modificatore (*addEpisode*).
- Class *Episode* è una classe immutabile, per cui tutti metodi sotto la class *Episode* sono puri, possiamo usare *getSeries* per predicare in JML.

Domanda a) Specificare in JML il metodo numNextEpisodes.

Seguiamo il pattern generale:

1. Quali sono i metodi ground (di base)?

   - Episode getFirstEpisode(String series)

   - Episode getNextEpisode(Episode ep)

2. \requires: l'episodio non può essere null, *ep!=null*
3. \signals: non ce ne sono
4. \ensures: per contare il numero degli episodi di una serie che ci sono dopo quell'episodio passato per parametro, bisogna innanzitutto scorrere tale serie fino alla fine.

Problema: non abbiamo un metodo get che ci permette di ritornare gli elementi di una certa posizione.

Unica via di uscita: usiamo la tecnica di ricorsione.

Ricorsione:

1) Trovare il caso base: l'episodio che ritorna il next è null, *getNextEpisode(ep)==null*
2) Definire il passo intuitivo: 1+ metodo da specificare chiamato sul successivo, *1+numNextEpisodes(getNextEpisode(ep))*.

Errore: *1+numNextEpisodes((ep))*, perché non ci consente di fare una mossa avanti verso il fondo, cioè genera un errore per la mancanza del passo intuitivo.

⚠ Attenzione!

Una definizione ricorsiva è ben formata quando abbiamo un caso base e un passo intuitivo ma una definizione per poter essere definita ricorsivamente deve essere una definizione di un metodo puro, altrimenti non possiamo usare tale metodo per definire sé stesso, perché solo i metodi puri sono utilizzabile per predicare in JML.
L'uso della definizione ricorsiva dipende dai metodi pubblici puri che vengono dati dalla classe: se la classe permettesse di muoverci arbitrariamente con l'indice, allora potremmo usare il metodo classico utilizzando get, altrimenti ci servirebbe a definire una ricorsione.

JML - getNextEpisode

```
//@ requires ep != null
//@ ensures \result == (getNextEpisode(ep) == null)? 0:
//@ 1+numNextEpisodes(getNextEpisode(ep))
   // "Se dopo l'episodio ep non esiste nessun altro episodio, allora il risultato ritorna 0, altrimenti result+=1 e invoca il metodo numNextEpisodes sul prossimo episodio"
```

Domanda b): specificare in JML il metodo addEpisodes

Seguiamo il pattern generale:

1. Quali sono i metodi ground (di base)?

   - *Episode getFirstEpisode(String series)*

   - *Episode getNextEpisode(Episode ep)*

2. \requires: non serve specificare *ep!=null* perché c'è già \signals nel caso di *ep==null*
3. \signals:

   3.1. innanzitutto, *ep==null*

⚠ Attenzione!

Non basta *ep==null*, perché il metodo *addEpisodes* è un metodo non puro; quindi, ci dobbiamo ricordare di specificare che **niente cambia**, cioè lo stato delle serie prima della chiamata del metodo è uguale allo stato delle serie dopo la chiamata. Se non lo specificassimo, potrebbe generare caos (potrebbe togliere tutte le serie che ci sono, potrebbe rigirare tutti gli episodi ecc.).

   3.2. I valori di quei metodi ground non cambiano: per ogni valore di input, l'output che ci dà *getFirstEpisode* prima della chiamata del metodo *addEpisodes* è uguale all'output che ci dà dopo la chiamata, specifichiamo nello stesso modo anche per *getNextEpisode*.

   N.B.: Nel caso che ci siano **più metodi modificatori** in cui dobbiamo dire più volte che tutto non cambia, possiamo usare *unchanged( )* per definire l'insieme delle cose che non variano:

   unchanged( )== ("le formule logiche") oppure **unchanged( ) sta per ("le formule logiche")**

   Ad esempio:

   ```
   unchanged( ) = = (\forall Episode e; e! = null; getNextEpisode(e) == \old(getNextEpisode(e))
   ```

4. \ensures:

   4.1. \ensures è in mutua esclusione con \signals: mettere le condizioni disgiunti (tutte le condizioni che non fanno lanciare \signals)

   4.2. Per tutte le serie diverse da quella serie scelta, dobbiamo specificare che non cambiano niente: il primo episodio di quelle serie rimane uguale alla prima e non cambia il prossimo episodio.

JML code - addEpisode

```
//@ ensures ep != null &&
//@ (\forall String series; series != null &&
//@ !series.equals(ep.getSeries());
/*tutte le serie non null e diverse dalla serie ep*/
//@ getFirstEpisode(series) == \old(getFirstEpisode(series)))&&
//@ (\forall Episode e; e != null &&
//@ !e.getSeries().equals(ep.getSeries());
//@ getNextEpisode(e) == \old(getNextEpisode(e))) &&
/*speciifca che non cambia niente dopo la chiamata del metodo
addEpisode*/
//@ getFirstEpisode(ep.getSeries()) ==
//@ \old(getFirstEpisode(ep.getSeries())) == null ?
//@ ep : \old(getFirstEpisode(ep.getSeries()) &&
/* se \old getFirstEpisode di quella serie ritorna null allora ep è il
primo episodio, altrimenti il first episodio non cambia*/
//@ getNextEpisode(ep) == null &&
//@ (\forall Episode e; e.getSeries().equals(ep.getSeries()) &&
//@ !e.equals(ep);
//@ getNextEpisode(e) == (\old(getNextEpisode(e))) == null ?
//@ ep : \old(getNextEpisode(ep.getSeries())) )
/* se \old getNextEpisode di quella serie ritorna null, allora ep è il
successivo episodio di e, altrimenti rimane il valore
(getNextEpisode(ep.getSeries()) che c'era prima */
//@ signals (IllegalEpisodeException exc) ep == null &&
//@ (\forall Episode e; e != null; getNextEpisode(e) ==
//@ \old(getNextEpisode(e))) &&
//@ (\forall String series; series != null;
//@ getFirstEpisode(series) == \old(getFirstEpisode(series)))
/* == sottolinea che non cambiano neanche i puntatori (non cambia la
struttura).*/
```

Lezione 21: Specifiche e ereditarietà (23/11/2023 Benedetti Gabriele, Canepari Michele)

L'ultima parte di specifica riguarda l'ereditarietà (estensioni in Java), ossia quali sono i legami delle specifiche della sottoclasse rispetto alle specifiche della superclasse.

Estensioni pure

Con il termine *pure* vengono identificate le estensioni che non modificano la specifica dei metodi ereditati, ma possono solo modificarne l'implementazione (a pari specifica) e/o aggiungere nuove operazioni (nuovi metodi).

Vediamo un primo esempio di estensione pura *MaxIntSet*, erede della classe *IntSet* vista nelle lezioni precedenti, che aggiunge il metodo *max()* per ricavare il massimo elemento:

MaxIntSet.java

```
public class MaxIntSet extends IntSet {

    // MaxIntSet è sottotipo di IntSet con aggiunta metodo max


    //@ ensures (*inizializza this come il MaxIntSet vuoto*)

    public MaxIntSet() { … }


    //@ ensures this.size()>0 &&

    //@ \result == (\max int i; this.isIn(i); i)

    //@ signals(EmptyException e) this.size() == 0;

    public int /*@ pure @*/ max() throws EmptyException { … }

}
```

N.B. Sarebbe ridondante aggiungere nella *signals* il fatto che l'oggetto non viene modificato, in quanto *max()* è un metodo puro, ossia non modificatore

Come detto precedentemente una estensione pura non può modificare la specifica di un metodo ereditato ma può solo modificarne l'implementazione, come possiamo vedere nel suo codice, dove i metodi *insert*() e *remove()* aggiungono la funzionalità di tenere traccia del valore massimo:

MaxIntSet.java

```
public class MaxIntSet extends IntSet {
    private int biggest; // Memorizza il max. Non definito per this=vuoto
    public MaxIntSet(){ super(); }

    public int max() throws EmptyException {
        if (size() == 0) throw new EmptyException("MaxIntSet.max");
        return biggest;
    }

    @Override
    public void insert(int x) {
        if (size() == 0 || x > biggest) biggest = x;
```

```
        super.insert(x); // Chiama metodo insert di IntSet
    }

    @Override
    public void remove(int x) {
        super.remove(x);
        if (size() == 0|| x < biggest) return;
        for (Iterator<Integer> i = this.iterator(); i.hasNext();) {
            Integer z = i.next();
            if (z > biggest) biggest=z;
        }
    }
}
```

Come vengono modificate le AF e RI?

- AF: l'abstract function della sottoclasse non varia rispetto a quella della superclasse, in quanto gli unici metodi considerati nella AF, ossia i metodi "ground" tipicamente non cambiano (di solito vengono solo aggiunti dei nuovi "metodi accessori" per le implementazioni di nuove funzionalità)
- RI: a differenza delle AF, i RI possono variare. I representation invariant vengono ereditati senza modifiche con l'aggiunta di un nuovo RI dovuto alle nuove funzionalità e alla modifica della implementazione. Per implementare questo nuovo RI si utilizza la parola chiave "also". Per l'esempio precedente l'RI della sottoclasse diventerà il seguente:

```
//@ also
//@ private invariant this.size > 0
//@                       ==> this.isIn(this.biggest) &&
//@     (\forall int x; this.isIn(x); x <= this.biggest);
```

N.B. le nuove specifiche aggiunte sono in AND a quelle della superclasse

N.B. valgono le solite regole di visibilità -> il nuovo RI può utilizzare tutti gli attributi e metodi puri (anche privati) definiti all'interno della stessa classe e attributi e metodi puri ereditati (public e protected)

Estensioni non pure

Con il termine *non pure* vengono identificate le estensioni che modificano la specifica di uno o più metodi ereditati. Nell'esempio seguente vediamo come l'estensione *SortedIntSet* vada a modificare la specifica del metodo iterator, aggiungendo l'ordine crescente all'iterator ritornato

SortetIntSet.java

```
public class SortedIntSet extends IntSet {
```

```
    public SortedIntSet { super(); }


    @Override
    //@ ensures (*restituisce un iteratore su tutti
    //@ e soli gli elementi di this,
    //@ ciascuno una sola volta, in ordine crescente *);
    public Iterator<Integer> iterator() { … }
}
```

Quando posso definire una classe sottoclasse di un'altra? Mediante il **principio di sostituzione**

Intuizione di Barbara Liskov -> la sostituzione con un sottotipo all'interno di ogni punto del codice non deve causare danno all'utilizzatore. Per comprendere al meglio possiamo rifarci ai contratti telefonici: un contratto telefonico è sostituibile se non chiede di più (come costo) e/o mi deve offrire di più (maggiori offerte e garanzie). Possiamo fare lo stesso ragionamento con le sottoclassi

Formalmente -> Gli oggetti della sottoclasse devono rispettare il contratto della superclasse, cioè la sottoclasse non deve richiedere di più e/o offrire di meno della superclasse. Questo garantisce che dei moduli che utilizzano oggetti di un tipo "non devo accorgersi della differenza" ad usare oggetti di tipo derivato.

Vediamo ora due esempi che mostrano il corretto e non corretto utilizzo del principio di sostituzione:

- La classe *MaxIntSet* soddisfa il contratto di *IntSet* (superclasse) in quanto contiene tutti i suoi metodi con la stessa specifica e un nuovo metodo max. Il metodo Iterator in questo caso non darà problemi poiché la sua specifica non subisce variazioni
- Se invece modificassi il metodo iterator in modo che iteri solo sugli elementi dispari non rispetterei il contratto del metodo *iterator()* di *IntSet* poiché non andrei più ad iterare su tutti gli elementi ma solo una parte

Regole per la specifica di sottoclassi

Per verificare che il principio di sostituzione venga rispettato si utilizzano le seguenti regole:

REGOLA DELLE SIGNATURE: le specifiche e le signature dei metodi del sottotipo devono essere compatibili con quelle del supertipo, ossia garantisce che il contratto della superclasse sia ancora applicabile.

Questa regola è garantita dal compilatore di Java, non potendo scrivere sottoclassi con metodi ereditati aventi signature differenti.

N.B. Dalle versioni successive alla 1.5 di Java è possibile restringere il tipo del valore di ritorno facendo override (covarianza del risultato).

REGOLA DEI METODI: le chiamate ai metodi del sottotipo devono comportarsi come le chiamate ai metodi corrispondenti del supertipo. Si utilizza la seguente frase -> *Require no more, promise no less*

-> per riassumere come può essere la specifica di un metodo ereditato. In parole semplici, il metodo della sottoclasse non deve richiedere di più, in termini di *requires*, e non deve promettere di meno, in termini di *ensures*.

Digressione sulla logica
Regola forte -> regola più restrittiva, vera in "meno casi"gn
Regola debole -> regola meno restrittiva, vera in "più casi"
N.B. Una regola è più forte di un'altra se vale l'implicazione

Devono valere le seguenti regole:

- La precondizione della sottoclasse deve essere più debole della superclasse
  - *pre_super ==> pre_sub*
- La postcondizione della sottoclasse deve essere più forte della superclasse
  - *post_sub ==> post_super*

Violando una delle due implicazioni violo anche il principio di sostituzione come negli esempi seguenti:

Stack.java	BoundedStack.java

public class Stack<T> {
    // Una pila di elementi di tipo T
    //@ requires true;
    //@ ensures
    //@ (*inserisce v in cima a this*);
    public void push(T v)
}
	public class BoundedStack<T> {
	    // Una pila di elementi di tipo T
	    // di dimensione max pari a cento
	    //@ requires this.size() <=100;
	    //@ ensures
	    //@ (* inserisce v in cima alla pila *);
	    public void push(T v)
	}

Come detto prima per rispettare la regola dei metodi è necessario avere una pre-condizione più debole o uguale. In questo caso possiamo notare come, al contrario, l'estensione *BoundedStack* abbia una pre-condizione più forte; infatti, richiede che la dimensione non superi i 100 elementi. Di conseguenza violiamo il principio di sostituzione in quanto, ad esempio avendo un codice che aggiunge 110 elementi in uno *Stack*, sostituendolo con un *BoundedStack* il codice non avrebbe lo stesso comportamento.

CharBuffer.java	LowerCaseCharBuffer.java
public class CharBuffer { // Un buffer di caratteri	public class LowerCaseCharBuffer { // Un buffer di caratteri minuscoli

```
    //@ ensures this.isIn(x) && …
    //@ …
    //@ …
    public void insert(char x) {
    …
    }
}
```

```
    //@ ensures
minuscolo(x)==>this.isIn(x) && …
    // Se x e' minuscolo inserisce x in
this,
    // se no non assicura nulla
    public void insert(char x) {
    …
    }
}
```

In questo caso altro esempio la post-condizione della sottoclasse è più debole della post-condizione della superclasse. La classe *LowerCaseCharBuffer* inserisce il carattere in *this* solo se minuscolo mentre la classe *CharBuffer* garantisce l'inserimento in qualunque caso. Anche in questo esempio il principio di sostituzione viene violato.

JML -> per aggiungere le nuove pre-condizioni e post-condizioni utilizzo la parola chiave "*also*"

- La pre-condizione (pre_sub) è in OR con quella della superclasse -> "requires indebolita"
- La post-condizione (post_sub) è in AND con quella della superclasse -> "ensures rafforzata"
  N.B. La post_sub può anche essere più debole della post_super se prevale la pre_sub sulla pre_super, cioè la precondizione della sottoclasse è più forte di quella della superclasse, in quanto, non rispettando la regola dei metodi, siamo già fuori dal dominio delle regole della superclasse.

La nuova specifica sarà la seguente:

```
//@requires pre_super || pre_sub
//@ensures (\old(pre_super) ==> post_super)) &&
//@        (\old(pre_sub) ==> post_sub)
```

Eccezioni -> Le eccezioni in Java sono delle post-condizioni "eccezionali" e per quanto detto precedentemente NON è possibile eliminarle (solo aggiungerne), salvo casi specifici -> possono essere eliminate solo se all'interno della sottoclasse l'eccezione definita non può più verificarsi

REGOLA DELLE PROPRIETA': il sottotipo deve preservare tutte le proprietà (invarianti ed evolutive) del supertipo. Le proprietà definite nella superclasse devono continuare ad essere rispettati da tutti i metodi nuovi o ridefiniti all'interno dell'estensione

Vediamo un esempio in cui viene violata la proprietà invariante;

FatSet.java

```
//@ public invariant this.size() >= 1;

//@ ensures (*costruisce insieme {i}*)

public FatSet(int i)


//@ ensures \old(this.size()>=2) ==> !this.isIn(x);

public void removeNonEmpty(int x)
```

```java
public class IntSet {

//@ ensures !this.isIn(x);

    public void remove (int x) {…}

}
```

È possibile definire *IntSet* come sottoclasse di *FatSet*? No.
Possiamo notare come nella sottoclasse non rimanga vero che la dimensione è sempre maggiore o uguale di 1 -> infatti notiamo come il metodo *remove()* definito vada ad eliminare l'elemento anche se è l'unico presente nell'*IntSet* non rispettando la proprietà *invariant*. In realtà la proprietà verrebbe già violata con l'utilizzo del costruttore *IntSet()* che definisce un Set vuoto

Vediamo un esempio sulla violazione della proprietà evolutiva. Prendiamo la classe *Stack* definita in un esempio precedente e creiamo una sua sottoclasse *MirrorStack* con un nuovo metodo *mirror()* che inverte l'ordine degli elementi. Anche in questo caso *MirrorStack* non può essere definita come sottoclasse di *Stack* in quanto la proprietà evolutiva della classe *Stack* (politica LIFO -> l'ultimo elemento inserito è il primo estratto) non viene mantenuta diventando una politica FIFO -> il primo elemento inserito è il primo estratto

MirrorStack.java

```java
public class MirrorStack extends Stack {

    //@ ensures (*this diventa il riflesso di \old(this) *);

    public void mirror() { … }

}
```

N.B. È importante notare che anche un'estensione pura può violare il principio di sostituzione

N.B. A differenza della prima regola che, come detto, è garantita dal compilatore, la seconda e la terza devono essere rispettate dal programmatore

Ereditarietà e collezioni

Le stesse considerazioni fatte precedentemente devono essere fatte anche con l'utilizzo di *Collections* all'interno delle superclassi e sottoclassi. Vediamo un esempio:

```java
class Automobile {...}


class AutomobileElettrica extends Automobile {...}


class ParcheggioAutomobili {

    //@ ensures (*inserisce auto a in parcheggio*)

    public void aggiungi(Automobile a) { … }
```

```
}


class ParcheggioElettriche extends ParcheggioAutomobili {...}
```

È corretto definire *ParcheggioElettriche* (parcheggio che contiene solo auto elettriche) sottoclasse di *ParcheggioAutomobili*? No.

Infatti, il metodo *aggiungi()* della sottoclasse *ParcheggioElettriche* non rispetta le specifiche perché l'idea è che possa aggiungere solo auto elettriche (posso aggiungere una *requires* più restrittiva, o una *ensures* meno restrittiva) -> In entrambi i casi sto violando il principio di sostituzione.

N.B. Il principio di Liskov è anche il motivo per cui i generici e le *collections* vengono definiti come invarianti, ossia ad esempio *Collection<String>* non è sottotipo di *Collection<Object>* perché viene violata nello specifico la terza regola

Messaggio finale -> i costrutti non vanno utilizzati a "caso" ma con un senso logico, rispettando le regole definite precedentemente. È molto importante ricordare che le estensioni non devono essere utilizzate solo per riutilizzare del codice già scritto ma le relazioni che si creano hanno un vero e proprio significato

Lezione 22: Specifiche e ereditarietà (27/11/2023 Omar Chaabani, Andrea Biasion Somaschini)

Tema d'esame 10/07/2023

Ingegneria del Software — Soluzione del Tema 10/07/2023

Esercizio 1

Si consideri la seguente classe immutabile `Message`. Ogni messaggio ha un contenuto, un mittente, un destinatario e un timestamp. Mittente e destinatario sono individuati univocamente da un identificatore, per semplicità un long. Ancora per semplicità, il timestamp è un long (anziché un `Date`).

```
public /*@ pure @*/ class Message {
  public Message(byte[] content, long recipientID, long senderID, long timestamp) {...}
  public byte[] content() {...}     // returns the content of this
  public long recipientID() {...}   // returns the recipientID of this
  public long senderID() {...}      // returns the senderID of this
  public long timestamp() {...}     // returns the timestamp of this
}
```

La seguente classe `MessageManager` descrive un gestore di messaggi. È possibile inviare ed estrarre messaggi, vedere elenchi dei messaggi e degli utenti; in particolare, è possibile fondere un altro gestore messaggi con quello corrente.

```
public class MessageManager {
  // Builds a new empty manager
  public MessageManager() {...}

  // Returns the set of all valid ids, i.e., ids of recipients or senders of some message in the past.
  public /*@ pure @*/ Set<Long> allIds() {...}

  // Returns all messages yet to be delivered to the recipient having the given recipientId,
  // from the oldest to the newest.
  // Returns an empty list if the recipient is unknown or if there is no message to be delivered it.
  public /*@ pure @*/ List<Message> toId(long recipientId) {...}

  // Returns all messages, yet to be delivered, sent from the sender having the given senderId.
  // Throws an UnknownIdException if the sender is unknown.
  public /*@ pure @*/ Set<Message> fromId(long senderId) throws UnknownIdException {...}

  // Inserts a new message in the manager, with the timestamp at the instant of the call.
  public void addMessage(byte[] content, long senderId, long recipientId) {...}

  // Returns and deletes the oldest message added for recipientId and not yet delivered.
  // Throws a NoMessageExpetion if no message is present.
  // Throws an UnknownIDException if the id is unknown.
  public Message deliverTo(long recipientId) throws NoMessageException, UnknownIDException{...}

  // Copies all messages and ids in the manager other to this.
  // The manager other is unchanged.
  public void merge(MessageManager other) {...}
```

Overview

Message è una classe **pure** (immutabile) e contiene un array di byte, un recipientID, recipientID, senderID e timestamp, tutti long per semplicità.

Abbiamo:

- un **costruttore** che costruisce un nuovo manager vuoto.
- metodo **allIds** restituisce un set di long di tutti gli id validi: se un messaggio che aveva come sender o receiver quell'Id è apparso e stato aggiunto a questo manager allora allIds contiene quell'Id
- metodo **toId** di tipo List<message>, prende in ingresso un recipientId e ritorna una lista di messaggi, ordinati dal più vecchio al più nuovo, che devono ancora essere mandati al recipient aventi quel recipientId. Ritorna una lista vuota se recipientId non è noto o non ci sono messaggi che devono essere inviati.
- metodo **fromId**: ritorna il set di tutti i messaggi, ancora da mandare (ovvero all'interno del messaggio), inviati dal sender col suo senderId che lancia un'eccezione se il senderId è sconosciuto
- **addMessage** inserisce un nuovo messaggio con content senderId e recipientId

- **deliverTo** è quello che mi invia il messaggio, ritorna e cancella il messaggio più vecchio aggiunto per recipientId che non è ancora stato mandato
- **merge** copia tutti i messaggi e gli Id dell'other manager (passato come parametro) e li copia all'interno di this, non tocca l'altro ma copia soltanto

Domanda a) specificare in JML il metodo puro fromId

La prima cosa da fare con questo tipo di esercizi dobbiamo capire quali sono i metodi ground:
Ci sta dicendo che i messaggi possono essere in due stati: aggiunti ma non ancora mandati oppure anche mandati, tutti i messaggi lasciano delle tracce tramite gli Id.

Abbiamo solo **3 metodi puri**:
- **toId()**
- **fromId()**
- **allIds()**

In questo caso dobbiamo specificare solo fromId, se usassi solo allIds() avrei solo l'insieme di tutti i messaggi passati. Perciò, mi serve qualcosa per accedere ai messaggi.

I miei **metodi ground** sono:
- **allIds()**
- **toId()**

Soluzione completa

```
//@ requires true
//@ ensures allIds().contains(senderId) && \result!=null &&
//@ (\forall long recipientId; allIds().contains(recipientId);
//@  (\forall Message m; toId(recipientId).contains(m) &&
m.senderID()==senderId;
//@       \result.contains(m) ) ) &&
//@ (\forall Message m; \result.contains(m);
//@  m.senderID()==senderId && allIds.contains(m.recipientId()) &&
//@  toId(m.recipientId()).contains(m) )
//@
//@ signals(UnknownIdException e) !allIds().contains(senderId)
public /*@ pure @*/ Set<Message> fromId(long senderId) throws UnknownIdException
{...}
```

Set<Message> /*@ pure @*/ fromId(long senderId) throws UnknownIdException ritorna tutti i messaggi aggiunti ma non ancora inviati e lancia un'eccezione nel caso il senderId non sia noto.

Spiegazione soluzione

Requires

Non ci sono requires da specificare perché non è un oggetto e inoltre tutto il dominio è valido, non abbiamo nessuna restrizione sugli Id.

```
//@ requires true
```

Signals

Se il sender non è noto dobbiamo aggiungere la signals (non cambia nulla perché è un metodo puro)

```
//@ signals(UnknownIdException e) !allIds().contains(senderId)
```

Ensures

Devo mettere in disgiunzione come sempre con la signals:

```
//@ ensures allIds().contains(senderId)
```

Poi, devo guardare per tutti gli Ids possibili, quali sono i messaggi che sono destinati a quell'id (perché il mio metodo ground è toId) perciò controllo quale è il sender e se è quello corretto allora fa parte del risultato:

Il risultato contiene tutti i messaggi validi

```
//@ (\forall long recipientId; allIds().contains(recipientId);
(\forall Message m; toId(recipientId).contains(m);
                  m.getSenderId()==senderId ⇒ ;\result.contains(m) )
&&
```

Ora dobbiamo dire che contiene solo quelli:
Per fare ciò deve essere vero che il suo sender deve essere quello corretto, poi allIds deve contenere il recipientId (mi permette di chiamare toId) e toId di m.recipientId() deve contenere il mio messaggio.

Il risultato contiene solo i messaggi validi

```
//@ (\forall Message m; \result.contains(m);
      /* il suo sender deve essere quello corretto */
      //@ m.senderID()==senderId &&
/* allIds deve contenere il recipientId (mi permette di chiamare
toId) e toId di m.recipientId() deve contenere il mio messaggio */
      //@ allIds().contains(m.recipientId()) &&

//@ toId(m.recipientId()).contains(m) )
```

Complessivamente la ensures completa:

```
//@ ensures allIds().contains(senderId)
&&
//@ (\forall long recipientId; allIds().contains(recipientId);
(\forall Message m; toId(recipientId).contains(m);
                m.getSenderId()==senderId ⇒ ;\result.contains(m) )

&&
//@ (\forall Message m; \result.contains(m);
      //@ m.senderID()==senderId &&
      //@ allIds().contains(m.recipientId()) &&
//@ toId(m.recipientId()).contains(m) )
```

Osservazione:

Sul **lato logico** la soluzione è **corretta**, tuttavia sul **lato pratico non sarebbe ideale** perchè **manca** l'aggiunta di **\result!=null**. Infatti, tutte queste clausole in un certo senso ci stanno dicendo che il risultato non può essere nullo. **(Nel compito non viene considerato errore non metterlo)**

Essendo tutte queste clausole in &&, gli esecutori di JML funzionano come Java, perciò, terminano appena una di queste è falsa. Infatti, se il risultato è nullo esce e non controlla il resto. Invece, omettendo questa clausola si rischia che, se il risultato dovesse essere nullo, al momento dell'invocazione di \result.contains(m) l'esecutore si pianta.

Domanda b) specificare in JML il metodo merge

Soluzione completa

```
//@ requires other!=null;
//@ containsAll(\old(other.allIds()) &&
//@ other.allIds().size()==\old(other.allIds().size())
&&
//@ (\forall long id; ;
//@  other.toId(id).size()==\old(other.toId(id).size())) &&
//@  (\forall int i; i>=0 && i<other.toId(id).size();
//@  other.toId(id).get(i)==\old(other.toId(id).get(i)) ) )
 ensures
//@ (\forall long id; ;
//@  allIds().contains(id) <==> (\old(allIds()).contains(id) ||
\old(other.allIds()).contains(id)) )
//@  &&
//@ (\forall long recipientId; ;
//@  toId(recipientId).containsAll(\old(toId(recipientId))) &&
//@  toId(recipientId).containsAll(\old(other.toId(recipientId)))
&&
//@  toId(recipientId).size() == \old(toId(recipientId)).size() +
other.toId(recipientId).size())
//@  &&
//@  (\forall int i; 0<=i && i<toId(recipientId).size()-1;
//@       toId(recipientId).get(i).timestamp() <=
toId(recipientId).get(i+1).timestamp()) );
//@ other.allIds().
public void merge(MessageManager other) {...}
```

Il metodo merge non è un metodo puro perchè modifica this in modo da aggiungere tutti i messaggi e tutti gli Ids che ci sono in other anche in this e non modifica other.

N:B: Bisogna specificare la mutabilità o immutabilità dei parametri perchè stiamo fornendo la specifica della classe MessageManager e other è di tipo MessageManager, quindi bisogna dire specificare come modifica la classe come conseguenza dell'invocazione di quel metodo.

Spiegazione soluzione

Requires

Richiedo la pre-condizione:

```
//@ requires other != null
```

Signals

Non sono presenti **Signals**

Ensures:

Metodo valido per le liste ma non per i set:

```
//@ (\forall int index; index >= 0 &&
//@ index < other.allIds().size();
//@ other.allIds().get(i).equals(\oId(other.allIds().get(i)) &&
//@ \toId(other.allIds()).size() == other.allIds().size()
```

è sbagliato perché non si tratta di una lista, che ha elementi in i-esima posizione, mentre set non ha il metodo get().

Alternative corrette per dire che other.allIds() non cambia:
1)

Verifico che tutti gli Id nello stato precedente ci siano anche nello stato futuro.

```
//@ \old(other.allIds()).containsAll(other.allIds()) &&
//@ other.allIds()).containsAll(\old((other.allIds()))
```

2)

Verifico che un insieme contiene l'altro e che la loro dimensione sia uguale

```
//@ \old(other.allIds()).containsAll(other.allIds()) &&
//@ \toId(other.allIds()).size() == other.allIds().size()
```

3)

Guardo tutte le componenti attraverso un forall:

```
//@ \forall(long Id; \old(other.allIds());
//@ \old(other.allIds().contains(id)) <==>
other.allIds().contains(id)
```

Ora affermo che toId non cambia:

Stessa dimensione delle liste ritornate e in ogni posizione il valore non è cambiato:

```
(\forall long id; ;
other.allIds.contains(id); other.toId(id).size() ==
\old(other.toId(id).size())
/*per lo stesso destinatario mi devi dare una lista che è lunga
uguale*/
(\forall int x; x>=0 && x<other.toId(id).size();
other.toId(id).get(x) == \old(other).toId(id).get(x)
     /*il valore di ogni posizione deve rimanere uguale per ogni
posizione*/
```

i messaggi sono entità immutabili quindi se sono "==" sono anche equals, in più non solo devi avere messaggi uguali ma non deve cambiare anche reference. Se i massaggi non fossero stati immutabili non avrei potuto usare "==" ma ".equals".

N.B: Bisogna specificare la mutabilità (immutabilità) sia toId() che allIds() perché sono entrambi metodi ground.

ora bisogna dire come cambia this:
Deve contenere tutti e soli i messaggi che conteneva this o che conteneva other.

Alternative corrette:
1)

this.allIds() contiene tutti e soli i vecchi allIds() di this e tutti i vecchi allIds() di other

```
(\forall long id; ;
allIds().contains(id) <==> \old(allIds().contains(id) ||
\old(other.allIds()).contains(id))
```

// this.toId(id) contiene la lista di tutti i messaggi che c'erano in this.toId(id) e tutti i messaggi che c'erano in other.toId(id)

Idea: adesso contengo sia tutti i messaggi che c'erano in toId sia tutti quelli che c'erano in other.toId e verifico che la dimensione non sia variata:

“per tutti gli id la lista contiene tutti i messaggi che c’erano prima o da una parte o dall’altra”

```
(\forall long id; ;
toId(id).containsAll(\old(toId(id) &&
toId(id).containsAll(\old(other.toId(id) &&
toId(id).size() == \old(toId(id).size() +
\old(other.toId(id).size())
//Aggiorno la dimensione
```

2)

Mantengo comunque il messaggio quando è contenuto da entrambe le parti:

```
(\forall long id; ;
toId(id).containsAll(\old(toId(id) &&
toId(id).containsAll(\old(other.toId(id) &&
(\forall Message m; ; toId(id).contains(m) < == >
\old(toId(id)).contains(m) ||
\old(other.toId(id).contains(m))
/*scorro tutti i messaggi e affermo che se erano contenuti in
toId() o in other.toId(), allora sono contenuti ora in toId() e
viceversa.*/
```

Idea: l’ordine deve rimanere uguale. Presa una posizione e quella successiva, la posizione di quella successiva deve rimanere quella precedente +1.

toId() rimane ordinato per timestamp:

```
(\forall long id; ;
\forall int x; x >= 0 && x < toId(x).size() -1;
  toId(id).get(x).timestamp() <=
toId(id).get(x+1).timestamp())
```

: Esercizi specifiche(28/11/2023 Alessandro Di Maria, Tommaso Crippa)

****Continuazione esercizio dell'ultima lezione****

Versione alternativa di fromId, che predica su tutti i messaggi e mette il se e solo se in un'unica formula; per un decisore automatico però la precedente soluzione sarebbe migliore in quanto il dominio dei messaggi da controllare è limitato.

fromId - specifica

```
1 //@ (\forall Message m; ;
2 //@     \result.contains(m) <==>
3 //@     m.getSenderId() == senderID &&
4 //@     allIds().contains(m.reciepentId()) &&
5 //@     allIds().contains(m.senderId()) &&
6 //@     toId(m.recipientId()).contains(m))
7 //@
8 //@ (\forall long id; ;
9 //@     (\forall int x; x >= 0 && x < toId(id).size - 1;
10 //@         toId(id).get(x).timestamp() <==>
11 //@         toId(id).get(x+1).timestamp() &&
12 //@         (\forall int y; y > x && y < toId(id).size();
13 //@             !toId(id).get(x).equals(toId(id)).get(y) )))
```

Tema d'esame 23/07/2023

Si consideri la seguente classe immutabile `Message`. Ogni messaggio ha un contenuto, un mittente, un destinatario e un timestamp. Mittente e destinatario sono individuati univocamente da un identificatore, per semplicità un long. Ancora per semplicità, il timestamp è un long (anziché un `Date`).

```
public /*@ pure @*/ class Message {
  public Message(byte[] content, long recipientID, long senderID, long timestamp) {...}
  public byte[] content() {...}     // returns the content of this
  public long recipientID() {...}   // returns the recipientID of this
  public long senderID() {...}      // returns the senderID of this
  public long timestamp() {...}     // returns the timestamp of this
}
```

La seguente classe `MessageManager` descrive un gestore di messaggi. È possibile inviare ed estrarre messaggi, vedere elenchi dei messaggi e degli utenti; in particolare, è possibile fondere un altro gestore messaggi con quello corrente.

```
public class MessageManager {
  // Builds a new empty manager
  public MessageManager() {...}

  // Returns the set of all valid ids, i.e., ids of recipients or senders of some message in the past.
  public /*@ pure @*/ Set<Long> allIds() {...}

  // Returns all messages yet to be delivered to the recipient having the given recipientId,
  // from the oldest to the newest.
  // Returns an empty list if the recipient is unknown or if there is no message to be delivered it.
  public /*@ pure @*/ List<Message> toId(long recipientId) {...}

  // Returns all messages, yet to be delivered, sent from the sender having the given senderId.
  // Throws an UnknownIdException if the sender is unknown.
  public /*@ pure @*/ Set<Message> fromId(long senderId) throws UnknownIdException {...}

  // Inserts a new message in the manager, with the timestamp at the instant of the call.
  public void addMessage(byte[] content, long senderId, long recipientId) {...}

  // Returns and deletes the oldest message added for recipientId and not yet delivered.
  // Throws a NoMessageExpetion if no message is present.
  // Throws an UnknownIDException if the id is unknown.
  public Message deliverTo(long recipientId) throws NoMessageException, UnknownIDException{...}

  // Copies all messages and ids in the manager other to this.
  // The manager other is unchanged.
  public void merge(MessageManager other) {...}
```

Domanda c)

Si consideri la seguente semplice implementazione di MessageManager.

Implementazione MessageManager

```
public class MessageManager {


   private Set<Long> validIds;

    private List<Message> pendingMessages;

    ...


}
```

I messaggi sono inseriti in ordine di arrivo nella List, aggiornando anche validIds se necessario. Una volta inviati (mediante invocazioni di deliverTo) i messaggi sono cancellati da pendingMessages. Scrivere in JML l'invariante di rappresentazione e la funzione di astrazione.

Identifichiamo gli attributi ed i metodi su cui dovremo predicare durante la specifica dell'invariante di rappresentazione (RI) e la funzione di astrazione (AF)

Metodi ground (Metodi pubblici)

- `Set<Long> allIds;`
- `List<Message> toId(long recipientId)`

Implementazione (Attributi privati)

- `private Set<Long> validIds;`
- `private List<Message> pendingMeassages;`

Definiamo l'invariante di rappresentazione: l'insieme delle formule logiche che descrivono quando un oggetto concreto è correttamente implementato.

N.B. Durante la specifica del RI possiamo predicare solo sugli attributi privati

Dobbiamo definire:

- (righe 2-3) validIds non deve essere nullo / non deve contenere valori nulli
- (righe 4-6) Se un messaggio è in pendingMessages allora il suo senderId e il suo RecipientId sono in validIds
- (righe 7-11) pendingMessages deve essere ordinata secondo il timestamp e non deve contenere duplicati

Invariante di rappresentazione

```
1 //@ private invariant
2 //@ validIds != null && !validIds.contains(null) &&
3 //@ pendingMessages != null && !pendingMessages.contains(null)
4 //@ (\forall Message m; pendingMessages.contains(m);
5 //@     validIds.contains(m.senderId()) &&
6 //@     validIds.contains(m.recipientId()) ) &&
7 //@ (\forall int x; x>=0 && x<pendingMessages.size();
8 //@     (\forall int y;  y>x && y<pendingMessages.size();
9 //@         !pendingMessages.get(x).equals(pendingMessages.get(y)) &&
10 //@ pendingMessages.get(x).timestamp() <=
11 //@ pendingMessages.get(y).timestamp() ))
```

⚠ Attenzione!

Long != **l**ong
Il primo è una **classe**, può quindi avere valore null
Il secondo è un tipo **primitivo**, non può essere null
validIds è un set di **L**ong
(perché le Collections non sono definite per i tipi primitivo),
è per questo che devo specificare: !validIds.contains(null)

N.B. Non esiste una soluzione unica per l'invariante di rappresentazione, ma le decisioni prese su di esso influenzano la specifica della funzione di astrazione

Esempio: Se validIds fosse implementata come una Lista, si poteva:

1. Specificare nel RI che non si possono avere duplicati: allora durante la costruzione della abstract function non dobbiamo considerarlo
2. Non specificare l'assenza di duplicati nel RI -> DOBBIAMO specificarlo in AF, altrimenti la specifica sarebbe incompleta

Ora per creare la funzione di astrazione dobbiamo specificare come i metodi puri ground si integrano con l'implementazione definita

- (righe 2-3) Specifichiamo allIds
- (righe 4-5) Specifichiamo toId()

Funzione di astrazione

```
1 //@ private invariant
2 //@ (\forall long id; ;
3 //@     allIds().contains(id) <==> validIds.contains(id) &&
4 //@     (\forall Message m; ;
5 //@         toId(m.recipientId()).contains(m) <==>
6 //@ PendingMessages.contains(m) ))
```

Domanda d)

Si consideri una classe ClearableMessageManager che aggiunge alla classe MessageManager il metodo clear che cancella tutti gli id e tutti i messaggi pendenti. Puo` ClearableMessageManager essere definita come sottoclasse di MessageManager in accordo con il principio di sostituzione di Liskov? Si motivi la risposta

Dobbiamo verificare se la sottoclasse viola le regole dei metodi o delle proprietà della classe originaria.

- Prima domanda da porsi: **È un'estensione pura?**

Sì, il metodo "clear" non cambia la specifica di alcun metodo esistente, dunque non può violare la regola dei metodi.

- Seconda domanda: **Può cambiare qualche proprietà invariante o evolutiva?**

Sì, perché rimuove gli id, cosa che non era possibile nella classe originaria, dunque viola la regola delle proprietà.

La sottoclasse non può quindi essere definita.

Tema d'esame 26/08/2022

Si consideri la seguente classe `SeriesStore` per memorizzare episodi di serie TV. Gli episodi sono rappresentati dalla classe immutabile `Episode` che espone tra gli altri un metodo per ottenere la serie di appartenenza di un episodio.

```
public class SeriesStore {
  // Ritorna il primo episodio della serie passata per parametro.
  // Ritorna null se la serie non esiste.
  public /*@ pure @*/ Episode getFirstEpisode(String series);

  // Ritorna l'episodio successivo a quello passato per parametro.
  // Ritorna null se tale episodio non esiste, ovvero se quello passato per parametro
  // e' l'ultimo della serie.
  public /*@ pure @*/ Episode getNextEpisode(Episode ep);

  // Ritorna il numero di episodi della serie successivi a quello passato per parametro.
  // Ritorna 0 se la serie non esiste.
  public /*@ pure @*/ int numNextEpisodes(Episode ep);

  // Aggiunge l'episodio in fondo alla sua serie di appartenenza. Se la serie ancora
  // non esiste, l'episodio viene aggiunto come il primo di quella serie.
  // Lancia una IllegalEpisodeException se l'episodio passato come parametro e' null.
  public void addEpisode(Episode ep) throws IllegalEpisodeException;
}

public /*@ pure @*/ Episode {
  // Ritorna la seria di appartenenza dell'episodio.
  public /*@ pure @*/ String getSeries();
... }
```

Domanda c)

Si consideri la seguente implementazione della classe SeriesStore che usa una mappa per associare a ogni serie la lista dei suoi episodi. Si ricorda che una Map e una struttura dati che associa a chiavi di tipo K (String in questo caso) valori di tipo V (List in questo caso). Una Map fornisce, fra gli altri, i metodi osservatori get(K key), che restituisce il valore univoco associato alla chiave key, e containsKey(K key), che restituisce true se esiste un valore associato alla chiave key. Altri metodi sono ad esempio size() e isEmpty(), di significato ovvio.

Implementazione SeriesStore

```
public class SeriesStore {


private Map<String, List<Episode>> episodes;

...


}
```

Per tale implementazione, si forniscano l'invariante di rappresentazione e la funzione di astrazione.

Individuiamo i componenti principali per rispondere alla domanda

I **metodi ground** sono:

- `Episode getFirstEpisode(String series)`
- `Episode getNextEpisode(Episode ep)`

L'implementazione è

- `private Map<String, List<Episode>> episodes>`

Le caratteristiche del Representation Invariant devono essere:

- (righe 2-3) La struttura dati non deve essere nulla e non deve contenere chiavi e valori nulli
- (righe 4-5) La Lista di episodi non deve essere vuota e per ogni episodio nella lista la serie è quella della chiave
- (righe 6-8) non ci sono duplicati (scelta ragionevole data la specifica)

Invariante di rappresentazione

```
1 //@ private invariant
2 //@ episodes != null && !episodes.containsKey(null) &&
3 //@ !episodes.containsValue(null) &&
4 //@ (\forall String s; episodes.containsKey(s);
5 //@     !episodes.get(s).isEmpty() &&
6 //@     (\forall int x; x>=0 && x<episodes.get(s).size();
7 //@         episodes.get(s).get(x) != null &&
8 //@         episodes.get(s).get(x).getSeries().equals(s) ))
```

Nel definire la Abstraction Function invece dobbiamo definire i metodi puri a partire da episodes.

(Possiamo fare l'assunzione che gli episodi nella lista siano in ordine)

- (righe 2-6) Specifico getFirstEpisode per ogni valore di s.
- (righe 3-4) Se episodes ha s come chiave, allora getFirstEpisode restituisce il primo episodio
- (righe 5-6) Se episodes non ha s come chiave allora non ci sono episodi di s, quindi getFirstEpisode(s) == null.
- (righe 8-15) Specifico nextEpisode.

Il valore di getNextEpisode(e) dipende dall'esistenza della serie di e nelle chiavi di episodes (riga 10) e dal fatto che e non sia l'ultimo elemento della lista (righe 11-12). Se entrambe le condizioni sono soddisfatte allora getNextEpisode(e) è uguale all'elemento successivo della lista (righe 13-14), altrimenti è null.

Funzione di Astrazione

```
1 //@ private invariant
2 //@ (\forall String s; s!=null;
3 //@     episodes.containsKey(s) ==>
4 //@     getFirstEpisode(s).equals(episodes.get(s).get(0)) &&
5 //@     !episodes.containsKey(s) ==>
```

```
6 //@     getFirstEpisode(s) == null) &&
7 //@
8 //@ (\forall Episode e; e!=null;
9 //@     getNextEpisode(e) ==
10 //@     (episodes.containsKey(e.getSeries()) &&
11 //@     (episodes.get(e.getSeries()).indexOf(e) <
12 //@     episodes.get(e.getSeries()).size()-1) ?
13 //@     episodes.containsKey(e.getSeries()).get(
14 //@     episodes.get(e.getSeries()).indexOf(e)+1)
15 //@     : null)
```

💡 Tip!
Definizione indexOf(e) Restituisce l'indice in base zero della prima occorrenza di e all'interno dell'istanza

Domanda d)

Si consideri una classe SeriesStore2 che aggiunge un metodo addEpisodes che prende in ingresso una lista di episodi e li aggiunge allo store. Puo' tale classe essere definita come sottoclasse di SeriesStore in accordo con il principio di sostituzione?

- Viola le specifiche degli altri metodi?

No, è un metodo nuovo, non cambia le definizioni degli altri metodi

- Viola le proprietà della classe?

No, chiamare addEpisodes corrisponde a una sequenza di chiamate di addEpisode nella classe originaria, non viola nessuna proprietà

Dunque SeriesStore2 può essere definita come sottoclasse.

Tema d'esame 28/08/2023

```
public class CityMap {
   // Ritorna l'insieme di tutti i punti di interesse memorizzati
   public /*@ pure @*/ Set<Site> getSites() { ... }

   // Aggiunge il punto di interesse passato per parametro
   public void addSite(Site site) { ... }

   // Ritorna tutti i punti di interesse del tipo passato per parametro,
   // ordinati per distanza crescente rispetto alla location loc indicata.
   // Lancia una NotFoundException se non esiste alcun punto di interesse del tipo indicato.
   public /*@ pure @*/ List<Site> getSitesByType(Type type, Location loc) throws NotFoundException { ... }

   // Ritorna i k punti di interesse piu' vicini alla location loc indicata
   // (per semplicita', si assuma che non possano esistere due punti equidistanti da loc).
   // Se i punti di interesse sono meno di k, vengono ritornati tutti.
   // La mappa ritornata contiene i punti di interesse come chiave e la loro distanza da loc come valore.
   public /*@ pure @*/ Map<Site, Double> getNearestSites(int k, Location loc) { ... }
}

public /*@ pure @*/ class Site {
   // Ritorna la posizione geografica del punto di interesse
   public Location getLocation() { ... }

   // Ritorna il tipo del punto di interesse (ristorante, hotel, museo, ...)
   public Type getType() { ...}
... }

public /*@ pure @*/ class Location {
   // Calcola la distanza rispetto alla Location passata per parametro
   public double getDistance(Location other) { ... }
... }
```

Domanda d) Si consideri una classe MergeableCityMap, che aggiunge un metodo void merge(CityMap other) che aggiunge tutti i punti di interesse presenti in other. Puo` MergeableCityMap essere definita come sottoclasse di CityMap in accordo con il principio di sostituzione di Liskov? Si motivi la risposta

Se non viola un invariante pubblico su other (se quindi non modifico lo stato di other), è un'estensione pura, perché aggiunge un nuovo metodo: "merge". Il risultato che ottengo chiamando merge è equivalente a chiamare addSite per ogni Site di other, quindi non viola neanche la regola delle proprietà.

Tema d'esame 13/02/2023

Si consideri la seguente classe `Championship` che memorizza i risultati di un campionato tra diversi `Team`. Si noti che il campionato distingue tra andata e ritorno di ogni partita.

```
public class Championship {
  // Ritorna l'insieme dei team che fanno parte del campionato.
  public /*@ pure @*/ Set<Team> getTeams() {...}

  // Cancella tutti i risultati delle partite fin qui giocate.
  public void clear() {...}

  // Ritorna il team vincente nella partita giocata tra t1 (in casa) e t2 (fuori casa), oppure null
  // se non si e' giocata nessuna partita tra t1 e t2 (t1 e t2 devono essere squadre del campionato).
  // Si noti che il valore restituito potrebbe non coincidere esattamente con t1 o t2, ma deve essere
  // uguale ("equals") a uno dei due.
  public /*@ pure @*/ Team winner(Team t1, Team t2) {...}

  // Inserisce il risultato della partita tra t1 (in casa) e t2 (fuori casa) con vincitore winner
  // (riguardo t1, t2 e winner valgono le stesse note indicate per il precedente metodo).
  // Solleva l'eccezione MatchAlreadyPlayedException se la partita e' gia' stata giocata.
  public void addMatch(Team t1, Team t2, Team winner) throws MatchAlreadyPlayedException {...}

  // Ritorna l'elenco dele squadre con cui la squadra t ha vinto giocando in casa.
  public /*@ pure @*/ List<Team> defeatedAtHome(Team t) {...}

  // Ritorna il punteggio accumulato dalla squadra t (la squadra vincente, in casa o fuori casa,
  // prende un punto).
  public /*@ pure @*/ int numPoints(Team t) {...}
```

Domanda a) Specificare in JML il metodo addMatch.

Scegliamo come **metodi ground**:

- `Public Set<Team> getTeams():` predica sulle squadre
- `Public Team Winner(Team t1, Team t2):` predica sulle partite

A partire da questi specifichiamo addMatch:

REQUIRES

- (righe 2-3) Richiediamo che le due squadre non siano nulle e siano contenute in getTeams.
- (righe 4) Il vincitore deve essere uno delle due squadre e le due squadre devono essere diverse fra loro.

addMatch - requires

```
1 //@ requires
2 //@ t1!=null && t2!=null && !t1.equals(t2) &&
3 //@ getTeams().contains(t1) && getTeams().contains(t2) &&
4 //@ (t1.eqals(winner)|| t2.equals(winner))
```

SIGNALS

- (righe 1-2) Lanciamo l'eccezione MatchAlreadyPlayedException quando il valore di winner prima della chiamata è diverso da null.
- (righe 3-7) Non essendo un metodo puro dobbiamo anche specificare che niente cambi
- (righe 3-4) Il risultato di getTeam rimane invariato.
- (righe 5-7) Per ogni coppia team il risultato di winner rimane invariato.

addMatch - signals

```
1 //@ signals (MatchAlreadyPlayedException e)
2 //@ \old(winner(t1, t2)) != null &&
3 //@ getTeams().containsAll(\old(getTeams())) &&
4 //@ getTeams().size() == \old(getTeams()).size() &&
5 //@ (\forall Teams tt1; getTeams().contains(tt1);
6 //@     (\forall Teams tt2; getTeams().contains(tt2);
7 //@         winner(tt1, tt2) == \old(winner(tt1, tt2))
```

ENSURES

- (riga 2) Mettiamo la condizione della signals negata, il vecchio risultato di winner deve essere null mentre, dopo la chiamata del metodo, il nuovo vincitore tra t1 e t2 è pari a winner.

- (riga 3-8) Come nella signals, non essendo un metodo puro dobbiamo specificare che niente cambi. Tranne il caso in cui consideriamo il risultato di winner(t1, t2) (coperto già dalla specifica alla riga 2).

addMatch - ensures

```
1 //@ ensures
2 //@ \old(winner(t1, t2)) == null && winner(t1, t2)) == winner &&
3 //@ getTeams().containsAll(\old(getTeams())) &&
4 //@ getTeams().size() == \old(getTeams()).size() &&
5 //@ (\forall Teams tt1; getTeams().contains(tt1) ;
6 //@     (\forall Teams tt2; getTeams().contains(tt2);
7 //@         (!tt1.equals(t1) || !tt2.equals(t2)) ==>
8 //@         winner(tt1, tt2) == \old(winner(tt1, tt2))
```

Domanda b) Specificare in JML i metodi defeatedAtHome e numPoints

N.B. La specifica di defeatedAtHome è un po' ambigua, consideriamo l'output di defeatedAtHome come la lista di quelli battuti da t mentre giocava in casa(sua).

A partire dai metodi ground (definiti sopra) specifichiamo defeatedAtHome:

REQUIRES

- (riga 2) t deve essere diverso da null e contenuto in getTeams.

ENSURES

- (righe 5-7) Per ogni team tt il risultato contiene tt se e solo se e è presente in getTeams ed è stato battuto giocando in trasferta dal team t.
- (riga 8) precondizione per chiamare i metodi su result.
- (righe 9-11) la lista risultato non contiene team duplicati.

defeatedAtHome - specifica

```
1 //@ requires
2 //@ t!=null && getTeams().contains(t)
3 //@
4 //@ ensures
5 //@ (\forall Team tt; ;
6 //@     \result.contains(tt) <==> getTeams().contains(tt) &&
7 //@     winner(t, tt).equals(t)) &&
8 //@ \result != null &&
```

```
9 //@     (\forall int x; 0<=x && x<\result.size();
10 //@         (\forall int y; y>x && y<\result.size();
11 //@             !\result.get(x).equals(\result.get(y) )))
```

numPoints

REQUIRES

- (riga 2) t deve essere diverso da null e contenuto in getTeams.

ENSURES

- (righe 5-12) Il risultato è pari al numero di team tt contenuti in getTeams e diversi da t battuti da t in casa più il numero di quelli battuti da t in trasferta.

N.B. Non possiamo mettere un'unica numof con all'interno un or per i casi di vittoria in trasferta o in casa perché considereremmo meno punti per t nel caso di vittoria in entrambe le giornate.

numPoints - specifica

```
1 //@ requires
2 //@ t!=null && getTeams().contains(t)
3 //@
4 //@ ensures
5 //@ \result ==
6 //@ (\numof Team tt;
7 //@     getTeams().contains(tt) && !tt.equals(t);
8 //@     winner(t,tt).equals(t))
9 //@     +
10 //@     (\numof Team tt;
11 //@     getTeams().contains(tt) && !tt.equals(t);
12 //@     winner(tt,t).equals(t)
```

Domanda c) Si consideri una implementazione della classe Championship che usa una lista di Team per memorizzare le squadre del campionato e un array bidimensionale di Team per memorizzare le partite giocate

Implementazione Championship

```
public class Championship {


    private List teams;
```

```
    private Team[][] res;

    …


}
```

La posizione res[i][j] vale null se la partita tra l'i-esima squadra (in casa) e la j-esima squadra (fuori casa) nella lista teams non si è ancora giocata, mentre contiene un riferimento alla squadra vincente se la partita si è giocata. Per questa implementazione scrivere l'invariante di rappresentazione e la funzione di astrazione

Per quanto riguarda il Representation Invariant:

Non vogliamo che:

- (riga 2) teams sia nullo
- (riga 2) teams contenga elementi nulli
- (riga 2) res sia nullo
- (riga 5) res[i] sia nullo

N.B res[i][j] può essere nullo (la partita non è stata ancora giocata)

E vogliamo che:

- (riga 3) la lunghezza res sia uguale alla lunghezza teams
- (riga 5) Lunghezza res[i] sia uguale alla lunghezza teams
- (righe 6-8) Dentro res[i][j] posso avere: null || teams[i] vincitore || teams[j] vincitore

N.B. Non consideriamo il caso in cui i == j perché nella specifica di addMatch richiedo che t1 != t2, quindi non deve essere valutato dall'invariante di rappresentazione

Invariante di rappresentazione

```
1 //@ private invariant
2 //@ teams!= null && !teams.contains(null) && res != null &&
3 //@ res.length==teams.size() &&
4 //@ (\forall int i; i>= 0 && i < teams.size();
5 //@     res[i] != null && res[i].length==teams.size() &&
6 //@     (\forall int j; j>= 0 && j < teams.size() && i!=j
7 //@             res[i][j]==null || res[i][j]==teams.get(i) ||
8 //@ res[i][j]==teams.get(j) ))
```

Funzione di astrazione

I **metodi ground** sono:

- Public Set<Team> getTeams()
- Public Team Winner(Team t1, Team t2)

Nella funzione di astrazione:

- (riga 2) getTeams e teams contengono gli stessi elementi.
- (righe 3-5) verifichiamo la corretta implementazione di res cioè per ogni team il vincitore sia uguale al risultato di winner tra gli stessi team.

Funzione di astrazione

```
1 //@ private invariant
2 //@ getTeams().containsAll(teams) && getTeams.size() == teams.size()
3 //@ (\forall int i; i >=0 && i < teams.size();
4 //@     (\forall int j; j >=0 && j < teams.size();
5 //@         winner(teams.get(i), teams.get(j)).equals(res[i][j])
```

Domanda d) Si consideri una classe ReplayableChampionship che aggiunge alla classe Championship il metodo pubblico: void replayMatch(Team t1, Team t2, Team winner) ReplayableChampionship puo essere definita come sottoclasse di ` Championship senza violare il principio di sostituzione di Liskov?

È un'estensione pura, perché la sottoclasse aggiunge un nuovo metodo: "replayMatch" (senza violare la regola dei metodi) e inoltre non viola neanche la regola delle proprietà grazie al metodo clear. Il risultato che ottengo chiamando replayMatch (t1, t2, winner) corrisponde a chiamare successivamente clear() e winner(t1, t2, winner) (ovviamente andrebbero reinseriti anche i risultati precedenti).

N.B. Se il metodo clear non fosse presente nella classe Championship, verrebbe invece violata la regola delle priorità, dato che non sarebbe possibile replicare replayMatch con i metodi di Championship.

Lezione 24: Testing (30/11/2023 Davide Ciamacco, Simone Checchia)

Q1. Verifica vs Validazione	
1)	La verifica determina la correttezza del software rispetto alla specifica
2)	La validazione non richiede mai l'intervento dell'utente
3)	La validazione risponde alla domanda "stiamo costruendo il prodotto giusto"
4)	La verifica può solo dimostrare che il prodotto sia corretto, non che colga i reali bisogni dell'utente

Verifica = Controlla che il software sia corretto rispetto alla specifica. Risponde alla domanda: "Stiamo costruendo il prodotto correttamente"

Validazione = Controlla che il software sia coretto rispetto alla richiesta dell'utente. Risponde alla domanda: "Stiamo costruendo il prodotto giusto"

1) Corretta perché corrisponde con la definizione di verifica

2) No, perché la validazione deve andare a verificare rispetto all''utente

3) Corretta perché corrisponde con la definizione di validazione

4) Corretta perché la verifica si concentra solo sulla correttezza del software rispetto alla specifica

Q2. Verifica statica	
1)	Si può utilizzare per individuare problemi nei documenti di progetto prima ancora di aver realizzato il prodotto finito
2)	Si applica solo a prodotti eseguibili
3)	Può essere autorizzata se il semilavorato sottoposto a verifica è espresso con un linguaggio formale
4)	Può essere automatizzata per trovare alcuni difetti nel codice

L'analisi statica è una tecnica che consente di analizzare dei semilavorati software senza eseguirli. È eseguita attraverso ispezioni della rappresentazione dei diversi semilavorati software per scoprire problemi. Attraverso l'esecuzione di strumenti automatici di analisi solo se la rappresentazione è espressa con un linguaggio formale.

1) Corretta perché l'analisi statica non necessariamente si riferisce al codice, ma a qualunque facente parte del mio processo di sviluppo (es. documenti di specifica)

2) No, per definizione

3) Si per definizione. JML è un esempio di forma per la verifica.

4) Si, per definizione

Q3. Quale delle seguenti affermazione sono vere per il testing?	
1)	In generale non è possibile testare in maniera esaustiva un software
2)	La bontà di una attività di testing si può misurare in funzione del numero di casi di test utilizzati
3)	Permette di identificare problemi che non sarebbe possibile trovare tramite verifica statica
4)	Se ben eseguito permette di trovare tutti i problemi di un software

Il testing è una verifica dinamica. Si applica al risultato finale dello sviluppo, ovvero l'eseguibile, con lo scopo di far emergere quanti più difetti possibile. Attraverso il solo testing non potremo mai concludere che il software prodotto è corretto.

1) Si, in generale non è possibile testare in maniera esaustiva un software. Per esempio, una funzione che riceve in ingresso due parametri int avrebbe bisogno di 2^{32} casi di test.

2) No, il numero di casi di test non è una metrica significativa. La qualità dei casi di test è più importante della quantità.

3) Si, perché la verifica statica non può catturare problemi che avvengono a runtime

4) No, perché in generale non è possibile testare in maniera esaustiva un software

Q4. L'ispezione del software	
1)	Richiede una specifica formale
2)	È una tecnica di verifica dinamica
3)	Permette di individuare in una sola sessione diversi errori
4)	Tende a trovare errori di pochi tipi ricorrenti

L'ispezione del software è una verifica secondo una lista di eventuali errori che potrebbe avere il nostro programma.

1) No, non richiede una specifica formale. Per esempio, l'ispezione del software si preoccupa se tutte le variabili sono state inizializzate

2) No, l'ispezione non avviene a runtime, ma solo sul codice

3) Si, fino a tutti i facenti parte della lista

4) Si, solo quelli nella lista

Q5. L'analisi statica automatica	
1)	Richiede che il semilavorato sotto analisi sia espresso in un linguaggio formale
2)	Permette di verificare qualsiasi proprietà del software
3)	Permette di rilevare il caso di variabili definite e mai usate
4)	Permette di trovare problemi di prestazioni

1) Si, per essere automatica ha bisogno di un linguaggio formale (vedere definizione analisi statica, Q2)

2) No, perché esistono problemi non decidibili

3) Si, perché è un problema decidibile

4) No, perché richiede l'esecuzione

Q6. Test di regressione

1)	Si riferisce solo alle singole unità (moduli) del software
2)	Si riferisce solo all'intero sistema
3)	Coincide con il beta test
4)	Serve a verificare che una nuova versione non comprometta caratteristiche soddisfatte nelle versioni precedenti

Il test di regressione verifica che le caratteristiche positive consolidate persistano anche nelle versioni nuove.

1) No, perché il test di regressione è traversale rispetto alla granularità del test ovvero che può essere eseguito a diversi livelli

2) No, come precedente

3) No, il beta test è un diverso tipo di test

4) Si, per definizione

Q7. Il test mira a trovare...	
1)	Malfunzionamenti (failure)
2)	Errori nel codice (error)
3)	Difetti nel codice (fault)

Error: Il programmatore sbaglia concettualmente il codice e causa un fault

Fault: Il codice è scritto in maniera errata (es. > invece di <=)

Failure: Se l'esecuzione raggiunge la riga di codice con un fault genera un malfunzionamento del programma

I test mirano a trovare i malfunzionamenti

Q8. Si consideri la funzione a parametri interi che calcola la radice n-esima di un numero m: radice(n, m). Tale funzione ammette valori negativi per il parametro m solo se n è dispari. Inoltre n deve essere diverso da 0. Ciò premesso, quali dei seguenti valori di input definiscono casi limite?	
1)	n=2, m=0
2)	n=0, m=2
3)	n=-2, m=2
4)	n=-2, m=0
5)	n=3, m=0
6)	n=3, m=-2

Per caso limite si intendono i valori dei parametri al limite del loro dominio. Non implica che i casi limite siano interessanti, perché è solo uno dei criteri applicati per verificare un software.

Escludiamo i casi tali per cui i valori dei parametri non ricadono all'interno del proprio dominio (es. n=0, m=2).

Q9. Quali dei seguenti valori di input definiscono casi limite per la funzione "boolean isCap (String s)" che verifica la stringa s sia un codice di avviamento postale...	
1)	s="20100"
2)	s=""
3)	s=null
4)	s="aaaaa"

Analogamente con quanto enunciato nel quesito precedente: i casi limite del dominio di una variale String sono il caso in cui la variabile non è definita (null) e il caso, quando è definita, nella sua forma minima (s="")

Q1. Il test strutturale	
1)	Testa sempre una funzionalità in maniera esaustiva
2)	Non richiede la conoscenza dei risultati attesi
3)	Offre diversi criteri di copertura, caratterizzati da diversi compromessi tra costi ed efficacia
4)	Garantisce l'assenza di malfunzionamenti una volta che sono state coperte tutte le decisioni e condizioni semplici

Il testing strutturale tiene conto della struttura interna del programma e i casi di test sono derivati da un'analisi diretta dell'implementazione (ovvero dal codice del programma).

1) No, coerentemente con quanto detto precedentemente il testing non è in grado di proporre una soluzione esaustiva

2) No, di norma una sequenza di test è un'invocazione di una funzionalità con determinati input e relativo risultato atteso

3) Si, ad esempio la copertura di decisioni, condizioni, cammini più o meno complesse da coprire caratterizzate da compromessi di costi ed efficacia

4) No, non dà garanzia totale: segnala se qualcosa non va, ma non può dimostrare in assoluto l'assenza di errori

Q2. Uno stub	
1)	Richiede una conoscenza approfondita dei dettagli implementativi del modulo che simula
2)	Simula un modulo che invoca funzioni di interfaccia del componente oggetto del test
3)	Richiede la conoscenza della specifica del modulo che deve simulare
4)	È sempre usato in una strategia di testing bottom-up

1) No, è necessario simulare lo stesso comportamento, ma non la medesima implementazione

2) No, è la definizione opposta (Simula la parte di programma che invoca l'unità oggetto del test)

3) Si, è necessario conoscere cosa viene ritornato, non come.

4) No, al contrario viene utilizzato nella programmazione top-down

Q3. Un Driver	
1)	Richiede la conoscenza dell'implementazione del modulo che deve stimolare
2)	Richiede la conoscenza della specifica del modulo che deve stimolare
3)	È sempre usato in una strategia di testing top-down
4)	Stimola il modulo oggetto del test in maniera diversa rispetto a come accadrebbe nel sistema finale

1) No, non è richiesta l'implementazione, ma la specifica

2) Si, per definizione, devo conoscere pre e post condizioni per riconoscere input e output validi

3) No, al contrario dello stub, è relativo alla strategia bottom-up

4) No, al contrario dovrebbe stimolarlo esattamente secondo l'uso inteso nella componente finale

Q4. Quali sono le caratteristiche della strategia di test "Big Bang"	
1)	Richiede la costruzione di un numero limitato di driver
2)	Richiede la costruzione di un numero limitato di stub
3)	Non richiede il test di unità dei singoli componenti software
4)	Rende difficile localizzare un errore, perché tutti i moduli vengono testati insieme

In una strategia di test "Big Bang" le componenti sono prima testate individualmente e poi testate insieme come un singolo sistema. Sebbene sia di semplice realizzazione è molto costoso e rende impossibile il processo di ricerca delle componenti fallacie in caso di errore.

1) No, in quanto non interviene un approccio ordinato che permette di evitare di costruire stub e driver riutilizzando software esistenti

2) No, come sopra

3) No, è necessario continuare a testarli in isolamento tutti e poi metterli insieme

4) Si, testando moduli separatamente, non è noto da cosa potrebbe venire il problema

Q5. Test bottom-up vs top-down	
1)	Nell'approccio bottom-up è necessario definire driver
2)	Nell'approccio bottom-up è necessario definire gli stub
3)	In entrambi i tipi di test è necessario definire sia driver che stub
4)	Nell'approccio top-down è necessario definire i driver

5)	Richiedono di norma la scrittura di più codice rispetto all'approccio big-bang

1) Si, bottom-up necessita i driver poiché partendo dal basso la sollecitazione avviene dall'alto

2) No, perché in contraddizione con 1)

3) No, perché bottom-up necessità di driver mentre il top-down necessità degli stub

4) No, perché in contraddizione con 1)

5) No, perché l'approccio Big Bang, che avviene in ordine sparso, richiede la sollecitazione di tutti i singoli moduli e quindi necessità di più codice

Q6. Nel test delle interfacce	
1)	Si valutano i singoli moduli, ma non la loro integrazione
2)	È necessario considerare le risorse condivise tra diversi moduli (ad esempio oggetti in memoria condivisa)
3)	Si tiene conto solo di requisiti funzionali
4)	Non si tiene conto dell'ordine con cui vengono invocati i metodi

1) No, non è sempre vero

2) Si, è possibile invocare metodi che modificano lo stato di una determinata risorsa. È necessario tenere in considerazione come queste risorse sono mutate

3) No, potrebbe essere richiesto di misurare requisiti non funzionali (es. prestazioni, memoria occupata, sicurezza, scalabilità)

4) No, deve tenere conto dell'ordine di invocazione

Q7. Qual è il numero minimo di casi di test necessari per garantire la copertura degli statement del metodo in figura?

```
public void foo(int a[], int x) {
        int i=0;
        while (i<a.length%3){
            if (a[i]>0 && a[i]<x) a[i]=x;
            a[i]=-x;
            i++;
        }
}
```

1)	1
2)	2
3)	3
4)	Non è possibile soddisfare il criterio

Per la copertura degli statement è necessario che i casi di test eseguano tutte le righe di codice del metodo. Prendendo un array di dimensione 1 è possibile entrare nel ciclo while (nella prima iterazione i=0 e a.lenght=1 quindi soddisfano le condizioni del while. Da questo si evince che l'array non dovrà essere vuoto). Scegliendo accuratamente il valore del singolo elemento (es. a[0] = 3) e il valore di x (es. 5) le condizioni all'interno dell'if verranno soddisfatte. Con questo unico caso di test è possibile quindi eseguire tutto il codice del modulo.

Q8. Qual è il numero minimo di casi di test necessari per garantire la copertura delle decisioni (edge) del metodo in figura?	
1)	1
2)	2
3)	3
4)	Non è possibile soddisfare il criterio

Per la copertura delle decisioni, i casi di test devono eseguire tutte le scelte di fronte a una decisione (es. un test che esegue if e un test che esegue then)

Con un singolo caso di test è possibile soddisfare contemporaneamente tutte le scelte nel codice. Nella prima iterazione esegue il codice del while e dell'if, con la seconda iterazione non entra né nel while né nell'if.

Q9. Qual è il numero minimo di casi di test necessari per garantire la copertura delle decisioni e condizioni (edge and condition) del metodo in figura?	
1)	1
2)	2
3)	3
4)	Non è possibile soddisfare il criterio

Per la copertura delle decisioni e condizioni i casi di test devono eseguire tutte le possibili combinazioni di condizioni all'interno dell'if. Nel nostro caso abbiamo tre combinazioni delle condizioni all'interno dell'if (quando la prima condizione risulta falsa, Java non verifica la correttezza della seconda quindi viene valutata come un *don't care*)

Con un solo caso di test è possibile eseguire al massimo due cicli while (per via della condizione i<a.length%3) quindi non sarà possibile svolgere nello stesso test le tre combinazioni. È necessario avere due casi di test separati.

Per esempio:

1) a=[1,-2] ; x=5

2) a=[3] ; x=1

Lezione 25: Testing (4/12/2023 Alessandro Bertelli, Mattia Belfiore)

Durante la lezione di oggi si vedranno alcuni vecchi temi d'esame che riassumono le diverse tipologie di esercizi che coprono l'argomento Testing con particolare focus su testing strutturale o white box (perché più frequenti all'esame).

I primi esercizi visti si concentrano sugli aspetti di testing funzionale, ovvero quello basato sulle specifiche, non guardando come è stata realizzate realmente l'implementazione. Il testing funzionale

è un metodo utile per andare a sollecitare un frammento di codice o un metodo basandosi su quella che è la sua specifica e dunque i risultati attesi. Partendo da queste informazioni si cerca di capire quali sono delle classi di casi di test che si comportano in modi simili. È bene non esagerare però con i casi di test in quanto questi devono in ogni caso essere eseguiti e quindi potrebbe essere una operazione costosa.

Esercizio 1

Considerando il metodo booleano bisestile, valido solo per gli anni d.c., si assuma che la nozione di anno bisestile sia definita informalmente come segue.
Un anno è bisestile se e solo se è multiplo di 4 ma non di 100, oppure è multiplo di 400. Sulla base di tale definizione si indichi un insieme di dati per il test funzionale del metodo.

```
boolean bisestile(int anno)
```

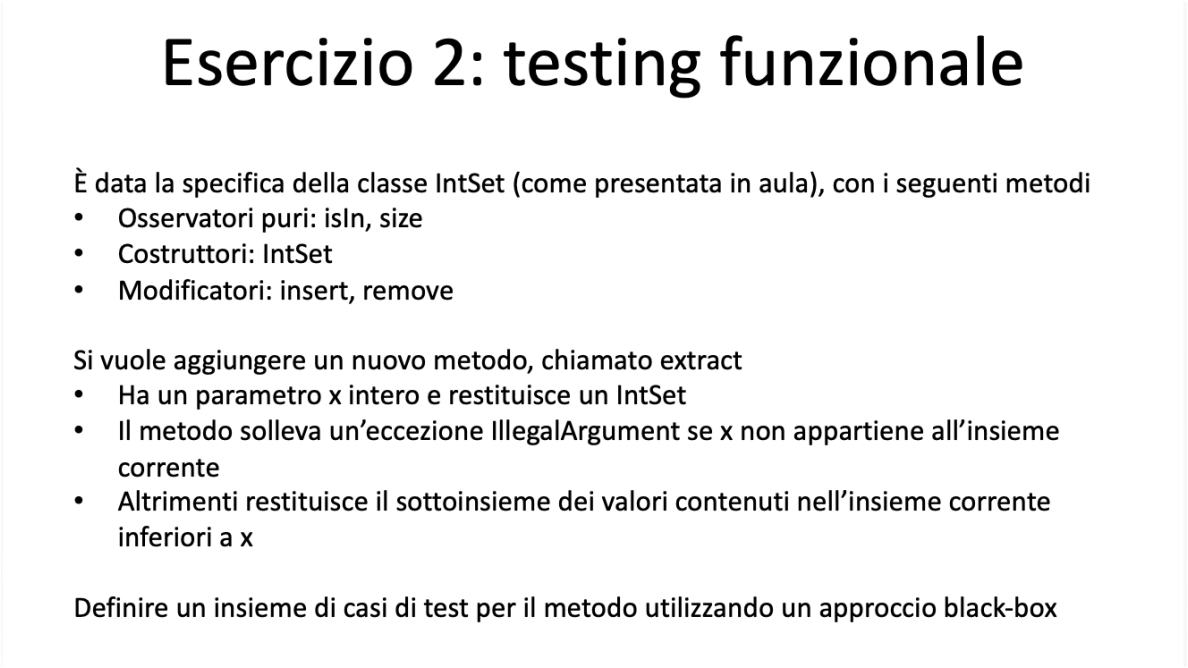
Il caso di test "-1" non è da utilizzare poiché al di fuori delle specifiche mentre vorremmo andare a trovare solo casi di test appartenenti al dominio della funzione o del metodo in considerazione.

In poche parole voglio testare tutte le casistiche che vengono sollevate o trattate nella specifica

Il caso di test limite "1" (o anno 0 nel caso esso sia incluso) potrebbe essere una scelta per il criterio dei casi limite, altri casi interessanti potrebbero essere: "1989" (non multiplo di 4), "1988" (multiplo di 4 ma non di 100), "1300" (multiplo di 100), "1600" (multiplo di 400).

Il caso "1" poteva essere utilizzato non solo come caso limite ma anche come caso di non multiplo di 4, questa scelta sarebbe stata corretta ma è sempre preferibile dividere i casi di limite dagli altri in quanto potrebbero avere comportamenti differenti nell'implementazione.

Esercizio 2: testing funzionale



Esercizio 2: testing funzionale

È data la specifica della classe IntSet (come presentata in aula), con i seguenti metodi

- Osservatori puri: isIn, size
- Costruttori: IntSet
- Modificatori: insert, remove

Si vuole aggiungere un nuovo metodo, chiamato extract

- Ha un parametro x intero e restituisce un IntSet
- Il metodo solleva un'eccezione IllegalArgument se x non appartiene all'insieme corrente
- Altrimenti restituisce il sottoinsieme dei valori contenuti nell'insieme corrente inferiori a x

Definire un insieme di casi di test per il metodo utilizzando un approccio black-box

Osservazioni preliminari:

Input e Stato dell'Oggetto:

- Il metodo extract prende un intero come parametro di input, e questo intero sarà fornito dal chiamante quando il metodo viene invocato.

- Tuttavia, il testing di un metodo non si limita solo all'input. È anche importante considerare lo stato attuale dell'oggetto su cui il metodo è chiamato. Gli oggetti possono mantenere uno stato interno che può influenzare il comportamento dei metodi. Infatti fare testing singifica portare l'oggetto in uno stato interessante a questo punto chiamare i metodi e verificare i risultati attesi sull'output e/o sul nuovo stato dell'oggetto.

N.B. In questo particolare esempio non essendo un metodo modificatore non devo proccuparmi di verificare lo stato successivo all'invocazione del metodo, ma dovro ragionare esclusivamente sull'output.

Tip!
Il testing non dovrebbe essere limitato esclusivamente al metodo considerato come un'entità isolata. Questo perché una funzione è strettamente connessa al suo contesto, e tale contesto di solito si riferisce allo stato corrente dell'oggetto. In altre parole, è fondamentale considerare non solo il comportamento del metodo in sé, ma anche il contesto circostante, specialmente lo stato dell'oggetto a cui appartiene.

Definizione dei casi di test:

Sapendo che l'input è: *int x* e *"l'input implicito"* che è *this* (lo stato precedente di this alla chiamata).

Testo sui casi limite:

- this come elemento vuoto
- this contenente un unico elemento (con x che è proprio quell'elemento / x non è quell'elemento)
- dimensione del set this arbitraria, con x si trova in mezzo o in ultima/prima posizione (condizione di bound: estremi del dominio)

Soluzione	
• this={1, 2} • this={} • this={1, 2, 5} • this={1, 2, 4, 5}	x=5 oppure x=0 (x più piccola, intermedia, più grande) x=5 (x arbitrario e this vuoto) x=5 oppure x=1 (x al limite del dominio) x=4 (x contenuto in this)

In generale ci si ricordi di provare anche i *casi estremi*: puntatore null, insieme vuoto etc...

Tip!
Il "testing dei casi limite" è una strategia comune nel testing funzionale che mira a testare i valori agli estremi, o limiti, delle condizioni di input. L'obiettivo è rilevare eventuali problemi o errori che potrebbero verificarsi in situazioni critiche o particolari. Sulle parti escluse dal dominio posiamo escludere il testing.

Esercizio 3

```
//@ requires true;
//@
//@ ensures elements != null &&
//@   (\forall int I; 0<=i && i<elements.size(); elements.get(i)!=0) &&
//@   \result <==> (\exists int i; 1<=i && i<elements.size();
//@      elements.get(i).equals(elements.get(i-1)) ||
//@      (elements.get(i).equals(elements.get(i-2)) && i>1))
//@
//@ signals (BadArgumentException e)
//@   elements == null ||
//@   (\exists int i; 0<=i && i<elements.size(); elements.get(i)==0)
Public static /*@ pure @*/
Boolean twoNext(ArrayList<Integer> elements) throws BadArgumentException
```

La ensures mi assicura che: deve esistere un intero tale che ci sono due elementi che sono uguali e al più distanti due posizioni l'uno dall'altro.

I casi interessanti potrebbero essere:

elements nullo o non contenente nemmeno un valore: null, {}

elements contenente un valore pari a 0(considero anche i casi in cui la funzione sarebbe soddisfatta proprio dagli 0): {0}, {1, 0}, {0, 1}, {0, 0}, {0, 1, 0}

Casi in cui i due elementi sono contigui, sono separati da al più un valore o non fanno in modo che la funzione ritorni false: {1, 1}, {1, 2, 1}, {1, 1, 1}, {1, 2, 3}, {1, 2, 3, 3}

Posso considerare anche dei casi limite come ad esempio {1, 2, 3, 1} dove la funzione ritornerebbe true se la condizione includesse i casi con due elementi in mezzo a quelli uguali

Le cose da ricordare sono dato un oggetto, sollecitarlo significa portarlo in uno stato interessante e da questo stato sollecitare tutti i possibili cambiamenti di stato con i valori degli input che sono considerati rilevanti per quel particolare stato.

Esercizio 4: testing strutturale

Testo esercizio 4

```
READ foo

READ boo

IF boo > 12 OR foo < 70

      PRINT "pippo "

END IF

PRINT "pluto "

IF foo > 75
```

```
      PRINT "topolino!"
END IF
```

L'esercizio ci richiede di verificare se dato un caso di test soddisfiamo i criteri di:

- **Criterio delle istruzioni:** ogni nodo del Control Flow Graph deve essere eseguito almeno una volta durante il testing; è un criterio di copertura debole
- **Criterio dei cammini:** L'insieme dei casi di test deve garantire che ogni possibile cammino(o percorso) che porti dal nodo iniziale al nodo finale del CFG sia percorso. L'idea è di stimolare tutti i possibili funzionamenti del programma
- **Criterio degli edge:** L'insieme dei casi di test deve essere definito in modo che ogni ramo del "Control Flow Graph" del programma venga attraversato almeno una volta
- **Criterio delle condizioni:** L'insieme dei casi di test deve essere definito in modo che ogni ramo del CFG venga attraversato almeno una volta e con tutti i possibili valori sottoespressioni che compaiono nelle condizioni composte

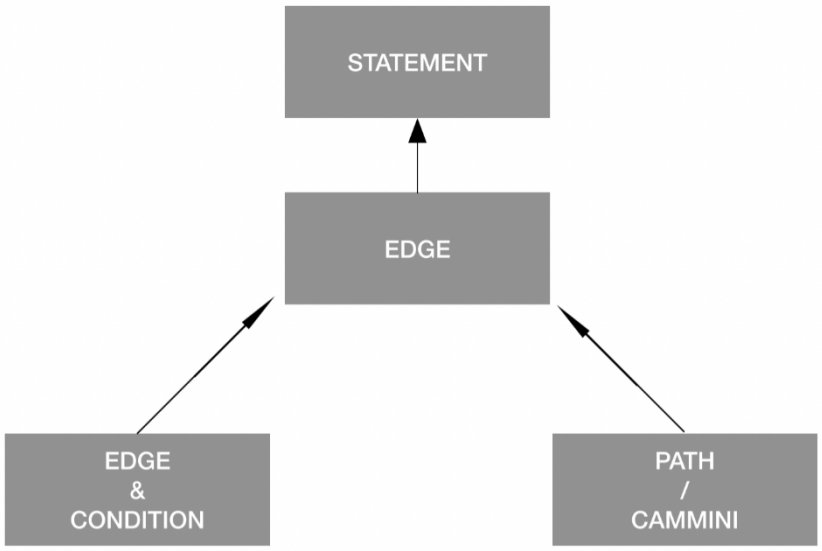
Caso di test 1

Testo esercizio 4	Caso di test 1	Control Flow Graph (CFG)
1. READ foo 2. READ boo 3. IF boo > 12 OR foo < 70 4. PRINT "pippo " 5. END IF 6. PRINT "pluto " 7. IF foo > 75 8. PRINT "topolino!" 9. END IF	boo = 13, foo = 77 Istruzioni OK Cammini NO Edge NO Condizioni NO	

- Criterio delle istruzioni: soddisfatto perche con il caso di test 1 entriamo sia nel *if* alla riga 3 che nell'*if* alla riga 5.
- Criterio dei cammini: ogni cammino, visto come percorso, puo essere sollecitato da solo un caso di test. Questo implica che per ogni cammino ci servirà un caso specifico di test. Per i cammini mi servono 4 casi di test. Con il caso in analisi copriamo V,V e ci manca verificare il cammino F,F nel CFG per cui la risposta è no.
- Criterio degli edge: in teoria mi servirebbero due casi di test (devo coprire tutte le frecce nel grafo almeno una volta) due che coprono V e V nel CFG e F,F nel CFG (oppure V e F oppure F e V andrebbeero bene).

- Criterio delle condizioni: dato che non copro il criterio degli edge posso escludere il criterio delle condizioni siccome questo criterio è più stringente rispetto al precedente (si veda il grafico delle implicazioni sotto).

Implicazioni tra i criteri di copertura



Osservazione sulle implicazioni tra i criteri di copertura: il criterio "statement" è meno stringente rispetto a "edge". In altre parole, se un insieme di casi di test copre tutte le decisioni, si può essere certi che tutte le possibili linee di codice siano state attraversate. Se un insieme di casi di test copre gli "edge", possiamo essere sicuri che anche il criterio "statement" è soddisfatto.

Lo stesso ragionamento si applica anche al passaggio da "edge" a "edge & condition". È necessario coprire gli "edge" e, inoltre, considerare le condizioni composte e tutte le possibili permutazioni di tali condizioni composte per garantire una copertura completa. In questo contesto, il criterio "edge & condition" è più rigoroso di "edge" poiché richiede la considerazione delle condizioni complesse e delle loro varie combinazioni.

Caso di test 2

Testo esercizio 4	Caso di test 2	Control Flow Graph (CFG)
1. READ foo 2. READ boo 3. IF boo > 12 OR foo < 70 4. PRINT "pippo " 5. END IF 6. PRINT "pluto " 7. IF foo > 75 8. PRINT "topolino!"	boo = 14, foo = 74 Istruzioni NO Cammini NO Edge NO	

9. END IF	Condizioni NO	

- Criterio delle istruzioni: il primo *if* viene valutato a *true* mentre il secondo *if* non viene valutato poiché la condizione *foo > 75* non è verificata
- Criterio dei cammini: visto che come illustrato nel Grafo il criterio delle istruzioni è il meno stringente sicuramente gli altri criteri non verranno soddisfatti
- Criterio degli edge: come per cammini
- Criterio delle condizioni: come per cammini

Caso di test 3

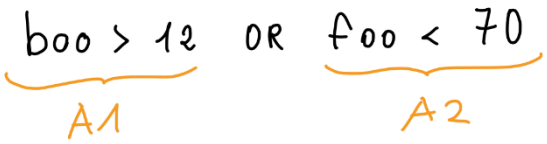
Testo esercizio 4	Caso di test 3	Control Flow Graph (CFG)
1. READ foo 2. READ boo 3. IF boo > 12 OR foo < 70 4. PRINT "pippo " 5. END IF 6. PRINT "pluto " 7. IF foo > 75 8. PRINT "topolino!" 9. END IF	boo = 10, foo = 80 boo = 13, foo = 80 Istruzioni OK Cammini NO Edge NO Condizioni NO	

- Criterio delle istruzioni: il secondo caso (boo = 13, foo = 80) mi rende vero il primo *if*, riga 3, mentre tutti e due i casi coprono il secondo *if* a riga 7
- Criterio dei cammini: no perche non viene soddisfatto il criterio degli edge che è meno stringente dei cammini.
- Criterio degli edge: non valuto mai a false il caso *dell'if* a riga 7.
- Criterio delle condizioni: no perche non viene soddisfatto il criterio degli edge che è meno stringente delle condizioni.

Caso di test 4

Testo esercizio 4	Caso di test 4	Control Flow Graph (CFG)
1. READ foo 2. READ boo 3. IF boo > 12 OR foo < 70 4. PRINT "pippo " 5. END IF 6. PRINT "pluto " 7. IF foo > 75 8. PRINT "topolino!" 9. END IF	boo = 10, foo = 80 boo = 13, foo = 80 boo = 13, foo = 73 Istruzioni OK Cammini NO Edge OK Condizioni NO	

- Criterio delle istruzioni: si per quanto esposto nel caso di test 3
- Criterio dei cammini: no perché dovrei avere al minimo 4 casi di test.
- Criterio degli edge: Devo verificare se gli if sono valutati in true e false. Il primo caso (boo = 10, foo = 80) valuta *false* nell'if a riga 3 e il secondo *if* viene valutato in *false* dal terzo caso (boo = 13, foo = 73)
- Criterio delle condizioni: guarda schema sotto per via delle condizioni composte. Le decisioni che richedono piu condizioni composte sono l'if a riga 3



A1	A2	Soddisfatto da
V	~	II e III caso
F	V	Non verificata
F	F	.....

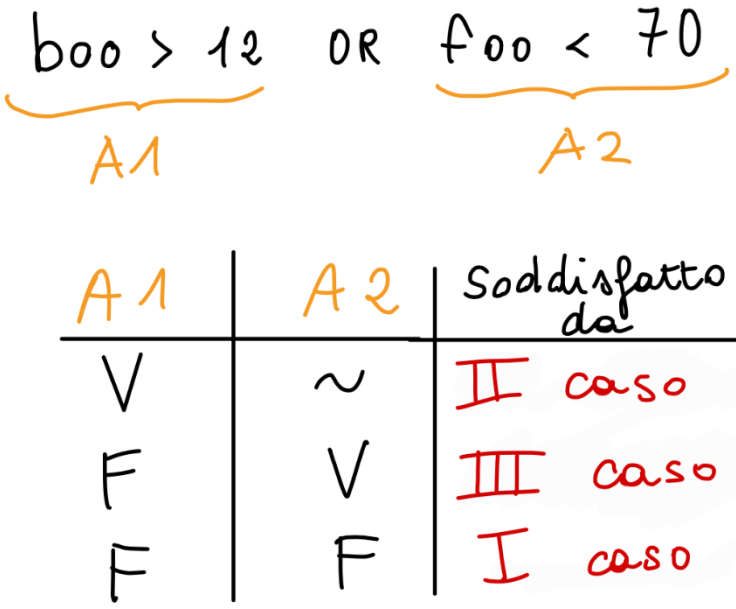
Visto che la condizione A1 Falso e A2 Vero non è mai verificata non soddisfo il criterio delle condizioni.

Caso di test 5

Testo esercizio 4	Caso di test 5	Control Flow Graph (CFG)
1. READ foo 2. READ boo	boo = 10, foo = 80	

3. IF boo > 12 OR foo < 70 4. PRINT "pippo " 5. END IF 6. PRINT "pluto " 7. IF foo > 75 8. PRINT "topolino!" 9. END IF	boo = 13, foo = 80 boo = 9, foo = 60 Istruzioni OK Cammini NO Edge OK Condizioni OK	

- Criterio delle istruzioni: si per quanto esposto nel caso di test 3
- Criterio dei cammini: no perché dovrei avere al minimo 4 casi di test.
- Criterio degli edge: Devo verificare se gli if sono valutati in true e false. Il primo caso (boo = 10, foo = 80) valuta *false* nell'if a riga 3 e il secondo *if* viene valutato in *false* dal terzo caso (boo = 9, foo = 60)
- Criterio delle condizioni: guarda schema sotto per via delle condizioni composte. Le decisioni che richedono piu condizioni composte sono l'if a riga 3



Tutte le condizioni sono verificate pertanto il criterio è soddisfatto.

```
public static int fun(int a, int b) {
    int temp = 0;
    if (a > b) temp++;
    if (a + b > 10) temp += a;
    else temp += b;
    return temp;
}
```

Primo quesito: Insieme di test che copre tutte le istruzioni:

In questa situazione non basta un caso di test solo in quanto è presente un else per il secondo if che va coperto con un caso di test separato:

- a=10, b=5 per coprire l'if con condizione true (a > b) ed il secondo if con condizione true (a + b > 10)
- a=1, b=1 per coprire il primo e il secondo if con condizione false entrando dunque nel ramo else dell'ultimo if poiché (a = b -> prima condizione falsa, a + b = 2 <= 10 seconda condizione falsa)

Secondo Quesito: Insieme di test che copre tutte le decisioni:

Devo comunque ad andare a valutare come false il primo if anche se non possiede un ramo else, per i casi di test che abbiamo considerato nel punto precedente questo controllo è gia stato considerato nel secondo caso di test ovvero quello con a = 1 e b = 1

- a=10, b=5
- a=1, b=1

Terzo Quesito: Insieme di test che copre tutti i cammini(copro cosi tutti i possibili cammini presenti):

Se posso coprire tutti i cammini possibili avrò bisogno di quattro casi di test differenti

- a=10, b=5 per coprire il primo e il secondo if con condizione true
- a=1, b=1 per coprire il primo ed il secondo if con condizione false (percorrendo il ramo else)
- a=2, b=1 per coprire il primo if con condizione true ed il secondo con condizione false
- a=10, b=20 per coprire il primo if con condizione false ed il secondo if con condizione true

Esercizio 6: Tema d'Esame del 18/02/2019

Esercizio 6 (18/02/2019)

```
static List<Integer> addX(List<String> nums, int x) {
   List<Integer> plusX = new LinkedList<>();
   for(String numString : nums) {
      number = Integer.valueOf(numString);
      if (number>0) plusX.add(number + x);
   }
   return plusX;
}
```

Primo quesito: Insieme minimo di test che copre decisioni e condizioni

Considerazioni per la soluzione:

→ *addX(List<String> nums, int x)* prende in ingresso due elementi una lista di stringhe e un intero. I casi di test saranno pertanto una coppia di valori e l'esercizio richiede di coprire decisioni e condizioni. Attenzione però che non fa alcuna differenza coprire criterio delle decisioni o condizioni perche non ho nessuna condizione composta. Infatti sia il *for* che *l'if* devono essere valutati solo per vero o falso.

→ Voglio dei casi di test con cardinalità 1 perché vedendo il corpo del ciclo *for* si capisce che basta entrare e iterare due volte il ciclo in modo da coprire sia il caso *dell'if* a *true* sia il caso *dell'if* a *false*. Concretamente un numero maggiore di 0 coprirà il caso true dell'if mentre un numero negativo coprirà il ramo false dell'if.

→ La soluzione consiste nell'utilizzare un array di dimensione 2, in cui un numero è positivo e l'altro è negativo, insieme a un valore arbitrario x (poiché x non è vincolato da alcuna istruzione decisionale o iterativa).

Soluzione proposta dal testo: nums={"1", "-1"}, x=1

Secondo quesito: Quanti sono i cammini nel codice, se num è una lista di 10 elementi che contiene esattamente 2 stringhe rappresentanti numeri positivi.

Considerazioni per la soluzione:

→ I parametri fissati dalla dal quesito sono: lista di 10 elementi e il numero di stringhe rappresentanti numeri positivi. Il ciclo verrà iterato 10 volte, mentre i cammini dipendono da quali iterazioni entro nell'if e quali non entro nell'if.

→ Abbiamo due posizioni da coprire con numeri positivi, come specificato nel testo. Supponiamo che la prima posizione positiva sia all'indice [0] della lista. In questo caso, abbiamo 9 possibilità per trovare il secondo elemento positivo nella lista. Se, invece, la prima posizione positiva è all'indice [1] della lista, abbiamo 8 possibilità per trovare il secondo elemento positivo. Continuando questo ragionamento, se la prima posizione positiva è all'indice [2], avremo 7 possibilità per trovare il secondo elemento positivo nella lista, e così via...

Soluzione proposta dal testo: Quindi avrò 9 + 8 + 7 + … + 1 = 45 cammini possibili.

TDE 16/07/2019

```
public static boolean m(Predicate<String> p, String s) {
  if (p == null) p = x -> true;
  if (s == null) s = "";
  if (s.length() < 2) return false;
  else if (p.test(s)) return true;
  else return false;
}
```

Analizziamo velocemente la funzione. Abbiamo una funzione che prende in ingresso una stringa ed un predicato. Un predicato è un interfaccia funzionale ovvero un interfaccia con un unico metodo, in questo caso test, esso prende in ingresso una stringa e ritorna un booleano in seguito ad una valutazione su una serie di if.

Primo Quesito: Insieme di test minimo per copertura istruzioni:

Ho bisogno di almeno tre casi di test per la copertura delle istruzioni a causa della presenza dell' else if e dell'else. Prenderò quindi tre test uno che copra l'ultimo if con condizone true (s.length() < 2), uno che renda true la condizione dell'else if (p.test(s)) e uno che abbia false sulle due condizioni precedentemente trattate

- p = null, s = null, copre I tre if con condizione true
- p = x -> true, s = "aaa", copre I primi due if con condizione false mentre rientra nella condizione dell'else if
- p = x -> false, s = "aaa", copre copre i vari if con condizione false, supera l'else if non soddisfacendolo e arrivando dunque all'ultimo else

Secondo Quesito: Insieme di test minimo per copertura decisioni:

Con i casi di test identificati prima copriamo gia tutte le decisioni presenti.

- p = null, s = null
- p = x -> true, s = "aaa"
- p = x -> false, s = "aaa"

Esercizio 8: Tema d'Esame del 16/01/2020

Esercizio 8 (16/01/2020)

```
int f(int a, int b) {
   if(a > b) System.out.println("A");
   else System.out.println("B");
   for (int i=0; i<1000; i++) {
      if (a > 0) break;
      else a = 1;
   }
}
```

Primo quesito: Insieme minimo di test per coprire istruzioni

Considerazioni per la soluzione:

→ Guardiamo innanzi tutto come è fatto il caso di test: ci sono un *if* e un *else* quindi minimo 2 condizioni di test per via del ramo *else*. Guardando il ciclo *for* poi capisco che se ***a**>0* esco, mentre se *a<0* allora ***a**=1* e quindi al prossimo ciclo uscirò.
→ Nel ciclo *for* quindi ho la possibilità di entrare 1 o 2 volte al massimo.
→ Quindi mi servirà per la soluzione un caso in cui "a" sia minore di 0 e un caso in cui *a>b* e in uno di questi due casi "a" deve essere minore di 0 (in modo che venga coperta anche il secondo if).

Soluzione proposta dal testo: primo caso: a=1, b=1 – secondo caso: a=-1, b=-2

Secondo quesito: Insieme minimo di test che coprire decisioni

Considerazioni per la soluzione:

→ Dimenticandoci inizlamente il for: *if (a>b)* è valutato con il secondo caso proposto (*a=-1, b=-2*). Il ramo else è valutato dal primo caso (*a=1, b=1*). Spostandoci dentro il for: If (a > 0) viene valutato con entrambi i casi proposti (*a=1, b=1 – a=-1, b=-2*), mentre il ramo else che esprime quando a<0 viene valutato con il secondo caso (*a=-1, b=-2*).
→ Posso fare meglio di cosi (posso coprire anche il *for*)? La decisione di uscire dal *for* non la prendo mai ma non posso prenderla perché è condizione non ***reachable***.

Soluzione proposta dal testo: Impossibile: "i < 1000" non viene mai valutata come falsa. Casi di test precedenti sono sufficienti per coprire tutte le decisioni possibili.

Terzo quesito: Insieme minimo di test per coprire tutti i cammini

Considerazioni per la soluzione:

→ Per coprire tutti i cammini possibili, è necessario considerare le diverse combinazioni di condizioni all'interno del blocco di codice.
→ I cammini possibili sono:
    - 1. if vero e if interno vero

- 2. if falso vero ed esco
- 3. vero falso e al giro dopo diventa vero necessariamente e esco

→ Intuitivamente sono 4, poiché, date le caratteristiche del ciclo for, l'uscita avviene immediatamente o al ciclo successivo.

Soluzione proposta dal testo: Esecuzione passa da if oppure da else e entra nel ciclo 0 o 1 volta -> servono 4 casi

• a=1, b=0

• a=1, b=2

• a=-1, b=0

• a=-1, b=-2

TDE 10/07/2023

```
public static int test(int x, int y) {
    int k = 0;
    if(x>0) {
        if(y>0) {
            k += y;
        } else {
            k += -y;
        }
    }
    if(x==10) {
        k += 10;
    }
    return k;
}
```

Primo Quesito: Insieme di test minimo per coprire istruzioni:

Ho bisogno di almeno due casi di test per coprire le istruzioni. Questo perché riesco a generare due casi di test che entrino nel primo if con condizione true (x > 0) gestendo le istruzioni derivate dalla risoluzione del secondo if sia con condizione true (y > 0) che false (y <= 0), questi due casi possono inoltre andare a coprire l'ultimo if con condizione true (x == 10)

- (10, 1), copro i due if annidati con condizione true e avendo x == 10 anche l'ultimo if viene gestito con condizione true
- (10, 0), copro i primi due if annidati considerando la condizione sul primo come true e la condizione sul secondo come false rientrando dunque nel ramo else. Copro infine l'ultimo if con condizione true grazie a x == 10

Secondo Quesito: Insieme di test minimo per coprire decisioni:

Mi basta aggiungere un caso che permette di valutare il primo if con condizione false e l'ultimo if anch'esso con condizione false

- (10, 1)
- (10, 0)

- (0, qualsiasi), copro il primo if con condizione false, facendo in modo che qualsiasi valore di y risulti valido, copro infine l'ultimo if con condizione false

Terzo Quesito: Insieme di test minimo per coprire cammini:

Avrei teoricamente 6 cammini possibili, tuttavia se scelgo un valore di x < 0 non posso avere x == 10 rendendo un caso irraggiungibile.

- (10, 1)
- (10, 0)
- (0, qualsiasi)
- (5, 1), copre I due if annidati con condizione true ma l'ultimo con condizione false
- (5, 0), copre tutti gli if con condizione false

Lezione 26: Programmazione distribuita – Socket/RMI (5/12/2023 Sara Berardinelli, Marco Apollonio)

Due elementi importanti per la programmazione distribuita, i nodi fisici e logici.

Il nodo fisico si riferisce a un dispositivo hardware reale: un computer o un server, che è fisicamente presente in una rete.

Il nodo logico si riferisce ad un'entità virtuale o concettuale all'interno di una rete che svolge funzioni specifiche. Lo si può rappresentare come un'astrazione di un nodo fisico, permessa da Java.

Da qui andremo a vedere due possibili implementazioni per la programmazione distribuita: i socket e la RMI.

Architettura client-server

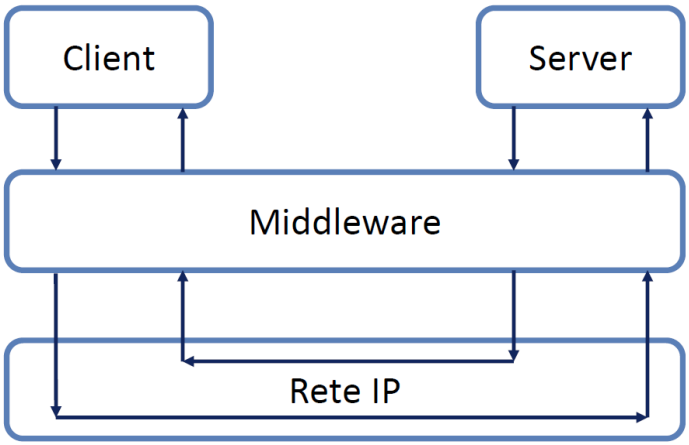
È il metodo classico di progettazione di applicazioni distribuite su rete. Questo tipo di architettura suddivide le funzionalità di un'applicazione o servizio in due categorie: i client, che richiedono e consumano i servizi e i server, che forniscono i servizi richiesti dai client.

Un client può essere un'applicazione o un dispositivo. Nel nostro caso saranno delle applicazioni scritte in Java.

Un server è un computer che esegue un programma che attende le richieste dei client, le elabora e restituisce le risposte appropriate.

Questo modello consente una distribuzione efficiente delle risorse e delle responsabilità, consentendo ai server di gestire risorse condivise in modo centralizzato e ai client di accedere e utilizzare tali risorse in modo distribuito.

Middleware



Il middleware è un software che si trova tra il sistema operativo e le applicazioni che fornisce servizi aggiuntivi e funzionalità di supporto per semplificare lo sviluppo e la gestione delle applicazioni distribuite.

Fornisce servizi comuni e funzionalità di supporto che consentono alle applicazioni di comunicare, condividere dati, gestire transazioni, garantire la sicurezza e scalare in un ambiente distribuito.

Questo software fa già parte del linguaggio Java, diversamente da altre soluzioni

Socket

Un socket rappresenta un'estremità di una connessione di rete bidirezionale tra due programmi in esecuzione su computer diversi o sulla stessa macchina.

Sono di tipo TCP(Transmission Control Protocol), attraverso uno stream, che rimane aperto finché la connessione non viene dichiaratamente bloccata, si mettono a disposizione due diversi processi con le seguenti caratteristiche: bidirezionalità, flusso ordinato di dati e la certezza che i dati arriveranno.

Java offre un servizio al di sopra del semplice invio di byte, il quale permette di inviare, con una serializzazione automatica, degli oggetti, attraverso delle classi; è quindi possibile inviare strutture complesse che possono essere ricostruite.

Si possono identificare i socket attraverso la combinazione:

- Indirizzo IP
- Numero di Porta

Come ad esempio 192.168.0.1:80

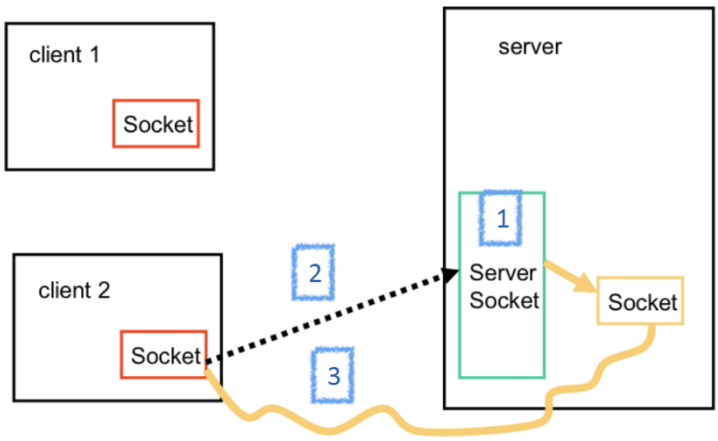
Sia per il client che per il server abbiamo bisogno di questi due elementi per l'identificazione.

Definizione di Socket dal Tutorial di Java

A socket is one end-point of a two-way communication link between two programs running on the network. Socket classes are used to represent the connection between a client program and a server

program. The java.net package provides two classes--Socket and ServerSocket--that implement the client side of the connection and the server side of the connection, respectively.

Comunicazione client-server



Attesa della Connessione

Si Crea un'istanza della classe java.net.ServerSocket specificando il numero di porta su cui rimanere in ascolto, settando così la porta:

EchoServer.java

```
ServerSocket sc = new ServerSocket(4567);
```

A questo punto il server si trova in ascolto di una determinata porta, nel nostro caso la 4567, dove la comunicazione è limitata in questo canale con una distruzione distribuita:

EchoServer.java

```
ServerSocket ServerSocket = null;
try {
   ServerSocket = new ServerSocket(port);
} catch (IOException e) {
   System.err.println(e.getMessage());
}
```

In seguito si chiama il metodo accept() che fa in modo che il server rimanga in ascolto di una richiesta di connessione:

EchoServer.java

```
Socket s = sc.accept();
```

Quando il metodo completa la sua esecuzione la connessione con il client è stabilita e viene restituita un'istanza di java.net.Socket connessa al client remoto.

Questo va a creare una primitiva bloccante, che quindi va a bloccare il chiamante e lo mette in attesa di un client. Una volta che il client si sblocca, al server ritorna una socket che avrà due metodi getter:

- InputStream
- OutputStream

EchoServer.java

```
in = new Scanner(socket.getInputStream());
out = new PrintWriter(socket.getOutputStream());
```

Che possono usare tutte le classi di utilità sia su file che su socket.

Questo codice deve gestire l'eccezione IOException.

A questo punto abbiamo la parte di codice in cui avviene la lettura dei byte provenienti dal client e la loro relativa stampa verso l'output:

EchoServer.java

```
while (true) {
   final String line = in.nextLine();
   if(line.equals("quit")) {
      break;
   } else {
      out.println("Received: " + line);
      out.flush();
   }
}
```

Anche il metodo nextLine() è bloccante.

Il metodo flush() è invece necessario, permette la pulizia del buffer nel caso in cui siano presenti politiche di buffering o di cashing.

Apertura della connessione

Ci troviamo ora nel codice di LineClient.java

Come per il server dovrà essere gestita l'eccezione IOException.

Si apre un Socket, non una server socket come nel caso del server, specificando l'indirizzo IP e il numero di porta del server (in questo caso, 127.0.0.1/4567).

LineClient.java

```
//Si sta instaurando una connessione con se stessi
```

```
//sulla porta 4567
Socket sc = new Socket("127.0.0.1", 4567);
```

Il numero di porta è compreso fra 1 e 65535. Le porte con numero inferiore al 1024 sono riservate a dei servizi standard. Ad esempio: 80 HTTP, 22 SSH, 25 SMTP.

Per far sì che la connessione venga stabilita, deve esserci in ascolto un processo server. Se la connessione ha successo si usano sia dal lato client che dal lato server gli stream associati al socket per permettere la comunicazione tra client e server e viceversa. Per ottenere InputStream e OutputStream utilizziamo questi due metodi getter dell'oggetto Socket:

LineClient.java

```
Scanner in = new Scanner(sc.getInputStream());
PrintWriter out = new PrintWriter(sc.getOutputStream());
```

Analogamente al caso del Server anche per il Client utilizziamo flush() per le stesse ragioni elencate prima.

LineClient.java

```
Final Socket socket = new Socket(ip, port);
```

In questo modo viene instaurato l'handshake del TCP, che va a stabilire una connessione con il server, se è attivo; se questa connessione viene stabilita ritorna un socket. Da questo punto in poi il codice è simmetrico a quello del Server.

Quindi lo Scanner e la PrintWriter vengono utilizzate per comunicazioni diverse, ad esempio lo Scanner per leggere da input del Socket, invece lo Scanner stdin da tastiera; stream quindi da ogni classe di utilità.

LineClient.java

```
final Scanner socketIn = new Scanner(socket.getInputStream());
final PrintWriter socketOut = new PrintWriter(socket.getOutputStream());
finel Scanner stdin = new Scanner(System.in);
```

Qui vediamo la porzione di codice dedicata alla comunicazione:

LineClient.java

```
try {
   while(true) {
      final String inputLine = stdin.nextLine();
      socketOut.println(inputLine);
      socketOut.flush();
      final String socketLine = socketIn.nextLine();
      System.out.println(socketLine);
   }
} catch (final NoSuchElementException e) {
   System.out.println("Connection closed");
```

```
} finally {
   stdin.close();
   socketIn.close();
   socketOut.close();
   socket.close();
}
```

Chiusura delle connessioni

Per chiudere un ServerSocket o un Socket si utilizza il metodo close().

Per ServerSocket, questo metodo fa terminare la accept() con IOException. Mentre per Socket, fa terminare le operazioni di lettura o scrittura del socket con eccezioni che dipendono dal tipo di reader/writer utilizzato.

Sia ServerSocket sia Socket hanno un metodo isClosed() che restituisce vero se il socket è stato chiuso.

Un server però dovrebbe essere in grado di accettare connessioni da diversi client e di dialogare con questi "contemporaneamente". Allora si può pensare a un server "multi-thread":

- All'interno del processo Server far eseguire le istruzioni dopo l'accept() in un nuovo thread
- In questo modo è possibile accettare più client contemporaneamente.

Quindi spostiamo la logica che gestisce la comunicazione con il client in una nuova classe ClientHandler che implementerà Runnable. La classe principale del server si occuperà solo di istanziare il ServerSocket, eseguire la accept() e di creare i thread necessari per gestire le connessioni accettate.

Dopo aver accettato la connessione con il Socket, delego la gestione della comunicazione ad un thread che farà quello che faceva prima.

In questo esempio utilizziamo un ExecutorService con un pool di 10 thread:

MultiEchoServer.java

```
executor.submit(() -> {
   try {
      final Scanner in = new Scanner(socket.getInputStream());
      final PrintWriter out = new PrintWriter(socket.getOutputStream());
      while(true) {
         final String line = in.nextLine();
         if (line.equals("quit")) {
            break;
         } else {
            out.println("Received. " + line);
            out.flush();
         }
      }
```

```
        in.close();
        out.close();
        socket.close();
    } catch (final IOExcpetion e) {
        System.err.println(e.getMessage());
    }
})
```

Se si cerca di lanciare una seconda volta il programma che fa da server si viene a generare una BindExecption, non potendo creare un ServerSocket su una porta già in utilizzo.

Passiamo ora al codice di MessageServer.java

Per poter inviare oggetti attraverso il canale di rete, gli oggetti devono essere serializzabili, per fare questo controllo Java deve implementare la classe Serializable, che è una Marker Interface.

Quello che cambia rispetto ai precedenti codici sono le classi che vanno a serializzare e deserializzare gli oggetti:

MessageServer.java

```
out = new ObjectOutputStream(socket.getOutputStream());
in = new ObjectInputStream(socket.getInputStream());
```

Se questo non risulta possibile Java lancia un'eccezione.

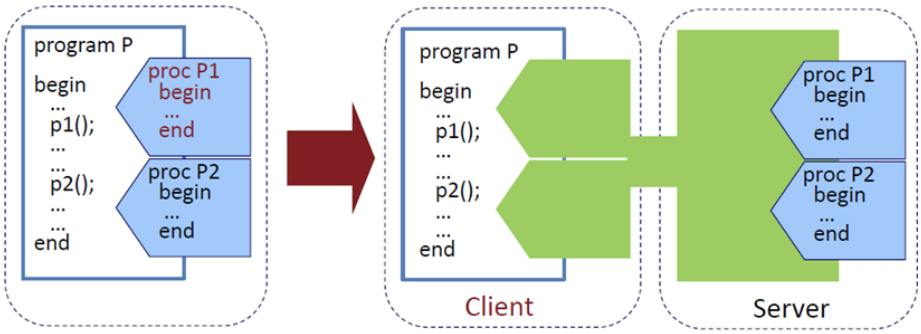
Successivamente nella lettura devo eseguire dei cast agli oggetti in lettura:

MessageServer.java

```
Message = (Message) in.readObject();
```

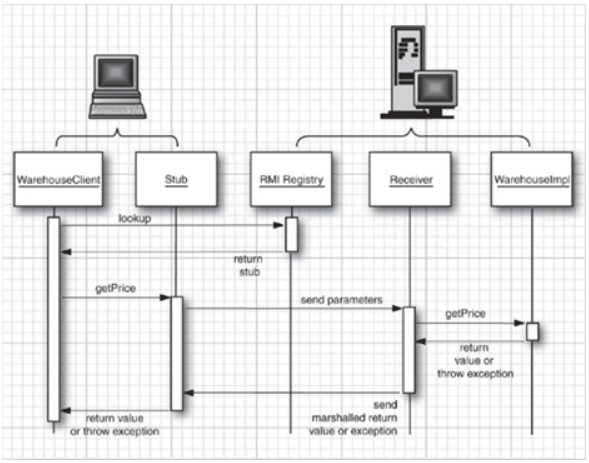
Remote Method Invocation (RMI)

La Remote Method Invocation (RMI) è una tecnologia di Java che consente a un programma Java di chiamare metodi su oggetti situati su un'altra macchina virtuale Java (VM). RMI offre l'illusione che un metodo locale venga chiamato, anche se l'oggetto a cui il metodo appartiene si trova su un altro sistema. Gli oggetti con i quali si desidera interagire tramite RMI devono implementare un'interfaccia remota e devono essere serializzabili, cioè devono poter essere convertiti in un flusso di byte in modo da essere trasmessi sulla rete. È inoltre utilizzato per creare applicazioni distribuite complesse, in cui i diversi componenti dell'applicazione risiedono su macchine diverse e devono comunicare tra loro.



Quello che cerchiamo di utilizzare è un meccanismo con il quale il programmatore del cliente esegue una normale chiamata a metodo, nella quale non c'è bisogno di preoccuparsi della presenza di una rete in mezzo.

RMI Registry



Si può identificare il RMI Registry come un documento in cui il server ha il permesso di registrare degli elementi, tramite nome. Il client può effettuare la ricerca su questo documento degli elementi disponibili.

Architettura interna

Il Client colloquia con un proxy locale del server, detto stub:

- Lo stub "rappresenta" il server sul lato client
- Implementa l'interfaccia del server
- È capace di fare forward di chiamate di metodi attraverso la rete

Esiste anche un proxy del client sul lato server, detto skeleton:

- È una rappresentazione del client
- Chiama i servizi del server
- Sa come fare forward dei risultati attraverso la rete

Stub e Skeleton dalla documentazione ufficiale

When a stub's method is invoked, it does the following:

- Initiates a connection with the remote JVM containing the remote object,
- Marshals (writes and transmits) the parameters to the remote JVM
- Waits for the result of the method invocation
- Unmarshals (reads) the return value or exception returned
- Returns the value to the caller

Each remote object may have a corresponding skeleton:

- Unmarshals (reads) the parameters for the remote method
- Invokes the method on the actual remote object implementation
- Marshals (writes and transmits) the result (return value or exception) to the caller

Come avviare il Registry RMI

- Su Linux e macOS
  $ rmiregistry

- Su Windows
  $ start rmiregistry

In generale è necessario che l'eseguibile rmiregistry sia presente nel path. Viene distribuito con Java.

Questo comando avvia un processo che si mette in attesa che qualcuno che aggiunga un oggetto al Registry. E un altro processo che richiederà le informazioni a riguardo di quell'oggetto.

Cosa posso aggiungere al Registry

Gli oggetti che possono essere aggiunti al Registry dovranno estendere la classe Remote.

Tutti i metodi dovranno eseguire il throws della RemoteException che verrà gestita dalla libreria stessa di RMI, come ad esempio nel caso di perdita di connessione.

Il chiamante vedrà solo i metodi implementati attraverso l'interfaccia, gli eventuali altri metodi implementati non possono essere invocati.

Vado quindi a definire un'interfaccia, che sarà quella esposta verso l'esterno e cui i Client possono accedere, che estende Remote, e tutti i metodi devono quindi lanciare la Remote Exception:

ChatServer.java

```
public interface ChatServer extends Remote {
   public void join(ChatClient client) throws RemoteException;
   public void leave(ChatClient client) throws RemoteException;
```

```
    public void sendMsg(Message msg) throws RemoteException;
}
```

In questo caso facciamo partire il registry direttamente dal codice, attraverso il metodo createRegistry():

ChatServerImpl.java

```
public static void main(String args[]) throws Exception {
    ChatServerImpl server = new ChatServerImpl();
    Registry registry = LocateRegistry.createRegistry(1099);
    registry.bind("ChatServer", server);
    System.out.println("Server bound and ready");
}
```

Successivamente si esegue il metodo bind(), che va ad associare la stringa passata come primo parametro del metodo all'oggetto passato come secondo parametro nel registry. Rendendolo disponibile per le eventuali richieste dei client.

Alla fine dell'esecuzione del main della classe ChatServerImpl.java il suo processo termina, ma quello del RMI Registry rimarrà in esecuzione con gli oggetti che sono stati aggiunti ad esso.

Come vengono passati gli oggetti attraverso RMI

Abbiamo tre possibili casi in cui verrà gestito l'oggetto:

1. Se l'oggetto non è serializzabile allora viene generata un'eccezione
2. Se l'oggetto è serializzabile e Remote allora questo viene passato come un riferimento
3. Se l'oggetto è solamente serializzabile allora viene fatta una copia

Esempio

Ipotizziamo di avere un server con un metodo che può essere richiamato attraverso RMI che esegue il settaggio di un parametro di un oggetto di tipo Person:

ServerObj

```
void RMI(Person p){
    p.setEta(10);

}
```

E abbiamo un Client che nel suo main possiede un oggetto Person p1 ed esegue

ClientObj

Person p1
…
p1.setEta(20)
 	
//Invocazione del metodo RMI del Server
ServerObj.RMI(p1)

Quanto varrà p1. Alla fine dell'esecuzione?

Dipende:

- Se p1 è serializzabile allora viene fatta una copia, che sono separate tra di loro quindi p1.eta = 20
- Se p1 è serializzabile ed estende Remote allora sarà passato per riferimento e quindi p1.eta = 10

Anche il server può avere dei riferimenti ai client attraverso l'estensione della interfaccia Remote.

ChatClient.java

```
public interface ChatClient extends Remote {
   public void printMsg(Message msg) throws RemoteExcpetion;
}
```

Quindi il metodo sendMsg() dell'interfaccia ChatServer verrà invocata da un client per inviare un messaggio e il metodo printMsg() dell'interfaccia ChatClient verrà invocata dal server per inoltrare il messaggio.

ChatClientImpl.java

```
public void doJob(String serverHost) throws Exception {
   ChatServer server;
   Message msg;

   // take a reference of the server from the registry
   server = (ChatServer) Naming.lookup("rmi://" + serverHost + "/ChatServer");

   //join
   server.join(this);
```

Naming.lookup() è il metodo che mi permette di contattare il RMI Registry e di accedere all'oggetto collegato al nome cercato.

Il metodo join() di server passa il riferimento del client al server.

RMI non è thread safe, per questo motivo è il programmatore che deve implementare la sincronizzazione, RMI infatti gestisce separatamente gli ingressi e le uscite utilizzando thread diversi.

Possiamo trovare nel codice:

ChatServerImpl.java

```
public void sendMsg(Message msg) throws RemoteException {
   List<ChatClient> clientsCopy = null;
   synchronized (this) {
      clientsCopy = new ArrayList<ChatClient>(clients);
   }
   for (ChatClient c : clientsCopy) {
      c.printMsg(msg);
   }
}
```

Vado ad eseguire la synchronize sulla copia perché nel ciclo sto eseguendo delle chiamate di rete che se, sincronizzate, potrebbero bloccare l'esecuzione di molteplici programmi o dispositivi.

Cosa viene effettivamente inviato con una richiesta RMI

Cosa succede se un client crea un nuovo sotto tipo dell'interfaccia accessibile dal RMI.

Il server non possiede la codebase di questa sotto classe, fino a Java 17 il dispositivo poteva scaricare la codebase remota, attraverso il Dynamic Code Loading. Dalle versioni successive per motivi di sicurezza è stato disattivato, e quindi non è più possibile avere codebase diverse tra i diversi dispositivi.

Java, inoltre, durante l'invio dei messaggi garantisce la referencial integrety, se, quindi, inviamo un grafo tramite la rete abbiamo la garanzia che due nodi se puntano allo stesso nodo successivo, anche al destinatario arriverà la stessa struttura.